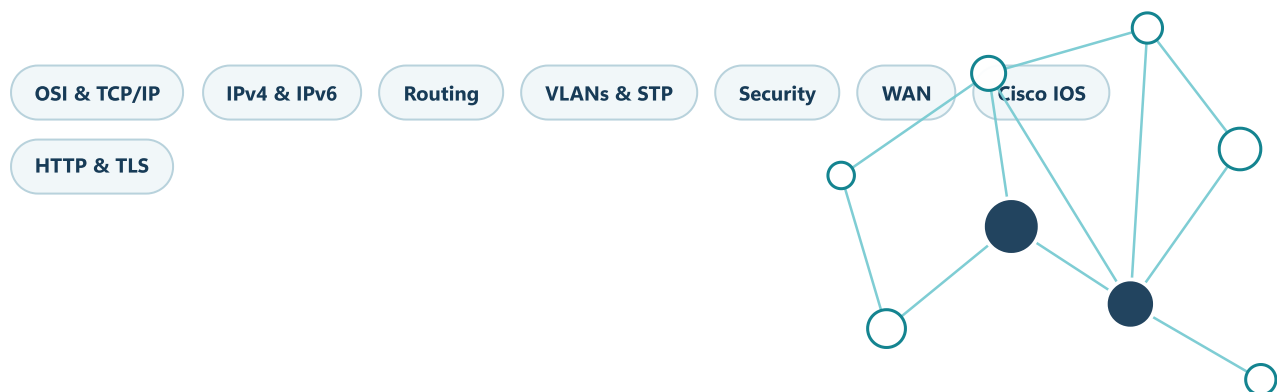


INTERVIEW STUDY GUIDE

COMPUTER NETWORKS

Interview Notes

A structured, practical guide from network fundamentals to routing, switching, security, WAN technologies, and the modern web stack.



CDAC DITISS · 16 integrated notes · Interview-oriented reference

NAVIGATION

Contents

Start with the roadmap, use the checklist to prioritize revision, then follow the chapters in order. Every entry and cross-reference is clickable in the PDF.

Start Orientation

GUIDE Computer Networks — Map of Content

00 Computer Networks — CDAC DITISS Syllabus

I Network Foundations

01A OSI Model — Complete Study Notes

01B OSI Layers: Layer 1 to Layer 7

02 TCP/IP Model — Study Notes

03 IPv4 & IPv6 Addressing, Subnetting, VLSM, and NDP

II Routing & Network Services

04A Network Routing — Detailed Study Notes

04B  Module 4: Routing — Complete Notes

05  VLAN — Complete Notes

06  Module 6: NAT (Network Address Translation) — Complete Notes

III Resilience & Infrastructure Security

08 Spanning Tree Protocol (STP)

09 Port Security, AAA & Access Control Lists (ACLs)

IV WAN, Switching & Router Operations

10 10. PPP and WAN Technologies

11 Switching Types, Ethernet Forwarding & Layer 2 Switching

13 Router Fundamentals: IOS, Boot Process, Memory, CLI Modes, Remote Access, Password Recovery & Troubleshooting

A Application-Layer Appendix

A1 HTTP Evolution — Interview Notes (0.9 → 1.0 → 1.1 → 2 → 3)

STUDY GUIDE · ORIENTATION

Computer Networks — Map of Content

A guided reading order and coverage map for the complete collection.

computer networks

moc

Master guide: [Syllabus and Interview Checklist](#)

Table of Contents

- Recommended Reading Order
- Supplemental Notes

Recommended Reading Order

1. OSI Model

Complete coverage across multiple notes: [Complete Notes](#), [Quick Reference](#)

2. TCP/IP Model

Complete coverage: [02 - TCP-IP Model](#)

3. IP Subnetting & VLSM

Complete coverage: [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)

4. Routing

Distributed complete coverage: [04A - Network Routing Fundamentals](#), [04B - Routing Protocols and Administrative Distance](#)

5. VLANs & Inter-VLAN Routing

Complete coverage: [05 - VLANs and Inter-VLAN Routing](#)

6. NAT (Network Address Translation)

Complete coverage: [06 - Network Address Translation](#)

7. IPv4 vs IPv6

Complete coverage: [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)

8. Spanning Tree Protocol (STP)

Complete coverage: [08 - Spanning Tree Protocol](#)

9. Infrastructure Security — ACL, AAA

Complete coverage: [09 - Infrastructure Security ACL AAA and Port Security](#)

10. PPP and WAN Technologies

Complete dedicated coverage: [Dedicated Notes](#); supporting coverage in [PPP/PPPoE](#) and [WAN link references](#)

11. Layer 2 Switching

Complete coverage: [11 - Layer 2 Switching and Ethernet Forwarding](#)

12. Ethernet and Wireless (IEEE Standards)

Distributed complete coverage: [Ethernet](#), [Wireless](#)

13. Router IOS and Management

Complete coverage: [13 - Router IOS and Management](#)

14. SDN — Software Defined Networking

Unavailable: No dedicated note provided.

15. OpenFlow and OpenDaylight

Unavailable: No dedicated note provided.

16. Virtual Networking

Unavailable: No dedicated note provided.

17. Advanced SDN / OpenDaylight Topics

Unavailable: No dedicated note provided.

Supplemental Notes

-
- [A1 - HTTP Evolution and TLS](#)
-

Source note: [Index.md](#)

CHAPTER 00 · ORIENTATION

Computer Networks — CDAC DITISS Syllabus

Interview priorities, completion tracking, and the essential ports and protocols.

computer networks

syllabus

interview preparation

moc

Topics, Interview Priority & Important Ports/Protocols

Table of Contents

- Topics, Interview Priority & Important Ports/Protocols
- Completion Checklist
-  PRIORITY 1 — MUST KNOW (Most Asked in Interviews)
-  PRIORITY 2 — IMPORTANT (Frequently Asked)
-  PRIORITY 3 — GOOD TO KNOW (Asked in Advanced Rounds)
-  INTERVIEW SPECIAL — IMPORTANT PORTS & PROTOCOLS
-  QUICK SYLLABUS TOPIC LIST (All Sessions)
- Related Notes

Completion Checklist

Priority 1 — Must Know

-  1. OSI Model — Complete coverage: [Complete Notes](#) · [Quick Reference](#)
-  2. TCP/IP Model — [Notes](#)
-  3. IP Subnetting & VLSM — [Notes](#)
-  4. Routing — [Fundamentals](#) · [Protocols](#)
-  5. VLANs & Inter-VLAN Routing — [Notes](#)
-  6. NAT (Network Address Translation) — [Notes](#)
-  7. IPv4 vs IPv6 — [Notes](#)

Priority 2 — Important

-  8. Spanning Tree Protocol (STP) — [Notes](#)
-  9. Infrastructure Security — ACL, AAA — [Notes](#)
-  10. PPP and WAN Technologies — Complete coverage: [Dedicated Notes](#) · Supporting coverage: [PPP/PPPoE](#) · [WAN link references](#)
-  11. Layer 2 Switching — [Notes](#)
-  12. Ethernet and Wireless (IEEE Standards) — Distributed coverage: [Ethernet](#) · [Wireless](#)

Priority 3 — Good to Know

-  13. Router IOS and Management — [Notes](#)

- ☐ 14. SDN — Software Defined Networking — No dedicated note provided
- ☐ 15. OpenFlow and OpenDaylight — No dedicated note provided
- ☐ 16. Virtual Networking — No dedicated note provided
- ☐ 17. Advanced SDN / OpenDaylight Topics — No dedicated note provided

Progress: 13 of 17 topics complete (76%).

PRIORITY 1 — MUST KNOW (Most Asked in Interviews)

1. OSI Model

- 7 Layers: Physical, Data Link, Network, Transport, Session, Presentation, Application
- Role of each layer with real-world examples
- PDU at each layer (Bits, Frames, Packets, Segments, Data)
- Encapsulation and De-encapsulation

2. TCP/IP Model

- 4-layer model vs OSI 7-layer mapping
- TCP vs UDP — differences, use cases
- Why video streaming uses UDP instead of TCP
- Three-way handshake (SYN → SYN-ACK → ACK)
- Four-way termination (FIN → FIN-ACK → FIN → ACK)

3. IP Subnetting & VLSM

- IPv4 address classes (A, B, C, D, E)
- Subnet mask, CIDR notation
- Variable Length Subnet Masking (VLSM)
- Network address, Broadcast address, Host range calculation
- Practice: 192.168.1.0/24, 10.0.0.0/8, 172.16.0.0/16

4. Routing

- Static Routing vs Dynamic Routing
- Distance Vector (RIP) vs Link State (OSPF)
- Administrative Distance concept
- IGP vs EGP
- Protocols: RIP, IGRP, EIGRP, OSPF, BGP

5. VLANs & Inter-VLAN Routing

- VLAN concept, benefits, port assignment
- VLAN tagging — IEEE 802.1Q
- Trunking — trunk ports vs access ports
- Inter-VLAN routing (Router-on-a-stick, Layer 3 switch)
- VTP modes: Server, Client, Transparent

6. NAT (Network Address Translation)

- Static NAT, Dynamic NAT, PAT (Port Address Translation / NAT Overload)

- Private IP ranges (10.x, 172.16.x, 192.168.x)
- Why NAT is used — IPv4 exhaustion
- NAT table concept

7. IPv4 vs IPv6

- Address format differences (32-bit vs 128-bit)
- IPv6 address types: Unicast, Multicast, Anycast
- IPv6 adoption challenges
- Dual stack, tunneling, translation methods

PRIORITY 2 — IMPORTANT (Frequently Asked)

8. Spanning Tree Protocol (STP)

- Loop problem in Layer 2 networks
- Root Bridge election — Bridge ID (Priority + MAC)
- Port States: Blocking, Listening, Learning, Forwarding, Disabled
- STP vs RSTP (Rapid STP) — convergence time difference
- Types: STP (802.1D), RSTP (802.1w), MSTP (802.1s), PVST+

9. Infrastructure Security — ACL, AAA

- Standard ACL vs Extended ACL (numbered and named)
- ACL placement: Standard — close to destination; Extended — close to source
- Port security — violation modes: Protect, Restrict, Shutdown
- AAA: Authentication, Authorization, Accounting
- TACACS+ vs RADIUS — key differences

10. PPP, WAN Technologies

- PPP (Point-to-Point Protocol) — authentication: PAP vs CHAP
- MLPPP (Multilink PPP) — link aggregation over WAN
- PPPoE — DSL broadband authentication
- GRE Tunneling concept
- MPLS vs Traditional WAN
- VPN fundamentals — site-to-site, remote access

11. Layer 2 Switching

- MAC address table, flooding, forwarding
- Cut-through vs Store-and-forward switching
- CSMA/CD concept (Ethernet)
- Half duplex vs Full duplex

12. Ethernet & Wireless (IEEE Standards)

- UTP vs STP cables — categories (CAT5e, CAT6, CAT7)
- IEEE 802.3 (Ethernet), 802.11 (Wi-Fi), 802.1Q (VLAN)
- Wi-Fi frequency bands: 2.4 GHz vs 5 GHz
- Wireless security: WEP, WPA, WPA2, WPA3

PRIORITY 3 — GOOD TO KNOW (Asked in Advanced Rounds)

13. Router IOS & Management

- Router boot sequence: POST → Bootstrap → IOS → Config
- IOS image storage: Flash, NVRAM, RAM, ROM
- Telnet vs SSH — security differences
- Configuration registers
- Debugging and logging importance

14. SDN — Software Defined Networking

- Traditional networking vs SDN
- Control Plane vs Data Plane separation
- SDN architecture: Application, Control, Infrastructure layers
- Use in cloud data centers, ISP automation
- Google's B4 SDN-based network

15. OpenFlow & OpenDaylight

- OpenFlow protocol — controller-switch communication
- OpenFlow message types: Hello, Features, Packet-in, Packet-out, Flow-mod
- OpenDaylight architecture — MD-SAL (Model-Driven Service Abstraction Layer)
- OpenDaylight clustering
- OpenVSwitch (OVS) concept
- Mininet — network emulation tool

16. Virtual Networking

- Virtual Switch vs Physical Switch
- Network Function Virtualization (NFV)
- NFV vs SDN relationship
- Use cases: NAC, Virtual Customer Edge, DC Optimization

17. Advanced SDN / OpenDaylight Topics

- Group Based Policy (GBP)
- Service Function Chaining (SFC)
- LISP Flow Mapping
- Virtual Tenant Networks (VTN)
- Multi-tenant isolation in data centers
- OVSDB Virtualization

INTERVIEW SPECIAL — IMPORTANT PORTS & PROTOCOLS

These are almost always asked in networking, DevSecOps, and security interviews.

Application Layer Protocols & Ports

Port	Protocol	Transport	Description
20	FTP Data	TCP	File Transfer — Data channel
21	FTP Control	TCP	File Transfer — Command channel
22	SSH	TCP	Secure Shell — encrypted remote login
23	Telnet	TCP	Unencrypted remote login (insecure)
25	SMTP	TCP	Send emails (Simple Mail Transfer Protocol)
53	DNS	TCP/UDP	Domain Name System — name resolution
67	DHCP Server	UDP	Dynamic Host Config — server port
68	DHCP Client	UDP	Dynamic Host Config — client port
69	TFTP	UDP	Trivial FTP — no auth, used in router IOS transfer
80	HTTP	TCP	Web traffic — unencrypted
110	POP3	TCP	Post Office Protocol v3 — receive emails
119	NNTP	TCP	Network News Transfer Protocol
123	NTP	UDP	Network Time Protocol — time sync
143	IMAP	TCP	Internet Message Access Protocol — receive emails
161	SNMP	UDP	Simple Network Management Protocol — monitoring
162	SNMP Trap	UDP	SNMP alerts/notifications
179	BGP	TCP	Border Gateway Protocol — inter-AS routing
389	LDAP	TCP/UDP	Lightweight Directory Access Protocol
443	HTTPS	TCP	Secure web traffic (HTTP over TLS/SSL)
445	SMB	TCP	Windows file sharing (Server Message Block)
465	SMTPS	TCP	SMTP over SSL — secure email sending
500	IKE/ISAKMP	UDP	IPSec VPN key exchange
514	Syslog	UDP	System logging
636	LDAPS	TCP	LDAP over SSL
993	IMAPS	TCP	IMAP over SSL
995	POP3S	TCP	POP3 over SSL
1433	MSSQL	TCP	Microsoft SQL Server
1521	Oracle DB	TCP	Oracle Database
3306	MySQL	TCP	MySQL Database
3389	RDP	TCP	Remote Desktop Protocol — Windows
5432	PostgreSQL	TCP	PostgreSQL Database
5900	VNC	TCP	Virtual Network Computing — remote GUI
6379	Redis	TCP	Redis in-memory database
8080	HTTP-Alt	TCP	Alternative HTTP / proxy port
8443	HTTPS-Alt	TCP	Alternative HTTPS port
9200	Elasticsearch	TCP	Elasticsearch REST API

Port	Protocol	Transport	Description
27017	MongoDB	TCP	MongoDB Database

 Network Layer Protocols

Protocol	Number	Description
ICMP	1	Internet Control Message Protocol — ping, traceroute
IGMP	2	Internet Group Management Protocol — multicast groups
TCP	6	Transmission Control Protocol — reliable, connection-oriented
UDP	17	User Datagram Protocol — fast, connectionless
GRE	47	Generic Routing Encapsulation — tunneling
ESP	50	Encapsulating Security Payload — IPSec encryption
AH	51	Authentication Header — IPSec integrity
OSPF	89	Open Shortest Path First — runs directly over IP

 Routing Protocols — Key Facts for Interview

Protocol	Type	Algorithm	AD	Metric
RIP v1/v2	IGP / Distance Vector	Bellman-Ford	120	Hop count (max 15)
IGRP	IGP / Distance Vector	Bellman-Ford	100	Bandwidth + Delay
EIGRP	IGP / Hybrid	DUAL	90 (internal)	Bandwidth + Delay
OSPF	IGP / Link State	Dijkstra (SPF)	110	Cost (bandwidth-based)
IS-IS	IGP / Link State	Dijkstra	115	Cost
BGP	EGP / Path Vector	Best Path	20 (eBGP) / 200 (iBGP)	AS Path, MED, etc.
Static Route	—	—	1	—
Connected	—	—	0	—

 TACACS+ vs RADIUS — Interview Comparison

Feature	TACACS+	RADIUS
Transport	TCP (port 49)	UDP (1812 auth / 1813 accounting)
Encryption	Full packet	Password only
AAA separation	Yes (separate A,A,A)	No (combined)
Used for	Device admin (Cisco)	Network access (VPN, Wi-Fi)
Vendor	Cisco proprietary	Open standard (RFC)

 STP Port States — Quick Reference

State	Duration	Action
Blocking	Indefinite	No forwarding, no learning, listens BPDUs
Listening	15 sec (FWD delay)	No forwarding, no learning
Learning	15 sec (FWD delay)	No forwarding, learns MACs
Forwarding	Indefinite	Forwards frames, learns MACs
Disabled	Admin down	Not participating in STP

 NAT Types — Quick Reference

Type	Description	Use case
Static NAT	1 private ↔ 1 public (fixed)	Servers behind firewall
Dynamic NAT	Pool of public IPs assigned dynamically	Medium enterprises
PAT / NAT Overload	Many private → 1 public (port-based)	Home router, ISP

 Private IP Address Ranges (RFC 1918)

Class	Range	Default Subnet
A	10.0.0.0 – 10.255.255.255	/8
B	172.16.0.0 – 172.31.255.255	/12
C	192.168.0.0 – 192.168.255.255	/16

 OSI Layer — Protocol Mapping (Interview Favourite)

OSI Layer	Layer Name	Protocols / Technologies
7	Application	HTTP, HTTPS, FTP, SSH, DNS, SMTP, SNMP
6	Presentation	SSL/TLS, JPEG, MPEG, ASCII
5	Session	NetBIOS, RPC, PPTP
4	Transport	TCP, UDP
3	Network	IP, ICMP, OSPF, BGP, ARP
2	Data Link	Ethernet, 802.1Q, STP, PPP, HDLC
1	Physical	Cables, Hubs, Repeaters, NIC

 QUICK SYLLABUS TOPIC LIST (All Sessions)

Session	Topics
1–2	Internetworking, OSI Model, Ethernet, Wireless (IEEE 802.11)

Session	Topics
3	Internet Protocol, TCP/IP Model, TCP vs UDP
4–5	IP Subnetting, VLSM
6	Router IOS, Security Device Manager, Subnetting practice
7	Managing Internetworking Router, Telnet, SSH, Debugging
8	Static Routing, Dynamic Routing, Routing Protocols
9–10	Implementing RIP, IGRP, EIGRP, OSPF
11	Layer 2 Switching, STP, STP types, Priority
12	VLANs, Inter-VLAN Routing, VTP
13	Infrastructure Security, ACL, TACACS+, RADIUS
14	NAT, IPv6, WAN Technologies
15	PPP, MLPPP, PPPoE, GRE, BGP basics
16	SDN Introduction, Architecture, Control vs Data Plane
17	Virtual Networking, NFV, Use Cases
18	OpenFlow, OpenDaylight Architecture
19	OpenDaylight Advanced, OVS, Mininet, L2Switch
20	AAA + ODL, OVSD, Group Policy, SFC, LISP, VTN

CDAC DITISS — PGCP-ITISS / Fundamentals of Computer Networks / Feb 2026 Total: 40T + 40L + 40SL = 120 hrs

Related Notes

- [Computer Networks Index](#)
- [01A - OSI Model](#)
- [01B - OSI Layers Quick Reference](#)
- [02 - TCP-IP Model](#)
- [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)
- [04A - Network Routing Fundamentals](#)
- [04B - Routing Protocols and Administrative Distance](#)
- [05 - VLANs and Inter-VLAN Routing](#)
- [06 - Network Address Translation](#)
- [08 - Spanning Tree Protocol](#)
- [09 - Infrastructure Security ACL AAA and Port Security](#)
- [10 - PPP and WAN Technologies](#)
- [11 - Layer 2 Switching and Ethernet Forwarding](#)
- [13 - Router IOS and Management](#)
- [A1 - HTTP Evolution and TLS](#)

Source note: 00 - Syllabus and Interview Checklist.md

PART I

Network Foundations

Build the mental models behind modern networking: layered architecture, transport behavior, and IP addressing.

[01A · OSI Model](#)[01B · OSI Quick Reference](#)[02 · TCP/IP Model](#)[03 · IP Addressing & Subnetting](#)

OSI Model — Complete Study Notes

A complete layer-by-layer guide to responsibilities, protocols, devices, and encapsulation.

- Topic 1
- Topic 10
- Topic 12
- computer networks
- osi model
- networking fundamentals

Table of Contents

- Layer 1 — Physical Layer
- Layer 2 — Data Link Layer
- Layer 3 — Network Layer
- Layer 4 — Transport Layer
- Layer 5 — Session Layer

- Layer 6 — Presentation Layer
- Layer 7 — Application Layer
- OSI Model — Complete Reference
- Data Encapsulation & Decapsulation
- Related Notes

Layer 1 — Physical Layer

1. Role & Overview

- Lowest OSI layer** — responsible for transmitting raw bits (0s and 1s) over a physical medium.
- Converts higher-layer units into electrical/optical/radio signals (**encoding/modulation**), and converts received signals back into bits (**decoding/demodulation**).
- Deals with mechanical, electrical, and functional details: connectors, cable types, voltage levels, timing (clocking), bit rates, line coding, and physical topology.

In short: Layer 1 is actual connectivity — it defines the physical characteristics of the network and handles raw bit flow over the medium (copper, fibre, or radio).

2. Why Layer 1 is Called a "Dumb Layer"

- Does **not** understand frames, packets, addresses, ports, or application data — it only moves bits.
- Devices at this layer (hubs, repeaters, CSU/DSU) make **no forwarding decisions** based on MAC/IP and maintain no routing/forwarding tables.
- Performs signal transmission/regeneration **only** — no filtering, no addressing logic.

3. Cables & Connectors

Cable Types

Cable	Description
UTP (Unshielded Twisted Pair)	Common for Ethernet (Cat5e / Cat6); no shielding

Cable	Description
STP (Shielded Twisted Pair)	Has foil/braid shielding to reduce EMI
Coaxial (coax)	Legacy use — 10BASE-2 / 10BASE-5
Fibre Optic	Transmits light; immune to EMI; used for long distances / high bandwidth

RJ45 / 8P8C Connector (Ethernet)

- Standard connector for twisted-pair Ethernet cabling.
- Uses wiring standards **T568A** or **T568B** (pinout order for 8 conductors).
- Poor crimping causes intermittent faults and signal reflections.

Interview Trap: T568A and T568B differ only in the swap of pairs 2 and 3 (orange ↔ green). Crossover cables use T568A on one end and T568B on the other.

4. Bits, Voltage Signals & Encoding

- Bits are represented by changes in **voltage**, **light intensity**, or **radio waveform** on the medium.
- **Line encoding** maps bits → physical signal shapes:

Encoding	How it Works
NRZ (Non-Return to Zero)	Steady voltage for 0 and 1 — simple but prone to clock drift
Manchester	Signal transitions in the <i>middle</i> of each bit interval — helps with clock recovery

- **Modulation** (for analogue carriers) and encoding together determine reliability and clock recovery at the receiver.

5. Communication Methods

Serial vs Parallel

Method	Description	Use Case
Serial	Bits sent one after another over one wire/pair	Networking, long-distance
Parallel	Multiple bits sent simultaneously over multiple wires	Short distances — internal buses, legacy printer ports

Parallel problem: Signal skew (timing differences across wires) and EMI make parallel impractical over longer runs.

Asynchronous vs Synchronous Serial

Type	Clock	Example
Asynchronous	No shared clock — each byte framed with start/stop bits	RS-232, old modems
Synchronous	Sender/receiver share a clock or embed it in the stream; continuous frames	Modern high-speed links

Bandwidth vs Throughput

Term	Definition
Bandwidth	Theoretical maximum capacity (Hz or bps) of the physical medium
Throughput	Actual data rate achieved after overheads, errors, retransmits, and protocol inefficiencies

Key Point: Throughput is always $\leq$ Bandwidth. Real-world throughput is significantly lower due to protocol overhead and noise.

6. Transmission Modes

Mode	Direction	Example
Simplex	One-way only	TV broadcast, keyboard
Half-Duplex	Both ways, but NOT simultaneously	Walkie-talkie, hub
Full-Duplex	Both ways simultaneously	Phone call, switch port

7. Physical Topologies

Topology	Key Characteristic
Mesh	Every device connected to every other; highest redundancy
Star	All devices connect to a central switch; most common today
Ring	Each device connects to two neighbours; one break = full failure
Bus	Single shared cable; legacy (old Ethernet 10BASE-2/5)

8. Layer 1 Devices

Rule: Physical layer devices are "dumb" — they deal only with signals and bits, never with addresses or data content.

Device	Layer	Role
Repeater	L1	2-port device; copies a weakening signal bit-by-bit and retransmits at full strength; zero intelligence — only understands voltage levels; sole job is extending cable distance
Hub	L1	Multi-port repeater; broadcasts incoming frame out of every other port regardless of destination; creates a single shared collision domain (half-duplex). Active hub — boosts/regenerates the signal. Passive hub — simply connects wires, no signal boosting
Cables	L1	Physical medium — UTP, STP, coaxial, fibre optic
Connectors	L1	RJ-45, BNC, LC, SC, DB-9 — physical interface points
NIC	L1/ L2	Converts digital bits to electrical/optical signals; has both L1 (signal) and L2 (MAC) functions

Device	Layer	Role
Modem	L1	Modulates/demodulates — converts digital bits ↔ analogue signals for telephone lines
CSU/DSU	L1	Channel Service Unit / Data Service Unit — connects router (DTE) to WAN digital line (DCE); provides clocking
Transceiver	L1	Converts between different physical media (e.g., copper ↔ fibre)

9. Layer 1 Protocols & Standards

Protocol / Standard	Description
Ethernet (IEEE 802.3)	Wiring & signaling for wired LAN (10Base-T, 100Base-TX, 1000Base-T)
Wi-Fi (IEEE 802.11)	Wireless physical layer — radio frequencies (2.4 GHz, 5 GHz)
Bluetooth (IEEE 802.15)	Short-range wireless, 2.4 GHz radio
USB	Serial physical interface for peripheral devices
RS-232	Serial communication standard (console cable, old modems)
DSL	Transmits digital data over telephone lines
SONET / SDH	Fibre-optic transmission standards for WAN backbone
V.35	WAN serial interface standard (router ↔ CSU/DSU)
DOCSIS	Cable internet physical standard
T1 / E1	Digital WAN transmission lines (1.544 / 2.048 Mbps)

Layer 2 — Data Link Layer

1. Role & Overview

- **Second lowest OSI layer** — transforms the raw bit stream from Layer 1 into a reliable **node-to-node (hop-to-hop)** link.
- Handles delivery of frames between **two directly connected devices only** — it cannot work across networks (that is Layer 3's job).
- Encapsulates data by adding a **Layer 2 header** (source + destination MAC) and a **Layer 2 trailer** (FCS) → the result is called a **frame**.
- Hides hardware complexities from upper layers, making network communication simpler for layers above.

In short: Layer 2 is the medium provision layer — it defines how data is formatted for transmission over the physical medium and handles physical MAC addressing.

2. Hop-to-Hop Delivery (Most Important Concept)

PC1 — R1 — R2 — PC2

Layer 3 (IP): PC1 ————→ PC2 (end-to-end, never changes)

Layer 2 (MAC): PC1→R1 | R1→R2 | R2→PC2 (hop-to-hop, changes at every router)

- The **IP packet (Layer 3) stays the same** from source to destination — never modified.
- The **Layer 2 frame is destroyed and rebuilt at every router hop** — each new frame uses the MAC addresses of that specific link only.

Think of it like a relay race — the baton (IP packet) stays the same, but each runner (frame) is different on each leg.

Full Frame Flow Example

Step 1: PC1 creates frame
 Src MAC = PC1_MAC, Dst MAC = R1_MAC (next hop only)

Step 2: R1 receives frame
 → Strips L2 header/trailer
 → Makes routing decision (looks at IP packet)
 → Creates NEW frame: Src MAC = R1_MAC, Dst MAC = R2_MAC

Step 3: R2 receives frame
 → Strips L2 header/trailer
 → Creates NEW frame: Src MAC = R2_MAC, Dst MAC = PC2_MAC

Step 4: PC2 receives frame
 → Strips frame → passes IP packet up to Layer 3

⚠️ **IP packet (Layer 3) = unchanged throughout | Frame (Layer 2) = recreated at every hop**

3. Two Sublayers of Layer 2

LLC (Logical Link Control)
IEEE 802.2

MAC (Media Access Control)
IEEE 802.3 / 802.11

↔ Interface with Network Layer (Layer 3)
Flow control, error notification, multiplexing

↔ Interface with Physical Layer (Layer 1)
Framing, MAC addressing, medium access control

Sublayer	Full Name	Key Responsibilities
LLC	Logical Link Control	Interface between L2 and L3; flow control; error notification; multiplexing (allows IP, IPX, AppleTalk to share the same NIC)
MAC	Media Access Control	Controls access to physical medium; adds MAC addresses to frames; converts packets → frames for Physical Layer

4. Frame Structure

Preamble	Dest MAC	Src MAC	Type/Len	Data (IP pkt)	FCS
----------	----------	---------	----------	---------------	-----

Field	Purpose
Preamble	Synchronization — alerts receiver that a frame is coming
Destination MAC	Receiver's hardware address
Source MAC	Sender's hardware address
Type / Length	Identifies the upper-layer protocol (e.g., 0x0800 = IPv4)
Payload	The IP packet from Layer 3
FCS / CRC	Error detection value — receiver recalculates and compares

Frame Types

- **Fixed size** — each frame is a fixed number of bytes (e.g., ATM cells = 53 bytes always)
- **Variable size** — delimiter flags mark start/end (e.g., Ethernet, HDLC)

5. Functions of the Data Link Layer

5.1 Framing

Divides the raw bit stream into manageable units called frames by adding headers and trailers around the Layer 3 packet.

5.2 Physical Addressing (MAC Addressing)

- **MAC address** = 48-bit (6-byte) hardware address burned into the NIC.
- Format: AA:BB:CC:DD:EE:FF (hexadecimal, colon-separated)
- **First 3 bytes** = OUI (Organizationally Unique Identifier — vendor/manufacturer)
- **Last 3 bytes** = Device-specific identifier
- MAC addresses are **only relevant on the local link** — replaced at every router hop.

5.3 Flow Control

Prevents a fast sender from overwhelming a slow receiver.

Method	How it Works
Stop-and-Wait	Sender sends one frame, waits for ACK before sending the next
Sliding Window	Sender can send multiple frames before needing an ACK (more efficient)

5.4 Error Control

Detects (and sometimes corrects) damaged, lost, or duplicate frames.

Mechanism	How it Works
FCS / CRC	Sender calculates checksum; receiver recalculates and compares — most common (used in Ethernet)
Parity Bit	Simple single-bit error detection
ACK + Retransmit	Receiver sends ACK; sender retransmits if no ACK received in time
Sequence Numbers	Detects duplicate or out-of-order frames

5.5 Media Access Control

When multiple devices share the same medium, access control prevents collisions.

Method	Used In	How it Works
CSMA/CD	Wired Ethernet (half-duplex)	Listen before transmit; detect and recover from collisions
CSMA/CA	Wi-Fi (IEEE 802.11)	Avoid collisions using random backoff before transmitting
Token Passing	Token Ring	Only the device holding the token can transmit
TDMA	Cellular, WAN	Fixed time slots assigned to each device

6. Layer 2 Protocols

Protocol	Used For
Ethernet (IEEE 802.3)	Wired LAN — most common L2 protocol
Wi-Fi (IEEE 802.11)	Wireless LAN
PPP	Point-to-point WAN serial links
HDLC	Cisco default serial encapsulation
Frame Relay	Legacy WAN packet switching
ATM	Fixed 53-byte cell WAN
ARP	Resolves IP address → MAC address (<i>sits at L2/L3 boundary — uses L3 IP addresses but operates at L2</i>)
STP (802.1D)	Prevents switching loops
VLAN (802.1Q)	Logical segmentation of L2 networks
PPPoE	PPP over Ethernet (DSL)
SDLC	Synchronous Data Link Control (IBM legacy)
SLIP	Serial Line Internet Protocol (old, replaced by PPP)

7. Layer 2 Devices

Device	Role
Switch	Forwards frames based on MAC address table; creates separate collision domains per port; most important L2 device
Bridge	Connects two LAN segments; filters traffic based on MAC address; 2-port predecessor to the switch
NIC	Contains MAC address; handles L2 framing and signaling (also touches L1)
Wireless AP	Bridges wireless (802.11) and wired (802.3) L2 networks

Layer 3 — Network Layer

1. Role & Overview

- **Layer 3** of the OSI model — responsible for **logical addressing, routing, and packet delivery** across different networks.
- Unlike Layer 2 (which only delivers between two directly connected devices), Layer 3 can route packets across **many different networks** to reach the final destination.

- Connects different networks that use different physical media by using logical IP addresses instead of hardware MAC addresses.
- Adds a **Layer 3 header (IP header)** to the data — the result is called a **packet**.
- The next upper-layer protocol (TCP, UDP, SCTP) is identified inside the IP header.

In short: Layer 3 is the logical addressing layer — it decides where packets go across networks and how they get there.

2. End-to-End Delivery (Most Important Concept)

PC1 — R1 — R2 — R3 — PC2
(Network A) (Network B) (Network C)

Layer 2 (MAC): PC1→R1 | R1→R2 | R2→R3 | R3→PC2 (hop-by-hop, changes every hop)
Layer 3 (IP): PC1 → PC2 (end-to-end, NEVER changes)

- **IP Source address** = always PC1's IP — never changes
- **IP Destination address** = always PC2's IP — never changes
- Layer 3 doesn't care about intermediate hops — it only cares about the **final destination**

At Every Router

- ✓ IP Header (L3) is READ to make routing decision
- ✓ L2 frame is STRIPPED and REBUILT with new MAC addresses
- ✗ IP Source / Destination NEVER changes
- ✗ Transport (L4) data is NOT touched

Direct vs Indirect Delivery

Type	When Used	Example
Direct Delivery	Source and destination on the same network — no router needed	192.168.1.10 → 192.168.1.20
Indirect Delivery	Source and destination on different networks — goes through one or more routers	192.168.1.10 → 10.0.0.5

3. Functions of the Network Layer

3.1 Logical Addressing

Physical (MAC) addresses only work within the same network. When a packet travels across multiple networks, a universal logical address (IP address) is needed.

- **IPv4** — 32-bit address (e.g., 192.168.1.10) — written as 4 dotted decimals
- **IPv6** — 128-bit address (e.g., 2001:DB8::1) — next-generation, far larger address space
- Structure: **Network ID + Host ID**
 - Example: 192.168.1.10/24 → Network = 192.168.1.0, Host = .10
- Every packet carries **Source IP** and **Destination IP** in its header

3.2 Packetization

The network layer encapsulates the segment from Layer 4 into a packet by adding the IP header.

Layer 4 Segment + IP Header = Layer 3 Packet

3.3 Routing

Determines the best path for packets to travel across multiple networks.

Method	Description	Example
Static Routing	Manually configured by admin	<code>ip route 10.0.0.0 255.255.255.0 192.168.1.1</code>
Dynamic Routing	Routers automatically exchange and update routes	RIP, OSPF, EIGRP
Default Routing	Single catch-all route for unknown destinations	<code>ip route 0.0.0.0 0.0.0.0 [next-hop]</code>

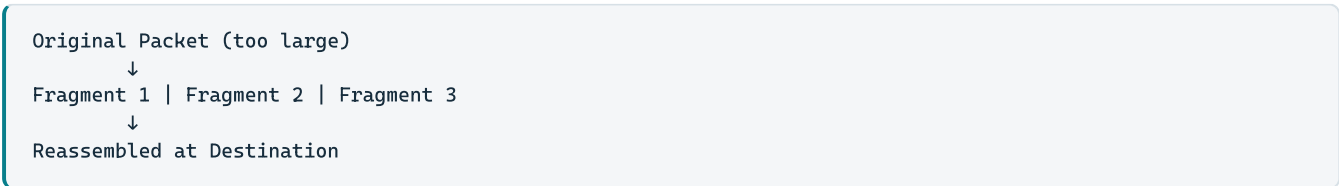
Routing vs Forwarding:

- **Routing** = Path determination (control plane) — deciding which way to send
- **Forwarding** = Packet movement (data plane) — actually sending it out the right interface

3.4 Fragmentation & Reassembly

If a packet is larger than the **MTU (Maximum Transmission Unit)** of the next link, the router splits it into smaller fragments.

- Default Ethernet MTU = **1500 bytes**
- Each fragment carries the same IP header with **Identification**, **Fragment Offset**, and **More Fragments (MF)** flag
- **Reassembly happens only at the final destination** — never at intermediate routers



3.5 NAT (Network Address Translation)

Converts private IP addresses ↔ public IP addresses — used in home routers to allow internet access.

Type	Description
Static NAT	One private IP ↔ one public IP (fixed mapping)
Dynamic NAT	Pool of public IPs assigned dynamically
PAT (Port Address Translation)	Many private IPs share one public IP using port numbers (most common — used in home routers)

4. IPv4 Packet Header

01234567890123456789012345678901

01234567890123456789012345678901

Version	IHL	ToS / DSCP	Total Length	
Identification			Flags	Fragment Offset
TTL (decrements per hop) drop at 0 → no loops			Protocol TCP=6 UDP=17 ICMP=1	Header Checksum

Source IP Address	
Destination IP Address	
Options (if IHL > 5)	Padding
Payload (TCP/UDP Segment)	

Field	Purpose
Version	IPv4 (4) or IPv6 (6)
IHL	Header Length — length of the IP header itself
ToS / DSCP	Type of Service — Quality of Service markings
Total Length	Total size of packet (header + payload)
Identification	Used to reassemble fragments belonging to the same original packet
Flags / Fragment Offset	Controls and tracks fragmentation
TTL	Time To Live — decremented by 1 at each router; dropped when TTL = 0 (prevents infinite loops)
Protocol	Identifies upper layer: TCP = 6, UDP = 17, ICMP = 1
Header Checksum	Error detection for the IP header only
Source IP	Sender's IP address
Destination IP	Receiver's IP address

5. Layer 3 Protocols

Protocol	Purpose
IPv4	32-bit logical addressing and packet delivery
IPv6	128-bit next-generation IP
ICMP	Error reporting and diagnostics (ping, traceroute, destination unreachable, time exceeded)
ICMPv6	ICMP for IPv6 + Neighbour Discovery Protocol (NDP)
ARP	Resolves IP → MAC address (<i>L2/L3 boundary — listed here because it resolves L3 addresses, but frames are L2</i>)
RARP	Resolves MAC → IP (obsolete, replaced by DHCP)
NAT	Translates private ↔ public IP addresses
IPSec	Encryption and authentication at the network layer
MPLS	Uses labels instead of IP addresses for faster forwarding — used by ISPs for traffic engineering
RIP	Distance-vector routing protocol
OSPF	Link-state routing protocol
EIGRP	Advanced distance-vector routing (Cisco proprietary)
BGP	Path-vector — inter-AS routing (internet backbone)

6. Routing Protocols

Protocol	Type	Metric	Max Hops	Best For
RIP	Distance-Vector	Hop count	15	Small, simple networks
OSPF	Link-State	Cost (bandwidth-based)	Unlimited	Large enterprise networks
EIGRP	Advanced Distance-Vector (Cisco)	Bandwidth + Delay + Load + Reliability	Unlimited	Cisco-only environments
BGP	Path-Vector	AS path attributes	Unlimited	Internet backbone, ISP ↔ ISP

Interview Trap — RIP max hops: RIP has a maximum hop count of **15**. A hop count of 16 = unreachable. This makes RIP unsuitable for large networks.

Interview Trap — OSPF algorithm: OSPF uses **Dijkstra's Shortest Path First (SPF)** algorithm. It builds a complete map of the network (LSDB) and calculates the shortest path to every destination.

7. Routing vs Flooding

Feature	Routing	Flooding
Path Selection	Best path chosen via routing table	All paths used simultaneously
Efficiency	Efficient	Very inefficient
Bandwidth Usage	Low	High
Loop Prevention	Routing algorithms (metrics, timers)	TTL or sequence numbers
Example	Router forwarding packets	ARP broadcast

8. Layer 3 Devices

Device	Role
Router	Primary L3 device — reads IP header, makes routing decisions, forwards packets between networks
Layer 3 Switch	Switch with routing capability — routes between VLANs using SVIs (Switched Virtual Interfaces)
Multilayer Switch	Can perform both switching (L2) and routing (L3)
Firewall (L3)	Filters packets based on IP addresses and protocols

Layer 4 — Transport Layer

1. Role & Overview

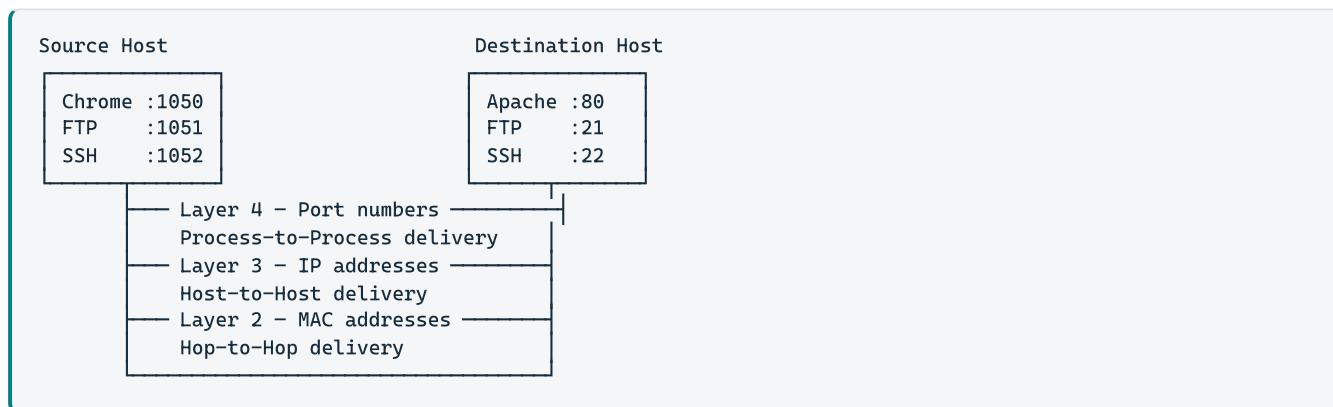
- **Layer 4** of the OSI model — responsible for **process-to-process (end-to-end)** delivery of the entire message.
- While Layer 3 delivers individual packets between hosts, Layer 4 ensures the **complete message arrives correctly at the right application process** on the destination host.
- Establishes, maintains, and terminates end-to-end connections.
- Responsible for end-to-end **error recovery, flow control**, and **QoS**.
- Adds a **Layer 4 header** to the packet — the result is called a **segment**.

- Allows multiple application processes to share the same physical connection simultaneously using **port numbers**.

In short: Layer 3 delivers to the right host (IP). Layer 4 delivers to the right application on that host (Port).

2. Process-to-Process Delivery (Most Important Concept)

A **process** is an application program running on a host (e.g., Chrome browser, FTP client, web server).



3. Functions of the Transport Layer

3.1 Port Addressing

Ensures the right application receives data on a host running multiple processes simultaneously.

- Port number** = 16-bit number (0–65535) identifying a specific process
- Combined with IP address → forms a **Socket**: `192.168.1.10:80`

Port Range	Type	Examples
0 – 1023	Well-known ports	HTTP=80, HTTPS=443, FTP=21, SSH=22, DNS=53, SMTP=25
1024 – 49151	Registered ports	MySQL=3306, RDP=3389
49152 – 65535	Dynamic / Ephemeral	Assigned temporarily to client processes

3.2 Segmentation & Reassembly

Divides a large message into smaller segments, each tagged with a sequence number.



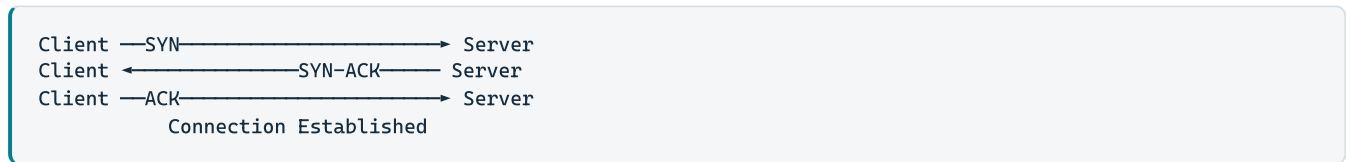
Sequence numbers allow the receiver to:

- Reassemble **out-of-order** segments in the correct order
- Detect **lost** segments (missing sequence number)
- Detect **duplicates** (same sequence number twice)

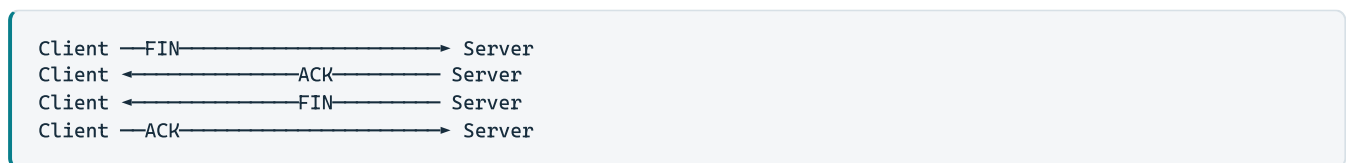
3.3 Connection Control

Type	Description	Protocol
Connection-Oriented	Establishes a connection first, transfers data, then terminates; reliable	TCP
Connectionless	No connection setup; each segment treated independently; fast but unreliable	UDP

TCP 3-Way Handshake (Connection Setup):



TCP 4-Way Termination:



3.4 Flow Control (End-to-End)

Prevents the sender from overwhelming a slow receiver — operates end-to-end between source and destination processes, not hop-to-hop.

- **TCP Sliding Window** — receiver advertises how much buffer space is available (**Window Size**)
- Sender can only transmit up to the window size before needing an ACK
- Receiver slow → window shrinks → sender slows down
- Receiver fast → window grows → sender speeds up

Sender: Window size = 3 → can send 3 unacknowledged segments
 Receiver: ACK 3, Window = 2 → sender can now only send 2 more

3.5 Error Control (End-to-End)

Ensures reliable process-to-process delivery using:

Mechanism	How it Works
Checksum	Detects corrupted segments
ACK	Receiver confirms receipt of segments
Retransmission	If no ACK received within timeout (RTO), sender retransmits
Sequence Numbers	Detect lost, duplicate, or out-of-order segments

⚠ **Layer 4 error control is end-to-end** (source process → destination process), unlike Layer 2 error control which is hop-to-hop only.

4. TCP vs UDP vs SCTP

Feature	TCP	UDP	SCTP
Full Name	Transmission Control Protocol	User Datagram Protocol	Stream Control Transmission Protocol
Connection	Connection-oriented (3-way handshake)	Connectionless	Connection-oriented (4-way handshake)
Reliability	Reliable — ACK + retransmit	Unreliable — no ACK	Reliable — selective ACK
Ordering	Guaranteed order (sequence numbers)	No ordering	Ordered and unordered transfer
Flow Control	Yes — sliding window	No	Yes
Congestion Control	Yes	No	Yes
Header Size	20–60 bytes	8 bytes	12 bytes + chunks
Speed	Slower (overhead)	Faster (no overhead)	Moderate
Security	SSL / TLS	DTLS	Built-in (protects from SYN flooding)
Data Unit	Segment (byte-oriented, combines messages)	Datagram (message-oriented)	Chunk (preserves message boundary)
Multihoming	No	No	Yes
Use Cases	HTTP, HTTPS, FTP, SSH, SMTP, Telnet	DNS, DHCP, VoIP, SNMP, TFTP, RIP	Telephony signaling (SS7 over IP)
IP Protocol No.	6	17	132

5. Segment Headers

TCP Header

Src Port	Dst Port	← 16-bit each
Sequence Number		← 32-bit – for ordering & reassembly
Acknowledgment Number		← 32-bit – next expected byte
Flags	SYN ACK FIN RST	← Control bits
Window Size		← Flow control
Checksum		← Error detection

UDP Header

Src Port	Dst Port	← 16-bit each
Length	Checksum	← Minimal overhead – only 8 bytes total

Interview Trap — Why is UDP's header only 8 bytes? UDP has no sequence numbers, no ACK, no window size, no connection state — nothing for reliability. It is a bare-minimum header for speed.

6. Layer 4 Protocols

Protocol	Type	Use Cases
TCP	Connection-oriented	HTTP, HTTPS, FTP, SSH, SMTP, Telnet
UDP	Connectionless	DNS, DHCP, SNMP, VoIP, TFTP, RIP
SCTP	Connection-oriented + multihoming	Telephony signaling (SS7 over IP)
DCCP	Connectionless + congestion control	Streaming media

7. Layer 4 Devices

Transport Layer is implemented in **software** — it operates on end hosts, not in networking hardware.

Device / Component	Role
Host OS (TCP/IP stack)	Every OS (Windows, Linux, macOS) implements TCP/UDP in the kernel
Stateful Firewall	Inspects TCP/UDP port numbers and connection state to allow/block traffic
Load Balancer	Distributes incoming connections across servers based on L4 port/protocol
Proxy Server	Terminates and re-establishes TCP connections on behalf of clients

Layer 5 — Session Layer

1. Role & Overview

- **Layer 5** of the OSI model — the network's **dialog controller**.
- Responsible for **establishing, managing, synchronizing, and terminating** communication sessions between two application processes on different machines.
- Sits above the Transport Layer and uses its services to create meaningful, organized conversations.
- Handles **session multiplexing** — allows multiple transport layer services to share the same connection.
- May coordinate identity checks during session setup, but actual **authentication and authorization** are typically handled by application-layer protocols (L7) — not the session layer itself.
- Operates almost entirely in **software** — primarily used by application developers, not network engineers.
- Logging is also performed at this layer.

In short: Layer 5 manages the conversation — who speaks when, how long, and how to recover if the link drops mid-transfer.

2. What is a Session?

A **session** is a logical, persistent connection between two application processes for the duration of their communication.

Browser (Client) ←—— Session ———→ Web Server
(established, maintained,

synchronized, terminated)

- Multiple sessions can exist **simultaneously** between the same two hosts (e.g., multiple browser tabs open to the same server).
- A login to a computer for file transfer is a classic example of a session layer job.

3. Session Lifecycle

Every session goes through three phases:

1. ESTABLISHMENT → Negotiate parameters, authenticate, select duplex mode
↓
2. DATA TRANSFER → Dialog exchange with synchronization checkpoints
↓
3. TERMINATION → Graceful session close after all data exchanged

4. Functions of the Session Layer

4.1 Dialog Control

Controls the direction and turn-taking of communication between two systems.

Mode	Description	Example
Half-Duplex	Only one side speaks at a time; other must wait	Walkie-talkie, HTTP/1.0
Full-Duplex	Both sides communicate simultaneously	Video call, HTTP/2

- Session layer **negotiates** which mode to use during session establishment.
- In half-duplex, it manages **tokens** — only the device holding the token can send.

4.2 Synchronization (Checkpoints)

The most unique and important function of the Session Layer.

Problem without checkpoints:

Transferring a 1 GB file → network fails at 900 MB
→ Must restart from 0 bytes – entire transfer wasted

Solution — Checkpoints: The session layer inserts **synchronization points** into the data stream at regular intervals.

[Data 0–100 MB] — CHECKPOINT 1 —
 [Data 100–200 MB] — CHECKPOINT 2 —
 [Data 200–300 MB] — CHECKPOINT 3 —
 ...
 [Data 800–850 MB] — CHECKPOINT 9 (in progress) — ← network fails here
 → Resume from CHECKPOINT 8 (800 MB) – only resend last 50 MB

Sync Point Type	Description
Major Sync Point	Explicit boundary; all data up to this point is committed and acknowledged; cannot go back beyond this
Minor Sync Point	More frequent, finer-grained checkpoints within major points; allows fine-grained recovery without full retransmission

4.3 Session Management

Function	Description
Authentication	May coordinate identity checks during session setup; in practice, authentication is handled by L7 application protocols (e.g. HTTP Basic Auth, OAuth, LDAP)
Authorization	Permissions are determined by the application layer — L5 may pass session tokens but does not enforce access control itself
Session ID	Unique identifier assigned to each session
Re-synchronization	Restores session to last known good checkpoint after a failure
Multiplexing	Multiple transport layer services share the same underlying connection

5. Session Layer in Context

```

Application (e.g., File Transfer)
    ↓
Session Layer – inserts checkpoints every X MB
    ↓
Transport Layer (TCP) – handles segment delivery
    ↓
Network Layer (IP) – handles routing
    ↓
...

Link fails mid-transfer:
  → TCP connection re-established (Layer 4)
  → Session resumes from last checkpoint (Layer 5) ✓
  → No need to restart the entire file transfer

```

6. Layer 5 Protocols

Protocol	Purpose
RPC (Remote Procedure Call)	Allows a program to execute procedures on a remote server as if local
NetBIOS	Session establishment for Windows file/printer sharing
SIP (Session Initiation Protocol)	Establishes and manages VoIP and video call sessions (<i>spans L5/L7 — session setup is L5, application signaling is L7</i>)
RTCP	Real-Time Control Protocol — monitors RTP media sessions
PPTP	Point-to-Point Tunneling Protocol — VPN session management
L2TP	Layer 2 Tunneling Protocol — VPN tunneling
PAP	Password Authentication Protocol — session-level authentication
H.245	Multimedia session control (video conferencing)

7. Layer 5 Devices

Session Layer operates entirely in **software** — no dedicated hardware devices exist at this layer.

Device / Component	Role
Application Servers	Create and maintain user sessions for web apps and databases
Proxy Servers	Manage sessions between clients and backend servers
Stateful Firewalls	Track and control active sessions; block unauthorized session establishment
Session Border Controllers (SBC)	Secure and manage VoIP/video call sessions
VPN Gateways	Establish and maintain encrypted VPN sessions (PPTP, L2TP)

Layer 6 — Presentation Layer

1. Role & Overview

- **Layer 6** of the OSI model — called the "**Translator**" or "**Syntax Layer**".
- Ensures that data sent by the application layer of one system is **readable by the application layer of another system**, regardless of internal data formats.
- Handles **translation, encryption/decryption, and compression/decompression** — purely software, no dedicated hardware.
- Different systems (IBM mainframe, PC, mobile) may use completely different data representations — Layer 6 converts everything into a common format for the network.

In short: Layer 6 is the translator — it converts data formats, encrypts/decrypts for security, and compresses/decompresses to save bandwidth.

2. Functions of the Presentation Layer

2.1 Translation (Data Conversion)

Different systems use different data formats — the Presentation Layer converts between them using a common intermediate format.

From	To	Example
EBCDIC	ASCII	IBM mainframe → PC communication
ASCII	Unicode	Older system → modern multilingual
Big-endian	Little-endian	Different CPU architectures
JPEG/PNG	Internal bitmap	Image display on screen

- Uses **ASN.1** (Abstract Syntax Notation One) as the common language for describing data structures.
- Handles **serialization** — converting complex objects (JSON, XML) into flat byte strings for transmission.
- String representations: **Pascal style** (length prefix) vs **C style** (null-terminated `\0`).

2.2 Encryption & Decryption

Provides confidentiality by encoding data before transmission and decoding it at the destination.

```
Sender:  Plaintext → [Encryption] → Ciphertext → network
Receiver:      Ciphertext → [Decryption] → Plaintext
```

- **SSL/TLS** is commonly taught at this layer for encryption/decryption functions. In practice, TLS runs on top of TCP (L4) and serves application protocols like HTTPS (L7) — it does not map cleanly to a single OSI layer.
- In the TCP/IP model, TLS is considered part of the Application layer.
- Protects against eavesdropping, unauthorized access, and replay attacks.

2.3 Compression & Decompression

Reduces the size of data before transmission to save bandwidth.

Type	Algorithms	Use
Lossless	ZIP, GZIP, LZW	Text files, documents — no data lost
Lossy (image)	JPEG	Photos — some data discarded
Lossy (video)	MPEG, H.264	Video streaming
Lossy (audio)	MP3, AAC	Music, voice

Interview Trap — Lossy vs Lossless: Lossy is acceptable for media (human perception can't detect small losses). Lossless is required for text and files — even a single bit flip corrupts data.

3. Layer 6 Protocols

Protocol	Purpose
SSL / TLS	Encryption for web (HTTPS), email, VPN (<i>commonly placed at L6; in practice spans L4–L6 — uses L4 TCP, performs L6 encryption, serves L7 applications. TCP/IP model places it in the Application layer</i>)
JPEG	Image compression standard
MPEG	Video/audio compression
GIF / PNG	Image formats
ASCII / EBCDIC / Unicode	Character encoding standards
XDR (External Data Representation)	Sun Microsystems data format standard
MIME	Email attachment encoding (Base64)
ASN.1	Abstract data structure description language

4. Layer 6 Devices

Presentation Layer is **purely software** — no dedicated hardware exists at this layer.

Component	Role
Web browsers	Handle SSL/TLS decryption, HTML/JSON parsing
Media players	Decompress JPEG, MPEG, MP3
Email clients	Decode MIME-encoded attachments
Encryption libraries	OpenSSL, Bouncy Castle

Layer 7 — Application Layer

1. Role & Overview

- **Topmost layer** of the OSI model — provides a **network interface directly to the user's application**.
- Does **not** refer to the applications themselves (Chrome, Outlook) — but to the **protocols and services** that allow those applications to use the network.
- The application itself does **not** belong to this layer — only the protocols that enable network communication do.
- Closest layer to the end user — everything begins here before data travels down through the lower layers.

In short: Layer 7 is the entry point where application data enters the OSI stack — it provides services like web browsing, email, file transfer, name resolution, and remote access.

2. Functions of the Application Layer

2.1 Network Virtual Terminal (NVT)

Allows a user on one host to connect to a remote computer as if it were a local terminal.

Local PC — SSH / Telnet —> Remote Server
(terminal emulator) (as if sitting at the server)

- Creates a software emulation of a terminal at the remote machine.
- **Telnet** — unencrypted (port 23); **SSH** — encrypted (port 22).

2.2 File Transfer, Access & Management (FTAM)

Allows users to access, transfer, and manage files on remote systems.

Protocol	Description	Port
FTP	Upload/download files	21 (control), 20 (data)
TFTP	Simplified FTP — UDP, no authentication	69
SFTP / SCP	Secure file transfer over SSH	22
NFS	Network File System — mount remote directories	2049
SMB / CIFS	Windows file and printer sharing	445

2.3 Mail Services

Provides electronic messaging between users across networks.

Protocol	Role	Port
SMTP	Sends email	25 / 587
POP3	Downloads email to client (deletes from server)	110
IMAP4	Reads email on server (keeps on server)	143
MIME	Handles attachments and non-ASCII content in emails	—

2.4 Directory Services

Provides a lookup system for network resources — translating names to addresses.

Protocol	Purpose
DNS	Resolves domain names → IP (e.g., <code>google.com</code> → <code>142.250.80.14</code>)
LDAP	Access directory services — user authentication, Active Directory
NIS	Unix/Linux directory service

2.5 Other Application Layer Services

Service	Protocol	Port
Web browsing	HTTP	80
Secure web	HTTPS	443
Remote admin	SSH	22
Remote terminal	Telnet	23
Network management	SNMP	161 / 162
Time synchronization	NTP	123
IP address assignment	DHCP	67 / 68

3. Layer 7 Devices

Device / Component	Role
End-user computers	Run application layer protocols — browsers, email clients
Web servers	Serve HTTP/HTTPS content
DNS servers	Resolve domain names to IP addresses
Mail servers	Handle SMTP/POP3/IMAP
DHCP servers	Assign IP addresses dynamically
Application Firewall (L7)	Deep packet inspection at the application layer
Load Balancer (L7)	Routes requests based on URL, cookies, HTTP headers

OSI Model — Complete Reference

Overview

Application (L7)	} → Data (Layers 5–7 work with raw data – no individual headers added)
Presentation (L6)	
Session (L5)	
	↓ + L4 header
Transport (L4)	→ Segment (src/dst port, seq no., ACK, window size)
	↓ + L3 header
Network (L3)	→ Packet (src/dst IP, TTL, protocol)
	↓ + L2 header + L2 trailer
Data Link (L2)	→ Frame (src/dst MAC, FCS/CRC)
	↓
Physical (L1)	→ Bits (0s and 1s as voltages / light / radio waves)

Note: Layers 5, 6, and 7 do not each add a distinct header the way L2/L3/L4 do. These upper layers operate together within the application process — only L4 (segment), L3 (packet), and L2 (frame) produce distinct encapsulation headers. This is why the TCP/IP model collapses L5/L6/L7 into a single **Application layer**.

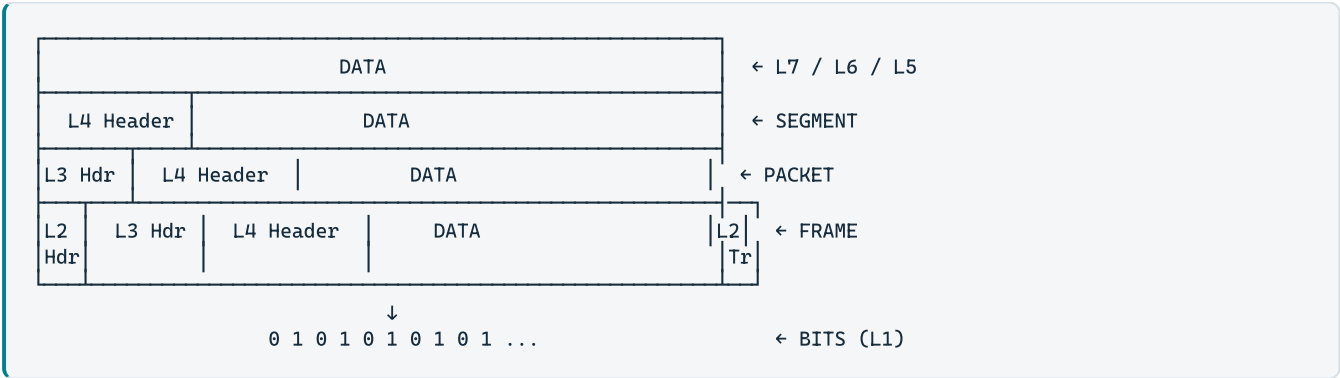
OSI vs TCP/IP: The OSI model is a **conceptual/teaching framework** — it is not a strict implementation. Modern internet stacks follow the simpler **TCP/IP model** (4 layers: Network Access, Internet, Transport, Application), which does not implement each OSI layer as a separate protocol. OSI is used for **troubleshooting, teaching, and vendor-neutral discussion** — not as a blueprint for how networks are actually built.

OSI Layer	TCP/IP Layer
L7 Application + L6 Presentation + L5 Session	Application
L4 Transport	Transport
L3 Network	Internet
L2 Data Link + L1 Physical	Network Access

Data Encapsulation & Decapsulation

1. Data Encapsulation (Sending Data)

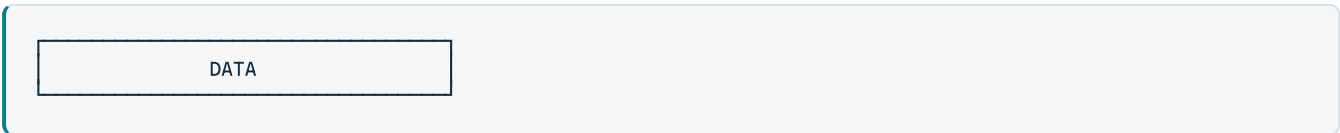
Encapsulation is the process of adding headers and trailers to data as it moves **down** from the Application layer to the Physical layer before transmission. Each layer wraps the data from the layer above with its own control information.



Step 1 — Application / Presentation / Session Layers (Data)

User generates data. Before passing down, it may be:

- **Compressed** (Presentation — L6)
- **Encrypted** (Presentation — L6)
- **Formatted / Serialized** (Presentation — L6)
- **Session managed** (Session — L5)



Examples

HTTP request from a browser

Email message body

File transfer data

Data unit at this stage: DATA**Step 2 — Transport Layer (Segment)**The Transport Layer adds a **TCP or UDP header** to the data.

TCP / UDP Header	
Source Port	→ identifies the sending application
Destination Port	→ identifies the receiving application
Sequence Number (TCP only)	→ ensures correct data order
Acknowledgement Number (TCP only)	→ confirms received bytes
Flags (TCP only)	→ SYN, ACK, FIN, RST
Window Size (TCP only)	→ flow control
Checksum	→ error detection
DATA	

Example values:

Source Port: 49152 (ephemeral – client side)
 Destination Port: 80 (HTTP server)
 Sequence Number: 1001

Data unit at this stage: SEGMENT**Step 3 — Network Layer (Packet)**The Network Layer adds an **IP header** to the segment.

IP Header	
Version	→ IPv4 or IPv6
Header Length (IHL)	
TTL (Time To Live)	→ prevents infinite loops
Protocol	→ TCP=6, UDP=17, ICMP=1
Header Checksum	
Source IP Address	→ sender device IP
Destination IP Address	→ receiver device IP
TCP Segment	

Example values:

```
Source IP:      192.168.1.10
Destination IP: 8.8.8.8
TTL:           64
Protocol:      6 (TCP)
```

Data unit at this stage: PACKET

Step 4 — Data Link Layer (Frame)

The Data Link Layer wraps the packet with a **MAC header** and appends a **CRC/FCS trailer** for error detection.

Dest MAC	Source MAC	Type	IP Packet (Payload)	CRC / FCS (Trailer)
----------	------------	------	------------------------	------------------------

Field	Purpose
Destination MAC	Next device on the local network
Source MAC	Sender's hardware address
Type	Identifies payload protocol (e.g., 0x0800 = IPv4)
CRC / FCS	32-bit error detection value

Example values:

```
Source MAC:      00:1A:2B:3C:4D:5E
Destination MAC: 98:76:54:32:10:AA
CRC:             32-bit value
```

Data unit at this stage: FRAME

Step 5 — Physical Layer (Bits)

The frame is converted into **binary bits** and transmitted over the physical medium.

010101010101110101010101010101110101010 ...

Transmitted through:

- Ethernet cable (electrical signal)
- Fibre optic (light pulses)
- Wi-Fi (radio waves)

Data unit at this stage: BITS

2. Data Decapsulation (Receiving Data)

Decapsulation is the reverse process — headers and trailers are **stripped off** as data moves **up** the OSI layers at the receiving device.

```

Bits (L1)
  ↓ reconstruct frame
Frame (L2) → check & strip MAC header + CRC trailer
  ↓
Packet (L3) → check & strip IP header
  ↓
Segment (L4) → check & strip TCP/UDP header, reorder, reassemble
  ↓
Data (L5/L6/L7) → session managed, decrypted, decompressed, delivered

```

Step 1 — Physical Layer

Receives raw binary bits from the network medium and reconstructs the bit stream into a frame for Layer 2.

```

010101010101010 ... → reconstructed frame

```

Step 2 — Data Link Layer

Checks:	Destination MAC	(is this frame for me?)
	CRC / FCS	(is the frame corrupted?)
If CRC fails → frame is DISCARDED (no retransmit at L2 – upper layers handle recovery)		
If CRC passes → strip MAC header + CRC trailer		
Remaining data → IP Packet passed up to L3		

Step 3 — Network Layer

Checks:	Destination IP	(is this packet for me?)
	TTL	(has it expired?)
	Protocol	(TCP=6 / UDP=17 / ICMP=1)
	Header Checksum	(header intact?)
Strips IP header		
Remaining data → TCP Segment passed up to L4		

Step 4 — Transport Layer

Checks:	Destination Port	(which app gets this?)
	Sequence Number	(order correct?)
	Checksum	(segment corrupted?)
Reorders out-of-sequence segments		
Detects and requests retransmission of missing data		
Strips TCP/UDP header		

Remaining → Original DATA passed up to L5

Step 5 — Session Layer

- Manages the active session.
- Applies synchronization checkpoints.
- Handles session re-synchronization if needed.

Step 6 — Presentation Layer

- **Decrypts** the data (e.g., TLS decryption).
- **Decompresses** the data (e.g., GZIP, JPEG decode).
- **Converts format** to the application's expected representation.

Step 7 — Application Layer

Final data is delivered to the correct application process.

Examples:

- Browser renders and displays the webpage (HTTP response)
- Email client displays the received message (IMAP/POP3)
- File manager saves the downloaded file (FTP)

Encapsulation vs Decapsulation — Side by Side

Layer	Encapsulation (Sending)	PDU	Decapsulation (Receiving)
L7 / L6 / L5	Generate, format, compress, encrypt data	Data	Deliver to app, decrypt, decompress
L4 — Transport	Add TCP/UDP header (ports, seq, checksum)	Segment	Check ports/seq, strip header, reassemble
L3 — Network	Add IP header (src/dst IP, TTL, protocol)	Packet	Check IP, strip header
L2 — Data Link	Add MAC header + CRC/FCS trailer	Frame	Check MAC + CRC, strip both
L1 — Physical	Convert frame → bits → transmit	Bits	Receive bits → reconstruct frame

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [01B - OSI Layers Quick Reference](#)
- [02 - TCP-IP Model](#)
- [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)
- [04A - Network Routing Fundamentals](#)
- [10 - PPP and WAN Technologies](#)
- [11 - Layer 2 Switching and Ethernet Forwarding](#)
- [A1 - HTTP Evolution and TLS](#)

Source note: 01A - OSI Model.md

CHAPTER 01B · NETWORK FOUNDATIONS

OSI Layers: Layer 1 to Layer 7

A compact seven-layer recall sheet for rapid revision before interviews.

Topic 1

computer networks

osi model

quick reference

interview preparation

Table of Contents

- Layer 1 – Physical Layer
- Layer 2 – Data Link Layer
- Layer 3 – Network Layer
- Layer 4 – Transport Layer
- Layer 5 – Session Layer
- Layer 6 – Presentation Layer
- Layer 7 – Application Layer
- Related Notes

Layer 1 – Physical Layer

- Raw-bit transmission
- Bit synchronization
- Physical topology
- Transmission mode
- Cables and connectors

Layer 2 – Data Link Layer

- Framing
- MAC addressing
- Hop-to-hop delivery
- Flow control
- Error control

Layer 3 – Network Layer

- Logical addressing
- Packetization
- Routing
- NAT

Layer 4 – Transport Layer

- Port addressing
- Connection control

Layer 5 – Session Layer

- Session establishment
- Session maintenance
- Session termination
- Checkpoints

Layer 6 – Presentation Layer

- Data translation
- Data formatting
- Encryption and decryption
- Compression and decompression

Layer 7 – Application Layer

- Web services
 - Email services
 - File transfer
 - Remote login
-

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [01A - OSI Model](#)

Source note: 01B - OSI Layers Quick Reference.md

CHAPTER 02 · NETWORK FOUNDATIONS

TCP/IP Model — Study Notes

The practical Internet stack, TCP versus UDP, handshakes, teardown, and checksums.

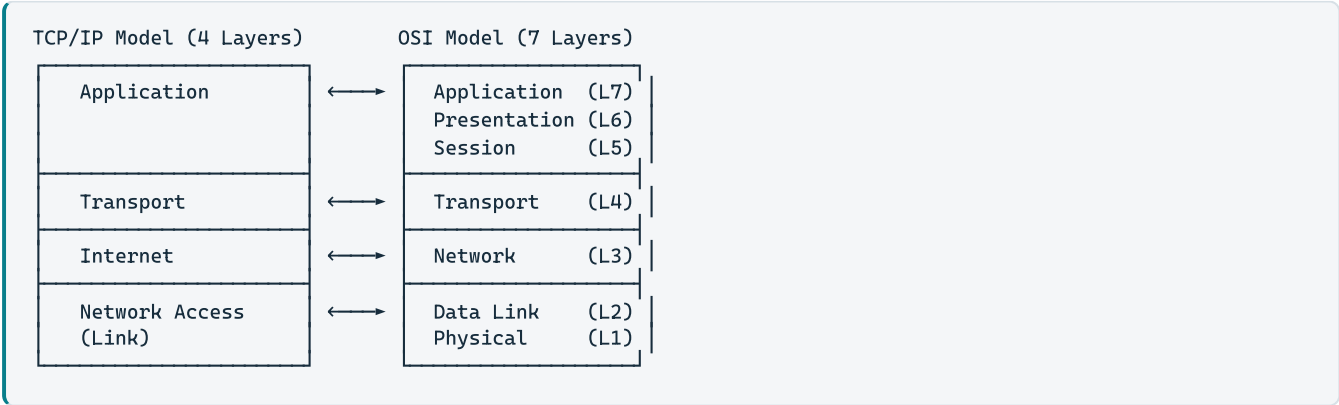
- Topic 2
- computer networks
- tcp ip
- transport layer

Table of Contents

- 1. TCP/IP Model — 4-Layer Overview
 - 2. TCP vs UDP — Full Comparison
 - 3. Why Video Streaming Uses UDP Instead of TCP
 - 4. TCP Three-Way Handshake (Connection Setup)
 - 5. TCP Four-Way Termination (Connection Teardown)
- 6. Quick Reference — Handshake vs Termination
 - 7. TCP/IP in Action — Full Data Flow
 - 8. Checksum — Error Detection
 - Related Notes

1. TCP/IP Model — 4-Layer Overview

The **TCP/IP model** (also called the **Internet Model** or **DoD model**) is the practical model used by the real internet. Unlike the 7-layer OSI model (a conceptual/teaching framework), the TCP/IP model is the **actual implementation** that networks run on today.



Layer-by-Layer Mapping

TCP/IP Layer	OSI Equivalent	Key Protocols
Application	L5 + L6 + L7	HTTP, HTTPS, FTP, DNS, SMTP, SSH, DHCP, SNMP
Transport	L4	TCP, UDP, SCTP
Internet	L3	IP (IPv4, IPv6), ICMP, ARP, NAT, Routing Protocols
Network Access	L1 + L2	Ethernet, Wi-Fi (802.11), MAC, ARP

Why 4 layers instead of 7? TCP/IP collapses OSI's upper three layers (Session, Presentation, Application) into a single **Application** layer — because in practice, most protocols handle all three concerns together (e.g., HTTP handles session management, formatting, and application logic in one protocol).

Similarly, TCP/IP collapses OSI's Physical and Data Link layers into **Network Access** — because the physical medium and framing are tightly coupled in real hardware (e.g., Ethernet).

2. TCP vs UDP — Full Comparison

Both TCP and UDP are **Transport Layer (L4)** protocols that live in the **Transport** layer of both the TCP/IP and OSI models.

Side-by-Side Comparison

Feature	TCP	UDP
Full Name	Transmission Control Protocol	User Datagram Protocol
Connection Type	Connection-oriented (handshake required)	Connectionless (no handshake)
Reliability	Reliable — ACK + retransmission	Unreliable — no ACK, no retransmit
Ordering	Guaranteed — sequence numbers used	No ordering — datagrams arrive as they come
Flow Control	Yes — Sliding Window	No
Congestion Control	Yes — slows sender if network is congested	No
Error Detection	Checksum + ACK + retransmit	Checksum only (no recovery)
Header Size	20–60 bytes	8 bytes (fixed)
Speed	Slower (due to overhead and handshake)	Faster (no setup, no acknowledgement)
Data Unit	Segment (byte-stream oriented)	Datagram (message-oriented)
IP Protocol Number	6	17
Use Cases	HTTP, HTTPS, FTP, SSH, SMTP, Telnet	DNS, DHCP, VoIP, SNMP, TFTP, Video Streaming, RIP

TCP Header (20–60 bytes)

Source Port	Destination Port	← 16-bit each – identifies sending/receiving process
Sequence Number		← 32-bit – byte position in stream; used for ordering
Acknowledgement Number		← 32-bit – next expected byte from other side
Flags	URG ACK PSH RST SYN FIN	← Control bits for handshake and connection state
Window Size		← Flow control – how much data receiver can accept
Checksum	Urgent Pointer	← Error detection

Interview Trap — Why is UDP's header only 8 bytes? UDP has no sequence numbers, no ACK, no window size, no connection state. It provides the bare minimum needed to identify source/destination processes and detect corruption. Everything else is stripped out for speed.

Use Cases — Choosing TCP vs UDP

3. Why Video Streaming Uses UDP Instead of TCP

The Core Problem with TCP for Streaming

TCP STOPS here and waits for segment 5 to be retransmitted
Segments 6, 7, 8... are buffered but NOT delivered to the app
until 5 arrives - even though they already reached the receiver

For video, this creates **visible freezing** — the player has frames 1–4 and frames 6–8, but it cannot display frame 6 until frame 5 is retransmitted and arrives. By the time frame 5 arrives, frame 6 is already late and causes a stutter.

↑
UDP delivers 6, 7, 8 immediately to the application
Missing frame 5 → application simply skips it or interpolates
Playback continues with minimal disruption

- A **missing frame** causes a brief visual artifact — barely noticeable to the human eye.
- A **delayed frame** causes freezing — very noticeable and annoying.
- UDP **skips the bad frame** and keeps playing; TCP **waits** for the bad frame, causing a freeze.

Why Latency Matters More Than Reliability in Streaming

Factor	TCP	UDP
Lost packet	Retransmitted — arrives late	Skipped — stream continues
Consequence	Freezing / stutter	Single corrupted/missing frame
User experience	Worse	Better

Real-world note: Modern streaming platforms (YouTube, Netflix, Zoom) often use **QUIC** (a protocol built on UDP by Google) which adds selective reliability on top of UDP — getting the best of both worlds. QUIC is the foundation of **HTTP/3**.

Summary — TCP vs UDP for Streaming

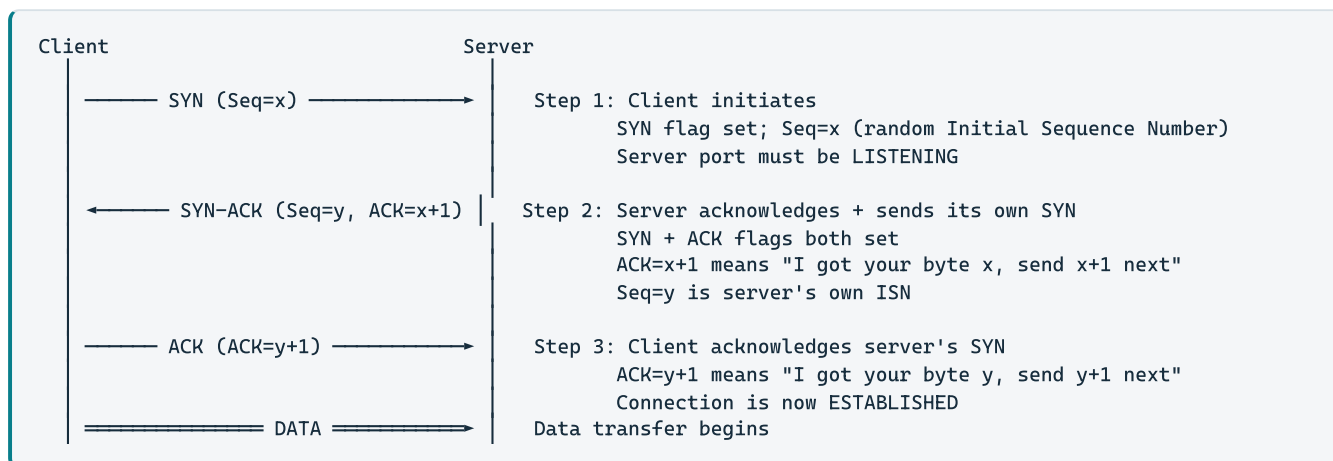
TCP streaming:	UDP streaming:
Segment lost	Segment lost
↓	↓
Wait for retransmit	Skip it, continue
↓	↓
All later frames stuck	Later frames delivered immediately
↓	↓
Video FREEZES ❌	Single glitch, playback continues ✔️

4. TCP Three-Way Handshake (Connection Setup)

What it Achieves

- Synchronizes **sequence numbers** on both sides (so each side knows where the other's byte stream starts)
- Confirms both sides can **send and receive** (bidirectionality verified)
- Allocates resources (buffers, socket state) on both sides

Handshake Steps



Flag Details

Step	Flags Set	Meaning
SYN (Client → Server)	SYN=1	"I want to connect; my starting sequence number is x"
SYN-ACK (Server → Client)	SYN=1, ACK=1	"Agreed; my starting sequence number is y; I acknowledge your x"
ACK (Client → Server)	ACK=1	"I acknowledge your y; connection established"

ISN — Initial Sequence Number

- Both sides pick a **random Initial Sequence Number (ISN)** — not starting from 0.
- Random ISN prevents **TCP sequence prediction attacks** (a security measure).
- After the handshake, sequence numbers increment with each byte sent.

Interview Trap — Why 3 steps and not 2? Two steps (SYN + SYN-ACK) would let the server confirm the client's ISN, but the client never gets a chance to confirm the **server's ISN**. A 3rd step (ACK) is required so the **server's ISN is also acknowledged**, making the connection truly bidirectional. Two steps would leave the server's sequence number unacknowledged.

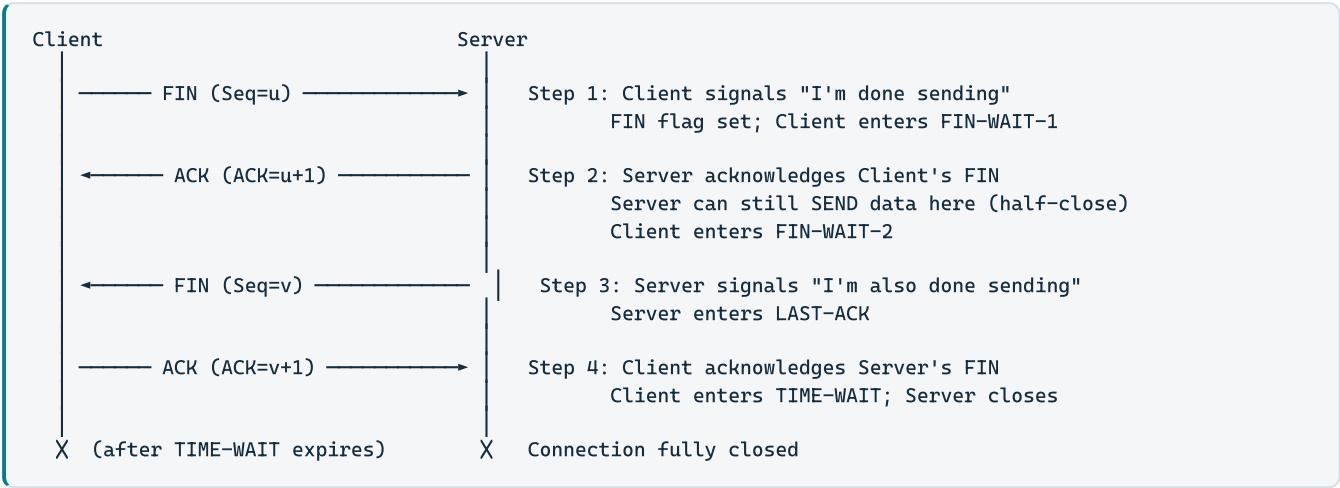
5. TCP Four-Way Termination (Connection Teardown)

TCP uses a **4-way handshake** to close a connection. It requires 4 steps (not 3) because **both sides must independently close their half of the connection** — TCP is full-duplex, so each direction is closed separately.

Why 4-Way (Not 3-Way)?

- After Client sends FIN — Client can no longer send, but can still **receive**.
- Server may still have data to send before it's ready to close its side.
- So Server sends **ACK immediately** (to acknowledge the FIN), but sends its own **FIN later** (when it's done sending).
- This creates 4 distinct steps instead of 3.

Termination Steps



TCP Connection States During Termination

Side	State	What's Happening
Client	FIN-WAIT-1	Sent FIN; waiting for ACK
Client	FIN-WAIT-2	Got ACK; waiting for Server's FIN
Server	CLOSE-WAIT	Got Client's FIN; still may be sending data
Server	LAST-ACK	Sent FIN; waiting for final ACK
Client	TIME-WAIT	Got Server's FIN; sent final ACK; waiting before fully closing
Both	CLOSED	Connection terminated

TIME-WAIT State

After sending the final ACK, the Client waits in **TIME-WAIT** for **2 × MSL (Maximum Segment Lifetime)** — typically 60–120 seconds.

Why TIME-WAIT exists:

- 1. If the final ACK is lost, the Server will retransmit its FIN — the Client must be alive to respond again.
- 2. Ensures old delayed packets from the closed connection don't interfere with a new connection using the same port pair.

Interview Trap — What if the final ACK is lost? The Server never receives the final ACK, so it retransmits its FIN. The Client is still in TIME-WAIT — it receives the retransmitted FIN and sends the ACK again. If the Client had already closed, the Server would be stuck in LAST-ACK forever. TIME-WAIT prevents this.

6. Quick Reference — Handshake vs Termination

Aspect	3-Way Handshake	4-Way Termination
Purpose	Establish connection	Close connection
Steps	3	4
Why not fewer?	Need to sync both ISNs	Each side closes independently

Aspect	3-Way Handshake	4-Way Termination
Key flags	SYN, SYN-ACK, ACK	FIN, ACK, FIN, ACK
Initiator	Client (usually)	Either side
End state	ESTABLISHED	CLOSED (after TIME-WAIT)

7. TCP/IP in Action — Full Data Flow

Putting it all together with the TCP/IP model:

```

Application Layer (HTTP request)
  | "GET /index.html HTTP/1.1"
Transport Layer (TCP Segment)
  | + Src Port: 49200 Dst Port: 80
  | + Seq No., ACK, Window Size, Checksum
Internet Layer (IP Packet)
  | + Src IP: 192.168.1.10 Dst IP: 8.8.8.8
  | + TTL: 64 Protocol: 6 (TCP)
Network Access Layer (Ethernet Frame)
  | + Src MAC: AA:BB:CC:DD:EE:FF Dst MAC: 11:22:33:44:55:66
  | + CRC/FCS
Physical → Bits → 010101010101010...

```

Key reminders:

- **IP addresses** — assigned at Internet Layer — don't change end-to-end.
- **MAC addresses** — assigned at Network Access Layer — change at **every router hop**.
- **Port numbers** — assigned at Transport Layer — identify the **application process**.
- **TCP connection** must be established (3-way handshake) **before** any HTTP data is sent.

8. Checksum — Error Detection

What is a Checksum?

A **checksum** is a calculated value added to data before transmission. It lets the receiver verify whether the data arrived intact or was corrupted in transit.

SENDER	RECEIVER
Data: [1, 2, 3, 4]	Receives: [1, 2, 3, 4] + Checksum: 10
Sum = 1+2+3+4 = 10	Recalculates: 1+2+3+4 = 10
Send data + checksum (10) →	Compares: 10 == 10 ✓ → Data is intact
	If data was corrupted: [1, 2, 9, 4]
	Recalculates: 1+2+9+4 = 16
	Compares: 16 ≠ 10 ✗ → Data corrupted → Discard

The receiver independently recalculates the checksum from the received data and compares it to the checksum that arrived with the packet. Match = intact. Mismatch = corrupted → discard.

Where Checksums Appear in the OSI Model

Layer	Field	What it Protects
L2 — Data Link	CRC / FCS (Frame Check Sequence)	The entire Ethernet frame
L3 — Network	IP Header Checksum	The IP header only (not the payload)
L4 — Transport	TCP / UDP Checksum	The TCP/UDP header + data

Each layer has its own checksum protecting its own PDU independently.

What Happens When Checksum Fails?

The action taken depends on **which layer** detected the corruption.

Layer 2 — Ethernet (CRC/FCS fails)

```

Frame arrives → L2 checks CRC → Mismatch ✗
      ↓
Frame is SILENTLY DROPPED
No notification to sender
No retransmit at L2
      ↓
Upper layers (TCP) notice the missing segment
and handle recovery themselves
  
```

Layer 4 — TCP (Checksum fails)

```

Segment arrives → TCP checks checksum → Mismatch ✗
      ↓
Segment is DROPPED (silently)
      ↓
Sender waits for ACK that never comes
      ↓
RTO (Retransmit Timeout) expires
      ↓
Sender RETRANSMITS the segment ✔
      ↓
Receiver gets clean copy → ACK sent
  
```

TCP recovers automatically — the application never knows it happened.

Layer 4 — UDP (Checksum fails)

```

Datagram arrives → UDP checks checksum → Mismatch ✗
      ↓
Datagram is DROPPED
      ↓
NOTHING happens.
No ACK, no retransmit,
no notification to sender.
Application never receives it.
  
```

This is why UDP is called "**fire and forget**" — it has no recovery mechanism.

Summary — What Happens on Corruption

Layer	Protocol	On Corruption	Recovery
L2	Ethernet (CRC/FCS)	Frame silently dropped	TCP (L4) handles it
L3	IP Header Checksum	Packet dropped	TCP (L4) handles it
L4	TCP Checksum	Segment dropped → RTO expires	Sender retransmits automatically ✓
L4	UDP Checksum	Datagram dropped	Nothing — gone forever ✗

Key Points for Interviews

- Checksum **detects** errors only — it does **not correct** them. A corrupted PDU is simply discarded.
- **Recovery only happens at TCP (L4)** — every other layer just discards and moves on.
- The IP header checksum covers **only the IP header**, not the payload — that's why TCP/UDP have their own checksums covering the data.
- Checksums catch **transmission errors** (bit flips from electrical noise) — they are not security tools. They cannot detect intentional tampering. That requires cryptographic hashes (SHA-256, HMAC).

Interview Trap — Does UDP use a checksum? Yes — UDP does have a checksum field in its 8-byte header. But it only **detects** corruption; it cannot **recover** from it. A corrupted UDP datagram is simply dropped with no follow-up. The checksum in UDP is also technically optional in IPv4 (though almost always used in practice), but mandatory in IPv6.

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [01A - OSI Model](#)
- [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)
- [A1 - HTTP Evolution and TLS](#)

Source note: 02 - TCP-IP Model.md

CHAPTER 03 · NETWORK FOUNDATIONS

IPv4 & IPv6 Addressing, Subnetting, VLSM, and NDP

IPv4, IPv6, CIDR, subnetting, VLSM, address scopes, EUI-64, and neighbor discovery.

- Topic 3
- Topic 7
- computer networks
- ip addressing
- subnetting
- ipv6

Table of Contents

- 1. What Is an IP Address?
- 2. IPv4 Address Classes
- 3. Subnetting
- 4. CIDR (Classless Inter-Domain Routing)
- 5. VLSM (Variable Length Subnet Mask)
- 6. Wildcard Mask & Wildcard IP
- 7. IPv6 Addressing

- 8. EUI-64 (Extended Unique Identifier)
- 9. IPv6 Address Scopes: Link-Local, ULA, and Global Unicast
- 10. IPv6 Enhancements Over IPv4
- 11. NDP — Neighbor Discovery Protocol (IPv6)
- 12. IPv4 vs IPv6 — Side-by-Side Comparison
- 13. Quick Revision Sheet
- Related Notes

1. What Is an IP Address?

An **IP (Internet Protocol) address** is a unique number given to every device (computer, phone, router, server) on a network so it can send and receive data. It does two jobs:

- 1. Identifies the device** — tells the network *who* this device is.
- 2. Locates the device** — tells routers *where* to send packets so they reach it.

IP Address = Network Portion + Host Portion
(which network?) (which device on it?)

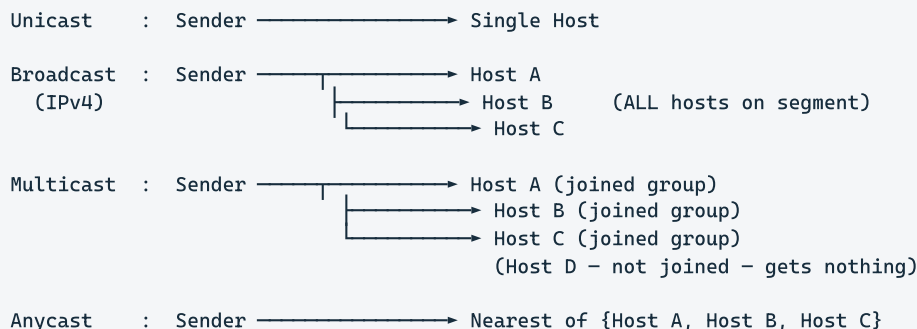
- IPv4** → 32 bits long, written as `192.168.1.10`
- IPv6** → 128 bits long, written as `2001:db8::1`

1.1 How Traffic Gets Delivered: Unicast, Broadcast, Multicast, Anycast

Every packet on a network travels using one of four delivery styles:

Type	Goes To	Works In	In Plain Words
Unicast	One specific device	IPv4 & IPv6	One sender, one receiver — normal web browsing traffic.
Broadcast	Every device on the local network	IPv4 only	One sender, everyone on the segment gets it (e.g. <code>255.255.255.255</code>). IPv6 has no broadcast at all — multicast does this job instead.
Multicast	Only devices that joined a group	IPv4 & IPv6	One sender, only "subscribed" devices receive it. IPv4 range: <code>224.0.0.0–239.255.255.255</code> . IPv6 range: <code>ff00::/8</code> .

Type	Goes To	Works In	In Plain Words
Anycast	The <i>nearest</i> device in a group	Mainly IPv6 (limited IPv4 use)	One sender, delivered to whichever member of the group is closest by routing distance.



One-liners to remember:

- Unicast = 1 → 1
- Broadcast = 1 → everyone (IPv4 only)
- Multicast = 1 → subscribed group (both IPv4 & IPv6)
- Anycast = 1 → nearest member of a group (mostly IPv6)

2. IPv4 Address Classes

IPv4 addresses are split into **5 classes**, identified by the value of the **first octet**.

```

Class A : 0xxxxxxx -> 1 - 126   (N.H.H.H)
Class B : 10xxxxxx -> 128 - 191 (N.N.H.H)
Class C : 110xxxxx -> 192 - 223 (N.N.N.H)
Class D : 1110xxxx -> 224 - 239 (Multicast, no subnet mask)
Class E : 1111xxxx -> 240 - 255 (Experimental / research, no subnet mask)

```

`127.x.x.x` is carved out of Class A and reserved for **loopback** testing.

Class	First Octet Range	Default Mask	# Networks	# Hosts / Network	Format
A	1 – 126	255.0.0.0	126 ($2^7 - 2$)	16,777,214 ($2^{24} - 2$)	N.H.H.H
B	128 – 191	255.255.0.0	16,384 (2^{14})	65,534 ($2^{16} - 2$)	N.N.H.H
C	192 – 223	255.255.255.0	2,097,152 (2^{21})	254 ($2^8 - 2$)	N.N.N.H
D	224 – 239	— (Multicast)	—	—	—
E	240 – 255	— (Experimental)	—	—	—

Key points:

- "N" = network bits, "H" = host bits.
- Class D is for **multicast only** — it has no host/network split and no subnet mask.
- Class E is **reserved for research** — also no subnet mask.
- Class-based addressing (A/B/C/D/E) is largely historical today — real networks use **CIDR** (Section 4), which ignores class boundaries. Classes are still worth knowing for exams and for reading legacy documentation.

3. Subnetting

Subnetting means splitting one large network into smaller networks by "borrowing" bits from the host portion to extend the network portion.

Default subnet masks (by class):

Class	Default Subnet Mask
A	255.0.0.0
B	255.255.0.0
C	255.255.255.0

Why subnet at all?

- ✓ Cuts down broadcast traffic (smaller broadcast domains)
- ✓ Gets around the fixed host-count limits of a single classful network
- ✓ Lets you expose only part of a network for security/segmentation (e.g. a guest Wi-Fi subnet)

Core formulas:

```
Usable subnets = 2^(subnet bits)
Usable hosts    = 2^(host bits) - 2
Block size      = 256 - (subnet mask octet value)
```

Older textbooks subtract 2 from the subnet count for an "all-zeros" and "all-ones" subnet. Modern routers (RFC 1878) allow both, so in practice **all 2ⁿ subnets are usable** — but some exam boards still test the old "–2" rule, so know both.

3.1 Class C Example — Mask 255.255.255.224 (/27)

Last octet in binary: 1110 0000 (3 subnet bits, 5 host bits)

- Subnet bits = 3 → up to **8 subnets** (6 if using the older –2 convention)
- Host bits = 5 → $2^5 - 2 = 30$ **usable hosts per subnet**
- Block size = $256 - 224 = 32$

Subnet	Broadcast	Valid Host Range
32	63	33–62
64	95	65–94
96	127	97–126
128	159	129–158
160	191	161–190
192	223	193–222

3.2 Class B Example — Mask 255.255.240.0 (/20)

- 4 bits borrowed → $2^4 = 16$ **subnets** (14 under the old convention)
- 12 host bits → $2^{12} - 2 = 4,094$ **hosts per subnet**
- Block size = $256 - 240 = 16$

Subnet	First Host	Last Host	Broadcast
.16.0	.16.1	.31.254	.31.255
.32.0	.32.1	.47.254	.47.255
.48.0	.48.1	.63.254	.63.255
.64.0	.64.1	.79.254	.79.255

3.3 Class A Example — Mask 255.240.0.0 (/12)

- 4 bits borrowed → $2^4 = 16$ **subnets** (14 under the old convention)
- 20 host bits → $2^{20} - 2 = 1,048,574$ **hosts per subnet**
- Block size = $256 - 240 = 16$

```

First subnet:
Subnet    : 10.0.0.0
Broadcast : 10.15.255.255
Hosts     : 10.0.0.1 - 10.15.255.254

Last subnet:
Subnet    : 10.240.0.0
Broadcast : 10.255.255.255
Hosts     : 10.240.0.1 - 10.255.255.254

```

4. CIDR (Classless Inter-Domain Routing)

CIDR replaces the rigid Class A/B/C system with a simple **slash notation** (`/n`) that states exactly how many bits, counting from the left, form the network portion. This lets a network be *any* size, not just a class-sized one.

```

192.168.1.0/24
    └─ "/24" = first 24 bits = NETWORK part
          = last 8 bits  = HOST part
          = subnet mask 255.255.255.0

```

4.1 Why CIDR Matters

- ☒ No more wasted addresses (a Class C used to force exactly 254 hosts even if you only needed 10)
- ☒ Enables **route summarization** — many small networks advertised as one big block, shrinking routing tables
- ☒ Is the foundation that makes **VLSM** possible (Section 5)

4.2 Prefix Length ↔ Subnet Mask ↔ Hosts (IPv4 Quick Reference)

CIDR (/n)	Subnet Mask	# Host Bits	Usable Hosts ($2^n - 2$)
/24	255.255.255.0	8	254
/25	255.255.255.128	7	126
/26	255.255.255.192	6	62

CIDR (/n)	Subnet Mask	# Host Bits	Usable Hosts ($2^n - 2$)
/27	255.255.255.224	5	30
/28	255.255.255.240	4	14
/29	255.255.255.248	3	6
/30	255.255.255.252	2	2 (point-to-point links)
/32	255.255.255.255	0	1 (single host route)

Formula:

Usable Hosts = $2^{(32 - n)} - 2$ where n = CIDR prefix length

4.3 Route Summarization (Supernetting)

CIDR also lets you go the *other* direction — combine several small networks into one larger advertised block:

```

192.168.0.0/24
192.168.1.0/24
192.168.2.0/24
192.168.3.0/24
└─ Summarized as 192.168.0.0/22
    (covers .0.0 - .3.255, 1024 addresses)

```

- **Shorter prefix** (fewer network bits) → bigger block, more hosts, fewer routes to advertise.
- **Longer prefix** (more network bits) → smaller block, fewer hosts, more granular control.

4.4 CIDR in IPv6

Same /n idea, e.g. `2001:db8::/32`. IPv6 almost always standardizes on a **/64** network prefix, leaving the remaining 64 bits for the device's own identifier (see Section 7).

One-liners:

- CIDR = slash notation that states the network-bit count, replacing rigid classes.
- Shorter prefix = bigger block. Longer prefix = smaller block.
- CIDR supports both subnetting (splitting) and summarization (combining).

5. VLSM (Variable Length Subnet Mask)

VLSM, also called **classless addressing**, lets you use *different* mask lengths for different subnets carved from the same network — long masks (small blocks) for small departments, short masks (big blocks) for large ones. It requires a **classless routing protocol** (e.g. OSPF, EIGRP) to work, since those protocols carry the mask along with each route.

Why use VLSM instead of one fixed subnet size for everyone? Because department sizes vary. A fixed subnet size forces you to either waste addresses on small departments or run out of addresses for big ones. VLSM sizes each subnet to fit its actual need, minimizing waste.

Golden rule: always allocate the largest requirement first, then carve the remaining space for smaller ones.

Worked Example — 192.168.1.0/24

Requirements (sorted largest → smallest):

Department	Hosts Needed
Sales	100

Department	Hosts Needed
Purchase	50
Accounts	25
Management	5

Steps:

1. Sort requirements from biggest to smallest.
2. Give the biggest requirement the smallest mask that still fits it (most hosts).
3. Move to the next unused block of address space and repeat for the next requirement.

Dept	Network	Prefix	Mask	Usable Hosts
Sales	192.168.1.0	/25	255.255.255.128	126
Purchase	192.168.1.128	/26	255.255.255.192	62
Accounts	192.168.1.192	/27	255.255.255.224	30
Management	192.168.1.224	/29	255.255.255.248	6

```

192.168.1.0/24
├── 192.168.1.0/25   → Sales      (126 hosts)
├── 192.168.1.128/26 → Purchase  (62 hosts)
├── 192.168.1.192/27 → Accounts  (30 hosts)
└── 192.168.1.224/29 → Management (6 hosts)
    └── 192.168.1.232 - 255 → still unused, kept for future growth
  
```

6. Wildcard Mask & Wildcard IP

A **wildcard mask** is the *inverse* of a subnet mask. It's used in **ACLs (Access Control Lists)** and **OSPF** network statements, and it flips the logic of a normal subnet mask:

```

Subnet Mask   : 1 = network bit (must match)  0 = host bit (varies)
Wildcard Mask : 0 = must match                 1 = don't care (any value OK)

Wildcard Mask = 255.255.255.255 - Subnet Mask
  
```

6.1 Quick Reference

CIDR (/n)	Subnet Mask	Wildcard Mask
/24	255.255.255.0	0.0.0.255
/25	255.255.255.128	0.0.0.127
/26	255.255.255.192	0.0.0.63
/27	255.255.255.224	0.0.0.31
/28	255.255.255.240	0.0.0.15
/29	255.255.255.248	0.0.0.7
/30	255.255.255.252	0.0.0.3
/32	255.255.255.255	0.0.0.0

CIDR (/n)	Subnet Mask	Wildcard Mask
/0	0.0.0.0	255.255.255.255

Worked example:

```

Subnet Mask   : 255.255.255. 0 → 11111111.11111111.11111111.00000000
Wildcard Mask :   0.  0.  0.255 → 00000000.00000000.00000000.11111111
                (each octet: 255 - subnet-octet = wildcard-octet)

```

6.2 Where Wildcard IPs Are Used

Use Case	Example Syntax	Meaning
ACL permit/deny	<code>access-list 10 permit 192.168.1.0 0.0.0.255</code>	Matches any host <code>192.168.1.0</code> — <code>192.168.1.255</code>
OSPF network statement	<code>network 10.0.0.0 0.0.0.255 area 0</code>	Enables OSPF on interfaces in <code>10.0.0.0/24</code>
Match a single host	<code>access-list 10 permit 192.168.1.5 0.0.0.0</code>	Matches only <code>192.168.1.5</code> — every bit must match
Match anything	<code>access-list 10 permit 0.0.0.0 255.255.255.255</code>	Matches any address — shortcut keyword: <code>any</code>

One-liners:

- Wildcard mask = inverse of subnet mask.
- `0` bit = must match exactly | `1` bit = don't care.
- All-`0.0.0.0` wildcard = match exactly one host.
- All-`255.255.255.255` wildcard = match every address.
- Wildcard masks configure **ACLs and OSPF**, not an interface's actual IP (that still uses a normal subnet mask).

7. IPv6 Addressing

- **128 bits** long (vs IPv4's 32 bits) — written as 8 groups of hex digits.
- Identifies a network interface and enables routing, exactly like IPv4 does, just with a vastly bigger address pool.
- **No broadcast in IPv6** — the all-nodes multicast group `ff02::1` covers most of that role, though protocol-specific multicast groups are preferred in practice.

7.1 IPv6 Address Types

Type	Delivered To	Notes
Unicast	One specific interface	Standard one-to-one delivery.
Multicast	All interfaces in a group	Replaces IPv4 broadcast; devices join a group to receive it.
Anycast	The nearest interface in a group	Same address <i>format</i> as unicast — the only difference is that the address is assigned to multiple devices, and routing sends traffic to whichever one is closest. A common real-world use is DNS root servers: many physical servers share one anycast address, and each client reaches the nearest one.

7.2 Writing IPv6 Addresses

Full form — 8 groups of 4 hex digits ("hextets"), separated by `:`:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

Rule 1 — drop leading zeros in each group (keep at least one digit):

```
2001:db8:85a3:0:0:8a2e:370:7334
```

Rule 2 — replace ONE run of consecutive all-zero groups with `::` (only once per address, to avoid ambiguity about how many groups it stands for):

```
2001:db8:85a3::8a2e:370:7334
```

Special addresses:

Address	Meaning
<code>::1</code>	Loopback (full: <code>0:0:0:0:0:0:0:1</code>)
<code>::</code>	Unspecified address (full: <code>0:0:0:0:0:0:0:0</code>)

7.3 Structure of an IPv6 Address: Network ID + Interface ID

A typical `/64` IPv6 address splits cleanly into two 64-bit halves:

```
3A:2B1C : 0000:0000 : 0009:0101:0000:007C /64
  |             |
  Network ID   Interface ID
  (first 64 bits) (last 64 bits)
```

Part	Bits	Also Called	Purpose
Network ID	First 64	Network Prefix, Subnet ID	Identifies <i>which network</i> — assigned by admin/ISP.
Interface ID	Last 64	Host ID	Identifies <i>which specific device</i> on that network.

The Interface ID can be set two ways:

Method	Description
Manual	Admin types the full 64-bit host part directly, e.g. <code>ipv6 address 3A:2B1C::1/64</code>
EUI-64	Auto-generated from the interface's 48-bit MAC address (see Section 8)

Same address, three equivalent forms:

Form	Address
Full	<code>003A:2B1C:0000:0000:0009:0101:0000:007C /64</code>
Leading zeros dropped	<code>3A:2B1C:0:0:9:101:0:7C /64</code>
Zero-run compressed	<code>3A:2B1C::9:101:0:7C /64</code>

⚠️ :: can appear **only once per address** — using it twice makes it impossible to tell how many zero groups each one represents.

7.4 Configuring IPv6 (Cisco IOS)

Step 1 – Enable IPv6 routing globally:
Router(config)# ipv6 unicast-routing

Step 2 – Enter the interface:
Router(config)# interface g0/0

Step 3a – Auto-generate the host part via EUI-64:
Router(config-if)# ipv6 address 3A:2B1C::/64 eui-64

Step 3b – OR manually assign a complete address:
Router(config-if)# ipv6 address 3A:2B1C::1/64

Command	Purpose
ipv6 unicast-routing	Enables IPv6 routing on the device
interface g0/0	Enters configuration mode for that interface
ipv6 address <prefix>/64 eui-64	Auto-generates the Interface ID from the MAC address
ipv6 address <full-address>/64	Manually assigns a complete IPv6 address

8. EUI-64 (Extended Unique Identifier)

EUI-64 automatically builds a 64-bit Interface ID (Section 7.3) out of a device's 48-bit MAC address, so no one has to type a host address by hand.

Algorithm:

1. Split the 48-bit MAC address into two 24-bit halves.
2. Insert FFFE in the middle → 48 bits become 64 bits.
3. Flip the 7th bit of the first byte (the universal/local bit).

Worked example — MAC = 00:1A:2B:3C:4D:5E

Step	Value
Original MAC	00:1A:2B:3C:4D:5E
Split into halves	00:1A:2B 3C:4D:5E
Insert FFFE	00:1A:2B:FF:FE:3C:4D:5E
Flip 7th bit (00 → 02)	02:1A:2B:FF:FE:3C:4D:5E

Full address example — prefix 3A:2B1C::/64 + this Interface ID:

3A:2B1C:02:1A:2B:FF:FE:3C:4D:5E

(7–8 bits) (40 bits, random) (16 bits) (64 bits)

9.4 Global Unicast Address (GUA) — 2000::/3

This is IPv6's equivalent of a public IPv4 address — used for direct, end-to-end communication with the Internet.

9.5 Summary Table

Address Type	Range	Scope	Routable?
Link-Local	FE80::/10	Same link/LAN only	✗ Not by routers
Site-Local (deprecated)	FEC0::/10	Old internal site	✗ Deprecated, don't use
ULA	FC00::/7	Private organization/network	✓ Within private networks only
Global Unicast (GUA)	2000::/3	Internet	✓ Yes

One-liners:

- Every IPv6 interface has a link-local address, even with no global address configured.
- Site-Local is dead — use ULA for private addressing instead.
- Neither Link-Local nor ULA is reachable from outside its own scope.

9.6 Full IPv6 Special Address Range Reference

Beyond the scopes above, a handful of other reserved ranges show up regularly in exams and configs:

IPv6 Address / Range	Name	Purpose
::/128	Unspecified	Means no address — used as a placeholder when an address isn't yet known (e.g. a host's source address before DHCPv6/SLAAC completes).
::1/128	Loopback	The device talking to itself — IPv6's equivalent of IPv4 127.0.0.1.
FE80::/10	Link-Local	Communication on the same LAN/link only (Section 9.1).
FC00::/7	Unique Local Address (ULA)	Private/internal IPv6 networks, not Internet-routable (Section 9.3).
FF00::/8	Multicast	One-to-many delivery — replaces IPv4 broadcast (Section 7.1).
2000::/3	Global Unicast	Public IPv6 addresses used on the Internet (Section 9.4).
2001:db8::/32	Documentation	Reserved specifically for use in examples, docs, and textbooks — never assigned to real devices, so it's safe to use in configs shown here.
::ffff:0:0/96	IPv4-mapped	Represents an IPv4 address inside an IPv6 packet/application, for dual-stack software that needs to handle both address families uniformly.
100::/64	Discard-Only	A "black hole" range — packets sent here are deliberately dropped, useful for testing or filtering.
FE00::/9	Reserved	Unassigned space held in reserve (part of what used to include the deprecated Site-Local range, Section 9.2).

FE00::/9 is a separate reserved block from FE80::/10 and FEC0::/10 (old Site-Local) — despite the similar-looking prefixes, the bit math places them in different, non-overlapping ranges. Don't assume nesting just because the hex looks similar.

9.7 Can a ULA-Only Device Reach the Internet?

No — not directly. Since ULA (`fc00::/7`) isn't routable on the public Internet, packets sourced from a ULA address get dropped as soon as they try to leave the private network. To actually reach the Internet, one of these is needed:

Option	How It Works	NAT Needed?
Dual addressing (<i>standard, recommended</i>)	Every device gets both a ULA (for internal traffic) and a GUA (for Internet traffic), typically via SLAAC or DHCPv6. Internet-bound packets simply use the GUA.	✗ No
NAT66 / NPTv6 (<i>uncommon</i>)	The edge router translates the ULA source prefix into a GUA prefix — conceptually similar to IPv4 NAT. Officially discouraged and rarely deployed in IPv6 networks.	✓ Yes

Standard design – dual addressing, no NAT:

```
PC — fd12:3456:789a:1::10  (ULA – internal LAN traffic)
    └─ 2001:db8:1::10      (GUA – traffic to the Internet)
```

Both addresses live on the SAME interface – no translation needed.

ULA-only LAN (no GUA assigned) – cannot reach the Internet:

```
PC (fd12::10 ULA only) —X→ Internet
                        |
                    dropped: ULA isn't
                        routable outside the
                        private network
```

This is why IPv6 is often described as "**NAT-free**" — the normal design gives every Internet-facing device its own GUA, so no translation is ever needed (contrast with IPv4 in Section 10). NAT66/NPTv6 exists only as a fallback for networks that insist on staying ULA-only.

Edge case: it's technically possible for only *one* device on a LAN to hold a GUA, with everyone else staying ULA-only. In that setup, the ULA-only devices must funnel their Internet traffic through that one device acting as a gateway/proxy — conceptually similar to IPv4-style NAT/PAT. This is **not** the recommended design; IPv6's huge address space makes giving every device its own GUA cheap and simple, so that's the standard approach.

10. IPv6 Enhancements Over IPv4

Function	IPv4 Approach	IPv6 Approach
Subnetting	Manual subnetting required to divide networks	Vast 128-bit space mostly avoids complex subnetting; a simple hierarchical <code>/64</code> prefix is the norm
Address Resolution	ARP — maps IP→MAC using broadcast	NDP — resolves addresses, detects duplicates, and discovers routers, using multicast instead of broadcast
Address Assignment	DHCP — server assigns IPs	SLAAC — device self-configures from the advertised network prefix; DHCPv6 available for stateful cases
Network Booting	BOOTP — bootstrap + IP assignment	Largely unnecessary — SLAAC (and DHCPv6 where needed) covers this

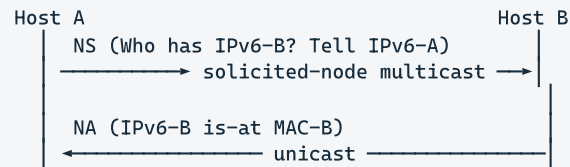
In short: IPv6 folds ARP, DHCP, and BOOTP's responsibilities into more efficient, integrated mechanisms (NDP, SLAAC), and its enormous address space removes most of the pressure to subnet carefully.

11. NDP — Neighbor Discovery Protocol (IPv6)

NDP replaces ARP (and adds more capabilities), using **ICMPv6 multicast** instead of broadcast.

Message	Direction	Purpose
RS (Router Solicitation)	Host → <code>ff02::2</code> (all-routers)	"Is there a router here?"
RA (Router Advertisement)	Router → <code>ff02::1</code> (all-nodes)	"I'm here — here's the network prefix and gateway."
NS (Neighbor Solicitation)	Host → solicited-node multicast	Resolves an IPv6 address to a MAC address; also used for Duplicate Address Detection (DAD).
NA (Neighbor Advertisement)	Host/Router → unicast reply	Answers an NS with the requested MAC address.

Address Resolution Flow (NDP):



DAD (Duplicate Address Detection): before a host starts using a new IPv6 address, it sends an **NS** targeting its own tentative address. If it gets an **NA** back, that address is already in use elsewhere — it must not be used.

12. IPv4 vs IPv6 — Side-by-Side Comparison

Feature	IPv4	IPv6
Size	32-bit	128-bit
Format	<code>192.168.1.10</code>	<code>2001:db8::1</code>
Total addresses	~4.3 billion	$\sim 3.4 \times 10^{38}$
Auto-config	DHCP only	SLAAC or DHCPv6
Address resolution	ARP (broadcast)	NDP (multicast)
Loopback	127.0.0.1	::1
Private / local addressing	RFC 1918 ranges	Unique Local Address, <code>fc00::/7</code>
Link-local	169.254.x.x (APIPA)	<code>fe80::/10</code> (mandatory on every interface)
NAT needed?	✔ Usually yes	✗ Rarely — only if a network stays ULA-only with no GUA (Section 9.6)

IPv6 adoption challenges: legacy hardware/software that doesn't support it, the added complexity of running dual stack, routing-table growth, a skills gap among network staff, and migration cost.

Transition mechanisms:

- **Dual Stack** — a device runs IPv4 and IPv6 side by side.
- **Tunneling** — IPv6 packets are wrapped inside IPv4 packets (e.g. 6to4, Teredo) to cross IPv4-only infrastructure.
- **Translation** — a device converts between IPv4 and IPv6 directly (e.g. NAT64) for interoperability.

13. Quick Revision Sheet

Class A : /8 default mask 255.0.0.0 → 126 nets / 16.7M hosts
 Class B : /16 default mask 255.255.0.0 → 16384 nets / 65534 hosts
 Class C : /24 default mask 255.255.255.0 → 2M nets / 254 hosts

VLSM rule : allocate the LARGEST requirement first
 IPv6 size : 128 bits = 8 hexets
 IPv6 ":::" : only ONE zero-compression allowed per address
 IPv6 split : /64 network part + /64 interface ID (host part)

EUI-64 : MAC (48-bit) → insert FFFE + flip 7th bit → 64-bit interface ID
 Anycast : same address on multiple interfaces, routed to the NEAREST one
 Link-Local : FE80::/10 (mandatory, auto-config, single-segment only)
 Site-Local : FEC0::/10 (deprecated – do not use)
 ULA (private): fc00::/7, usually fd00::/8 – IPv6's "private" address, not Internet-routable
 Global (GUA) : 2000::/3 – used for actual Internet communication

IPv6 replaces: ARP → NDP | DHCP → SLAAC/DHCPv6 | BOOTP → SLAAC (not needed)

NDP addresses: ff02::1 (all-nodes) / ff02::2 (all-routers)
 Resolution : IPv4 uses ARP (broadcast) | IPv6 uses NDP (multicast)

FCN PG-DITISS – IACSD Reference Notes

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [01A - OSI Model](#)
- [02 - TCP-IP Model](#)
- [04A - Network Routing Fundamentals](#)
- [06 - Network Address Translation](#)

Source note: 03 - IP Subnetting VLSM IPv4 IPv6 and NDP.md

PART II

Routing & Network Services

Move traffic between networks, segment broadcast domains, and translate addresses at the network edge.

[04A · Routing Fundamentals](#)[04B · Routing Protocols](#)[05 · VLANs & Inter-VLAN Routing](#)[06 · Network Address Translation](#)

CHAPTER 04A · ROUTING & NETWORK SERVICES

Network Routing — Detailed Study Notes

Routing tables, route sources, metrics, path selection, loop prevention, and load balancing.

Topic 4

Topic 10

computer networks

routing

routing table

cisco

Table of Contents

- 1) What is Routing?
- 2) Router
- 3) How Routing Works
- 4) Working Principle of Routing
- 5) Routing Table
- 6) Types of Routing
- 7) Administrative Distance
- Administrative Distance (AD) Table
- Notes
- Quick Memory Order (Low → High AD)
- 8) Routing Protocol Categories
- 9) Metrics and Path Choice
- 10) Distance Vector Routing
- 11) Link-State Routing
- 12) Hybrid Routing
- 13) Loop Prevention Techniques
- 14) Load Balancing
- 15) RIP vs OSPF vs EIGRP — Quick Comparison
- Related Notes

Administrative Distance (AD) — A number that tells a router how much it **trusts a route source**. Lower AD = more trusted. When two sources advertise the same destination, the router picks the one with the lower AD. Range: 0 (most trusted) to 255 (never used).

1) What is Routing?

Routing is the process of choosing the best path for data packets to travel from a source device to a destination device across one or more networks.

In simple words, routing decides:

- **Where a packet should go next**
- **Which path is best**
- **How to get data across different networks efficiently**

Routing is very important in packet-switched networks such as the **Internet**, where data is broken into packets and sent independently.

2) Router

A **router** is a Layer 3 device that connects different networks and forwards packets based on the **destination IP address**.

Main functions of a router

- Receives packets from one network
- Reads the destination IP address
- Checks the routing table
- Chooses the best next hop
- Forwards the packet toward the destination

OSI Layer

- **Layer 3: Network Layer**
-

3) How Routing Works

Routing works hop by hop. A packet does not usually go directly from sender to destination in one step. Instead, it passes through routers one by one.

Working process

1. A sender creates data and adds the **destination IP address** in the packet header.
2. The packet reaches the nearest router.
3. The router checks its **routing table**.
4. The router forwards the packet to the next router or next hop.
5. This process continues until the packet reaches the destination.

Important point

- Each router along the path is called a **hop**.
 - If a packet crosses too many hops, it may be dropped.
 - To prevent packets from looping forever, IP packets have a **TTL (Time To Live)** value.
 - TTL decreases by 1 at each hop.
 - When TTL becomes 0, the packet is discarded.
-

4) Working Principle of Routing

Routing is based on finding the **best path**, not just any path.

Step-by-step flow

1. **Communication starts** A device sends data to another device, often using an application protocol like HTTP, HTTPS, or SSH.
2. **Data is split into packets** Large data is divided into smaller packets for easier transmission.
3. **Destination IP is added** Each packet contains the destination IP address in its header.
4. **Router checks routing table** The router compares the destination IP with available routes.
5. **Best path is selected** The router uses metrics like hop count, bandwidth, delay, or cost.
6. **Packets move through hops** Packets travel through multiple routers until they reach the destination.
7. **Reassembly at destination** The destination device collects packets and rebuilds the original data.

5) Routing Table

A **routing table** is a database stored in a router that contains information about possible paths to different networks.

A routing table usually contains:

- Destination network
- Next hop
- Metric / cost
- Route source
- Interface to use

Why it matters

The router uses the routing table to decide where to send a packet next.

6) Types of Routing

Routing is mainly of **three types**.

Type	Meaning	What Router Learns / Uses	Best For	Advantages	Disadvantages
Static Routing	Manually configured routes	Nothing learned automatically; routes are manually entered (destination + next hop)	Small, simple networks	Simple, predictable, no protocol overhead	Not scalable, no automatic failover
Dynamic Routing	Routes learned automatically using protocols	Destination networks, next-hop addresses, and metrics via RIP, OSPF, BGP	Large and changing networks	Scalable, adapts to failures automatically	Complex, uses CPU, memory, and bandwidth
Default Routing	Uses a predefined route when no other route matches	Uses a single default path (default gateway) when no other route is available	Networks with a single exit path	Easy to configure, reduces routing table	Can cause inefficient routing in complex networks

6.1 Static Routing

Static routing means the network administrator manually enters the route into the router.

In static routing, routing entries are added manually in the routing table and typically include:

- Destination IP address
- Next-hop address (or exit interface)

Features

- Routes are configured by hand
- Does not change automatically
- Very simple and predictable
- Uses no routing protocol

Advantages

- Full control over the path
- More secure in small networks

- Low CPU and bandwidth usage

Disadvantages

- Hard to manage in large networks
- No automatic failover
- Not suitable when links change often

6.2 Dynamic Routing

Dynamic routing means routers automatically learn routes and update them when the network changes.

Features

- Routes are discovered automatically
- Routers exchange information
- Adapts to failures and new paths

Advantages

- Easier to manage in large networks
- Automatically reacts to network changes
- Scales better than static routing

Disadvantages

- Uses more CPU, memory, and bandwidth
- More complex than static routing
- Routing loops and convergence delays may occur

Issue	Meaning	Why It Happens in Dynamic Routing	Impact on Network	How It Is Reduced
Routing Loops	Packets circulate repeatedly between routers without reaching destination	Temporary inconsistent routing information between routers	Wastes bandwidth, increases delay, packet loss	Split horizon, route poisoning, hold-down timers
Convergence Delays	Time taken for all routers to update and agree on new network topology	Routers need time to exchange updates and recalculate routes	Temporary routing errors, packet drops	Faster protocols like OSPF/EIGRP, efficient updates

6.3 Default Routing

Default routing is used when the router does not know a more specific route.

Default route

- **0.0.0.0/0** in IPv4
- Means "send packets to this gateway if no other route matches"

Used when

- A network has only one way out
- A small branch office sends all unknown traffic to the main router or ISP

Example

If a router does not have a route for a destination, it sends the packet to the **default gateway**.

7) Administrative Distance

When a router learns the same destination from different routing sources, it uses **administrative distance** to choose the more trusted source. Lower AD means higher trust.

Administrative Distance (AD) Table

AD = Administrative Distance

Lower AD = More Trusted Route

Route Source	AD (Administrative Distance)	Meaning
Connected (Directly Connected)	0	Networks directly attached to router interfaces
Static Route	1	Manually configured routes
eBGP	20	Routes learned via External BGP
EIGRP Internal	90	Routes learned via EIGRP within same AS
IGRP	100	Routes learned via IGRP protocol
OSPF	110	Routes learned via OSPF protocol
IS-IS	115	Routes learned via IS-IS protocol
RIP	120	Routes learned via RIP protocol
EIGRP External	170	Routes redistributed into EIGRP
iBGP	200	Routes learned via Internal BGP
Unknown	255	Untrusted / unusable route (ignored)

Notes

- **Administrative Distance (AD)** indicates the **trustworthiness of a route source**.
- **Lower AD = Higher priority (more trusted).**
- A route with **AD 255** is **never used**.

Quick Memory Order (Low → High AD)

Connected → Static → eBGP → EIGRP → IGRP → OSPF → IS-IS → RIP → EIGRP External → iBGP → Unknown

0 → 1 → 20 → 90 → 100 → 110 → 115 → 120 → 170 → 200 → 255

Note - Default Routing != Directly connected

EIGRP internal routes are preferred over OSPF and RIP because their default administrative distance is lower.

8) Routing Protocol Categories

Routing protocols are rules used by routers to learn networks and choose the best path for packets. They are usually grouped into **distance vector**, **link-state**, and **hybrid** protocols. These are mainly **intradomain routing** protocols, meaning they are used inside an autonomous system rather than between different organizations or ISPs.

Category	Main idea	Example protocols	Typical use
Distance vector	Learns routes from neighbors only	RIP, IGRP	Small or simple networks
Link-state	Builds a full topology map	OSPF, IS-IS	Medium to large networks
Hybrid	Combines features of both	EIGRP	Enterprise and mixed-size networks

9) Metrics and Path Choice

A routing protocol decides the best path using a **metric**, which is its internal path cost. Different protocols use different metrics, so "best" can mean different things depending on the protocol.

Common metrics

- **Hop count:** number of routers crossed; RIP uses this.
- **Cost:** used by OSPF, usually based on bandwidth.
- **Bandwidth:** higher bandwidth links are preferred in OSPF cost calculation.
- **Delay:** used in some composite metrics such as EIGRP.
- **Reliability and load:** also part of composite path selection in EIGRP-style designs.

OSPF cost formula

OSPF commonly calculates cost as:

$$\text{cost} = \frac{\text{reference bandwidth}}{\text{interface bandwidth}}$$

With the default reference bandwidth of 100 Mbps, a 10 Mbps interface has a cost of 10.

10) Distance Vector Routing

Distance vector routing protocols choose routes based mainly on what their **neighbors** tell them. The router does not know the whole network map; it trusts information received from directly connected routers, which is why this is often called **routing by rumor**. RIP is the classic example, and its metric is hop count, meaning the route with the fewest routers is preferred.

Features

- Routers exchange updates periodically, often with the full routing table in RIP.
- Updates are shared only with neighboring routers, not flooded to the whole network.
- The main metric is usually **hop count**.
- RIP has a maximum hop count of **15**; a destination at 16 hops is unreachable.

Advantages

- Easy to configure and understand.
- Uses less CPU and memory than link-state protocols.
- Works well in small networks.

Disadvantages

- Converges slowly after a network change.
- More likely to suffer from routing loops and the **count-to-infinity** problem.
- Not ideal for large or fast-changing networks.

Distance vector vs link-state

Feature	Distance Vector	Link-State
Network view	Neighbor-based	Full topology-based
Updates	Periodic	Triggered + incremental
Algorithm	Bellman-Ford	Dijkstra SPF
Convergence	Slower	Faster
Loop issues	More common	Much less common
Example	RIP	OSPF, IS-IS

RIP overview

The **Routing Information Protocol (RIP)** is a dynamic routing protocol used by routers to find paths to destination networks. It operates at the **Network Layer (Layer 3)** and belongs to the **distance-vector** family. RIP is designed mainly for small to medium networks because it uses a very simple metric and has limited scalability.

Core idea

- Routers share route information with neighbors only.
- Each route is measured by **hop count**.
- The route with the lowest hop count is preferred.

Hop count

Hop count is the number of routers a packet must cross to reach a destination. In RIP, every router crossed adds **1 hop**, so the path with fewer routers is considered better. RIP allows a maximum hop count of **15**; a hop count of **16** means the destination is unreachable.

Why this matters:

- It prevents routes from circling forever in loops.
- It also limits RIP's usable network size.
- A path with better bandwidth is still ignored if it has more hops, because RIP does **not** consider bandwidth or delay.

How RIP works

RIP uses the **distance-vector** method, which means each router knows only what its neighbors tell it. Every **30 seconds**, routers send their routing table updates to neighboring routers, and the information spreads gradually until the network reaches **convergence**.

Operation flow

1. The router starts with directly connected networks.
2. It exchanges updates with neighbors every 30 seconds.
3. If a router learns a shorter path, it updates its table.
4. If updates stop arriving, the route becomes invalid after **180 seconds**.
5. The route is removed after **240 seconds** if it is not refreshed.

RIP features

- Periodic updates every 30 seconds.
- Shares the **full routing table** in updates.
- Uses **routing by rumor**, meaning routers trust information from neighbors.
- Uses loop-reduction methods like **split horizon** and **route poisoning**.
- Works with a simple hop-count metric.

RIP versions

Version	IP type	Update method	Subnet mask support	Authentication
RIPv1	IPv4	Broadcast to 255.255.255.255	No, classful only	No
RIPv2	IPv4	Multicast to 224.0.0.9	Yes, classless	Yes
RIPng	IPv6	Multicast to FF02::9	Yes, classless	Supports IPv6 security mechanisms; not the same as classic RIPv2 auth

RIP timers

RIP uses timers to control route freshness and stability. The default values are:

- **Update timer:** 30 seconds.
- **Invalid timer:** 180 seconds.
- **Hold-down timer:** 180 seconds.
- **Flush timer:** 240 seconds.

Timer meaning

- **Update timer:** when the next routing update is sent.
- **Invalid timer:** when a route is marked unreachable.
- **Hold-down timer:** prevents unstable routing changes from being accepted too quickly.
- **Flush timer:** removes the route from the table completely.

RIP advantages

- Very easy to configure.
- Low CPU and memory usage.
- Simple to understand and troubleshoot.
- Works well in small networks.
- Can support basic equal-cost load balancing in some cases.

RIP disadvantages

- Limited to **15 hops**, so it does not scale well.
- Slow convergence after failures.

- Periodic full-table updates waste bandwidth.
- Weak for modern enterprise needs like large topologies and advanced path selection.

Security concerns

RIP is weak in security because updates are easy to intercept and manipulate if authentication is not used. Problems include route spoofing, eavesdropping, and false updates that can trigger instability or denial of service.

Why RIP is considered old

RIP is often called "dead" in modern networks because it:

- uses only hop count, not bandwidth-aware metrics,
- sends updates frequently,
- supports only 15 hops,
- converges slowly,
- is less efficient than OSPF or EIGRP for real-world networks.

11) Link-State Routing

Link-state routing protocols give each router a much more complete view of the network. Routers share information about their links, form a topology database, and then run **Dijkstra's SPF algorithm** to calculate the best path. OSPF and IS-IS are the common examples.

Features

- Routers use **hello packets** to discover and maintain neighbors.
- Topology changes are sent as **triggered updates** rather than fixed periodic full-table advertisements.
- Each router keeps a copy of the network topology in a database.
- OSPF stores and uses **LSAs** (Link-State Advertisements) to build the topology view.

Tables used in link-state

Table	Purpose
Neighbor table	Stores directly adjacent routers and adjacency status
Topology table / LSDB	Stores the full network map and all learned link information
Routing table	Stores the best routes selected from the SPF calculation

Advantages

- Fast convergence after changes.
- More accurate because routers know the overall topology.
- Scales better than distance vector in larger networks.
- Avoids persistent routing loops more effectively.

Disadvantages

- Uses more memory and CPU because of topology calculations.
- Configuration and troubleshooting are more complex.
- Flooding link-state information can use more bandwidth.

OSPF overview

OSPF (Open Shortest Path First) is a **link-state routing protocol** used inside an autonomous system. Each router independently runs OSPF, learns the topology, and calculates the best routes based on **cost**, not hop count.

What OSPF does

- Discovers neighbors with Hello packets.
- Builds a link-state database (LSDB).
- Runs SPF (Dijkstra) to select shortest paths.
- Uses areas to reduce flooding and improve scalability.

Key OSPF terms

Term	Meaning
Router ID	A unique 32-bit number used to identify an OSPF router.
Hello packet	A packet used to find neighbors and keep the OSPF relationship alive.
Area ID	A number that identifies which OSPF area an interface belongs to. Routers in the same area exchange OSPF information with each other.
DR	Designated Router. It reduces the number of OSPF adjacencies on multi-access networks.
BDR	Backup Designated Router. It takes over if the DR fails.
LSA	Link-State Advertisement. This is the message OSPF uses to share network topology details.
LSDB	Link-State Database. This is the table where a router stores all received LSAs.

Router ID

The Router ID is like the **name tag** of an OSPF router. It must be unique inside the OSPF network.

How it is chosen

1. If the router-id is manually configured, OSPF uses that first.
2. If not, OSPF uses the highest loopback IP address.
3. If there is no loopback, it uses the highest active interface IP address.

Why it matters

- It identifies the router in OSPF.
- It is used in DR/BDR election when priorities are equal.
- If no valid Router ID exists, OSPF may not start properly.

Hello packet

Hello packets are OSPF's way of saying, "Are you there?" They are sent regularly to discover neighbors and maintain the relationship between routers.

Main job of Hello packets

- Discover OSPF neighbors.
- Keep the neighbor relationship alive.
- Help in DR/BDR election on broadcast and multi-access networks.

Important idea

If a router stops receiving Hello packets for the dead interval, it assumes the neighbor is down.

OSPF broadcast and multicast addresses

OSPF does not use broadcast to send updates. Instead, it uses **multicast** so only OSPF-enabled routers receive the packets, not every device on the network.

Address	Who receives it	Used for
224.0.0.5	All OSPF routers	General Hello packets and LSU flooding — every OSPF router listens on this address
224.0.0.6	DR and BDR only	DROTHERs send updates to this address so only the DR and BDR process them

How it works on a LAN segment

- All OSPF routers send Hello packets to **224.0.0.5** to discover neighbors.
- When a DROTHER needs to share a route update, it sends it to **224.0.0.6** (DR/BDR only).
- The DR then re-floods the update to **224.0.0.5** so all routers on the segment receive it.

This two-step process reduces unnecessary processing on routers that are not the DR or BDR.

Why not broadcast?

- Broadcast goes to **every device** on the segment, including non-routers like PCs and switches.
- Multicast reaches only devices that have joined that multicast group — so only OSPF routers are involved.

Area ID

The Area ID tells OSPF which **group** or **section** of the network an interface belongs to. Routers with the same area ID can exchange detailed OSPF information directly.

Why areas are used

- They divide a large network into smaller parts.
- They reduce flooding and make OSPF easier to scale.
- They keep LSDB size smaller inside each area.

Important note

- Area 0 is the backbone area.
- In general, other areas must connect to area 0 somehow.

DR and BDR in OSPF

On **broadcast** and **multi-access** networks (like Ethernet segments), OSPF elects two special routers to reduce traffic:

- **DR (Designated Router)** → the main router for distributing routing information on that segment.
- **BDR (Backup Designated Router)** → the backup; it becomes DR if the original DR fails.

Instead of every router forming full adjacency with every other router, all other routers (called **DROTHER**) form adjacencies mainly with the DR and BDR. This reduces the number of adjacencies and LSA flooding, which saves bandwidth and CPU.

How DR/BDR election works

OSPF uses two main things to decide which router becomes DR and which becomes BDR:

1. **OSPF priority (per interface)**
2. **Router ID (RID)**

Election rules

1. **Highest priority wins as DR**

- Each router has an **OSPF priority** on the interface, from `0` to `255`.
- The router with the **highest priority** becomes **DR**.
- The router with the **second-highest priority** becomes **BDR**.

2. If priorities are equal, highest Router ID wins

- If two routers have the **same priority**, the router with the **highest Router ID** wins the DR role.
- The router with the next-highest Router ID becomes BDR.

3. Priority 0 → not eligible

- A router with `priority 0` cannot become DR or BDR.
- It will only form adjacencies as a **DROTHER**.

4. Default priority = 1

- If nobody configures priority, every router has **priority 1** by default.
- In that case, DR/BDR is chosen purely by **Router ID**.

OSPF network types

OSPF behaves differently depending on the **network type** of an interface. The network type controls two things: whether a **DR/BDR is elected**, and how **neighbors are discovered**.

Network Type	DR/BDR Elected?	Neighbor Discovery	Typical Use
Broadcast	Yes	Automatic (multicast)	Ethernet (LAN)
Point-to-Point	No	Automatic (multicast)	Serial links, PPP, HDLC
NBMA	Yes	Manual (unicast)	Frame Relay, ATM
Point-to-Multipoint	No	Automatic (multicast)	Frame Relay hub-and-spoke

Broadcast

- The most common type in modern networks — used on **Ethernet**.
- OSPF automatically discovers neighbors using **multicast** (224.0.0.5 / 224.0.0.6).
- A **DR and BDR are elected** to reduce the number of adjacencies on the segment.
- All other routers on the segment are called **DROTHERs** and only form full adjacency with the DR and BDR.

Point-to-Point

- Used on links with only **two routers** (e.g., serial links, PPP, HDLC WAN connections).
- No DR/BDR election** — not needed because there are only two endpoints on the link.
- The two routers form a **direct full adjacency** with each other automatically.
- Simple and fast — this is the preferred type for serial WAN links.

NBMA (Non-Broadcast Multi-Access)

- Used on older WAN technologies like **Frame Relay or ATM**, where multiple routers share the same network but the medium **does not support native broadcast**.
- DR/BDR is still elected** (same logic as broadcast), but neighbors must be **manually configured** because multicast does not work natively here.
- Requires `neighbor` statements under the OSPF process to tell OSPF where to send updates.

Point-to-Multipoint

- Also common on **hub-and-spoke WAN** designs (Frame Relay).
- No DR/BDR election** — OSPF treats each connection to a spoke as a separate point-to-point link.

- Neighbors are discovered **automatically** where multicast is supported, or can be configured manually.
- Simpler to set up than NBMA because it avoids the DR/BDR complexity.

Quick rule to remember

- **DR/BDR elected** → Broadcast, NBMA
- **No DR/BDR** → Point-to-Point, Point-to-Multipoint

LSA

LSA stands for **Link-State Advertisement**. It is the message OSPF uses to share information about links, routers, and network changes.

What an LSA contains

- Router information.
- Link/interface information.
- Cost and connectivity details.
- Topology changes.

Why it matters

- LSAs are how OSPF learns the full network topology.
- When something changes, new LSAs are flooded so all routers can update their view.

LSDB

LSDB means **Link-State Database**. It is the collection of all LSAs a router knows about.

Simple meaning

Think of the LSDB as OSPF's **network map**. Every router in the same area keeps a similar map so they can calculate the best route.

Why it is important

- OSPF uses the LSDB as input for SPF calculation.
- A correct LSDB is needed for correct route calculation.
- If LSDBs do not match, routing problems can happen.

OSPF neighbor states

When two routers start talking OSPF, they do not become fully connected immediately. They move through a set of states step by step until they finally reach **Full**, which means their databases are synchronized.

State	Meaning
Down	No Hello packet has been received yet. OSPF has not started forming a relationship.
Init	A Hello packet was received, but the router has not yet seen its own Router ID in the neighbor's Hello packet.
Two-Way	Both routers can see each other in Hello packets, so bidirectional communication is confirmed.
ExStart	The routers decide who will be master and who will be slave for database exchange.
Exchange	Routers exchange DBD packets, which contain summaries of their LSDB contents.
Loading	Routers request missing LSAs using LSR packets and receive them using LSU packets.
Full	The LSDBs are synchronized. At this point, the OSPF adjacency is complete.

What the states really mean

You can think of the OSPF process like two routers getting to know each other.

- **Down** means they have not started talking yet.
- **Init** means one router has heard the other, but the connection is not confirmed both ways.
- **Two-Way** means both routers recognize each other.
- **ExStart** and **Exchange** are about preparing and sharing database summaries.
- **Loading** is when missing information is requested.
- **Full** means both routers now have the same view of the network.

Neighbor vs adjacency

A **neighbor** is just a router that has been discovered by Hello packets. It means the routers know each other exists.

An **adjacency** is a stronger relationship. It means the routers have gone beyond discovery and have fully exchanged and synchronized their LSDBs.

Simple example

- If Router A and Router B send Hello packets to each other, they are **neighbors**.
- If they also exchange database information and finish synchronizing, they are **adjacent**.

Why not every neighbor becomes adjacent

On broadcast networks like Ethernet, OSPF uses the **DR/BDR** mechanism to reduce overhead. That means routers do not always form full adjacencies with every other router on the same network segment. Instead, many routers only fully adjacency with the DR and BDR, which keeps the number of relationships lower and makes the network more efficient.

Why adjacency can fail

OSPF may stop before Full if some values do not match. Common problems include:

- Area ID mismatch.
- Hello/dead timer mismatch.
- Authentication mismatch.
- MTU mismatch.
- Wrong network type in some designs.

Easy memory trick

- **Neighbor** = "I know you."
- **Adjacency** = "I know you, and I share my routing database with you."
- **Full** = "We are completely synchronized."

OSPF message types

OSPF uses **5 types of packets** to discover neighbors, synchronize databases, and update topology.

1. Hello

- **Purpose:** Discover and maintain OSPF neighbors.
- **What it does:**
 - Says "I am here" (neighbor discovery).
 - Keeps the neighbor relationship alive (like a heartbeat).
 - Helps in **DR/BDR election** on broadcast networks.

- **Used in:** Down → Init → Two-Way states.

2. DBD (Database Description)

- **Purpose:** Exchange a **summary** of the LSDB (not the full routes).
- **What it does:**
 - Each router shows what LSAs it knows (like a table of contents).
 - Routers compare DBD packets to see **which LSAs are missing**.
 - Used in **ExStart** and **Exchange** states.

3. LSR (Link-State Request)

- **Purpose:** Ask for **specific missing LSAs**.
- **What it does:**
 - After comparing DBDs, a router says, "You have something I don't; send me those LSAs."
 - Used in the **Loading** state.

4. LSU (Link-State Update)

- **Purpose:** Send the **actual LSAs** that were requested.
- **What it does:**
 - Contains full **Link-State Advertisements** (routing info).
 - Can be used to answer an LSR or to flood new topology changes.
 - Sent in **Loading** state for sync, and anytime an LSA is updated (flooding).

5. LSAck (Link-State Acknowledgment)

- **Purpose:** Confirm that LSU packets were received.
- **What it does:**
 - Each router sends an LSAck to say "Yes, I got your LSU and its LSAs."
 - This makes OSPF **reliable** (if no LSAck, the LSU is re-sent).

OSPF cost

OSPF uses **cost** as its only metric to decide which path is the best. Lower cost is always preferred, and by default, cost is based on **interface bandwidth**.

What "cost" means

- **Cost = how "expensive" a link is to use.**
- Low-bandwidth links have **high cost** (not preferred).
- High-bandwidth links have **low cost** (preferred).

For example, OSPF should prefer a 100 Mbps link over a 10 Mbps link for the same path.

The cost formula

$$\text{Cost} = \frac{\text{Reference Bandwidth}}{\text{Interface Bandwidth}}$$

- **Reference Bandwidth** is a fixed value set by the vendor (by default **100 Mbps** on Cisco).
- **Interface Bandwidth** is the actual speed of the link (e.g., 10 Mbps, 100 Mbps, 1 Gbps).

Example values (Cisco default)

Link type	Bandwidth	Formula (100 Mbps / BW)	Cost
10 Mbps Ethernet	10 Mbps	100 / 10	10
100 Mbps FastEthernet	100 Mbps	100 / 100	1
1 Gbps GigabitEthernet	1 Gbps	100 / 1000	1 (since cost is rounded to integer, any value < 1 becomes 1)

How OSPF uses cost

- Each router assigns a **cost to every outgoing OSPF interface** using this formula.
- When choosing a path to a destination, OSPF **adds up all the link costs** along the path.
- The path with the **lowest total cost** becomes the best route.

Simple example

Suppose Router A has two paths to a server:

- Path 1: A → B → Server, with link costs 10 + 10 = **20**
- Path 2: A → C → Server, with link costs 1 + 1 = **2**

Even if both paths have the same number of hops, OSPF chooses **Path 2** because its **total cost (2)** is lower than cost 20.

LSDB and SPF

Each router stores LSAs in the **LSDB**, and that database becomes the input to the SPF algorithm. SPF does not directly read packets to build the routing table; instead, it calculates the best route tree from the LSDB from the local router's perspective.

Why OSPF areas matter

Areas exist mainly to **reduce load** and **improve convergence**:

- **Limit LSA flooding** to just the area.
- **Reduce RAM** because LSDBs are smaller.
- **Reduce CPU** because SPF runs on smaller LSDBs.
- **Improve convergence** because changes in one area do not force full SPF runs everywhere.

OSPF packet flow (step-by-step)

This is the usual OSPF "conversation" on a segment:

1. **Hello** → Routers send Hello packets.
2. **Two-Way** → Neighborships form (bidirectional communication).
3. **DR/BDR election** → On broadcast/multi-access links, DR and BDR are elected.
4. **ExStart & Exchange** → Routers negotiate master/slave and exchange DBD packets (LSDB summaries).
5. **Loading** → Routers ask for missing LSAs using **LSR**, receive them with **LSU**, and confirm with **LSAck**.
6. **Full** → LSDBs are synchronized; adjacency is complete and SPF can run.

OSPF areas (types explained)

OSPF breaks the network into **areas**. Area 0 is the **backbone**, and everything else connects to it.

1. Backbone area (Area 0)

- The **core** of the OSPF network.
- **All other areas must connect** to Area 0 (directly or via virtual links).

- It carries routing information between non-backbone areas.
- Supports **all LSA types** (1, 2, 3, 4, 5).

2. Standard (normal) area

- A regular OSPF area that is not special.
- **Any LSA type is allowed** (1, 2, 3, 4, 5).
- No special restrictions; it behaves like the classic OSPF design.
- This is the **default** if you don't configure a special area type.

3. Stub area

- A **simplified** area that:
 - **Blocks Type 5 external LSAs** (no external routes from other AS).
 - Instead of all externals, it uses a **default route** toward the backbone.
- Still accepts:
 - Internal LSAs (1, 2)
 - Inter-area LSAs (3).
- Goal: reduce LSDB size and routing table size in edge areas.

4. Totally stubby area (Cisco-style)

- Even stricter than a stub area:
 - **Blocks Type 5 (external) and Type 3 (inter-area) LSAs.**
- The router only knows:
 - Internal routes in the area (Types 1, 2).
 - One **default route** advertised by the ABR.
- This keeps the LSDB and routing table **very small**.

5. NSSA (Not-So-Stubby Area)

- Like a **stub** but more flexible:
 - **Blocks Type 5 external LSAs from outside.**
 - **Allows an ASBR inside the NSSA** to generate **Type 7 LSAs** for external routes.
 - At the ABR, **Type 7 LSAs are translated into Type 5** so the rest of the OSPF domain can see them.
- Use case: an edge area that needs to redistribute external routes without becoming a normal area.

OSPF router types

Backbone Area (Area 0)

- Identifier = **Area 0** (backbone area).
- Acts as the **core** of the OSPF domain.
- **All other areas must connect to Area 0** (directly or via ABRs/virtual links).
- Carries routing traffic between non-backbone areas.

Backbone router

- Any router that has **at least one interface in Area 0**.
- It may also have interfaces in other areas.
- Participates in the backbone routing but does not necessarily connect multiple areas itself.

Area Border Router (ABR)

- A router with **interfaces in multiple areas** (for example, Area 0 + Area 10).
- Connects an **internal area to the backbone**.
- **Summarizes routes** between areas and controls which LSAs are sent where.
- Plays a key role in hierarchical design and route-table size control.

Internal router

- A router whose **all interfaces belong to a single area**.
- Operates only within that area's LSDB.
- Generally **simpler** (no ABR/ASBR features).

Autonomous System Boundary Router (ASBR)

- A router that **connects an OSPF AS to another routing domain** (e.g., another AS, BGP, RIP, EIGRP).
- **Redistributes** external routes into OSPF as **Type 5 LSAs** (or **Type 7 in NSSA**).
- Typically located at the edge of the OSPF network.

OSPF addressing and summarization

- OSPF is **classless** and supports **VLSM** and **CIDR**.
- It allows **route summarization** at ABRs and ASBRs.

How summarization helps

- Instead of advertising each small subnet, ABRs can advertise **a single network ID or summary prefix**.
- This:
 - Reduces the **routing-table size**.
 - Reduces **LSA traffic and LSDB size**.
 - Hides small topology changes from the rest of the domain.

OSPFv2 vs OSPFv3

Feature	OSPFv2	OSPFv3
IP version	IPv4	IPv6
Addressing	IP addresses	Link-local addresses for neighbor relationships
Security	Traditionally plain text or MD5	Uses IPv6 security mechanisms such as IPsec support
Scope	Area and AS scope	Adds more flexible flooding scopes

12) Hybrid Routing

Hybrid routing combines ideas from distance vector and link-state routing. EIGRP is the best-known example in Cisco environments. It learns routes from neighbors like a distance-vector protocol, but it also uses fast convergence techniques, neighbor discovery, and partial updates that make it behave more like a modern advanced protocol.

Features

- Uses **hello packets** for neighbor discovery.
- Sends **partial, triggered updates** when the topology changes.
- Uses the **DUAL algorithm** to find loop-free paths.
- Supports backup paths called **feasible successors**.

EIGRP terms

Term	Meaning
Successor	Best route to the destination, installed in the routing table
Feasible successor	Backup route that can replace the successor immediately if needed
Feasibility condition	Rule used to ensure the backup route is loop-free

Advantages

- Fast convergence.
- Efficient bandwidth use because it does not flood full tables constantly.
- Supports scalable networks.
- Can provide backup routes quickly through feasible successors.

Disadvantages

- More complex than RIP.
- Uses more CPU and memory than distance vector protocols.
- Cisco-focused in practice, so it is less universal than OSPF.

EIGRP – Core idea

EIGRP (Enhanced Interior Gateway Routing Protocol) is a **hybrid routing protocol**: it behaves mostly like an **advanced distance-vector** protocol but borrows features from link-state routing. It runs at **Layer 3 (Network Layer)** and uses **IP protocol 88**.

- Used to discover and maintain **best paths inside an AS**.
- Fast convergence, efficient bandwidth use, and unequal-cost load balancing make it popular in **enterprise networks**.

Administrative Distance in EIGRP

Administrative Distance (AD) tells the router **how much it trusts a route**. Lower AD = more trusted.

Type	AD Value
EIGRP summary routes	5
EIGRP internal routes	90
EIGRP external routes (redistributed)	170

- So, **EIGRP internal routes** are trusted more than most other IGPs (for example, OSPF = 110, RIP = 120).

Key EIGRP messages

EIGRP uses several message types, sent as **multicast (224.0.0.10)** or **unicast**, depending on purpose.

Message	Purpose
Hello	Neighbor discovery and keep-alive. Sent every 5 seconds by default. If no Hello arrives within 15 seconds (Dead/hold time) , the neighbor is declared down.
Update	After adjacency, routers send full updates (all routes). On topology changes, partial updates (only changed routes) are sent.
Query	When a route disappears and there is no backup, EIGRP queries neighbors for a new path (multicast).

Message	Purpose
Reply	Sent in response to a Query , giving alternative route info.
Ack	A Hello-like packet with no data , used to acknowledge Updates, Queries, and Replies (these are reliable messages).
NULL Update	Used internally for measuring SRTT (Smooth Round Trip Time) and RTO (retransmission timeout) .

Reliability note:

- **Hello and Ack** are **not** reliable (no separate acknowledgment).
- **Update, Query, Reply** are **reliable** and must be acknowledged.

EIGRP broadcast and multicast

EIGRP does **not** use broadcast. It uses **multicast** for most messages, and switches to **unicast** in specific situations.

Address / Type	Used for
224.0.0.10 (multicast)	Hello packets, Updates, Queries sent to all EIGRP neighbors on the segment
Unicast	Retransmissions of reliable messages if a neighbor does not acknowledge; also used on NBMA links where multicast is not supported

Why multicast instead of broadcast?

- **Broadcast** goes to every device on the network — including PCs, printers, and switches that do not run EIGRP.
- **Multicast 224.0.0.10** only reaches routers that have joined the EIGRP multicast group, which saves processing on non-EIGRP devices.

When EIGRP falls back to unicast

- If a neighbor does not **acknowledge** a reliable message (Update, Query, Reply), EIGRP retransmits it as **unicast** directly to that neighbor.
- On **NBMA links** (e.g., Frame Relay) where multicast may not work, EIGRP uses unicast with manually configured neighbor statements.

Composite metric (EIGRP metric)

EIGRP chooses the best path using a **composite metric** built from **five K-values**. By default, only **K₁ = Bandwidth** and **K₃ = Delay** are used.

K-value meanings

- **K₁ = Bandwidth**
- **K₂ = Load**
- **K₃ = Delay**
- **K₄ = Reliability**
- **K₅ = MTU**

Default K-values

$$K_1 = 1, \quad K_2 = 0, \quad K_3 = 1, \quad K_4 = 0, \quad K_5 = 0$$

So **only bandwidth and delay contribute** by default. Lower metric = better route.

Classic metric mindset

- **Bandwidth:** lower bandwidth → higher metric.
- **Delay:** higher delay → higher metric.

EIGRP's metric can be **scaled** so large bandwidths (10 Gbps, 100 Gbps) are still distinguished.

Neighbor adjacency requirements

For two routers to become **EIGRP neighbors**, these must match:

- **K-values** (all 5) on the link.
- **AS number** (same value in `router eigrp <AS>`).
- **Subnet mask / network** (same Layer-3 segment).
- **Authentication** (if enabled; EIGRP supports **MD5** only).

If any of these differ, the routers **will not form a neighbor relationship**, even if physically connected.

EIGRP timers

Timer	LAN (Ethernet) Default	WAN / Slow-link Default	Purpose
Hello Timer	5 seconds	60 seconds	Interval between Hello packets.
Dead / Hold Time	15 seconds	180 seconds	If no Hello is received for this time, the neighbor is declared dead.

Important: The 5s/15s values are the defaults on **LAN (Ethernet)** interfaces. On **WAN or slow-speed links** (T1 speed and below), EIGRP defaults to **60 seconds Hello / 180 seconds Hold**. Hello and Hold timers **must match** between neighbors — a mismatch will prevent the neighbor relationship from forming.

The LAN timers allow **fast failure detection** on high-speed links. The longer WAN timers reduce Hello overhead on slower, more sensitive connections.

Pros of EIGRP

- **Fast convergence** (via **DUAL algorithm** and feasible successors).
- **VLSM and CIDR support** for efficient IP design.
- **Partial updates** reduce bandwidth usage vs full-table distance-vector protocols like RIP.
- **Scalable to large enterprise networks**.
- **Unequal-cost load balancing** (via `variance` command).

DUAL (Diffusing Update Algorithm)

DUAL is the **core of EIGRP's loop-free convergence**. It uses:

- **Topology table** (all routes and their metrics).
- **Feasibility condition** to choose **loop-free backup routes (feasible successors)**.

Key distance terms

Before understanding DUAL, you need to know two distance values EIGRP uses:

Term	Short form	Meaning
Reported Distance	RD	The metric a neighbor reports for reaching the destination — it is the neighbor's own cost to get there. Also called Advertised Distance (AD).
Feasible Distance	FD	The total metric from the local router to the destination through a specific neighbor. It equals the link cost to the neighbor plus the neighbor's RD.

Simple way to remember:

- **RD** = what your neighbor tells you its cost is.
- **FD** = your total cost = (your link to neighbor) + (neighbor's RD).

Feasibility condition

A backup path (feasible successor) is loop-free if:

RD of the backup path < FD of the current best path

This rule guarantees the backup neighbor is closer to the destination than you are, so it cannot be routing traffic back through you — meaning no loop is possible.

Key ideas

- **Successor** = the current best path (in routing table). Has the lowest FD.
- **Feasible successor** = a loop-free backup path ready to take over **immediately**. Its RD must be less than the successor's FD.
- If no feasible successor exists, EIGRP runs a **diffusing computation** (queries and replies) to find a new route; this usually takes a few seconds.

Example

Suppose Router A has two paths to a destination:

- Path via B: FD = 100, RD reported by B = 60
- Path via C: FD = 120, RD reported by C = 80

Path via B is the **successor** (lowest FD = 100). Path via C: is C's RD (80) < successor's FD (100)? Yes → C is a **feasible successor** (loop-free backup).

This is why EIGRP is often said to support an **"immediate backup"**, unlike OSPF.

EIGRP tables

Table	Purpose
Neighbor table	Stores EIGRP neighbors and their metrics (like RTT, SRTT, RTO).
Topology table	Stores all routes advertised by neighbors, including feasible successors.
Routing table (RIB)	Contains the best routes (successors) selected by DUAL.

OSPF style: **LSDB** → **SPF** → **routing table**. EIGRP style: **Topology table** → **DUAL** → **routing table**.

Auto-summarization and bandwidth management

Auto-summarization

By default, EIGRP **auto-summarizes** routes at **classful boundaries**.

- Example: 192.168.1.0/24 and 192.168.2.0/24 may be summarized to **192.168.0.0/16**.
- This can cause **incorrect routing** if the subnets are in different parts of the network.

To disable it (modern best practice):

```
router eigrp 10
no auto-summary
```

BASH

Bandwidth management

EIGRP limits routing traffic to a percentage of the interface bandwidth (default **50%**).

- Prevents routing updates from **flooding low-bandwidth links**.
- On high-speed links, this can be too conservative, so you can adjust it and tune **bandwidth** and **delay** metrics manually.

PDM (Protocol Dependent Module)

EIGRP is designed to support **multiple network-layer protocols**, not just IP. This is possible because of its modular design — each supported protocol gets its own **Protocol Dependent Module (PDM)**.

What PDM does

- Each PDM handles the routing logic for **one specific protocol** (e.g., IPv4, IPv6, IPX).
- The DUAL algorithm and the core EIGRP engine remain the same — only the PDM layer changes depending on the protocol being routed.
- Think of it like a **compartment** or plug-in: one compartment for IPv4, another for IPv6, and so on.

Why it matters

- EIGRP can maintain **separate neighbor tables, topology tables, and routing tables** for each protocol independently.
- A problem in one protocol's routing does not affect the others.
- In modern networks, the most relevant PDMs are **IPv4** and **IPv6 (EIGRPv6)**.

Simple way to think about it

DUAL is the engine. PDM is the gear selector — it tells the engine which protocol's routes to process.

Basic EIGRP commands (Cisco)

```
R(config)# router eigrp 10
R(config-router)# network 192.168.1.0 0.0.0.255
R(config-router)# network 10.0.0.0
```

BASH

- **AS number (10)** must be the same on all routers in the EIGRP domain.

Common verification commands

```
show ip eigrp neighbors      → EIGRP neighbor table
show ip eigrp topology      → Topology table (includes feasible successors)
show ip route eigrp         → EIGRP routes in routing table
```

BASH

13) Loop Prevention Techniques

A **routing loop** happens when a packet goes in circles between routers instead of reaching the destination.

- Wastes **bandwidth**.
- Can cause **black-holes** and slow convergence.
- Common in **distance-vector** protocols (like RIP, EIGRP) if loop-prevention rules are missing.

The goal of all these techniques is: **Stop packets from looping endlessly** and **prevent bad routing info from spreading**.

1. TTL (Time To Live)

- Every IP packet has a **TTL** field (or **Hop-Limit** in IPv6).
- Every time the packet passes a router, **TTL = TTL - 1**.
- When **TTL = 0**, the packet is **dropped** and not forwarded.

Purpose

- **Last-resort safety net**: even if a loop exists, the packet dies after a limited number of hops.
- Does **not fix** the routing bug; it just limits the damage.

2. Split Horizon

In distance-vector protocols, a router **does not advertise a route back out the same interface** it learned it on.

Example:

- Router A tells B about network X.
- Router B will **not** tell A again about X.

Why it helps

- Prevents **back-loops** between two neighboring routers.
- Basic rule in **RIP**, **EIGRP**, and similar protocols.

3. Route Poisoning

When a route fails (link goes down), the router **advertises it with "infinite" metric**, i.e., unreachable.

Example:

- Network X goes down.
- Router sends: "Distance to X = ∞ " (16 for RIP).
- Neighbors immediately mark it as **unreachable**.

Purpose

- Faster convergence than waiting for hop-count to slowly increase.
- Prevents **count-to-infinity** and routing loops.

4. Poison Reverse

A **stronger version** of Split Horizon.

- Instead of just **not advertising** the route, the router **explicitly advertises it as unreachable** back to the neighbor.

Example:

- Router A tells B about X.
- When X goes down, B **replies to A** with: "Distance to X = ∞ ".

Why it helps

- Neighbor knows **for sure** this path is bad, not just silent.
- More aggressive at preventing loops; used in **RIP with poison-reverse** and EIGRP.

5. Hold-down Timers

When a route disappears, routers **temporarily ignore** new updates for that destination unless the new route comes from the same source or is clearly better.

What it prevents

- **Flapping** (route going up and down).
- Accepting **old/wrong** info from a late-arriving update.

Trade-off

- Slower convergence in some cases, but safer against instability.

6. Triggered (Flash) Updates

Instead of waiting for **periodic updates** every 30 seconds (RIP style), a router **immediately sends an update** when a route changes.

Purpose

- **Faster convergence**: neighbors learn topology changes quickly.
- Reduces chance that others still use an old path, which could cause loops.

7. Sequence Numbers

Some protocols attach a **version number** to each route or message.

- Neighbors accept only **newer** sequence numbers.
- Older numbers are discarded.

Loop-prevention benefit

- Prevents **stale** information from being used after a failure or change.
- Avoids using old routes that might have created loops.

8. Link-State Routing (OSPF style)

In **link-state protocols (OSPF, IS-IS)**:

- Each router builds a **complete map (topology)** of the network using **LSAs**.
- All routers run **Dijkstra's SPF algorithm** on the same topology to find the best path.

Why loops are less likely

- Routers do **not guess** routes from neighbors alone.
- Every router independently computes **loop-free shortest paths** from the shared map.

9. Path-Vector (BGP style)

In **BGP**, each route carries a **full path list** (list of AS numbers or routers).

- When a router sees **its own ID** (AS number) in the path, it **rejects** that route.

Example

- Router R1 advertises a route via AS1 → AS2 → AS3.
- If AS2 tries to send it back to AS1, AS1 sees **its own AS** in the path and **drops** it.

Purpose

- Straight-forward **loop detection**: no router can import a path that includes itself.

Quick conceptual summary

Technique	Main idea	Where it's used
TTL	Eventually kill looping packets	All IP networks
Split Horizon	Do not advertise back the way you learned it	Distance-vector (RIP, EIGRP)
Route Poisoning	Mark failed routes as unreachable immediately	Distance-vector
Poison Reverse	Tell neighbor "this route is dead"	Distance-vector
Hold-down Timer	Wait before accepting new updates for a failed route	Distance-vector
Triggered Updates	Send updates immediately on change	Distance-vector
Sequence Numbers	Use version numbers so only newer routes are accepted	Some protocols
Link-State (OSPF)	Build global map and use Dijkstra for loop-free paths	OSPF, IS-IS
Path-Vector (BGP)	Check path; if it includes me → reject	BGP

Simple way to remember

- Distance-vector** → Prevent loops with **rules and timers** (split horizon, poisoning, hold-down, triggers).
- Link-state** → Avoid loops with **complete topology + Dijkstra**.
- BGP** → Detect loops with **path history** (AS-PATH / router ID list).

14) Load Balancing

Load balancing in OSPF

How it works

- OSPF uses **cost** as its metric.
- If a router has **two or more routes with exactly the same total cost** to a destination, it can install **multiple next-hops** in the routing table.
- Traffic is then **shared across these equal-cost paths** (usually per-flow or per-packet, depending on platform/CEF).

This is called **ECMP – Equal-Cost Multi-Path** routing.

Limits

- On Cisco IOS, by default, OSPF usually uses up to **4 equal-cost paths**.

- You can change this with the `maximum-paths` command under the OSPF process, up to higher values (often up to 16 or 32, depending on platform/IOS).

Example:

```
router ospf 1
maximum-paths 4
```

BASH

- OSPF does **not** support unequal-cost load balancing in classic implementations: Only routes with **identical cost** are used together.

Load balancing in EIGRP

EIGRP is more flexible: it supports **equal-cost** and **unequal-cost** load balancing.

1. Equal-cost load balancing (default)

- If EIGRP learns multiple routes to a destination with the **same composite metric**, it can install multiple next-hops and share traffic across them.
- By default, Cisco routers use up to **4 equal-cost paths**, controlled by `maximum-paths`.

Example:

```
router eigrp 100
maximum-paths 4
```

BASH

! Can be raised up to 16 or 32 depending on platform

2. Unequal-cost load balancing (variance)

This is the big EIGRP advantage over OSPF.

- EIGRP can also use **paths with different metrics** if they are "**good enough**" compared to the best path.
- This is controlled by the `variance` command (a multiplier).

Logic (simplified):

- EIGRP finds the **best path** (successor) with metric = **M**.
- You configure `variance n`.
- Any **feasible successor** (loop-free backup) with metric $\leq n \times M$ can also be used for forwarding.

Example:

```
router eigrp 100
variance 2
maximum-paths 4
```

BASH

- Here, EIGRP will use:
 - The best path (metric M), and
 - Any feasible successors with metric $\leq 2 \times M$, for **unequal-cost load sharing**.

Important:

- Only **feasible successors** are used for unequal-cost load balancing (must pass feasibility condition, so they are loop-free).

Traffic-share ratios

When EIGRP does unequal-cost load balancing, it does **not** split traffic equally. Instead, it sends **more traffic through the better path** and less through the worse one, in proportion to their metrics.

The traffic share is calculated as:

Traffic share $\propto 1 / \text{metric}$ — the path with the lower metric carries more traffic.

Example:

Suppose three paths have metrics of 100, 200, and 500:

Path	Metric	Inverse (1/metric)	Traffic share (simplified)
Path A (successor)	100	1/100 = 10	10 parts
Path B (feasible successor)	200	1/200 = 5	5 parts
Path C (feasible successor)	500	1/500 = 2	2 parts

So the ratio is **10 : 5 : 2** — Path A carries roughly 10 packets for every 5 sent via Path B and 2 via Path C.

This is called **proportional load sharing** — faster/better paths carry more of the load automatically.

Quick comparison

Feature	OSPF	EIGRP
Metric	Cost (bandwidth-based)	Composite (bandwidth, delay, etc.)
Equal-cost load balancing	Yes (ECMP, <code>maximum-paths</code>)	Yes (<code>maximum-paths</code>)
Unequal-cost load balancing	No (classic OSPF)	Yes (<code>variance</code> with feasible successors)

Mental shortcut:

- **OSPF** → "Equal only" (ECMP over same-cost paths).
- **EIGRP** → "Equal + Unequal" (tune with `maximum-paths` and `variance`).

15) RIP vs OSPF vs EIGRP — Quick Comparison

Feature	RIP	OSPF	EIGRP
Protocol type	Distance vector	Link-state	Hybrid (advanced distance vector)
Metric	Hop count (max 15)	Cost (based on bandwidth)	Composite (bandwidth + delay by default)
Administrative Distance	120	110	90 (internal) / 170 (external)
Convergence speed	Slow	Fast	Very fast (DUAL + feasible successors)
Max hops / scale	15 hops hard limit; not suitable for large networks	No hop limit; scales well using areas	No hop limit; scales to large enterprise networks
Authentication	RIPv2 supports MD5; RIPv1 has none	MD5 (OSPFv2); OSPFv3 uses IPsec	MD5
Update type	Periodic full-table every 30 seconds	Triggered partial updates on change	Triggered partial updates on change
Open standard	Yes (RFC 1058 / RFC 2453)	Yes (RFC 2328)	Cisco proprietary (later published as RFC 7868)

Feature	RIP	OSPF	EIGRP
Load balancing	Equal-cost only	Equal-cost only (ECMP)	Equal-cost and unequal-cost (via <code>variance</code>)

Simple memory shortcut

- **RIP** → Simple, slow, small networks only. Hop count is the only metric. Good for learning; rarely used in production today.
- **OSPF** → Open standard, cost-based metric, scales well with areas. The most common choice in real-world networks.
- **EIGRP** → Cisco-focused, fastest convergence via DUAL, flexible load balancing with `variance` . Best in Cisco-only environments.

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [01A - OSI Model](#)
- [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)
- [10 - PPP and WAN Technologies](#)
- [13 - Router IOS and Management](#)
- [04B - Routing Protocols and Administrative Distance](#)
- [05 - VLANs and Inter-VLAN Routing](#)
- [09 - Infrastructure Security ACL AAA and Port Security](#)

Source note: 04A - Network Routing Fundamentals.md

CHAPTER 04B · ROUTING & NETWORK SERVICES



Module 4: Routing — Complete Notes

Static and dynamic routing with RIP, OSPF, EIGRP, BGP, and administrative distance.

Topic 4

computer networks

routing protocols

administrative distance

cisco

Table of Contents

- 4.1 Static Routing vs Dynamic Routing
- 4.2 Distance Vector vs Link State (RIP vs OSPF)
- 4.3 Administrative Distance (AD) Concept
- 4.4 IGP vs EGP
- 4.5 RIP (Routing Information Protocol)
- 4.6 IGRP (Interior Gateway Routing Protocol) — Cisco Legacy
- 4.7 OSPF (Open Shortest Path First)
- 4.8 EIGRP (Enhanced Interior Gateway Routing Protocol)
- 4.9 BGP (Border Gateway Protocol)
- 4.10 🎯 Master Comparison Table — RIP vs OSPF vs EIGRP vs BGP
- ⭐ Quick Revision — All Protocols Ultra-Short
- 🎯 Interview-Level Key Differences
- Related Notes

4.1 Static Routing vs Dynamic Routing

◆ Static Routing

- Routes are **manually configured** by the network administrator.
- Router does **not** automatically adapt to topology changes.
- No routing protocol overhead (no CPU/bandwidth used for updates).

Command (Cisco):

```
ip route <destination-network> <subnet-mask> <next-hop-ip>
```

Advantages:

- No CPU overhead — no calculations
- No bandwidth used between routers (no updates exchanged)
- More secure (no route advertisement to intercept)
- Predictable — admin has full control

Disadvantages:

- Not scalable for large networks
- Manual reconfiguration needed on topology change
- Prone to human error
- No automatic failover

◆ Dynamic Routing

- Routers **automatically learn** and update routes using routing protocols.
- Adapts to topology changes (link failure, new network) automatically.

Advantages:

- Scalable for large/complex networks
- Automatic failover / self-healing
- Less administrative overhead

Disadvantages:

- Consumes CPU, memory, and bandwidth
- More complex to configure/troubleshoot initially
- Slight security risk (routes advertised, can be spoofed without auth)

◆ Comparison Table

Parameter	Static Routing	Dynamic Routing
Configuration	Manual	Automatic
Scalability	Poor (small networks)	Good (large networks)
CPU/Bandwidth Usage	None	Used for updates
Adapts to Failure	No	Yes (auto convergence)
Security	More secure	Less secure (unless auth)
Administrative Effort	High (manual updates)	Low (self-managing)
Best For	Small/stub networks	Large/enterprise networks

✓ **Exam Trap:** Static routes have a default **Administrative Distance (AD) = 1** (better/more trusted than any dynamic protocol, so static always wins if configured for same destination).

4.2 Distance Vector vs Link State (RIP vs OSPF)

◆ Distance Vector Routing (e.g., RIP)

- Each router shares its **entire routing table** with directly connected neighbors only.
- **"Routing by rumor"** — router trusts what neighbor says without seeing full topology.
- Uses **Bellman-Ford Algorithm**.
- Metric = **Hop Count**.
- Slower convergence, prone to routing loops (needs Split Horizon, Hold-down timers).

◆ Link State Routing (e.g., OSPF)

- Each router builds a **complete map (topology) of the network** using Link State Advertisements (LSAs).
- Every router has **identical view** of the network (LSDB — Link State Database).
- Uses **Dijkstra's Algorithm (SPF – Shortest Path First)**.
- Metric = **Cost** (based on bandwidth).
- Faster convergence, more CPU/memory intensive.

◆ Comparison Table

Parameter	Distance Vector (RIP)	Link State (OSPF)
Info Shared	Entire routing table	Only link-state info (LSAs)
Shared With	Directly connected neighbors	All routers in area (flooded)
Algorithm	Bellman-Ford	Dijkstra (SPF)
Network View	Partial ("routing by rumor")	Full topology map
Convergence	Slow	Fast
Metric	Hop Count	Cost (bandwidth-based)
Loop Prevention	Split Horizon, Hold-down	Not needed (full topology known)
Resource Usage	Low	High (CPU/Memory)
Scalability	Small networks only	Large networks

4.3 Administrative Distance (AD) Concept

◆ What is AD?

- AD = a value that represents the **trustworthiness/reliability** of a routing information source.
 - Range: **0 – 255**
 - **Lower AD = More trusted = More preferred**
 - Used when a router learns the **same destination network** from **more than one routing protocol/source** — the router picks the route from the source with the **lowest AD**.
- ✦ AD is used **before** the metric. Metric is only compared **within the same protocol**.

◆ Default AD Table (Very Important — Memorize!)

Route Source	Administrative Distance
Connected Interface	0
Static Route	1
EIGRP (Summary Route)	5
eBGP (External BGP)	20
EIGRP (Internal)	90
OSPF	110
IS-IS	115
RIP	120
EIGRP (External)	170
iBGP (Internal BGP)	200
Unknown / Unreachable	255 (never used)

✓ **Memory Trick: 0 (Connected) → 1 (Static) → 5 (EIGRP Summary) → 20 (eBGP) → 90 (EIGRP) → 110 (OSPF) → 115 (IS-IS) → 120 (RIP) → 170 (EIGRP Ext) → 200 (iBGP)**

◆ Example Scenario (Common Exam/Viva Question)

If a router learns route to 192.168.1.0/24 via both OSPF (AD 110) and RIP (AD 120), which will it install in the routing table?

Answer: OSPF — because it has the **lower AD (110 < 120)**, regardless of hop count or cost.

4.4 IGP vs EGP

◆ IGP (Interior Gateway Protocol)

- Used for routing **within a single Autonomous System (AS)**.
- Examples: **RIP, IGRP, EIGRP, OSPF, IS-IS**

◆ EGP (Exterior Gateway Protocol)

- Used for routing **between different Autonomous Systems**.
- Example: **BGP** (the only EGP in practical/modern use)

◆ Comparison Table

Parameter	IGP	EGP
Full Form	Interior Gateway Protocol	Exterior Gateway Protocol
Scope	Within one AS	Between different AS
Examples	RIP, IGRP, EIGRP, OSPF, IS-IS	BGP
Algorithm Type	Distance Vector / Link State / Hybrid	Path Vector
Convergence	Fast to Medium	Slow (designed for stability)
Used By	Enterprises, campus networks	ISPs, Internet backbone
Metric	Hop count / Cost / Composite	Path attributes (AS-Path, etc.)

✦ **What is an Autonomous System (AS)?** A group of networks/routers under a **single administrative control**, identified by a unique **AS Number (ASN)**, e.g., an ISP or large organization.

4.5 RIP (Routing Information Protocol)

◆ RIP Basics

- **Full Form:** Routing Information Protocol
- **Type:** Distance Vector Routing Protocol (IGP)
- **Algorithm:** Bellman-Ford
- **Metric Used:** Hop Count
- **Maximum Hop Count:** 15 → 16 = Unreachable
- **Administrative Distance:** 120
- **Transport Protocol:** UDP
- **Port Number:** 520

- **Update Type:** Broadcast (RIPv1) / Multicast (RIPv2)
- **Multicast Address (RIPv2):** 224.0.0.9

◆ RIP Timers

Timer	Value	Meaning
Update Timer	30 sec	Sends routing updates every 30 seconds
Invalid Timer	180 sec	Route becomes invalid if no update received
Hold-down Timer	180 sec	Prevents accepting bad updates for that route
Flush Timer	240 sec	Route is removed from routing table

✓ **Memory Trick: 30 — 180 — 180 — 240**

◆ RIP Versions

Version	Name	Features
RIP v1	Classful	No subnet mask support
RIP v2	Classless	Supports VLSM, CIDR, Auth
RIPng	IPv6	Used for IPv6 routing

◆ RIP Loop Prevention Methods

1. **Hop Count Limit** — Max 15, 16 = unreachable
2. **Split Horizon** — Don't advertise a route back out the interface it was learned from
3. **Hold-down Timer** — Ignore new updates about a route for a period after it's marked invalid
4. **Triggered Updates** — Send an update immediately when a route changes (instead of waiting for timer)
5. **Route Poisoning** — Advertise a failed route with metric 16 (infinity) instead of just removing it

◆ Very Important Exam Notes ★

- RIP is **Distance Vector Protocol**
- Uses **Bellman-Ford Algorithm**
- Metric = **Hop Count** — lowest hop count is preferred
- Max hops = **15**; **AD = 120**; **UDP Port = 520**
- Sends full-table updates every **30 sec**
- TTL exceeded → ICMP message sent

4.6 IGRP (Interior Gateway Routing Protocol) — Cisco Legacy

- **Full Form:** Interior Gateway Routing Protocol
- **Developer:** Cisco (proprietary, now **obsolete/deprecated** — replaced by EIGRP)
- **Type:** Distance Vector
- **Metric:** Composite (Bandwidth + Delay + Reliability + Load — configurable)
- **Max Hop Count:** 100 (default), up to 255
- **Administrative Distance:** 100
- **Update Timer:** 90 sec

★ **Exam Note:** IGRP is Cisco-proprietary and largely **retired**; it's the historical predecessor to EIGRP. Mentioned mainly for legacy/theory questions — EIGRP is what's actually configured today.

Parameter	IGRP	EIGRP
Type	Distance Vector	Hybrid (Advanced DV)
Metric	Composite	Composite (same base formula)
AD	100	90 (Internal)
Classless	No (Classful)	Yes
VLSM Support	No	Yes
Status	Obsolete	Actively used

4.7 OSPF (Open Shortest Path First)

◆ OSPF Basics

- **Full Form:** Open Shortest Path First
- **Type:** Link-State Routing Protocol (IGP)
- **Algorithm:** Dijkstra (SPF — Shortest Path First)
- **Metric Used:** Cost = $10^8 / \text{Bandwidth}$
- **Administrative Distance:** 110
- **Transport Protocol:** IP Protocol Number **89**
- **Update Type:** Multicast, Triggered (not periodic)
- **Multicast Addresses:** 224.0.0.5 (All OSPF routers), 224.0.0.6 (DR/BDR)

◆ OSPF Timers

Timer	Default Value	Meaning
Hello Timer	10 sec	Sends Hello packets to discover neighbors
Dead Timer	40 sec	Neighbor declared down if no Hello received
Wait Timer	40 sec	Waits for DR/BDR election
Retransmit Timer	5 sec	Time between LSA retransmissions

✓ **Memory Trick:** 10 — 40 — 40 — 5

◆ OSPF Packet Types (5 types)

#	Packet Name	Purpose
1	Hello	Discover neighbors
2	DBD	Database Description
3	LSR	Link State Request
4	LSU	Link State Update
5	LSAck	Link State Acknowledgment

✓ **Memory Trick:** Hello → DBD → LSR → LSU → LSAck

◆ OSPF Areas

Area	Meaning
Area 0	Backbone Area (Mandatory)
Area 1,2...	Regular Areas
ABR	Area Border Router
ASBR	Autonomous System Boundary Router

✦ **All areas must connect to Area 0.**

◆ OSPF DR/BDR Election

Purpose: On broadcast/multi-access networks, DR & BDR reduce adjacency count, LSA flooding, and bandwidth/CPU usage. Other routers (DROTHERs) peer only with DR/BDR instead of full mesh.

Election Rules (in order):

1. **Highest Priority wins** (range 0–255) → DR; second-highest → BDR
2. **Tie-breaker:** Highest **Router ID** → DR; second-highest → BDR
3. **Priority = 0** → Router is ineligible, becomes DROTHER only
4. **Default priority = 1** for all interfaces → if unchanged, election goes purely by Router ID

◆ OSPF Neighbor States (7 states — Common Exam Question)

1. **Down** — No Hello received yet
2. **Init** — Hello received, but own Router ID not seen in neighbor's Hello
3. **Two-Way** — Bidirectional communication confirmed
4. **ExStart** — Master/slave roles decided for DB exchange
5. **Exchange** — DBD packets exchanged (LSDB summaries)
6. **Loading** — Missing LSAs requested (LSR) and received (LSU)
7. **Full** — LSDBs fully synchronized; adjacency complete

✓ **Memory Trick:** Down → Init → 2-Way → ExStart → Exchange → Loading → Full

◆ Very Important Exam Notes ★

- OSPF Hello = **10 sec**, Dead = **40 sec**
- Uses **Dijkstra Algorithm**
- Protocol Number = **89**
- Backbone area = **Area 0**
- Metric = **Cost**

4.8 EIGRP (Enhanced Interior Gateway Routing Protocol)

◆ EIGRP Basics

- **Full Form:** Enhanced Interior Gateway Routing Protocol
- **Type:** **Hybrid** (Distance Vector + Link-State features)

- **Algorithm: DUAL (Diffusing Update Algorithm)**
- **Metric Used:** Composite Metric (Bandwidth + Delay by default)
- **Administrative Distance:** 90 (Internal), 170 (External)
- **Protocol Number:** 88
- **Transport Protocol:** IP
- **Update Type:** Multicast, Triggered (no periodic updates)
- **Multicast Address:** 224.0.0.10

◆ EIGRP Metric Formula (Composite Metric)

$$\text{Metric} = 256 \times (10^7 / \text{Bandwidth} + \text{Delay})$$

- Bandwidth = lowest bandwidth in the path
- Delay = sum of delays in the path
- Optional (rarely used): Load, Reliability

◆ EIGRP Timers

Timer	Default Value	Meaning
Hello Timer	5 sec	Sends Hello packets to maintain neighbors
Hold Timer	15 sec	Neighbor declared down if no Hello received
Update Timer	No periodic updates	Sends updates only when changes occur (triggered)
Active Timer	3 min (180 sec)	Time allowed to find an alternative route

✓ **Memory Trick: 5 — 15 — (Triggered) — 180**

◆ EIGRP Tables (3 tables — Very Important)

Table	Purpose
Neighbor Table	Stores neighbor routers
Topology Table	Stores all learned routes
Routing Table	Stores best routes

◆ EIGRP Terminologies

Term	Meaning
Successor	Best path to destination
Feasible Successor	Backup path
Feasible Distance (FD)	Best metric to reach network
Reported Distance (RD)	Metric reported by neighbor

◆ EIGRP Packet Types (5 types)

#	Packet Name	Purpose
1	Hello	Discover neighbors

#	Packet Name	Purpose
2	Update	Send routing information
3	Query	Ask for alternate path
4	Reply	Respond to Query
5	Acknowledgment	Confirm receipt

✓ **Memory Trick:** Hello → Update → Query → Reply → Ack

◆ EIGRP Features

- Fast convergence
- Supports **VLSM and CIDR**
- Uses **Triggered Updates**
- Supports **Equal AND Unequal Cost Load Balancing** (unique advantage over RIP/OSPF)

◆ Very Important Exam Notes ★

- EIGRP uses **DUAL Algorithm**
- **AD = 90 (Internal)**, 170 (External)
- **Protocol Number = 88**
- **Hello = 5 sec, Hold = 15 sec**
- Uses **Composite Metric**
- Only IGP that supports **Unequal Load Balancing**

4.9 BGP (Border Gateway Protocol)

◆ BGP Basics

- **Full Form:** Border Gateway Protocol
- **Type: EGP (Exterior Gateway Protocol)** — connects Autonomous Systems
- **Algorithm Type: Path Vector** (not Distance Vector or pure Link-State)
- **Transport Protocol: TCP** (Port 179)
- **Administrative Distance:** 20 (eBGP), 200 (iBGP)
- **Convergence Speed:** Slow (designed for internet-scale stability, not speed)

◆ Characteristics of BGP

- Provides communication **between two Autonomous Systems**
- Supports the **Next-Hop Paradigm**
- Path Information included in advertisements (destination + next-hop pair)
- **Policy-based routing** — admin can configure custom route policies
- Runs over **TCP** (reliable, connection-oriented)
- Conserves bandwidth (incremental updates only, not periodic full updates)
- Supports **CIDR**
- Supports **security** via authentication (MD5, TCP-AO)

◆ Types of BGP

Type	Full Form	Used Between
eBGP	External BGP	Routers in different AS
iBGP	Internal BGP	Routers within the same AS

◆ BGP Path Attributes (Elements) — Used for Best Path Selection

Attribute	Meaning
Weight	Cisco-proprietary; higher value preferred
Local Preference	Selects outbound path; higher value preferred
AS Path	Shorter AS-Path length preferred
Origin	How the route was originated into BGP
Next Hop	IP address used as the next hop
MED (Multi-Exit Discriminator)	Suggests preferred entry point into AS; lower preferred

◆ BGP Functions (Peer-to-Peer)

1. **Peer Acquisition & Authentication** — TCP connection established, message exchange to agree on communication
2. **Reachability Exchange** — send positive/negative reachability info
3. **Verification** — confirm peers and connection are functioning correctly

◆ BGP Route Information Management

Function	Description
Route Storage	Stores info on how to reach other networks
Route Update	Determines when/how to update routes from peer info
Route Selection	Chooses best routes from database to each network
Route Advertisement	Regularly tells peers what it knows about reachable networks

◆ Why BGP Matters

- **Security:** Authenticates peer messages with passwords, filters unauthorized traffic
- **Scalability:** Manages massive route tables (entire internet)
- **Multihoming:** Allows connecting to multiple ISPs/networks simultaneously
- **Best Path Calculation:** Determines optimal path across AS boundaries

◆ BGP vs OSPF Comparison

Parameter	BGP	OSPF
Algorithm	Path Vector	Link-State
Convergence Speed	Slow	Fast
Scope	Inter-domain (between AS)	Intra-domain (within AS)
Routing Operation	Between two AS	Inside an AS
Transport Protocol	TCP	IP (Protocol 89)

◆ Very Important Exam Notes ★

- BGP = **only EGP** in practical use
- Uses **Path Vector algorithm**
- Runs over **TCP port 179**
- eBGP AD = **20**, iBGP AD = **200**
- Best path selection uses **Weight** → **Local Preference** → **AS Path** → **Origin** → **MED** → ...

4.10 🎯 Master Comparison Table — RIP vs OSPF vs EIGRP vs BGP

Parameter	RIP	OSPF	EIGRP	BGP
Category	IGP	IGP	IGP	EGP
Type	Distance Vector	Link-State	Hybrid	Path Vector
Algorithm	Bellman-Ford	Dijkstra (SPF)	DUAL	Path Vector (Best Path Selection)
Metric	Hop Count	Cost (Bandwidth)	Composite (BW+Delay)	Path Attributes (AS-Path etc.)
Max Hop Count	15	No limit	No fixed limit	N/A
Administrative Distance	120	110	90 (Int) / 170 (Ext)	20 (eBGP) / 200 (iBGP)
Transport	UDP (Port 520)	IP (Protocol 89)	IP (Protocol 88)	TCP (Port 179)
Update Type	Periodic (30 sec)	Triggered	Triggered	Triggered (incremental)
Convergence Speed	Slow	Fast	Very Fast	Slow
Load Balancing	Equal only	Equal only	Equal + Unequal	N/A (Policy-based)
Network Size	Small	Large	Medium–Large	Internet-scale (between AS)
Area/Hierarchy Concept	No	Yes (Areas)	Partial (Autonomous Systems)	Yes (AS-based)

★ Quick Revision — All Protocols Ultra-Short

RIP: DV | Bellman-Ford | Hop Count | Max 15 | AD 120 | UDP 520 | 30 sec updates

OSPF: Link-State | Dijkstra | Cost | AD 110 | Protocol 89 | Hello 10/Dead 40 | Area 0 mandatory

EIGRP: Hybrid | DUAL | Composite (BW+Delay) | AD 90/170 | Protocol 88 | Hello 5/Hold 15 | Unequal LB unique

BGP: Path Vector | TCP 179 | AD 20(e)/200(i) | Inter-AS | Slow convergence | Policy-based

AD Priority Order (lower = better): Connected(0) → Static(1) → EIGRP Summary(5) → eBGP(20) → EIGRP(90) → OSPF(110) → IS-IS(115) → RIP(120) → EIGRP Ext(170) → iBGP(200)

🎯 Interview-Level Key Differences

- **RIP** → Simple but **slow**
- **OSPF** → Structured and **scalable**, uses **Areas**
- **EIGRP** → Fast and **efficient**, supports **Unequal Load Balancing**
- **BGP** → Only protocol for **inter-AS/internet routing**, policy-driven, slow but stable

- **Static routing** → Best for small/stub networks, most secure, no overhead
 - **Dynamic routing** → Best for large networks, self-healing, more overhead
 - **AD decides between different protocols; Metric decides within the same protocol**
-

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [13 - Router IOS and Management](#)
- [04A - Network Routing Fundamentals](#)

Source note: 04B - Routing Protocols and Administrative Distance.md

CHAPTER 05 · ROUTING & NETWORK SERVICES



VLAN — Complete Notes

Segmentation, access and trunk links, 802.1Q tagging, router-on-a-stick, and VTP.

Topic 5

computer networks

v lans

inter vlan routing

switching

Table of Contents

- 1. VLAN Concept
- 1.1 VLAN ID (VLAN Number) and VLAN Range
- 2. Benefits / Advantages of VLAN
- 3. VLAN Port Assignment / Connection Types
- 4. VLAN Tagging — IEEE 802.1Q
- 5. Inter-VLAN Routing
- 6. VTP (VLAN Trunk Protocol)
- ★ Quick Revision — VLAN Module
- 🎯 Most Likely Exam/Viva Questions
- Related Notes

1. VLAN Concept

- **VLAN (Virtual Local Area Network)** = a **logical grouping** of networking devices, regardless of their physical location.
- Creating a VLAN **breaks one large broadcast domain into multiple smaller broadcast domains**.
- Think of a VLAN like a **subnet**: just as two different subnets cannot communicate without a router, **two different VLANs also require a router (or Layer 3 device) to communicate**.
- Devices in the same VLAN can communicate directly (Layer 2); devices in different VLANs need Layer 3 routing.

✦ **Exam Trap:** VLAN = Broadcast Domain = Subnet (conceptually mapped 1:1 in most designs).

🔑 **Key Point — Trunk Link:** A **trunk link carries traffic for multiple VLANs over a single physical connection** (one cable, many VLANs — each frame tagged with its VLAN ID so the far end can sort it back out). An **access link**, by contrast, carries traffic for **only one VLAN**.

1.1 VLAN ID (VLAN Number) and VLAN Range

Every VLAN is identified by a unique **VLAN ID** (a number) so switches can distinguish one VLAN's traffic from another's, especially over a trunk link.

◆ VLAN ID Field

- Carried inside the **802.1Q tag** in the Ethernet frame (12-bit field).
- 12 bits → theoretical range = **0 – 4095** → but 0 and 4095 are reserved, so usable range = **1 – 4094**.

◆ VLAN ID Ranges (Cisco Switches)

Range	Type	Notes
0	Reserved	Not used (priority tagging only, no VLAN)
1	Default VLAN	Exists automatically on every switch; cannot be deleted or renamed; carries CDP, VTP, PAgP, STP traffic by default
2 – 1001	Normal Range	User-configurable VLANs; stored in <code>vlan.dat</code> ; advertised via VTP
1002 – 1005	Reserved (legacy)	Auto-created for legacy Token Ring / FDDI; cannot be deleted
1006 – 4094	Extended Range	User-configurable; NOT advertised by VTP (in VTP versions 1 & 2); stored in running-config, not <code>vlan.dat</code>
4095	Reserved	Used internally, not assignable

✓ **Memory Trick:** 1 = Default | 2–1001 = Normal | 1002–1005 = Legacy Reserved | 1006–4094 = Extended | 4095 = Reserved

◆ Normal Range vs Extended Range VLANs

Parameter	Normal Range VLAN	Extended Range VLAN
VLAN ID	1 – 1001	1006 – 4094
Stored In	<code>vlan.dat</code> (flash memory)	<code>running-config</code> only
VTP Advertisement	Yes (VTPv1/v2)	No (VTPv1/v2) — VTPv3 does support it
Typical Use	Most enterprise deployments	Service provider / very large networks

✳ **Exam Trap:** VLAN 1 is the **default VLAN** and also commonly the **native VLAN** — both can be changed except VLAN 1 itself can never be deleted.

2. Benefits / Advantages of VLAN

Benefit	Explanation
Solves broadcast problem	Limits broadcast traffic to within the VLAN only
Reduces broadcast domain size	Smaller domains = less congestion, better performance
Additional layer of security	Sensitive systems can be isolated into their own VLAN
Easier device management	Devices grouped logically, easier to administer centrally
Logical grouping by function	Group by department/role instead of physical location (e.g., all HR PCs in one VLAN regardless of floor)

✓ **Memory Trick:** S-R-A-M-L → Solve broadcast, Reduce domain, Add security, Manage easily, Logical grouping

3. VLAN Port Assignment / Connection Types

When configuring VLANs on a switch port, we must define the **connection type**. Switches support **two types** of VLAN connections:

1. Access Link
2. Trunk Link

◆ Access Link

- Connects a switch port to an end device with a **standard Ethernet NIC**.
- Standard NICs only understand **IEEE 802.3 / Ethernet II** frames (no VLAN tag).
- An access port can be assigned to **only ONE VLAN** at a time.
- All devices connected via that access port belong to the **same broadcast domain (VLAN)**.

Example:

20 users connected to a hub → hub connected to a switch access port → all 20 users are in the **same VLAN**. To put 10 users in a different VLAN, you'd need a **separate hub** connected to a **different access port**.

◆ Trunk Link

- Connects a switch port to a device **capable of understanding multiple VLANs** (usually **switch-to-switch** or **switch-to-router**).
- Allows VLAN information to be carried **across the network** — this is how a VLAN can "span" multiple switches.
- Requires the original Ethernet frame to be **modified** to carry VLAN membership information (**tagging**).

◆ Access vs Trunk — Comparison Table

Parameter	Access Link	Trunk Link
Connects to	End devices (PC, printer, standard NIC)	Switches, routers
VLANs Supported per Port	Only 1 VLAN	Multiple VLANs
Frame Type	Standard Ethernet (untagged)	Tagged (ISL / 802.1Q)
Purpose	Connect end-user devices	Carry VLAN traffic between network devices
Requires Tagging Protocol	No	Yes (Dot1q or ISL)

4. VLAN Tagging — IEEE 802.1Q

◆ What is Tagging?

- In trunking, a **separate logical connection** is created for each VLAN (instead of one physical connection per VLAN).
 - The switch **adds the source port's VLAN identifier** to the Ethernet frame — this is called **tagging**.
 - This tag tells the receiving switch **which VLAN the frame originated from**, allowing intelligent forwarding decisions based on **both** the destination MAC address **and** the source VLAN ID.
 - Tagging is performed in **hardware** by **ASICs (Application-Specific Integrated Circuits)** for speed.
- ✦ Since the frame is modified, a **standard NIC will NOT understand tagged frames** and will typically **drop them**. Both ends of a trunk link **must support and be configured with the same trunking protocol**.

◆ Trunking Protocols (2 types)

Protocol	Full Form	Vendor
ISL	Inter-Switch Link	Cisco- proprietary
Dot1q	IEEE 802.1Q	Open industry standard

◆ IEEE 802.1Q Details

- Industry-standard trunking protocol (works across vendors, unlike ISL).
- Inserts a **4-byte tag** into the Ethernet frame header, containing:
 - **TPID (Tag Protocol Identifier)** — identifies the frame as 802.1Q tagged
 - **VLAN ID (12 bits)** — supports VLAN IDs from **1 to 4094**
 - **Priority bits (3 bits)** — for QoS (CoS — Class of Service)
- **Native VLAN concept:** 802.1Q does **NOT tag** frames belonging to the **native VLAN** (default VLAN 1 unless changed) — these are sent **untagged** across the trunk.

✓ Exam Trap:

- ISL = Cisco proprietary, **tags all frames including native VLAN**, now largely obsolete.
- 802.1Q = Industry standard, **does NOT tag native VLAN traffic**, widely used today.

5. Inter-VLAN Routing

Since different VLANs are like different subnets, communication **between VLANs requires Layer 3 routing**. There are **three options**:

1. **Router with one physical interface per VLAN** (traditional method — typically NOT used; wastes physical ports)
2. **Router-on-a-Stick** (single router interface, VLAN trunk to switch)
3. **Layer 3 Switch** (Switched Virtual Interfaces — SVIs, most common in modern networks)

◆ Option 1: One Router Interface per VLAN

- Requires a **dedicated physical router interface for each VLAN**.
- **Disadvantage:** Not scalable — limited by number of physical router ports; wasteful and expensive.

◆ Option 2: Router-on-a-Stick

- A configuration that allows routing of traffic **between VLANs using a single physical router interface**.
- The router has **one physical interface**, but that interface is divided into multiple **logical sub-interfaces** — one per VLAN.
- Router connects to the switch via a **VLAN trunk** (802.1Q).
- Each subnet's hosts use the router's sub-interface IP (in that VLAN) as their **default gateway**.

Configuration Steps (Cisco):

```
! Step 1: Create a sub-interface for each VLAN
interface <type><number>.<subinterface-number>
```

```
! Step 2: Enable 802.1Q trunking and associate the sub-interface with a VLAN
encapsulation dot1q <vlan_id>
```

! Step 3: Assign IP address to the sub-interface
 ip address <address> <mask>

Example:

```
interface fastethernet0/0.10
  encapsulation dot1q 10
  ip address 192.168.10.1 255.255.255.0

interface fastethernet0/0.20
  encapsulation dot1q 20
  ip address 192.168.20.1 255.255.255.0
```

✅ **Exam Trap:** Router-on-a-stick needs only **ONE physical interface** but **multiple logical sub-interfaces**, each tied to a VLAN via `encapsulation dot1q`.

◆ Option 3: Layer 3 Switch (Preferred in Modern Networks)

- A switch capable of performing **both Layer 2 switching AND Layer 3 routing**.
- Uses **SVI (Switched Virtual Interface)** — a virtual interface representing a VLAN, assigned an IP address, acting as the default gateway for that VLAN.
- **Faster** than router-on-a-stick (hardware-based switching/routing, no trunk bottleneck).
- Most commonly used method in **enterprise networks today**.

Basic Configuration Concept:

```
interface vlan 10
  ip address 192.168.10.1 255.255.255.0
  no shutdown
```

◆ Comparison of Inter-VLAN Routing Methods

Method	Physical Interfaces Needed	Speed	Scalability	Common Usage
One interface per VLAN	Many (1 per VLAN)	Fast (dedicated)	Poor	Rare / legacy
Router-on-a-Stick	1 (with sub-interfaces)	Slower (bottleneck at trunk)	Medium	Small/medium networks, labs
Layer 3 Switch (SVI)	0 (virtual interfaces)	Fastest (hardware-based)	Excellent	Modern enterprise networks

6. VTP (VLAN Trunk Protocol)

◆ What is VTP?

- **VTP (VLAN Trunk Protocol)** reduces administrative overhead in a switched network.
- When a new VLAN is configured on **one VTP server**, that VLAN information is **automatically distributed to all switches** in the same VTP domain.
- Eliminates the need to **manually configure the same VLAN on every switch**.
- **VTP is Cisco-proprietary**, available mainly on Cisco Catalyst series switches.

◆ VTP Features

- Advertises VLAN configuration information across the network
- Maintains **VLAN configuration consistency** throughout a common administrative domain

- Sends advertisements **only on trunk ports**

♦ VTP Modes (3 Modes — Very Important)

Mode	Can Create/Modify/Delete VLANs?	Forwards VTP Advertisements?	Saves VLAN Info in NVRAM?
Server	✓ Yes	✓ Yes	✓ Yes
Client	✗ No	✓ Yes	✗ No (learns from Server)
Transparent	✓ Yes (local only, not advertised)	✓ Yes (forwards, doesn't process)	✓ Yes (local database only)

Details:

- **Server Mode (default mode):**

- Full control — can create, modify, and delete VLANs.
- Advertises VLAN info to other switches in the domain.
- Stores VLAN configuration in NVRAM (persists across reboot).

- **Client Mode:**

- **Cannot** create/modify/delete VLANs locally.
- Only **receives and forwards** VTP advertisements from servers.
- Does **not** save VLAN info to NVRAM — relies on Server for updates on every boot.

- **Transparent Mode:**

- Can create/modify/delete VLANs, but changes are **local only** — NOT advertised to other switches.
- **Forwards** VTP advertisements it receives from other switches (acts as a pass-through) but does **not process/act on them**.
- Saves its own VLAN configuration locally in NVRAM.

✓ **Memory Trick: Server = Full Control + Advertises Client = No Control, Just Listens Transparent = Local Control Only, Just Relays**

♦ VTP Operation

- VTP advertisements are sent as **multicast frames**.
- VTP **servers and clients** synchronize using the **latest Configuration Revision Number** — higher revision number = more recent = gets adopted.
- Advertisements are sent **every 5 minutes**, or immediately when a change occurs (triggered).

⚠ **Critical Danger (Common Real-World/Exam Scenario):** If a **new switch with a higher VTP revision number** (even with wrong/empty VLAN data) is added to the domain, it can **overwrite the VLAN database** of the entire network — causing major outages!

♦ VTP Configuration Guidelines

Configuration items required for VTP setup:

- **VTP Domain Name** — must match across all switches in the domain to communicate
- **VTP Mode** — Server mode is default
- **VTP Pruning** — restricts unnecessary VLAN traffic on trunk links where not needed (saves bandwidth)
- **VTP Password** — secures the domain from unauthorized VTP changes

♦ Best Practice When Adding a New Switch to an Existing VTP Domain

To avoid accidentally overwriting the domain's VLAN database:

1. **Add the new switch in Client mode first** → allows it to safely learn/synchronize the latest VLAN information from the domain without risk of overwriting anything.
2. Once synced, **convert it to Server mode** if it needs to make changes.

Alternative approach:

1. Add all new configurations to the switch while it is in **Transparent mode** first.
2. Thoroughly **check/verify the configuration**.
3. Then **convert it to Server mode** — this prevents the switch from propagating incorrect/incomplete VLAN information to the rest of the domain during setup.

✓ **Exam Trap:** Never add a brand-new switch directly in **Server mode** with default settings blindly — the real danger is adding a switch that was **previously configured elsewhere** with a **higher revision number** — it can silently wipe out the domain's VLANs.

★ Quick Revision — VLAN Module

- VLAN = logical broadcast domain, needs a **router/L3 device** to talk between VLANs
- Benefits: **S-R-A-M-L** (Solve broadcast, Reduce domain, Add security, Manage easily, Logical grouping)
- **Access link** = 1 device, 1 VLAN, untagged, standard NIC
- **Trunk link** = multiple VLANs, tagged, switch-to-switch/router
- Trunk protocols: **ISL** (Cisco, tags native VLAN too) vs **802.1Q** (Standard, native VLAN untagged)
- Inter-VLAN routing: **Router-on-a-Stick** (1 interface, multiple sub-interfaces, `encapsulation dot1q`) OR **Layer 3 Switch** (SVI, faster, modern standard)
- VTP = Cisco-proprietary, auto-distributes VLAN config
- VTP modes: **Server** (full control + advertise) / **Client** (no control, listen only) / **Transparent** (local control, just relay)
- VTP uses **revision number** to determine latest config — **higher wins** (danger zone!)
- Safe practice: add new switches in **Client** or **Transparent** mode first, verify, then promote to **Server**

🎯 Most Likely Exam/Viva Questions

- What problem does VLAN solve? → **Broadcast domain size / broadcast storms**
- Difference between Access and Trunk ports? → 1 VLAN vs multiple VLANs
- What is tagging and why is it needed? → Identifies source VLAN of frame across trunk
- ISL vs 802.1Q? → Proprietary vs Standard; native VLAN tagging difference
- What is Router-on-a-Stick? → Single router interface + VLAN sub-interfaces via `dot1q` trunk
- What is an SVI? → Virtual Layer 3 interface for a VLAN on a Layer 3 switch
- 3 VTP modes and their differences? → Server / Client / Transparent (see table above)
- What determines which VTP update is accepted? → **Highest revision number**
- Why is VTP risky when adding new switches? → A switch with higher revision number can overwrite existing VLAN database

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [04A - Network Routing Fundamentals](#)
- [11 - Layer 2 Switching and Ethernet Forwarding](#)
- [08 - Spanning Tree Protocol](#)
- [09 - Infrastructure Security ACL AAA and Port Security](#)

Source note: 05 - VLANs and Inter-VLAN Routing.md

CHAPTER 06 · ROUTING & NETWORK SERVICES



Module 6: NAT (Network Address Translation) — Complete Notes

NAT terminology and operation, including static NAT, dynamic NAT, PAT, and NAT64.

Topic 6

computer networks

nat

ip addressing

network security

Table of Contents

- 6.1 What is NAT?
- 6.2 Why NAT is Used — IPv4 Exhaustion
- 6.3 Private IP Address Ranges (RFC 1918)
- 6.4 NAT Addressing Terminology (Very Important)
- 6.5 NAT Table Concept
- 6.6 Types of NAT
- 6.7 Static NAT
- 6.8 Dynamic NAT
- 6.9 PAT (Port Address Translation) / NAT Overload
- 6.10 NAT Types — Full Side-by-Side Comparison
- 6.11 NAT64
- ★ Quick Revision — NAT Module
- 🎯 Most Likely Exam/Viva Questions
- Related Notes

6.1 What is NAT?

Network Address Translation (NAT) is the process where a network device — usually a **router or firewall** — translates a **private IP address** into a **public IP address** (and vice versa) as traffic crosses the boundary between a private network and the outside network (typically the Internet).

- NAT is typically performed on a **border router/firewall** sitting between the inside (private) network and the outside (public) network.
- Assigns a **public address** to a computer (or group of computers) inside a private network.

6.2 Why NAT is Used — IPv4 Exhaustion

- IPv4 has only **~4.3 billion** addresses total — nowhere near enough for every device on Earth to have a unique public IP.
- NAT was introduced as a **short-term fix** to slow down the exhaustion of public IPv4 addresses (long-term fix = IPv6).

◆ Main Reasons NAT is Used

Reason	Explanation
IPv4 Address Conservation	Limits the number of public IPs an organization must purchase/use
Economy	Public IPs cost money (leased from ISP); NAT lets many private hosts share few public IPs
Security	Hides internal IP addressing scheme from the outside world (internal topology not exposed)

Reason	Explanation
Flexibility	Internal network can be renumbered/restructured without affecting external-facing addresses
Multihoming/ISP independence	Internal addressing doesn't need to change even if ISP or public IP changes

✦ **Exam Trap:** NAT is fundamentally an **IPv4 exhaustion workaround** — it does NOT eliminate the need for public IPs, it just drastically **reduces how many are needed**.

6.3 Private IP Address Ranges (RFC 1918)

These ranges are **reserved for private/internal use only** — they are **not routable on the public Internet**. Any device using these must go through NAT to reach the internet.

Class	Private IP Range	Total Addresses	CIDR
Class A	10.0.0.0 – 10.255.255.255	~16.7 million	10.0.0.0/8
Class B	172.16.0.0 – 172.31.255.255	~1 million	172.16.0.0/12
Class C	192.168.0.0 – 192.168.255.255	65,536	192.168.0.0/16

✓ **Memory Trick:** 10.x.x.x → 172.16.x–172.31.x → 192.168.x.x (Class A = biggest range for large enterprises; Class C = smallest, most common in homes/small offices)

✦ **Exam Trap:** Common mistake — thinking **all** of 172.x.x.x is private. Only **172.16.0.0 to 172.31.255.255** is private; e.g., 172.32.0.0 is a **public** address.

◆ Other Special/Reserved Ranges (Bonus)

Range	Purpose
127.0.0.0/8	Loopback (localhost)
169.254.0.0/16	APIPA (Automatic Private IP Addressing)

6.4 NAT Addressing Terminology (Very Important)

NAT uses **4 specific address terms**, based on the combination of **Inside/Outside** and **Local/Global**:

Term	Meaning
Inside Local	The actual private IP address assigned to a host inside the enterprise network (the "real" internal address)
Inside Global	The public/translated IP address that represents an inside host to the outside network (what the internet sees)
Outside Global	The actual real public IP address of a host that resides outside the enterprise (e.g., a web server on the internet)
Outside Local	The address used to represent an outside host as seen from inside the private network (rarely different from Outside Global unless doing NAT on both sides)

◆ Simple Way to Remember

- **"Inside"** = belongs to your private network
- **"Outside"** = belongs to the internet/external network
- **"Local"** = how the address appears **inside** the private network
- **"Global"** = how the address appears **outside**, on the public network

Direction of Translation:

- **Inside → Outside:** Source IP changes from **Inside Local** → **Inside Global**
- **Outside → Inside:** Destination IP changes from **Inside Global** → **Inside Local**

✓ **Memory Trick:** **Inside Local** = "my real private IP" **Inside Global** = "how I look to the outside world" **Outside Global** = "the real IP of the outside server" **Outside Local** = "how the outside server looks to me" (usually same as Outside Global unless double NAT)

6.5 NAT Table Concept

The **NAT Table** (also called the **NAT Translation Table**) is a table maintained by the NAT router/firewall that keeps track of the **mappings between inside local and inside global addresses** (and ports, in PAT).

◆ What the NAT Table Stores

- Inside Local IP (+ port, for PAT)
- Inside Global IP (+ port, for PAT)
- Outside Global IP (+ port, if applicable)
- Protocol (TCP/UDP)
- Idle timer (how long the entry stays before being removed if unused)

◆ Example NAT Table (PAT/Overload)

Inside Local	Inside Global	Outside Global	Protocol
192.168.1.10:5001	203.0.113.5:10001	8.8.8.8:80	TCP
192.168.1.11:5002	203.0.113.5:10002	142.250.1.1:443	TCP
192.168.1.12:5003	203.0.113.5:10003	93.184.216.34:80	TCP

- When a return packet arrives at the router, it checks the NAT table to see **which inside host** the packet belongs to, based on the **destination port**, and forwards it accordingly.
- **Static NAT** entries → **permanent** in the table (manually configured, never age out).
- **Dynamic NAT / PAT** entries → **temporary**, stay in the table only as long as traffic flows; removed after an **idle timeout**.

6.6 Types of NAT

There are **4 main types**:

1. **Static NAT**
2. **Dynamic NAT**
3. **PAT (Port Address Translation) / NAT Overload**

4. NAT64 (IPv6 ↔ IPv4 transition mechanism)

◆ Quick Comparison Table

Type	Mapping	Public IPs Used	Use Case
Static NAT	1-to-1, permanent	One public IP per private IP	Server that must be reachable from outside
Dynamic NAT	Many-to-many (pool)	Pool of public IPs	Multiple hosts; pool can run out
PAT / NAT Overload	Many-to-1 (using ports)	One public IP, many ports	Homes, businesses (most common NAT type)
NAT64	IPv6 ↔ IPv4	NAT64 prefix	IPv4-to-IPv6 migration/transition

6.7 Static NAT

- Defines a **one-to-one, permanent mapping** between one private (inside local) IP and one public (inside global) IP.
- Mapping includes **destination IP translation** in one direction and **source IP translation** in the reverse direction.
- **Manually configured** — does NOT age out or change automatically.
- **A public IP address must be allocated for every single private IP** that needs static NAT — no address pools involved.
- Allows connections to be **originated from either side** (inside → outside OR outside → inside) — this is important because it's the only NAT type that reliably supports **inbound connections initiated from the internet**.

◆ Static NAT Diagram

Static NAT:
 Inside: 192.168.1.10 → Router → Outside: 203.0.113.5

◆ Use Case

- **Servers that must be reachable from the internet** — e.g., a web server, mail server, or any host that external users need to initiate connections to.

✓ **Exam Trap:** Static NAT does **NOT conserve IP addresses** — it's still 1-to-1 — its purpose is **reachability/accessibility**, not conservation.

6.8 Dynamic NAT

- Also creates a **one-to-one mapping** between inside local and inside global addresses, BUT the mapping is chosen **dynamically** from a **pool of available public IPs**, rather than being manually fixed.
- Router defines:
 - A **pool of possible inside global (public) addresses**
 - **Criteria** (via ACL) for which inside local addresses should be translated
- The dynamic entry stays in the NAT table **only as long as traffic is flowing occasionally** — it **ages out** after a period of inactivity.

◆ Dynamic NAT Diagram

```
Dynamic NAT:
192.168.1.10 → 203.0.113.5
192.168.1.11 → 203.0.113.6
192.168.1.12 → 203.0.113.7 } NAT Pool
```

◆ Key Limitation

- If the **number of inside hosts needing translation exceeds the pool size**, some hosts will **fail to get a public IP** and be unable to reach the internet until an address frees up.

✓ **Exam Trap:** Dynamic NAT is still **1-to-1 at any given moment** — it just **automates address assignment** from a pool. It does NOT let multiple hosts share ONE IP simultaneously (that's PAT).

6.9 PAT (Port Address Translation) / NAT Overload

- Also called **NAT Overloading** or **NAPT (Network Address Port Translation)**.
- A modified form of Dynamic NAT where the **number of inside local addresses is greater than the number of inside global addresses**.
- Typically, **just ONE single public IP** provides internet access for **ALL inside hosts**.
- Distinguishes between multiple internal hosts sharing the same public IP by using **different source port numbers** for each translated connection.
- **The only NAT type that actually conserves IP addresses** — because many private hosts share just one public IP.
- **Most popular/common form of NAT** used today (homes, small businesses, most SOHO routers).

◆ PAT Diagram

```
PAT (NAT Overload):
192.168.1.10 → 203.0.113.5 : 10001
192.168.1.11 → 203.0.113.5 : 10002
192.168.1.12 → 203.0.113.5 : 10003 } Same public IP, different ports
```

◆ Why PAT Works

- TCP/UDP has **65,536 possible port numbers** per IP address.
- By mapping each internal host's connection to a **unique port** on the single shared public IP, the router can track and correctly route return traffic back to the right internal host.

✓ **Exam Trap:** PAT = **Many-to-One** using **ports** to differentiate; this is what makes home routers work with just 1 public IP for an entire household of devices.

6.10 NAT Types — Full Side-by-Side Comparison

Parameter	Static NAT	Dynamic NAT	PAT / NAT Overload
Mapping Type	1-to-1 (fixed)	1-to-1 (from pool)	Many-to-1 (via ports)
Public IP Requirement	1 per private IP	Pool of public IPs	Just 1 public IP (typically)

Parameter	Static NAT	Dynamic NAT	PAT / NAT Overload
Conserves IP Addresses?	✗ No	⚠ Partially	✓ Yes (best conservation)
Connection Origination	Either direction	Inside → Outside only	Inside → Outside only
Table Entry	Permanent	Temporary (ages out)	Temporary (ages out)
Common Use Case	Public-facing servers	Medium orgs with IP pool	Homes, small/large businesses
Popularity	Less common	Less common today	Most widely used

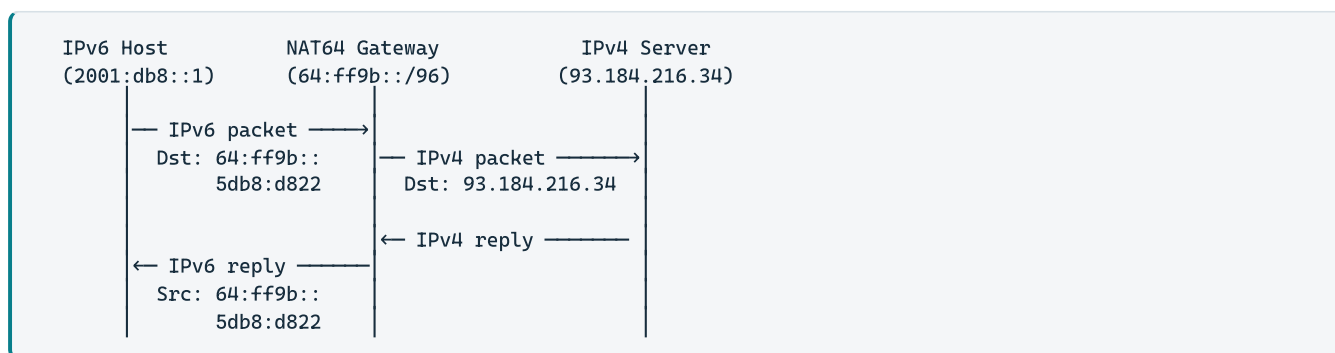
6.11 NAT64

NAT64 allows **IPv6-only hosts** to communicate with **IPv4-only servers**, by translating between IPv6 and IPv4 addresses at the network boundary. Works **alongside DNS64**.

◆ How NAT64 Works (Step-by-Step)

1. An **IPv6-only host** queries **DNS64** for an IPv4-only domain (e.g., `example.com`).
2. DNS64 finds **only an IPv4 A record** (e.g., `93.184.216.34`). Since no AAAA (IPv6) record exists, DNS64 **synthesizes** one by embedding the IPv4 address into the **NAT64 prefix** (`64:ff9b::/96`):
 - Synthesized AAAA: `64:ff9b::5db8:d822`
3. The host sends an **IPv6 packet** to `64:ff9b::5db8:d822`.
4. The **NAT64 gateway** intercepts the packet, **extracts the embedded IPv4 destination** (`93.184.216.34`), translates **IPv6 → IPv4**, and forwards it to the real IPv4 server.
5. The **IPv4 server replies** → the NAT64 gateway translates **IPv4 → IPv6** → sends the reply back to the original IPv6 host.

◆ NAT64 Flow Diagram



◆ NAT64 vs Traditional NAT

Feature	Traditional NAT	NAT64
Translates	Private IPv4 ↔ Public IPv4	IPv6 ↔ IPv4
Purpose	Address conservation	Protocol transition
Needs DNS?	✗ No	✓ Yes (DNS64 required)
Direction	IPv4 client → IPv4 server	IPv6 client → IPv4 server
Use Case	Home/office internet sharing	IPv4-to-IPv6 migration

◆ NAT64 Key Terms

Term	Meaning
NAT64	Translates between IPv6 and IPv4
DNS64	Synthesizes AAAA records from existing IPv4 A records
NAT64 Prefix	<code>64:ff9b::/96</code> — the last 32 bits of this prefix hold the embedded IPv4 address
Stateful NAT64	Many-to-one mapping (like PAT) — many IPv6 hosts share one IPv4 address
Stateless NAT64	One-to-one mapping — requires a dedicated block of IPv4 addresses

✓ **Exam Trap:** NAT64 is fundamentally different in **purpose** from traditional NAT — traditional NAT conserves addresses **within the same protocol (IPv4↔IPv4)**; NAT64 exists purely to let **two different protocol versions (v6 and v4) talk to each other** during the IPv6 migration period.

★ Quick Revision — NAT Module

- **NAT** = translates private ↔ public IPs at the network boundary (router/firewall)
- **Why:** IPv4 exhaustion, address conservation, cost savings, hides internal topology (security)
- **Private ranges (RFC 1918):**
 - `10.0.0.0 – 10.255.255.255` (Class A)
 - `172.16.0.0 – 172.31.255.255` (Class B)
 - `192.168.0.0 – 192.168.255.255` (Class C)
- **4 Address Terms:** Inside Local (real private IP) / Inside Global (translated public IP) / Outside Global (real public IP of external host) / Outside Local (how external host looks from inside)
- **NAT Table:** tracks Local↔Global mappings (+ports for PAT); static = permanent, dynamic/PAT = ages out
- **Static NAT:** 1-to-1, permanent, manual, allows inbound connections — used for public-facing servers
- **Dynamic NAT:** 1-to-1 from a pool, automatic, pool can exhaust
- **PAT/Overload:** Many-to-1 using **ports**, only NAT type that truly **conserves IPs**, most common
- **NAT64:** IPv6 ↔ IPv4 translation for protocol transition, requires DNS64, prefix `64:ff9b::/96`

🎯 Most Likely Exam/Viva Questions

- Why was NAT introduced? → **IPv4 address exhaustion**
- Name the 3 private IP ranges → 10.x / 172.16–172.31.x / 192.168.x
- Difference between Inside Local and Inside Global? → Real private IP vs how it appears externally (translated)
- Which NAT type allows inbound connections from the internet? → **Static NAT**
- Which NAT type conserves the most IP addresses? → **PAT / NAT Overload**
- What differentiates hosts sharing the same public IP in PAT? → **Port numbers**
- What happens to a Dynamic NAT/PAT entry when traffic stops? → It **ages out / times out** from the NAT table
- What is NAT64 used for? → Allowing **IPv6-only hosts** to reach **IPv4-only servers**
- What is the NAT64 prefix? → `64:ff9b::/96`
- Difference between Stateful and Stateless NAT64? → Many-to-1 (like PAT) vs strict 1-to-1

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [03 - IP Subnetting VLSM IPv4 IPv6 and NDP](#)
- [09 - Infrastructure Security ACL AAA and Port Security](#)

Source note: 06 - Network Address Translation.md

PART III

Resilience & Infrastructure Security

Keep Layer 2 paths stable and enforce who can connect, authenticate, and pass traffic.

[08 · Spanning Tree Protocol](#)[09 · Infrastructure Security](#)

CHAPTER 08 · RESILIENCE & INFRASTRUCTURE SECURITY

Spanning Tree Protocol (STP)

Layer 2 loop prevention through BPDUs, root election, port roles, states, and STP variants.

- Topic 8
- computer networks
- stp
- switching
- layer 2

Table of Contents

- 0. Full Forms / Abbreviations
- 1. Why STP Is Needed — The Layer 2 Loop Problem
- 2. What Is STP, In Plain Terms?
- 3. How Switches Discover Loops — BPDUs
- 4. Root Bridge Election
- 5. STP Timers & BPDU Types

- 6. Port Roles
- 7. STP Protocol Types (Variants)
- 8. Port States
- 9. Quick Revision — Key Facts
- Related Notes

0. Full Forms / Abbreviations

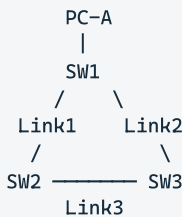
Abbreviation	Full Form
STP	Spanning Tree Protocol
RSTP	Rapid Spanning Tree Protocol
MSTP	Multiple Spanning Tree Protocol
PVST+	Per-VLAN Spanning Tree Plus
RPVST+	Rapid Per-VLAN Spanning Tree Plus
BPDU	Bridge Protocol Data Unit
TCN (BPDU)	Topology Change Notification (BPDU)
BID	Bridge ID
RP	Root Port
DP	Designated Port
TTL	Time To Live
ARP	Address Resolution Protocol
MAC	Media Access Control (address)
VLAN	Virtual Local Area Network
IEEE	Institute of Electrical and Electronics Engineers

1. Why STP Is Needed — The Layer 2 Loop Problem

Real-world networks deliberately use **redundant links** between switches for fault tolerance (a backup path if one link fails). The problem: **Ethernet frames have no TTL** (unlike IP packets) — nothing stops a frame from circulating forever if a loop exists. Without a loop-prevention mechanism, three problems happen **simultaneously**:

#	Problem	What Happens
①	Broadcast Storm	A single ARP request loops endlessly → CPU on all switches hits 100% → network collapses completely
②	MAC Table Instability (Flapping)	The same source MAC appears to arrive from multiple ports in rapid succession → switches keep re-learning it → forwarding becomes unreliable
③	Duplicate Frame Delivery	The same unicast frame arrives at the destination multiple times → breaks upper-layer protocols like TCP

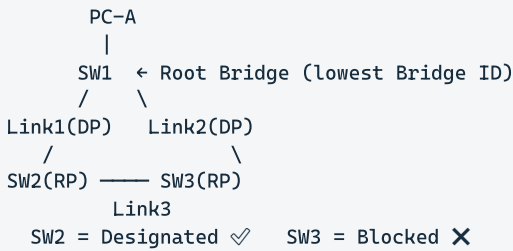
1.1 Without STP — Loop Present



- Step 1: PC-A sends a broadcast → SW1 floods it out BOTH Link1 and Link2.
Step 2: SW2 (via Link1) floods it onward via Link3.
SW3 (via Link2) floods it back via Link3.
Step 3: Both switches receive the SAME broadcast again → and flood it again → infinite loop.

RESULT: BROADCAST STORM – network CRASHES

1.2 With STP — Loop Broken



Loop BROKEN – one port blocked, backup path ready to activate automatically if SW2 fails

1.3 Problem → STP Solution

Problem Without STP	How STP Solves It
Broadcast storms crash the network	Blocks redundant ports to eliminate loops
MAC table instability / flapping	Stable topology → stable, predictable MAC learning

Problem Without STP	How STP Solves It
Duplicate frame delivery	Enforces a single forwarding path per network segment
Ethernet has no Layer-2 TTL	STP itself acts as the loop-prevention mechanism

2. What Is STP, In Plain Terms?

- STP (**IEEE 802.1D**) was created to **prevent Layer 2 loops** while still keeping redundant physical links available as backups.
- STP ensures there is only **one logical (active) path** between any two points on the network, by intentionally **blocking** redundant paths that could otherwise cause a loop.
- A **blocked port** stops **user data** from entering or leaving it — but it still sends/receives **BPDUs** (the control messages STP itself uses to detect and manage loops).
- The physical cabling/redundant paths still **exist** — they're just administratively/logically disabled. If the active path fails, STP **recalculates** and automatically unblocks the standby port to restore connectivity.

Key idea:

Physical topology = has loops (redundant links, by design)

Logical topology = loop-free (STP blocks the redundant ports)

3. How Switches Discover Loops — BPDUs

- Switches discover the presence of loops (and each other) by sending **probes** into the network.
- These probes are called **BPDUs** — **Bridge Protocol Data Units**.
- Each BPDU carries specific information identifying the sending switch (its Bridge ID, path cost, etc. — see §4.2).
- Every switch **multicasts** BPDU probes **every 2 seconds** (the Hello Time, §5.1).
- **Loop detection logic:** if a switch ever receives **its own BPDU back**, that confirms a loop exists in the network.

BPDUs serve **two purposes**:

1. **Loop detection** — as above.
2. **Root Bridge election** — BPDUs carry the Bridge ID used to elect the Root Bridge (§4).

Once the Root Bridge is elected, every other switch calculates the **best (lowest-cost) path** to reach it, and **blocks** any redundant/extra links (based on port cost — see §6). Those blocked links become **active automatically** only if the currently active link or port fails.

4. Root Bridge Election

4.1 Bridge ID (BID)

Every switch has a **Bridge ID**, which is what gets compared during root bridge election.

Classic 802.1D BID structure:

Priority (2 B)

System ID / MAC Address (6B)

Extended System ID (used in PVST+ / Rapid-PVST+, which run one STP instance per VLAN):

Priority field = Bridge Priority + VLAN ID
 Example: 32768 + 10 = 32778

4.2 Election Rule

Root Bridge Election Rule:

1. LOWEST Bridge ID (BID) wins.
2. If there's a tie on Priority → LOWEST MAC address wins.

Example — three switches with equal priority (32768):

32768.1111.1111.1111 ← Lowest MAC → becomes ROOT BRIDGE
 32768.2222.2222.2222
 32768.3333.3333.3333

Since all three share the same default priority (32768), the tie-breaker is the MAC address — **1111.1111.1111** is numerically lowest, so that switch wins the election and becomes the **Root Bridge**.

BPDUs (Bridge ID) contents used in the election:

- Bridge Priority
- MAC Address of the switch

5. STP Timers & BPDU Types

5.1 Key STP Timers

Timer	Default Value	Purpose
Hello Time	2 s	How often the Root Bridge sends BPDUs
Max Age	20 s	How long a switch waits before assuming the Root Bridge is gone (BPDU not heard)
Forward Delay	15 s	Time spent in the Listening state + time spent in the Learning state (15s each)

💡 Total 802.1D Convergence Time:

$\text{Max Age} + (2 \times \text{Forward Delay}) = 20 + (2 \times 15) = 50 \text{ seconds}$

5.2 BPDU Types

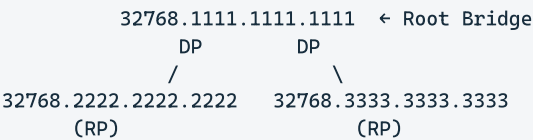
Type	Purpose
Configuration BPDU	Carries the Root BID, path cost, sender's BID, port ID, and timers. Sent every 2 seconds (Hello Time), originated by the Root Bridge and relayed downstream

Type	Purpose
TCN BPDU (Topology Change Notification)	Sent when a port changes state (e.g., link up/down); triggers the rest of the network to update its topology info faster

6. Port Roles

Once the Root Bridge is elected, every other ("non-root") switch assigns roles to its own ports:

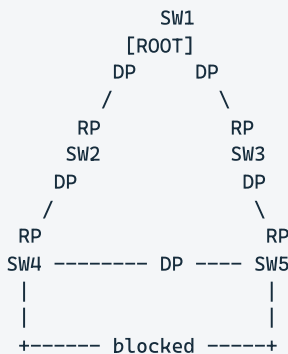
Port Role	Meaning
Root Port (RP)	The port used to reach the Root Bridge via the lowest-cost path — exactly one per non-root switch
Designated Port (DP)	The forwarding port for a given network segment/link — exactly one per link (both ends of a link can't both be DP)
Blocking / Non-Designated Port	Neither root nor designated — kept in blocking state to prevent a loop; does not forward user data



- **DP (Designated Port):** forwards traffic for its segment — the Root Bridge's ports are always DP.
- **RP (Root Port):** each non-root switch picks the port with the best path back to the Root as its Root Port.
- Any additional redundant port that is neither RP nor DP goes into **Blocking** state to eliminate the loop, but stays ready to take over if needed.

6.1 Final Simplified View — 5-Switch Example

A larger topology follows the exact same logic — every non-root switch has one Root Port, every link has one Designated Port, and any leftover redundant port is blocked:



Reading this diagram:

- **SW1** = Root Bridge → both its ports are **DP** (Root Bridge ports are always Designated).
- **SW2** and **SW3** each use their uplink to SW1 as their **Root Port (RP)** — best path to the Root.
- **SW2** → **SW4** and **SW3** → **SW5** each work the same way: SW2/SW3 side = **DP**, SW4/SW5 side = **RP**.

- **SW4 ↔ SW5** is a direct link between two non-root switches — one side becomes **DP** (forwarding), but since this link creates a **loop** back to the root via two paths, the extra connection between SW4 and SW5 (the bottom link shown) is put into **Blocking** state.
- Result: exactly **one loop-free active path** from every switch back to the Root, with the blocked link kept in reserve as an automatic backup.

7. STP Protocol Types (Variants)

Protocol	Vendor	Instances	Convergence Time	Key Trait
802.1D STP	IEEE	1 tree for ALL VLANs	30–50 s	Original STP; 5 port states
802.1w RSTP	IEEE	1 tree for ALL VLANs	1–6 s	3 states; adds Alternate/Backup port roles
802.1s MSTP	IEEE	Grouped VLANs → instances	1–6 s	Load-balances traffic per instance
PVST+	Cisco	1 tree per VLAN (802.1D-based)	Slow	Per-VLAN root bridge election
RPVST+	Cisco	1 tree per VLAN (802.1w-based)	Fast	Fastest convergence; highest resource usage

Same 3-switch topology, how each protocol treats it:

```
SW1 (Root) — SW2 — SW3  (identical physical topology for all variants below)

802.1D / 802.1w → ONE tree covers ALL VLANs
802.1s (MSTP)  → Instance 1: VLANs 10, 20 | Instance 2: VLAN 30
PVST+ / RPVST+ → A SEPARATE tree per VLAN (VLAN 10, VLAN 20, VLAN 30, ...)
```

Exam tip: MSTP groups multiple VLANs into a smaller number of instances (efficient), while PVST+/RPVST+ run a completely separate spanning tree per VLAN (more resource-intensive but allows finer per-VLAN load balancing).

8. Port States

8.1 802.1D — 5 States

```
Blocking → Listening → Learning → Forwarding
                        ↳ (or) Disabled
```

State	What Happens
Blocking	Receives BPDUs only; does not forward data or learn MAC addresses
Listening	Processes BPDUs to determine port role; still no data forwarding, no MAC learning
Learning	Starts learning MAC addresses into the table; still no data forwarding yet
Forwarding	Fully operational — forwards data and learns MAC addresses
Disabled	Administratively shut down; not participating in STP at all

8.2 802.1w RSTP — 3 States

```
Discarding → Learning → Forwarding
```

RSTP consolidates 802.1D's **Blocking + Listening + Disabled** into a single **Discarding** state, making convergence dramatically faster (1–6 seconds vs 30–50 seconds).

802.1D State	Maps To (RSTP)
Blocking	Discarding
Listening	Discarding
Disabled	Discarding
Learning	Learning
Forwarding	Forwarding

9. Quick Revision — Key Facts

STP Purpose : Prevent Layer-2 loops while keeping redundant links as backup
 Discovery mechanism: BPDU (Bridge Protocol Data Unit), multicast every 2s (Hello Time)
 Loop detection : A switch receiving its OWN BPDU back = loop exists
 Root Bridge election: LOWEST Bridge ID wins → tie broken by LOWEST MAC address
 BID structure : Priority (2 bytes) + MAC Address (6 bytes)
 Extended System ID : Priority + VLAN ID (used in PVST+/RPVST+)

Port Roles : Root Port (best path to Root) | Designated Port (forwards, 1 per link)
 | Blocking/Non-Designated Port (loop prevention)

Timers : Hello = 2s | Max Age = 20s | Forward Delay = 15s
 Convergence : Max Age + (2 × Forward Delay) = 50s (802.1D)

802.1D states (5): Blocking → Listening → Learning → Forwarding → (Disabled)
 802.1w states (3): Discarding → Learning → Forwarding

802.1D = original, slow (30–50s), 1 tree for all VLANs
 802.1w = RSTP, fast (1–6s), 1 tree for all VLANs
 802.1s = MSTP, fast, VLANs grouped into instances
 PVST+ = Cisco, 1 tree per VLAN, based on 802.1D (slow)
 RPVST+ = Cisco, 1 tree per VLAN, based on 802.1w (fastest, most resource-heavy)

STP Reference Notes

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [11 - Layer 2 Switching and Ethernet Forwarding](#)
- [05 - VLANs and Inter-VLAN Routing](#)

Source note: 08 - Spanning Tree Protocol.md

CHAPTER 09 · RESILIENCE & INFRASTRUCTURE SECURITY

Port Security, AAA & Access Control Lists (ACLs)

Port security, AAA, RADIUS, TACACS+, and access control lists in one interview-focused module.

- Topic 9
- computer networks
- port security
- aaa
- access control lists

Table of Contents

- 0. Full Forms / Abbreviations
- 0.1 Topics Covered in These Notes
- PART A — Port Security
- 1. What Is Port Security?
- 2. Violation Modes
- 3. MAC Address Learning / Types of MAC Address Entries
- 4. Aging Timers
- 5. Port Security — Full Example Diagram
- PART B — AAA Framework (Authentication, Authorization, Accounting)

- 6. What Is AAA?
- 7. RADIUS
- 8. TACACS+
- 9. RADIUS vs TACACS+
- PART C — Access Control Lists (ACLs)
- 10. ACL Types
- 11. Quick Revision — Key Facts
- Related Notes

0. Full Forms / Abbreviations

Abbreviation	Full Form
AAA	Authentication, Authorization, Accounting
RADIUS	Remote Authentication Dial-In User Service
TACACS+	Terminal Access Controller Access-Control System Plus
ACL	Access Control List
MAC	Media Access Control (address)
UDP	User Datagram Protocol
TCP	Transmission Control Protocol
VPN	Virtual Private Network
ISP	Internet Service Provider

0.1 Topics Covered in These Notes

Topic	Covered In
Standard ACL vs Extended ACL (numbered and named)	\$10
ACL placement — Standard close to destination, Extended close to source	\$10.1
Port Security — violation modes: Protect, Restrict, Shutdown	\$2
MAC Address Learning — Static, Dynamic, and why Sticky MAC is needed (with use case)	\$3
AAA — Authentication, Authorization, Accounting	\$6
TACACS+ vs RADIUS — key differences	\$9

PART A — PORT SECURITY

1. What Is Port Security?

Port security limits *which* MAC addresses are allowed to send traffic through a switch **access port** — preventing unauthorized devices (rogue laptops, rogue switches, MAC-spoofing attacks) from connecting to the network.

Normal port : any device can plug in and send traffic
 Port-security enabled : only APPROVED MAC address(es) can send traffic
 everything else = VIOLATION

2. Violation Modes

When a port-security violation occurs (an unapproved MAC tries to send traffic, or the max MAC count is exceeded), the switch reacts based on the configured **violation mode**:

Mode	Action	Logs ?	Port Stays Up?
Protect	Drops violating frames silently	✗ No	✓ Yes
Restrict	Drops frames + logs the event + increments violation counter	✓ Yes	✓ Yes
Shutdown	Err-disables the port; stops all traffic on that port (not just the violating frame). Needs to be manually re-enabled by the administrator	✓ Yes	✗ No

⚠ **Default on Cisco IOS = Shutdown.** If you don't explicitly specify a violation mode, the port will be **err-disabled** on the very first violation.

Severity ranking (mildest → harshest):

Protect < Restrict < Shutdown
 (silent) (logged) (port goes down completely)

Recovering a Shutdown (err-disabled) port:

```
A Shutdown-mode violation requires manual intervention:
Router(config-if)# shutdown
Router(config-if)# no shutdown
(or configure err-disable auto-recovery)
```

3. MAC Address Learning / Types of MAC Address Entries

A switch needs to know **which MAC address is allowed on which port**. There are three ways this gets configured:

3.1 Static MAC

Static = the administrator manually tells the switch exactly which MAC address is allowed on a port.

```
Switch(config-if)# switchport port-security mac-address aabb.cc00.0100
```

- You type the MAC address in yourself.
- It's saved in the running-config right away (and survives reload automatically, since it's a normal config line).
- Best for: a small number of known, fixed devices (e.g., a server that never changes NICs).

3.2 Dynamic MAC

Dynamic = the switch learns it automatically. You don't configure anything.

```
Switch(config-if)# switchport port-security
(no mac-address specified → switch learns whatever connects first, up to the max)
```

- The switch watches incoming frames and learns the source MAC on its own.
- **Downside:** this learned entry is **NOT saved** anywhere — it's lost the moment the switch reloads, meaning it has to re-learn from scratch (and briefly allow whatever connects next) after every reboot.

3.3 Why Is Sticky MAC Needed?

Static and Dynamic each have a real drawback:

Problem	Static MAC	Dynamic MAC
Manual effort	✗ You must type in every MAC address by hand — painful on a switch with dozens/hundreds of ports	✓ No manual effort
Survives reload	✓ Yes	✗ No — has to re-learn after every reboot

Sticky MAC exists to get the best of both: it **learns automatically** like Dynamic MAC (no manual typing), but **writes the learned MAC into the running-config** like Static MAC (so it can persist across reloads) — **provided** you save the config with `write memory`.

```
Static   : ✓ persists      ✗ manual effort
Dynamic : ✓ no effort      ✗ lost on reload
Sticky   : ✓ no effort      ✓ persists (IF you save config)
          = "best of both worlds"
```

3.4 Sticky MAC — Use Case Scenario

Scenario: You're securing 40 access ports in an office, each connected to a known employee laptop that rarely changes. You want port security enabled, but typing 40 MAC addresses by hand is slow and error-prone, and you also don't want the switch to have to blindly re-learn (and briefly trust) a new device every time it reboots.

Step 1: Enable sticky learning on each port:
 Switch(config-if)# switchport port-security mac-address sticky

Step 2: Employee laptop connects → switch DYNAMICALLY learns its MAC
 → automatically converts it into a STICKY entry in running-config

Step 3: Admin runs:
 Switch# write memory
 → sticky entries are now saved to startup-config

Result: After a reboot, the switch already "remembers" every laptop's MAC –
 no re-learning window, no manual typing of 40 addresses.

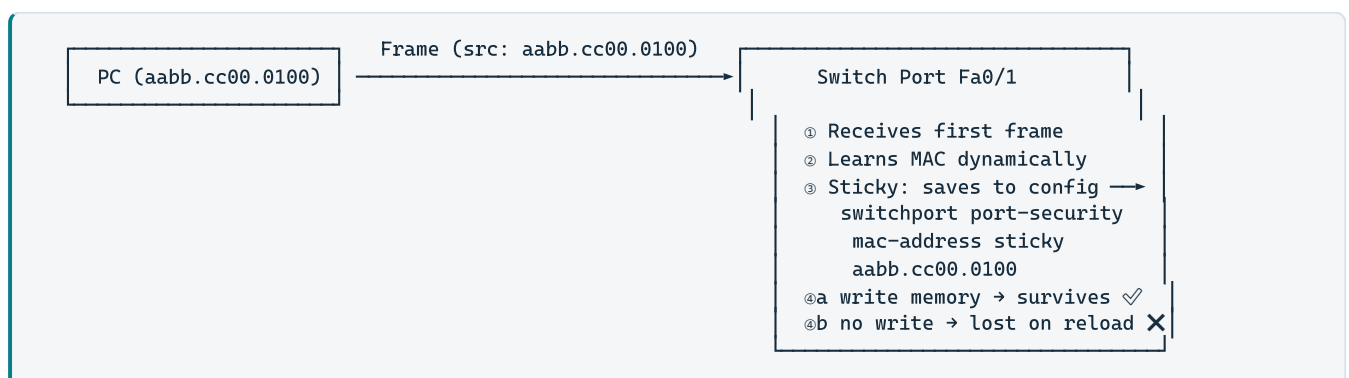
Why this matters in practice:

- Saves the admin from manually configuring every single MAC address (unlike pure Static).
- Avoids the reload-vulnerability window where a Dynamic-only port would have to re-learn (and briefly trust) a new device after every switch reboot.
- Ideal for environments where devices are relatively fixed (offices, server rooms) but the sheer number of ports makes manual static configuration impractical.

3.5 Summary Table

Type	How It's Learned	Survives Reload?
Static MAC	Manually configured by the admin	✅ Yes, automatically
Dynamic MAC	Learned automatically from incoming traffic	❌ No — lost on reload
Sticky MAC	Learned automatically (like dynamic) but automatically written into the running-config	✅ Only if <code>write memory</code> (saved to startup-config) is run — otherwise lost on reload

3.6 Sticky MAC — Step-by-Step Flow



Key takeaway: Sticky MAC gives you the *convenience* of dynamic learning with the *persistence* of static configuration — but only if you remember to save the config.

4. Aging Timers

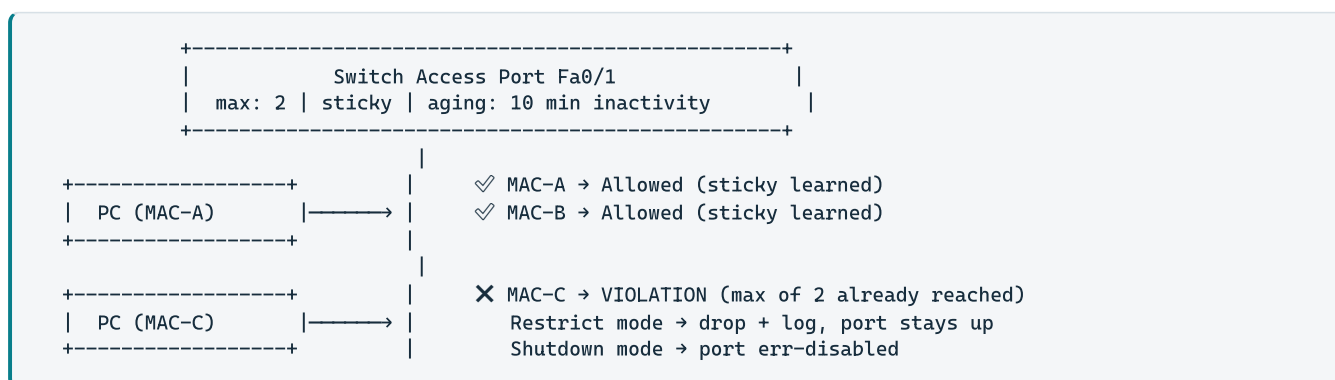
Aging timers control **how long** a dynamically learned secure MAC address stays bound to a port. **By default, secure MACs never age out.**

Type	Behaviour
Absolute	Timer starts the moment the MAC is learned; the MAC is removed when the timer expires — even if the device is still actively sending traffic
Inactivity	Timer resets on every frame received from that MAC; the MAC is only removed after a period of no traffic (idle time)

i Static MACs never age out. Only **dynamic** and **sticky** MACs are subject to aging timers.

Absolute timer : |---timer running---| → MAC removed at expiry, REGARDLESS of activity
 Inactivity timer: frame...frame...frame...[idle gap = timer]→ MAC removed only after idle period

5. Port Security — Full Example Diagram



Reading this example:

- Port Fa0/1 is configured for a **max of 2** MAC addresses, using **sticky** learning, with a **10-minute inactivity** aging timer.
- MAC-A and MAC-B are learned first and allowed (sticky-learned, within the max of 2).
- MAC-C attempts to connect — but the port has already reached its max of 2 allowed MACs → **violation**.
- What happens next depends on the configured **violation mode** (§2): silently dropped (Protect), dropped+logged (Restrict), or the whole port goes down (Shutdown).

PART B — AAA FRAMEWORK (AUTHENTICATION, AUTHORIZATION, ACCOUNTING)

6. What Is AAA?

Function	Question Answered	Examples
Authentication	"Who are you?"	Username/password, certificates, tokens
Authorization	"What can you do?"	Exec-privilege level, which commands are allowed
Accounting	"What did you do?"	Session logs, command history, session duration

AAA Flow:

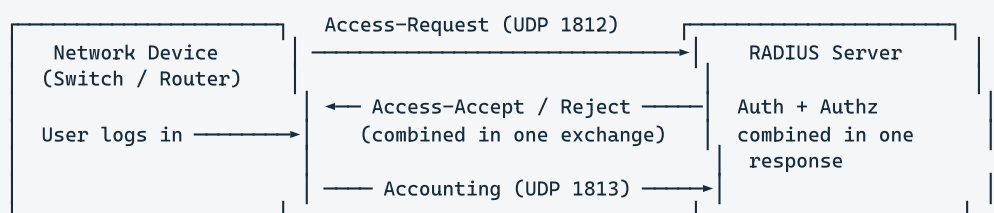
1. Authentication → prove identity (login)
2. Authorization → determine permitted actions
3. Accounting → record what was actually done

AAA is implemented using one of two main protocols: **RADIUS** or **TACACS+**.

7. RADIUS

RADIUS = Remote Authentication Dial-In User Service. An **open standard**, client-server AAA protocol.

- **Transport:** UDP
- **Ports:** 1812 (Authentication) and 1813 (Accounting)
- **Key trait:** Authentication + Authorization are **combined** into a single exchange (not separated)

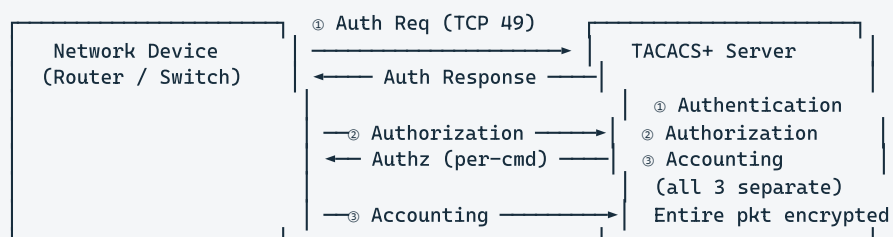


Common use cases: Wireless authentication, VPN access, ISP subscriber authentication.

8. TACACS+

TACACS+ = Terminal Access Controller Access-Control System Plus. A **Cisco-enhanced/proprietary** AAA protocol.

- **Transport:** TCP
- **Port:** 49
- **Key trait:** Authentication, Authorization, and Accounting are **fully separated** into three independent exchanges — enabling **per-command authorization**



Common use case: Device administration (Cisco environments) — e.g., controlling exactly which CLI commands a network admin is allowed to run.

9. RADIUS vs TACACS+

Feature	RADIUS	TACACS+
Standard	Open (RFC 2865)	Cisco proprietary
Transport	UDP	TCP
Ports	1812 / 1813	49
Separates A/A/A?	✗ No (Auth + Authz combined)	✓ Yes (all three fully separate)
Per-command authorization	✗ Limited	✓ Full support
Encrypts packet body?	✗ Password only	✓ Entire packet
Common use	Wireless, VPN, ISP	Device administration (Cisco)

Exam one-liners:

- RADIUS = UDP, open standard, combines Auth+Authz, encrypts only the password.
- TACACS+ = TCP, Cisco proprietary, fully separates AAA, encrypts the entire packet, supports per-command control.
- Choose **TACACS+** for fine-grained device-administration control; choose **RADIUS** for general network access (Wi-Fi, VPN, ISP).

PART C — ACCESS CONTROL LISTS (ACLs)

10. ACL Types

Type	Filters By	Number Range	Placement
Standard ACL	Source IP only	1–99, 1300–1999	Close to the destination
Extended ACL	Source/Destination IP, protocol, port	100–199, 2000–2699	Close to the source
Named ACL	Same filtering as Standard/Extended, but identified by name instead of number	N/A	Same placement rules as its Standard/Extended equivalent

10.1 ACL Configuration Commands

A. Standard ACL (Numbered) — filters by source IP only

Step 1 – Create the ACL:

```
Router(config)# access-list 10 permit 192.168.1.0 0.0.0.255
Router(config)# access-list 10 deny any
```

Step 2 – Apply it to an interface (close to the DESTINATION):

```
Router(config)# interface g0/1
Router(config-if)# ip access-group 10 out
```

Part	Meaning
<code>access-list 10</code>	ACL number 10 → falls in the Standard range (1–99)
<code>permit 192.168.1.0 0.0.0.255</code>	Allow traffic sourced from the 192.168.1.0/24 network (wildcard mask — see earlier notes)
<code>ip access-group 10 out</code>	Apply ACL 10 to traffic leaving this interface

B. Extended ACL (Numbered) — filters by source/dest IP, protocol, port

Step 1 – Create the ACL:

```
Router(config)# access-list 110 permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 80
Router(config)# access-list 110 deny ip any any
```

Step 2 – Apply it to an interface (close to the SOURCE):

```
Router(config)# interface g0/0
Router(config-if)# ip access-group 110 in
```

Part	Meaning
<code>access-list 110</code>	ACL number 110 → falls in the Extended range (100–199)
<code>permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 80</code>	Allow TCP traffic from 192.168.1.0/24 to host 10.0.0.5, destination port 80 only
<code>ip access-group 110 in</code>	Apply ACL 110 to traffic entering this interface

C. Named ACL — same logic, human-readable name instead of a number

Standard Named ACL:

```
Router(config)# ip access-list standard BLOCK-SALES
Router(config-std-nacl)# deny 192.168.10.0 0.0.0.255
Router(config-std-nacl)# permit any
```

Extended Named ACL:

```
Router(config)# ip access-list extended ALLOW-WEB
Router(config-ext-nacl)# permit tcp any host 10.0.0.5 eq 443
Router(config-ext-nacl)# deny ip any any
```

Apply (same as numbered):

```
Router(config-if)# ip access-group BLOCK-SALES in
Router(config-if)# ip access-group ALLOW-WEB in
```

Exam one-liners:

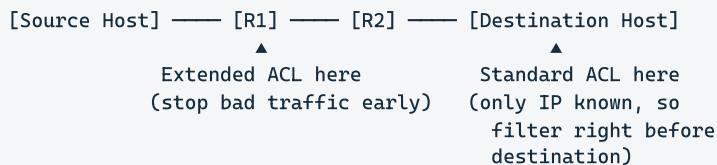
- Standard ACL syntax: `access-list <1-99> {permit|deny} <source-ip> <wildcard-mask>`
- Extended ACL syntax: `access-list <100-199> {permit|deny} <protocol> <source> <destination> [eq port]`
- `ip access-group <ACL> {in|out}` binds the ACL to an interface, in a specific direction.
- Named ACLs use `ip access-list {standard|extended} <name>` instead of a number, then enter a sub-mode to add `permit` / `deny` lines.

10.2 Why Placement Matters

Standard ACL (source IP only) → place NEAR the DESTINATION

Reason: it can only match source IP, so placing it near the source would block traffic from reaching OTHER destinations it shouldn't affect.

Extended ACL (full 5-tuple match) → place NEAR the SOURCE
 Reason: it can match precisely what it needs to block, so it's safe
 (and more efficient) to stop unwanted traffic as early as possible.



10.3 Implicit Deny

⚠ **Implicit Deny:** Every ACL ends with an **invisible deny any** statement. If a packet doesn't match **any** configured rule, it is **automatically dropped**.

ACL processing order:

```

Rule 1 → match? → apply action, STOP
Rule 2 → match? → apply action, STOP
...
Rule N → match? → apply action, STOP
[implicit deny any] → no match found anywhere → DROP
  
```

Exam tip: If you want to allow general traffic through an ACL, you must explicitly add a **permit** statement — otherwise the implicit deny silently blocks everything not explicitly permitted.

11. Quick Revision — Key Facts

PORT SECURITY

```

Violation modes : Protect (silent drop) < Restrict (drop+log) < Shutdown (port down)
Default mode    : Shutdown
MAC types       : Static (manual, persists) | Dynamic (auto, lost on reload)
                  | Sticky (auto-learned, persists ONLY with write memory)
Aging types     : Absolute (fixed timer, ignores activity)
                  | Inactivity (resets on traffic, removes only when idle)
Static MACs     : NEVER age out
  
```

AAA

```

Authentication : Who are you?
Authorization   : What can you do?
Accounting      : What did you do?
  
```

```

RADIUS : UDP | ports 1812/1813 | open standard | Auth+Authz combined
        | encrypts password only | used for Wi-Fi/VPN/ISP
TACACS+ : TCP | port 49 | Cisco proprietary | Auth/Authz/Accounting fully separate
        | encrypts entire packet | per-command authorization | device admin
  
```

ACLs

```

Standard ACL : source IP only | 1-99, 1300-1999 | place near DESTINATION
Extended ACL : src+dst IP, protocol, port | 100-199, 2000-2699 | place near SOURCE
Named ACL    : same rules, uses a name instead of a number
Implicit Deny: every ACL ends with a hidden "deny any" - unmatched traffic is dropped
  
```

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [04A - Network Routing Fundamentals](#)
- [11 - Layer 2 Switching and Ethernet Forwarding](#)
- [05 - VLANs and Inter-VLAN Routing](#)
- [06 - Network Address Translation](#)

Source note: 09 - Infrastructure Security ACL AAA and Port Security.md

PART IV

WAN, Switching & Router Operations

Connect distant networks, understand Ethernet forwarding, and operate Cisco routers safely and confidently.

[10 · PPP & WAN Technologies](#)[11 · Layer 2 Switching](#)[13 · Router IOS & Management](#)

CHAPTER 10 · WAN, SWITCHING & ROUTER OPERATIONS

10. PPP and WAN Technologies

PPP, MLPPP, PPPoE, GRE, MPLS, VPNs, authentication, tunneling, and WAN comparisons.

Topic 10

computer networks

wan

ppp

interview preparation

Table of Contents

- 10.1 WAN Basics
- 10.2 PPP — Point-to-Point Protocol
- 10.3 PPP Connection Process
- 10.4 PAP — Password Authentication Protocol
- 10.5 CHAP — Challenge Handshake Authentication Protocol
- 10.6 PAP vs CHAP
- 10.7 MLPPP — Multilink PPP
- 10.8 Why MLPPP is Needed
- 10.9 MLPPP Working
- 10.10 MLPPP Full Flow
- 10.11 Benefits of MLPPP
- 10.12 MLPPP Requirements
- 10.13 MLPPP vs Normal PPP
- 10.14 MLPPP Example
- 10.15 PPPoE — Point-to-Point Protocol over Ethernet
- 10.16 Why PPPoE Exists
- 10.17 PPPoE Example
- 10.18 PPPoE Phases
- 10.19 PPPoE Authentication
- 10.20 Important PPPoE Ports / EtherTypes
- 10.21 PPP vs PPPoE
- 10.22 GRE Tunneling
- 10.23 What is Tunneling?
- 10.24 GRE Tunnel Example
- 10.25 GRE Packet Structure
- 10.26 GRE Source and Destination
- 10.27 Important Feature of GRE
- 10.28 Does GRE Provide Encryption?
- 10.29 GRE + IPsec
- 10.30 GRE Advantages
- 10.31 GRE Limitations
- 10.32 GRE Interview-Ready Answer
- 10.33 MPLS — Multiprotocol Label Switching
- 10.34 Traditional IP Routing
- 10.35 MPLS Forwarding
- 10.36 MPLS Basic Devices
- 10.37 MPLS Flow
- 10.38 Benefits of MPLS
- 10.39 MPLS Does Not Automatically Mean Encryption
- 10.40 Traditional WAN
- 10.41 MPLS vs Traditional Dedicated WAN
- 10.42 MPLS vs Internet VPN
- 10.43 MPLS Interview-Ready Answer
- 10.44 VPN Fundamentals
- 10.45 Main Security Features of a VPN
- 10.46 Site-to-Site VPN
- 10.47 Remote-Access VPN
- 10.48 Site-to-Site vs Remote-Access VPN
- 10.49 VPN Communication Flow
- 10.50 VPN Inner and Outer Packets
- 10.51 Full Tunnel vs Split Tunnel
- 10.52 GRE vs VPN
- 10.53 PPP, MLPPP, PPPoE, GRE, MPLS and VPN Comparison
- 10.54 Complete WAN Technology Overview
- 10.55 Common Interview Questions
- 10.56 Quick Revision Sheet
- Related Notes

This section covers important WAN technologies used to connect devices, offices, branches, and remote users over long-distance networks.

Main topics:

- PPP
- PAP vs CHAP
- MLPPP
- PPPoE
- GRE tunneling

- MPLS vs traditional WAN
- VPN fundamentals

10.1 WAN Basics

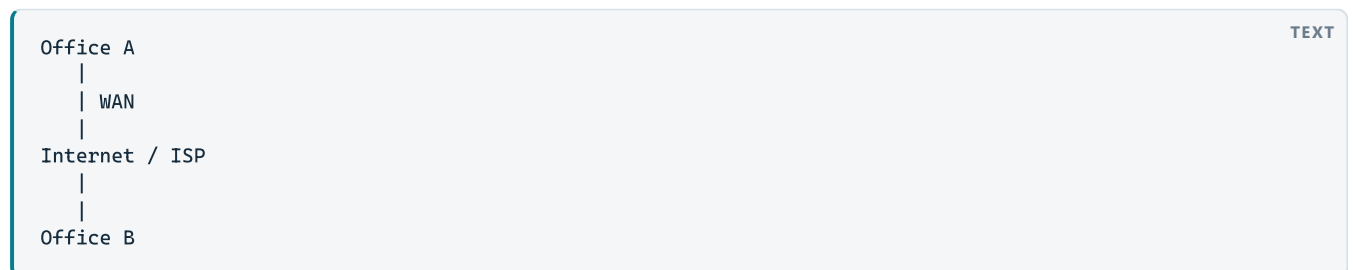
What is a WAN?

WAN (Wide Area Network) is a network that connects devices or networks over a large geographical area.

Examples:

- Connecting two company branches in different cities
- Connecting an office to a data center
- Connecting a home user to an ISP
- Connecting multiple corporate sites

Example:



A WAN can use technologies such as:

- PPP
- MPLS
- VPN
- leased lines
- broadband
- Metro Ethernet

10.2 PPP — Point-to-Point Protocol

What is PPP?

PPP stands for Point-to-Point Protocol.

It is a **Layer 2 protocol** used to create a direct communication link between two devices.

It was widely used on:

- Serial WAN links
- Dial-up connections
- Router-to-router links
- DSL connections through PPPoE

Example:

```

Router A
  |
  | PPP
  |
Router B

```

Main Functions of PPP

PPP provides:

- Link establishment
- Link configuration
- Authentication
- Error detection
- Support for multiple Layer 3 protocols
- Connection termination

Important components:

Component	Full Form	Purpose
LCP	Link Control Protocol	Establishes and manages PPP link
PAP	Password Authentication Protocol	Basic authentication
CHAP	Challenge Handshake Authentication Protocol	More secure authentication
NCP	Network Control Protocol	Configures Layer 3 protocols

10.3 PPP Connection Process

PPP works approximately like this:

```

Two Devices
  ↓
Physical Link Available
  ↓
LCP Negotiation
  ↓
Authentication
  ↓
PAP or CHAP
  ↓
NCP Configuration
  ↓
IP Communication

```

10.4 PAP — Password Authentication Protocol

What is PAP?

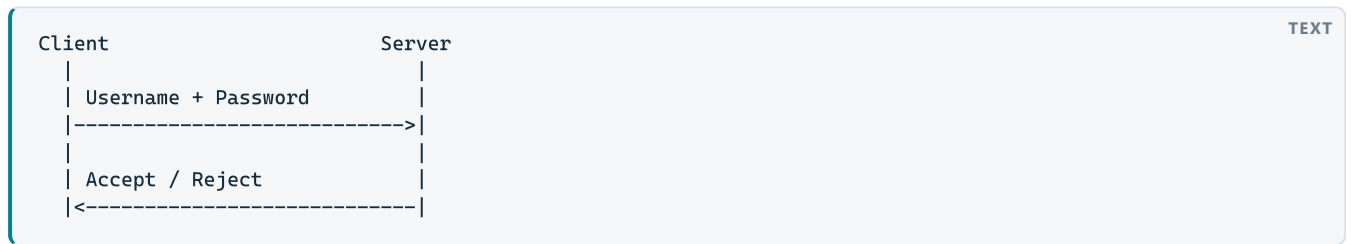
PAP is a simple PPP authentication method.

The client sends:

- Username
- Password

to the authenticator.

PAP uses a **2-way handshake**.



PAP Problem

The major weakness is that credentials are sent in a form that can be exposed if the traffic is intercepted.

Therefore:

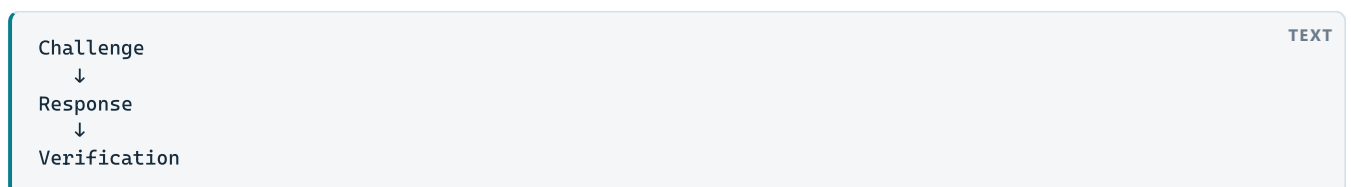
PAP provides weak authentication.

10.5 CHAP — Challenge Handshake Authentication Protocol

What is CHAP?

CHAP is a stronger PPP authentication protocol than PAP.

Instead of directly sending the password, CHAP uses a:

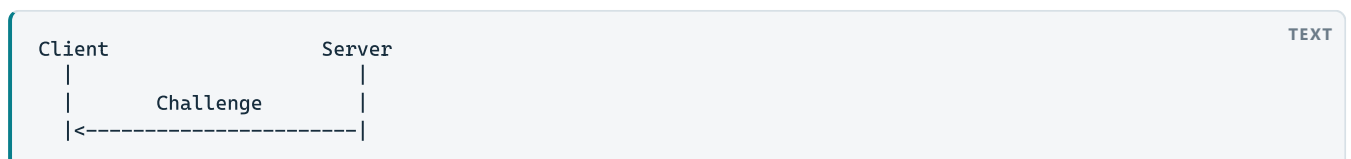


process.

CHAP uses a **3-way handshake**.

CHAP Working

Step 1: Server sends a challenge



The challenge contains a changing/random value.

Step 2: Client calculates a response

The client combines information such as:

```
Challenge
+
Shared Secret
+
Identifier
```

TEXT

Then creates a hash response.

```
Challenge + Secret
↓
Hash
↓
Response
```

TEXT

Step 3: Server verifies it

```
Client                                Server
|                                     |
| Hash Response                       |
|----->                             |
| Success / Failure                   |
|<-----                             |
```

TEXT

The actual password is not sent across the connection.

10.6 PAP vs CHAP

Feature	PAP	CHAP
Full Form	Password Authentication Protocol	Challenge Handshake Authentication Protocol
Handshake	2-way	3-way
Password sent directly	Yes	No
Challenge-response	No	Yes
Security	Weak	Better
Replay protection	Weak	Better
Reauthentication	Usually no	Can be performed
Preferred	No	Better than PAP

Interview-ready answer

PAP sends the username and password to the server and uses a two-way handshake. CHAP uses a three-way challenge-response process and does not transmit the actual password. Therefore, CHAP is more secure than PAP.

10.7 MLPPP — Multilink PPP

What is MLPPP?

MLPPP stands for Multilink Point-to-Point Protocol.

It allows multiple physical PPP links to be combined into **one logical connection**.

This is called:

Link aggregation

Example:

Suppose there are two WAN links:

```
Link 1 = 2 Mbps
Link 2 = 2 Mbps
```

TEXT

MLPPP can logically combine them:

```
2 Mbps + 2 Mbps
    ↓
Logical MLPPP Bundle
    ↓
Approximately 4 Mbps total capacity
```

TEXT

10.8 Why MLPPP is Needed

A single WAN link may not provide enough bandwidth.

Instead of replacing it with one larger circuit, multiple links can be combined.

Example:

```
Router A                                Router B
|----- PPP Link 1 -----|
|----- PPP Link 2 -----|
|----- PPP Link 3 -----|
```

TEXT

MLPPP combines them as:

```
MLPPP Bundle
Router A ===== Router B
```

TEXT

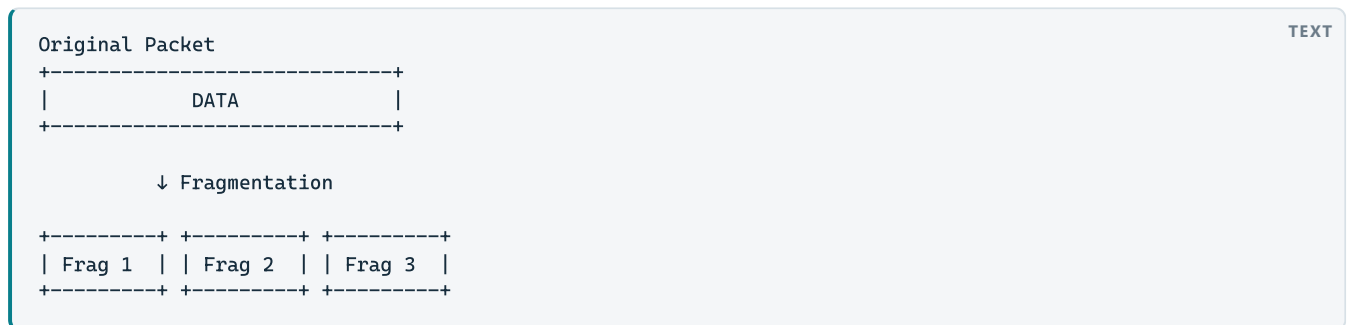
From a logical point of view, the multiple links behave like one connection.

10.9 MLPPP Working

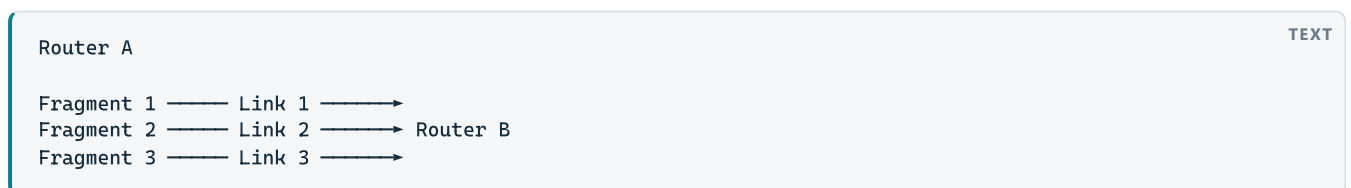
Suppose Router A wants to send a large packet.

Instead of sending the complete packet over one link, MLPPP can divide it into smaller fragments.

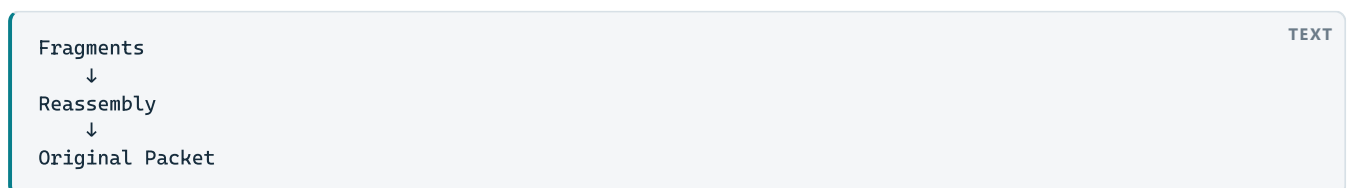
Example:



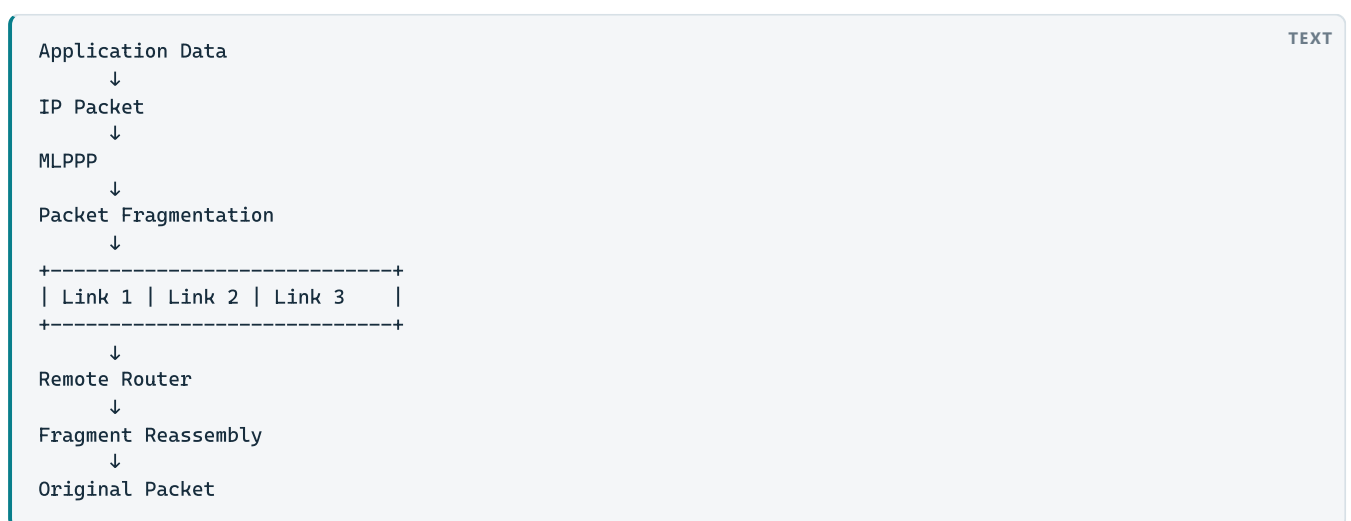
Fragments can be transmitted across different links.



Router B then rebuilds them.



10.10 MLPPP Full Flow



10.11 Benefits of MLPPP

1. Increased Bandwidth

Multiple links provide more total capacity.

Example:

```
Link 1 = 10 Mbps
Link 2 = 10 Mbps

Total logical capacity ≈ 20 Mbps
```

TEXT

2. Load Distribution

Traffic can be distributed across multiple physical links.

```
Traffic
  ↓
MLPPP
 /  \
L1   L2
```

TEXT

3. Better Link Utilization

Instead of leaving one link overloaded and another underused, traffic can be spread over the bundle.

4. Redundancy

If one link fails, the remaining links may continue carrying traffic.

Example:

```
Before Failure

Link 1 ✓
Link 2 ✓
Link 3 ✓
```

TEXT

If Link 2 fails:

```
Link 1 ✓
Link 2 ✗
Link 3 ✓
```

TEXT

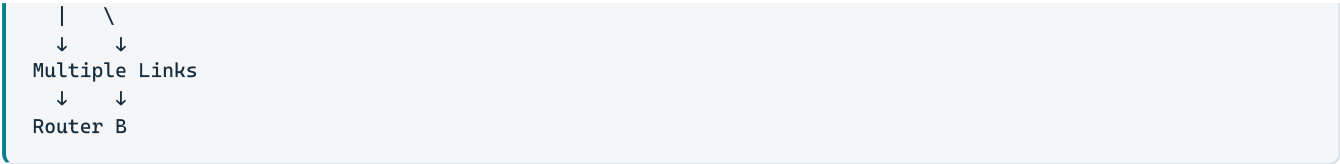
Communication may continue with reduced bandwidth.

10.12 MLPPP Requirements

Normally, the links in an MLPPP bundle should terminate between the same two logical endpoints.

```
Router A
 |  \
 |   \
```

TEXT



Both ends must support MLPPP.

10.13 MLPPP vs Normal PPP

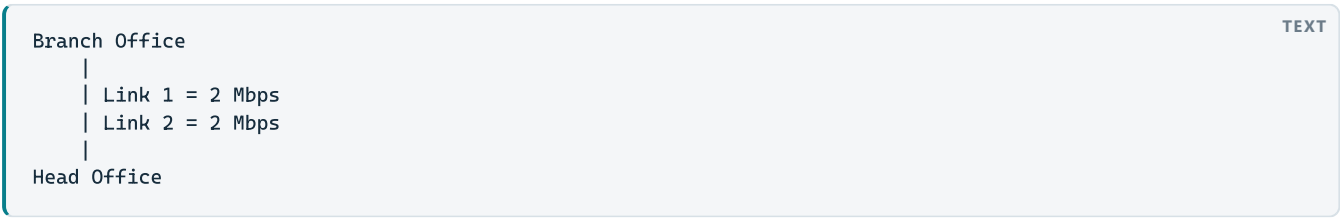
Feature	PPP	MLPPP
Number of links	Usually one	Multiple
Logical connection	One physical link	Bundle of links
Bandwidth	Limited to one link	Combined capacity
Fragmentation across links	No	Yes
Redundancy	Limited	Better
Load sharing	No	Yes

Interview-ready answer

MLPPP stands for Multilink PPP. It combines multiple PPP WAN links into one logical bundle. It increases available bandwidth and can distribute packet fragments over several links. If one member link fails, communication may continue using the remaining links with lower bandwidth.

10.14 MLPPP Example

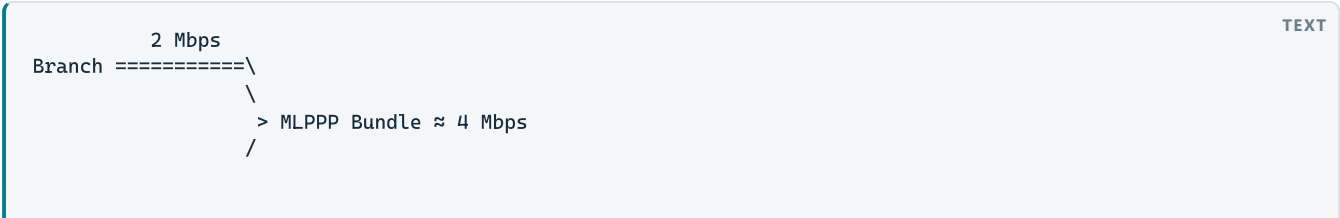
Suppose a company has:



Without MLPPP:

Each connection works separately.

With MLPPP:



```
Branch =====/
          2 Mbps
```

This gives more usable total bandwidth between the two routers.

10.15 PPPoE — Point-to-Point Protocol over Ethernet

What is PPPoE?

PPPoE stands for Point-to-Point Protocol over Ethernet.

It allows PPP communication to run over an Ethernet network.

It has been widely used by ISPs for:

- DSL broadband
- Subscriber authentication
- User session management
- Accounting

10.16 Why PPPoE Exists

Normal Ethernet does not provide traditional PPP-style user authentication by itself.

ISPs wanted to use PPP features such as:

- Username/password authentication
- Session management
- Customer identification
- Accounting

while using Ethernet infrastructure.

So PPP was carried inside Ethernet.

```
PPP
↓
PPPoE
↓
Ethernet
```

TEXT

10.17 PPPoE Example

Suppose a home user has:

```
PC / Router
↓
DSL Modem
↓
ISP
```

TEXT

The router may have credentials such as:

```
Username: customer123
Password: *****
```

TEXT

These credentials are used to create the PPP session.

Flow:

```
Home Router
  ↓
PPPoE Session
  ↓
DSL / Ethernet Access Network
  ↓
ISP Access Server
  ↓
Authentication
  ↓
Internet Access
```

TEXT

10.18 PPPoE Phases

PPPoE mainly has two phases:

1. **Discovery phase**
2. **Session phase**

Discovery Phase

The client searches for an available PPPoE server.

A simplified flow:

```
Client
  |
  | PADI
  ↓
Access Concentrator
  |
  | PADO
  ↓
Client
  |
  | PADR
  ↓
Access Concentrator
  |
  | PADS
  ↓
Session Established
```

TEXT

Common terms:

- **PADI** – PPPoE Active Discovery Initiation
- **PADO** – PPPoE Active Discovery Offer
- **PADR** – PPPoE Active Discovery Request

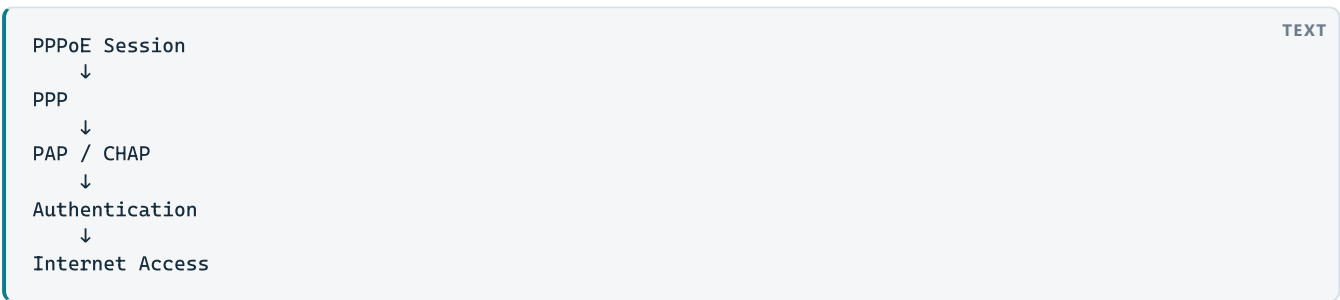
- **PADS** – PPPoE Active Discovery Session-confirmation

For many interviews, remembering the full message names is optional unless PPPoE is specifically asked in detail.

10.19 PPPoE Authentication

After the PPPoE session is created, PPP authentication can be used.

Example:



10.20 Important PPPoE Ports / EtherTypes

PPPoE does not use normal TCP/UDP port numbers for its basic operation.

It uses Ethernet protocol identifiers:

PPPoE Type	EtherType
Discovery	0x8863
Session	0x8864

10.21 PPP vs PPPoE

Feature	PPP	PPPoE
Full Form	Point-to-Point Protocol	PPP over Ethernet
Layer	Layer 2	Layer 2
Used over	Point-to-point links	Ethernet
Authentication	PAP/CHAP	PAP/CHAP through PPP
Common Use	Serial WAN	DSL / ISP broadband

Interview-ready answer

PPPoE stands for Point-to-Point Protocol over Ethernet. It allows ISPs to use PPP features such as authentication, session control, and accounting over Ethernet networks. It is commonly associated with DSL broadband connections.

10.22 GRE Tunneling

What is GRE?

GRE stands for Generic Routing Encapsulation.

GRE is a tunneling protocol used to carry one network packet inside another IP packet.

Think of it as:

```
Original Packet
  ↓
Put inside GRE packet
  ↓
Send across IP network
  ↓
Remove GRE header
  ↓
Original Packet recovered
```

TEXT

10.23 What is Tunneling?

Tunneling means:

Encapsulating one packet inside another packet so it can travel across another network.

Example:

```
Original Packet

[IP Header][Data]

  ↓ GRE Encapsulation

[Outer IP][GRE][Original IP][Data]
```

TEXT

10.24 GRE Tunnel Example

Suppose:

```
Branch A LAN
10.1.0.0/16

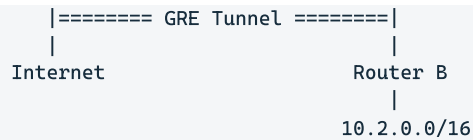
Branch B LAN
10.2.0.0/16
```

TEXT

The branches communicate across the Internet.

```
10.1.0.0/16
  |
Router A
  |
```

TEXT



10.25 GRE Packet Structure

Simplified packet:

```

+-----+
| Outer IP Header |
+-----+
| GRE Header      |
+-----+
| Original IP Header |
+-----+
| Original Data    |
+-----+
  
```

TEXT

10.26 GRE Source and Destination

The **outer IP header** contains the tunnel endpoint addresses.

Example:

```

Router A Public IP = 203.0.113.10
Router B Public IP = 198.51.100.20
  
```

TEXT

Outer packet:

```

Source      = 203.0.113.10
Destination = 198.51.100.20
  
```

TEXT

Inside it:

```

Original Source      = 10.1.1.10
Original Destination = 10.2.1.20
  
```

TEXT

10.27 Important Feature of GRE

GRE can carry different protocols through an IP network.

It can also support multicast traffic.

That makes it useful with routing protocols such as:

- OSPF
- EIGRP

in certain tunnel designs.

10.28 Does GRE Provide Encryption?

No.

This is very important.

GRE provides:

Encapsulation	✓
Tunneling	✓
Encryption	X

TEXT

Therefore GRE by itself does **not provide confidentiality**.

An attacker capable of reading the tunnel traffic may still inspect the payload.

10.29 GRE + IPsec

To provide security, GRE has traditionally been combined with IPsec.

```

Original Traffic
  ↓
GRE Tunnel
  ↓
IPsec Encryption
  ↓
Internet
  
```

TEXT

So:

- GRE provides tunneling
- IPsec provides encryption, integrity, and authentication

Easy memory:

GRE creates the tunnel; IPsec secures the tunnel.

10.30 GRE Advantages

- Simple tunneling
- Can carry various network protocols
- Supports multicast
- Useful for routing protocols across tunnels
- Can connect remote networks logically

10.31 GRE Limitations

- Does not provide encryption
- Adds additional headers
- Creates packet overhead
- Can cause MTU/fragmentation issues
- Requires separate security such as IPsec when confidentiality is needed

10.32 GRE Interview-Ready Answer

GRE stands for Generic Routing Encapsulation. It creates a logical tunnel by encapsulating the original packet inside a GRE packet and then an outer IP packet. GRE itself does not provide encryption, so it is often combined with IPsec when secure communication is required.

10.33 MPLS — Multiprotocol Label Switching

What is MPLS?

MPLS stands for Multiprotocol Label Switching.

It is a technology commonly used in service-provider networks to forward traffic using **labels**.

Instead of making every forwarding decision only from the destination IP address, an MPLS network can use a short label.

Simplified idea:

```

IP Packet
↓
Add MPLS Label
↓
Provider Network
↓
Forward Using Label
↓
Remove Label
↓
Destination Network
  
```

TEXT

10.34 Traditional IP Routing

In normal IP routing:

```

Packet
↓
Router receives packet
↓
Check Destination IP
↓
  
```

TEXT

```

Check Routing Table
↓
Find Next Hop
↓
Forward

```

This process happens across routers along the path.

10.35 MPLS Forwarding

With MPLS:

```

Incoming Packet
↓
MPLS Label Added
↓
Label = 100
↓
Provider Router
↓
Forward Based on Label

```

TEXT

Labels may change at each step.

Example:

```

Label 100
↓
Label 250
↓
Label 70
↓
Destination

```

TEXT

This process is called **label switching**.

10.36 MPLS Basic Devices

Important terminology:

PE Router

Provider Edge Router

It sits at the edge of the MPLS provider network and connects customers to the provider.

P Router

Provider Router

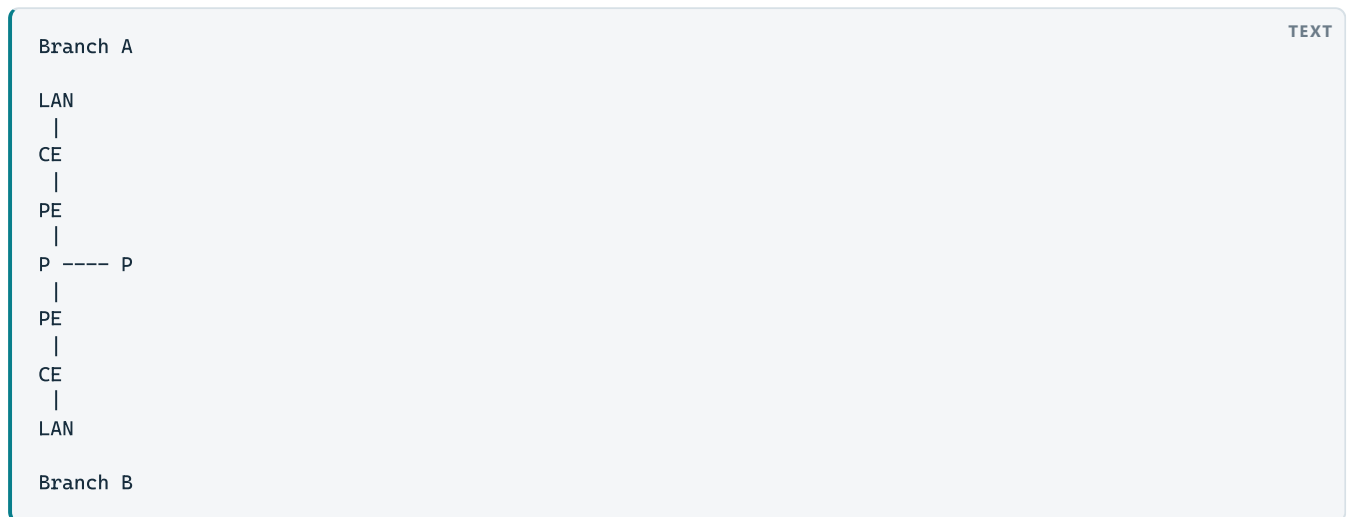
It works inside the MPLS core.

CE Router

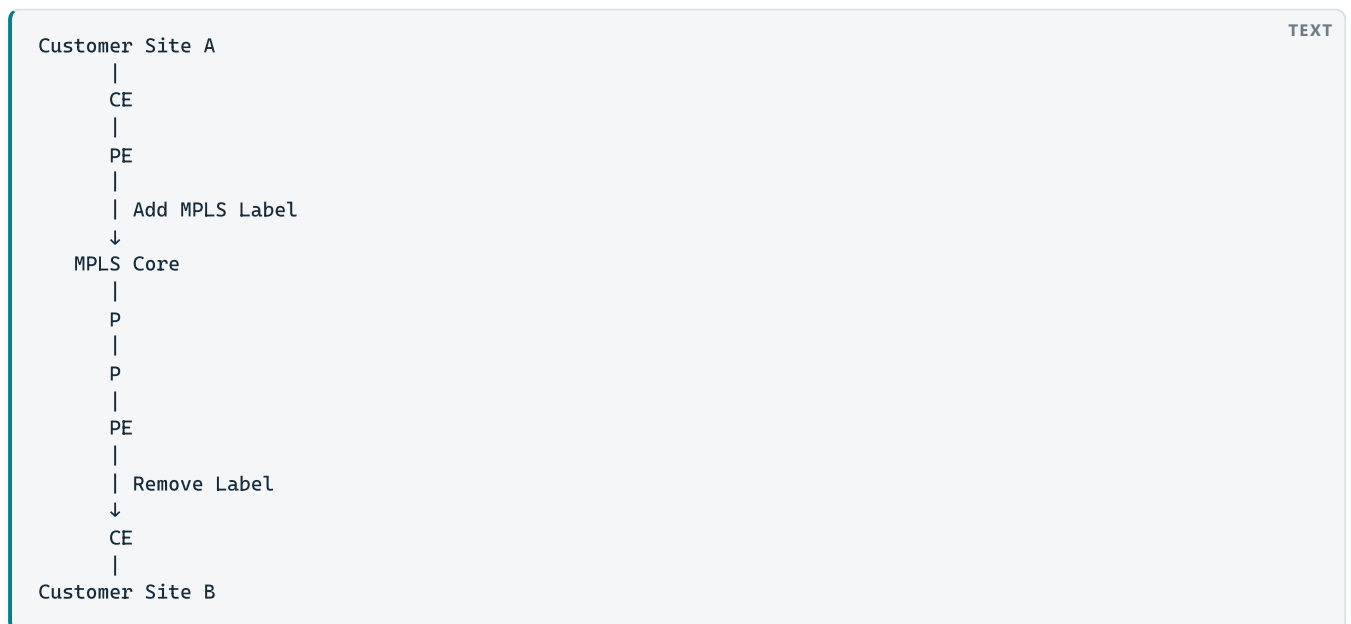
Customer Edge Router

It belongs to or serves the customer network and connects to the provider.

Example:



10.37 MPLS Flow



10.38 Benefits of MPLS

MPLS has historically been popular for enterprise WANs because it can provide:

- Predictable routing
- Traffic engineering
- Quality of Service
- Private provider-managed connectivity
- Support for many branches
- Good performance for voice and video

10.39 MPLS Does Not Automatically Mean Encryption

This is an important security point.

MPLS can provide traffic separation and private routing within a service provider network.

But:

MPLS itself does not automatically encrypt customer traffic.

If encryption is required, technologies such as IPsec can be added.

10.40 Traditional WAN

A traditional enterprise WAN could use:

- Leased lines
- Serial links
- Frame Relay historically
- ATM historically
- MPLS services
- Dedicated provider circuits

Example:

```
Office A
  |
Dedicated WAN
  |
Office B
```

TEXT

A major limitation is that dedicated private WAN services can be expensive.

10.41 MPLS vs Traditional Dedicated WAN

Feature	MPLS	Traditional Dedicated Link
Architecture	Provider label-switched network	Direct/dedicated circuits
Scalability	High	More difficult as sites increase
QoS	Strong support	Depends on service
Traffic engineering	Strong	More limited
Provider managed	Usually yes	Usually yes
Encryption by default	No	No
Cost	Can be high	Often high
Multi-site networking	Good	Can require many circuits

10.42 MPLS vs Internet VPN

This comparison is also important.

Feature	MPLS	Internet VPN
Underlying network	Service provider private network	Public Internet
Encryption	Not inherently	Commonly encrypted
Cost	Usually higher	Usually lower
Performance	More predictable	Depends on Internet
QoS	Better provider-level control	Limited across Internet
Deployment	Provider dependent	Easier
Security	Traffic separation	Encryption + authentication

10.43 MPLS Interview-Ready Answer

MPLS stands for Multiprotocol Label Switching. It is mainly used in provider networks to forward traffic using labels instead of relying only on destination IP lookups. MPLS is scalable and supports QoS and traffic engineering. However, MPLS does not inherently encrypt traffic.

10.44 VPN Fundamentals

What is a VPN?

VPN stands for Virtual Private Network.

A VPN creates a secure logical connection across an untrusted network such as the Internet.

Its main purpose is to protect communication between two endpoints.

Example:

```

User
|
| Encrypted VPN Tunnel
|
Internet
|
Company Network
  
```

TEXT

10.45 Main Security Features of a VPN

A secure VPN usually provides:

Confidentiality

Traffic is encrypted.

```

Readable Data
  ↓
Encryption
  ↓
Unreadable Ciphertext
  
```

TEXT

Integrity

Detects whether data was modified during transmission.

Authentication

Verifies the identity of the VPN endpoints or users.

10.46 Site-to-Site VPN

A **site-to-site VPN** connects entire networks.

Example:

```

Branch Office                                Head Office
10.1.0.0/16                                10.2.0.0/16
  |                                         |
Router A                                Router B
  |                                         |
|===== Encrypted VPN Tunnel =====|
                Internet
  
```

TEXT

Individual computers normally do not need to create separate VPN connections.

The routers/firewalls handle the VPN.

Site-to-Site VPN Use Case

A company has:

- Delhi office
- Mumbai office

Each location has its own LAN.

VPN gateways create a tunnel:

```

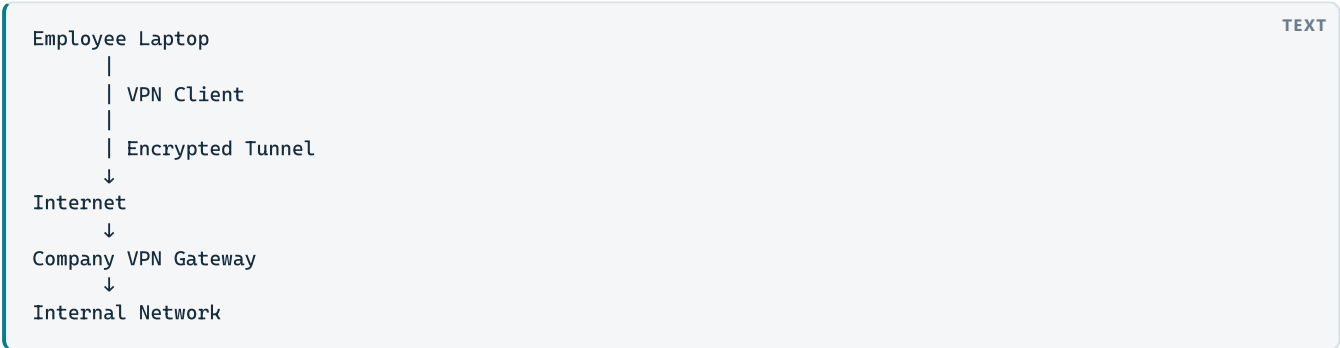
Delhi LAN
  ↓
Delhi Firewall
  ↓
=====
IPsec VPN Tunnel
=====
  ↓
Mumbai Firewall
  ↓
Mumbai LAN
  
```

TEXT

10.47 Remote-Access VPN

A **remote-access VPN** connects one individual user to an organization's network.

Example:



Common users:

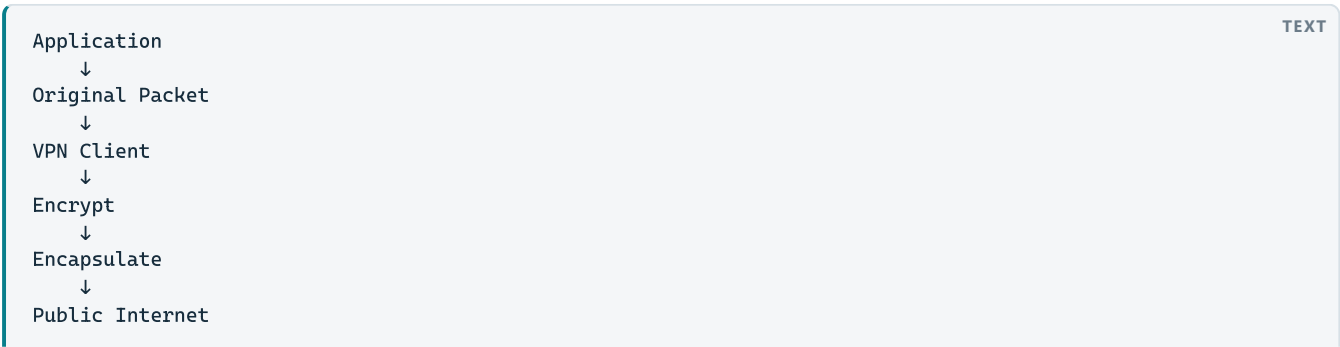
- Work-from-home employees
- Traveling employees
- Administrators
- Contractors

10.48 Site-to-Site vs Remote-Access VPN

Feature	Site-to-Site VPN	Remote-Access VPN
Connects	Network to network	User/device to network
Typical endpoints	Routers/firewalls	VPN client and VPN gateway
User action	Usually none	User normally connects
Example	Branch ↔ HQ	Laptop ↔ Office
Common protocol	IPsec	IPsec or TLS-based VPN

10.49 VPN Communication Flow

Example remote-access VPN:



```

↓
VPN Server / Gateway
↓
Decrypt
↓
Original Packet
↓
Private Network

```

10.50 VPN Inner and Outer Packets

Suppose:

```

Laptop Private IP = 192.168.1.10
VPN Server IP     = 203.0.113.10
Internal Server   = 10.10.10.20

```

TEXT

Before VPN:

Inner packet:

```

Source      = 192.168.1.10
Destination = 10.10.10.20

```

TEXT

After VPN encapsulation:

Outer packet:

```

Source      = User's public IP
Destination = 203.0.113.10

```

TEXT

Inside the encrypted tunnel is the original traffic.

```

[Outer IP]
|
+-- Encrypted [
    Inner IP
    TCP/UDP
    Data
]

```

TEXT

10.51 Full Tunnel vs Split Tunnel

Full Tunnel

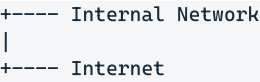
All user Internet traffic goes through the VPN.

```

Laptop
|
| All Traffic
↓
VPN Server
|

```

TEXT



Advantages

- Central security inspection
- Central logging
- Easier policy enforcement

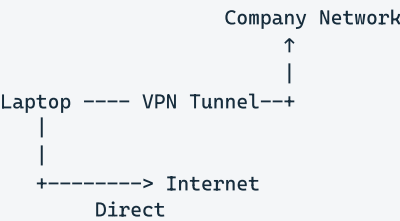
Disadvantages

- More VPN bandwidth required
- Can increase latency

Split Tunnel

Only company-related traffic goes through the VPN.

Other Internet traffic goes directly to the Internet.



TEXT

Advantages

- Lower VPN bandwidth usage
- Better Internet performance

Disadvantages

- Less centralized traffic control
- Can increase security risk if badly configured

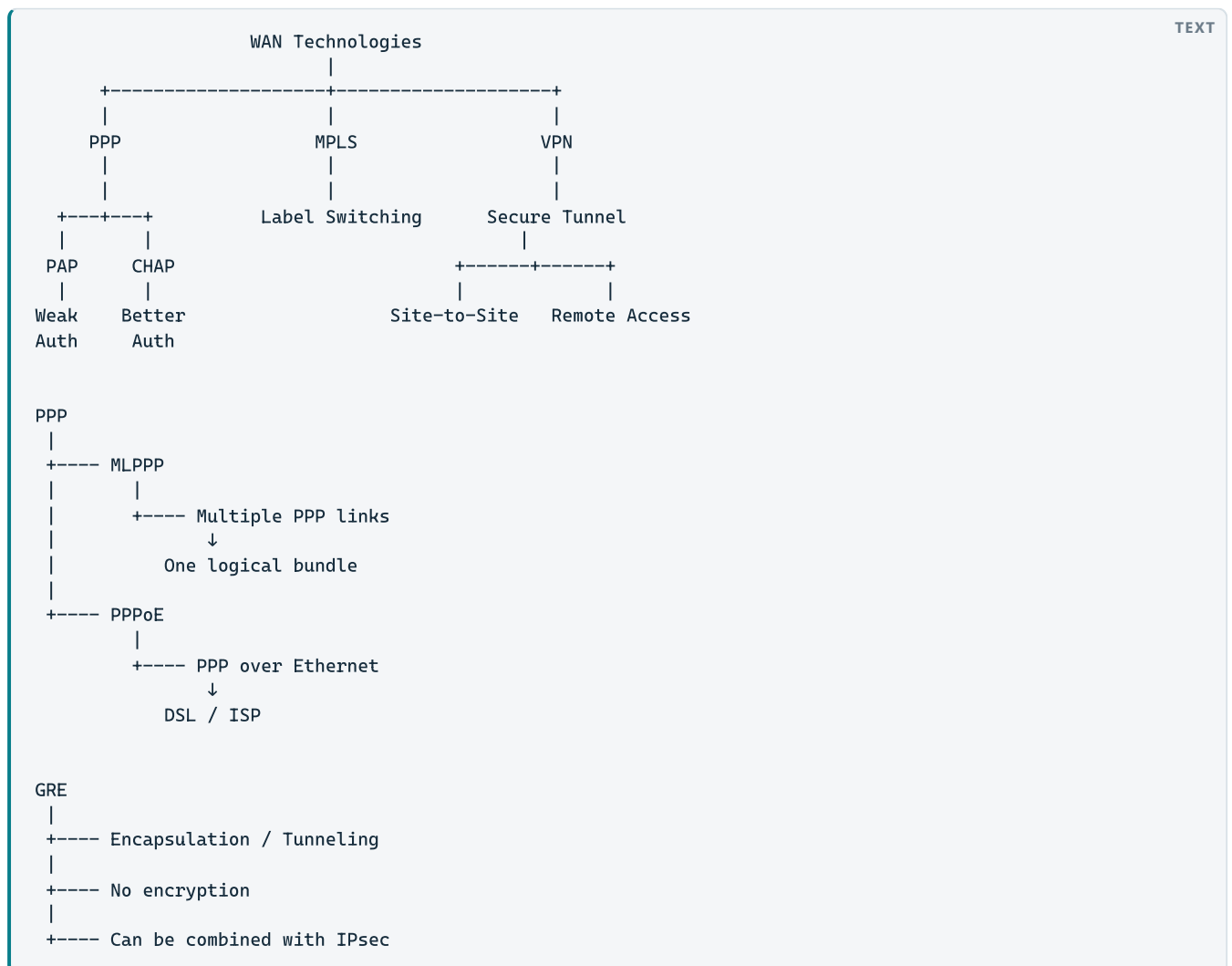
10.52 GRE vs VPN

Feature	GRE	Secure VPN
Tunneling	Yes	Yes
Encryption	No	Usually yes
Authentication	Limited/no inherent secure authentication	Yes
Integrity	No strong inherent protection	Usually yes
Main Purpose	Encapsulation	Secure communication
Safe alone over Internet	No	Generally intended for this

10.53 PPP, MLPPP, PPPoE, GRE, MPLS and VPN Comparison

Technology	Main Purpose	Encryption	Common Use
PPP	Point-to-point Layer 2 communication	No	Serial links
PAP	PPP authentication	No	Older authentication
CHAP	Challenge-response authentication	No traffic encryption	PPP authentication
MLPPP	Combine multiple PPP links	No	WAN link aggregation
PPPoE	PPP over Ethernet	No by itself	DSL broadband
GRE	Tunneling	No	Connect networks / carry routed protocols
MPLS	Label-based provider forwarding	No by itself	Enterprise/service-provider WAN
VPN	Secure tunnel	Yes, normally	Site-to-site / remote access

10.54 Complete WAN Technology Overview



10.55 Common Interview Questions

What is MLPPP?

MLPPP stands for Multilink PPP. It combines multiple PPP links into one logical WAN connection. It provides higher total bandwidth and can distribute traffic fragments across the member links.

Why is MLPPP used?

MLPPP is used to aggregate multiple WAN links to increase bandwidth, improve link utilization, and provide some redundancy if one link fails.

What is PPPoE?

PPPoE means Point-to-Point Protocol over Ethernet. It allows PPP authentication and session management to work over Ethernet and has been commonly used by DSL broadband providers.

What is GRE?

GRE is a tunneling protocol that encapsulates one packet inside another IP packet. It provides tunneling but not encryption, so it can be combined with IPsec when secure communication is required.

Does GRE encrypt data?

No. GRE only provides encapsulation. It does not provide confidentiality by itself.

What is MPLS?

MPLS is a provider-network technology that forwards traffic using labels. It supports scalable WAN connectivity, traffic engineering, and QoS.

Does MPLS encrypt traffic?

No. MPLS provides traffic separation and controlled forwarding, but it does not inherently encrypt the traffic.

What is a site-to-site VPN?

A site-to-site VPN creates an encrypted tunnel between two networks, such as a branch office and headquarters.

What is a remote-access VPN?

A remote-access VPN securely connects an individual user's device to an organization's network through the Internet.

10.56 Quick Revision Sheet

TEXT

PPP

- Layer 2 point-to-point protocol
- LCP establishes link
- PAP / CHAP authenticate
- NCP configures network protocol

PAP

- 2-way handshake
- Sends username/password
- Weak

CHAP

- 3-way handshake
- Challenge-response
- Password not directly transmitted
- Better than PAP

MLPPP

- Multilink PPP
- Combines multiple PPP links
- Link aggregation
- More bandwidth
- Can survive one link failure with reduced capacity

PPPoE

- PPP over Ethernet
- Common with DSL
- Authentication/session management for ISP customers

GRE

- Generic Routing Encapsulation
- Creates logical tunnel
- No encryption
- Often paired with IPsec

MPLS

- Multiprotocol Label Switching
- Label-based forwarding
- Provider WAN
- QoS and traffic engineering
- No inherent encryption

VPN

- Virtual Private Network
- Secure encrypted tunnel
- Site-to-site = network to network
- Remote access = user to network

Most important exam point

TEXT

- | | |
|-------|--------------------------------|
| PPP | → Point-to-point communication |
| MLPPP | → Multiple PPP links combined |
| PPPoE | → PPP carried over Ethernet |
| GRE | → Tunneling without encryption |
| MPLS | → Label-based WAN forwarding |
| VPN | → Secure encrypted tunnel |

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)

- [01A - OSI Model](#)
- [04A - Network Routing Fundamentals](#)

Source note: 10 - PPP and WAN Technologies.md

CHAPTER 11 · WAN, SWITCHING & ROUTER OPERATIONS

Switching Types, Ethernet Forwarding & Layer 2 Switching

Switching models, Ethernet forwarding, MAC learning, flooding, collision domains, and duplex.

- Topic 11
- Topic 12
- computer networks
- switching
- ethernet
- layer 2

This document covers three related but distinct layers of "switching":

- **Part A** — how data moves across an entire network (a WAN/network-layer concept)
- **Part B** — how one individual Ethernet switch decides *when* to start forwarding a frame
- **Part C** — how a Layer 2 switch actually learns addresses and forwards frames day-to-day (MAC table, flooding, collisions, duplex)

Table of Contents

- 0. Full Forms / Abbreviations
- PART A — Network Switching Types
- 1. Circuit Switching
- 2. Message Switching
- 3. Packet Switching
- PART B — Ethernet Forwarding Methods
- 4. Store-and-Forward Switching

- 5. Cut-Through Switching
- PART C — Layer 2 Switching Fundamentals
- 6. The MAC Address Table, Flooding & Forwarding
- 7. CSMA/CD — Carrier Sense Multiple Access with Collision Detection
- 8. Half-Duplex vs Full-Duplex
- 9. Quick Revision Sheet
- Related Notes

0. Full Forms / Abbreviations

Abbreviation	Full Form
VC / VCI	Virtual Circuit / Virtual Circuit Identifier
FCS	Frame Check Sequence
CRC	Cyclic Redundancy Check
CSMA/CD	Carrier Sense Multiple Access with Collision Detection
MAC	Media Access Control (address)
NIC	Network Interface Card

PART A — NETWORK SWITCHING TYPES

These describe the fundamental ways data can travel through a network of intermediate nodes. This is general networking theory — separate from how an individual Ethernet switch behaves internally, which is covered in Part B.

1. Circuit Switching

A **dedicated physical path** is set up end-to-end **before** any data is sent. The whole session then uses that **same fixed path** — exactly like a traditional telephone call.

Phases:

1. Setup — Call request and connection setup
2. Data Transfer — Data flows continuously over the fixed path
3. Teardown — Connection is released once transfer completes

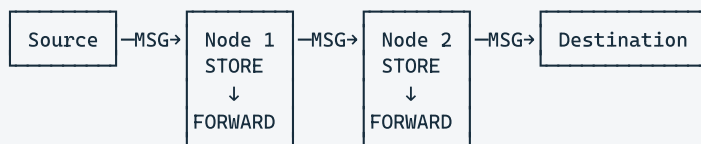


Dedicated path reserved for the ENTIRE call.
No other traffic can use it — and it sits idle
(wasting capacity) whenever no data is flowing.

✓ Pros	✗ Cons
Low delay during transfer — the path is always ready and reserved	Wastes bandwidth whenever the path is idle but still reserved

2. Message Switching

The **entire message** is treated as one unit. Each intermediate node **stores the whole message**, then forwards it onward once ready — a "**store-and-forward**" approach applied at the message level.



The entire message must be stored at EACH node before it can be forwarded.
This causes high delay — especially for large messages.

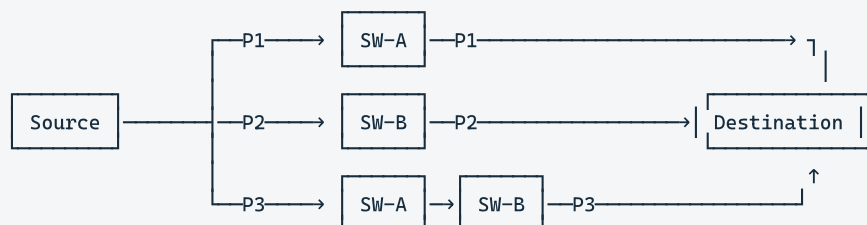
✓ Pros	✗ Cons
Efficient for bursty traffic (no dedicated path needed)	High delay from storing full messages at every hop; rarely used today

3. Packet Switching

The message is broken into smaller **packets** that travel independently or semi-independently through the network. There are two sub-types.

3.1 Datagram Switching (Connectionless)

Each packet is **routed independently** — there's no fixed path. Different packets from the same message may take **different routes**, and they're only reassembled at the receiver. Every packet must carry the **full source and destination address**, since each one is routed on its own.



- Each packet carries the FULL source & destination address.
- Packets can arrive OUT OF ORDER (e.g. P3 might arrive before P1).
- Reassembly happens only at the Destination.

✓ Pros	✗ Cons
Efficient, and resilient to link failures (packets simply reroute around a broken link)	Delay is unpredictable; out-of-order arrival is possible

3.2 Virtual Circuit Switching (Connection-Oriented)

A **logical path** (the virtual circuit) is set up **before** data transfer begins. Every packet then follows that **same pre-decided route**, identified by a short **Virtual Circuit Identifier (VCI)** instead of carrying a full address in every packet.

Phases:

1. VC Establishment – A setup packet reserves the logical path
2. Data Transfer – All packets follow the established VC
3. VC Teardown – The logical connection is released

Phase 1 – VC Establishment (logical path reserved):



Virtual Circuit VC1 (pre-established)

- Only a VCI label travels in each packet header
- No per-packet routing decision needed

Phase 2 – Data Transfer (all packets follow VC1, arrive in order):

[VC1, P1] → → → → → In-order delivery ✓

[VC1, P2] → → → → →

[VC1, P3] → → → → →

Phase 3 – VC Teardown: connection released, resources freed

✓ Pros	✗ Cons
Low per-packet delay; delivery is guaranteed in order	Less resilient to link failure (the whole VC breaks); has setup overhead

3.3 Quick Comparison — All Switching Types

Type	Path	Addressing	Connection-Oriented?	Arrival Order
Circuit Switching	Fixed, dedicated	None needed (path is pre-set)	Yes (end-to-end)	In order
Message Switching	Per message	Full address in the message	No	In order
Datagram (Packet)	Different per packet	Full address in every packet	No	May be out of order
Virtual Circuit	Same logical path	VCI only	Yes	In order

One-liners:

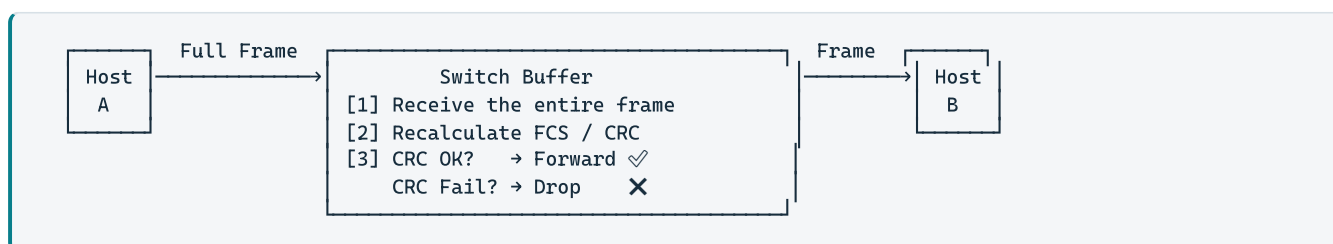
- **Circuit switching** = dedicated path, wasteful if idle, low delay once set up — think phone calls.
- **Message switching** = store-and-forward at the whole-message level, high delay, largely obsolete.
- **Datagram switching** = no fixed path, out-of-order possible, resilient — classic IP routing behavior.
- **Virtual Circuit switching** = fixed logical path + VCI, in-order, low per-packet overhead — think MPLS, Frame Relay, ATM.

PART B — ETHERNET FORWARDING METHODS

Part A described how data moves *across a network*. Part B zooms into a single Ethernet switch and asks: once a frame starts arriving on a port, **when does the switch begin forwarding it out the other side?**

4. Store-and-Forward Switching

The switch receives the **entire frame** into its buffer **before** making any forwarding decision. It checks the **FCS (CRC)** for errors and **drops invalid frames**.



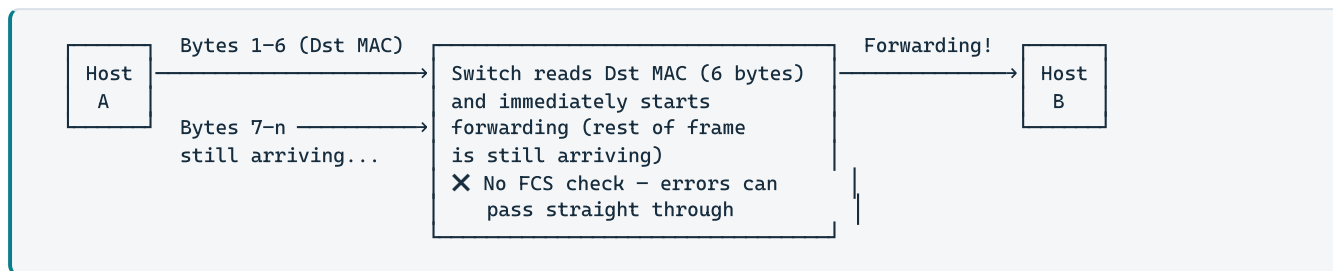
✓ Pros	✗ Cons
Most reliable — bad frames never leave the switch	Highest latency of all methods (has to wait for the whole frame)

5. Cut-Through Switching

Forwarding **starts before the whole frame has arrived**. There are two variants.

5.1 Fast-Forward Switching

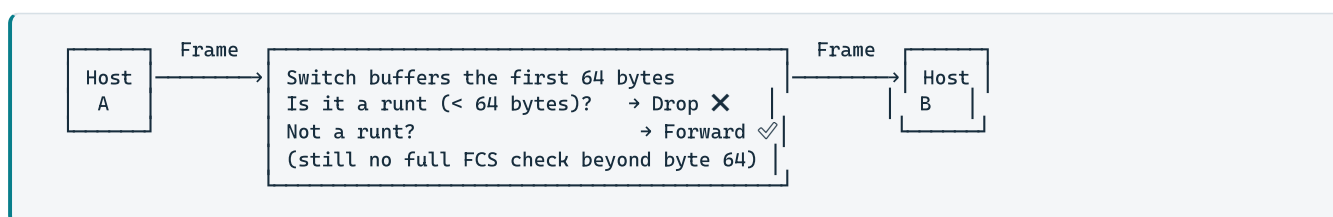
Starts forwarding **the instant the destination MAC address is read** — just the **first 6 bytes** of the frame. It does **not** check the FCS at all.



✓ Pros	✗ Cons
Lowest latency of all switching methods	Can forward corrupted frames — no error checking at all

5.2 Fragment-Free Switching

An improved cut-through variant: the switch buffers the **first 64 bytes** before forwarding. Ethernet collisions produce fragments **shorter than 64 bytes** (called "runt"), so this catches and filters those out. It still does **not** perform a full FCS check.



✓ Pros	✗ Cons
Filters out collision fragments (runt)	Doesn't catch bit errors that occur beyond byte 64

5.3 What Exactly Is a "Runt" (Collision Fragment)?

A **runt** is a very small, incomplete frame left over on the wire after a **collision**.

Why it happens: On a **shared, half-duplex** segment (Section 8), two devices can start transmitting at nearly the same instant. When their signals meet on the wire, they collide and corrupt each other. As soon as a device's CSMA/CD logic (Section 7) detects this, it **immediately stops transmitting** that frame. What's left behind on the wire is a **partial, truncated frame** — not a real, usable one.



What makes a runt a runt:

- **Size** — under the Ethernet minimum of **64 bytes** (often as small as ~40–60 bytes, depending on exactly when the collision hit).
- **Cause** — an aborted transmission from a collision, not a deliberately sent complete frame.
- **Validity** — meaningless, corrupted data; it's only part of a frame, so there's nothing usable in it.
- **Risk if forwarded** — a switch that doesn't check for runts wastes downstream bandwidth passing along garbage data.

Why this is exactly why Fragment-Free checks 64 bytes: a genuine collision fragment can only ever be shorter than 64 bytes — that's simply how fast CSMA/CD detects and aborts a transmission. So buffering the first 64 bytes before forwarding guarantees that anything shorter is, by definition, a runt — never a legitimate frame — and it's safe to drop.

- **Fast-Forward** (Section 5.1) skips this check entirely — since it starts forwarding after only 6 bytes, it can end up pushing runts onto the rest of the network.
- **Store-and-Forward** (Section 4) filters runts too, just as a side effect of waiting for and validating the complete frame anyway.

One-liners:

- Runt = collision fragment = a frame **shorter than the 64-byte Ethernet minimum**, caused by two devices transmitting at once and one aborting mid-frame.
- Runts only happen on **half-duplex/shared media** — they can't occur on collision-free full-duplex links.
- Fragment-Free switching buffers exactly 64 bytes because that's the guaranteed minimum size of any *legitimate* frame — which makes the runt check reliable at that exact cutoff.

5.4 Quick Comparison — Ethernet Forwarding Methods

Method	Buffers	Error Check	Latency	Best For
Store-and-Forward	Entire frame	Full FCS/CRC	Highest	Reliability
Fast-Forward	First 6 bytes (Dst MAC)	None	Lowest	Raw speed
Fragment-Free	First 64 bytes	Runt/collision check only	Medium–Low	Balance of speed and basic error filtering

One-liners:

- **Store-and-Forward** = safest, slowest — waits for the whole frame and runs a full CRC check.
- **Fast-Forward** = fastest, riskiest — forwards right after reading the destination MAC.
- **Fragment-Free** = middle ground — waits for 64 bytes to filter out runts, but skips the full CRC check.
- Most modern switches default to **Store-and-Forward**; cut-through variants show up mainly in very latency-sensitive environments (e.g. some data-center or high-frequency-trading switching).

PART C — LAYER 2 SWITCHING FUNDAMENTALS

Part B explained *when* a switch starts forwarding a frame. Part C covers the bigger picture of day-to-day switch operation: how it learns where devices are, what it does when it doesn't know, and how the underlying link (half- vs full-duplex) affects all of it.

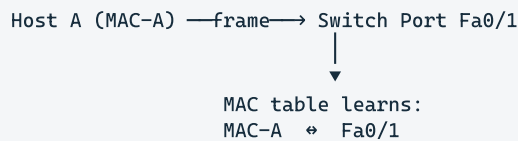
6. The MAC Address Table, Flooding & Forwarding

A Layer 2 switch builds and uses a **MAC address table** (also called a **CAM table** — Content Addressable Memory table) to decide where to send each frame.

6.1 How the Switch Learns (Building the MAC Table)

- Step 1: A frame arrives on a port.
Step 2: The switch reads the frame's SOURCE MAC address.

Step 3: It records {Source MAC ↔ Incoming Port} in its MAC address table.
 Step 4: That entry is then used for FUTURE frames destined to that MAC.



6.2 The Forwarding Decision

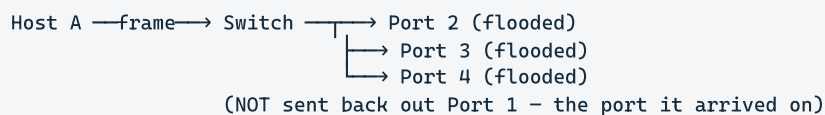
When a frame arrives, the switch checks its MAC table for the DESTINATION MAC:

Is the destination MAC already in the table?
 └ YES → Forward out that ONE specific port (efficient, unicast)
 └ NO → Flood out ALL ports except the one the frame arrived on

6.3 Flooding

Flooding happens when the switch doesn't yet know which port leads to the destination MAC — so it sends a copy of the frame out **every port** except the one it came in on. The hope is that the real destination is out there somewhere and will eventually reply, letting the switch learn its actual port.

Frame with an UNKNOWN destination MAC:



Flooding also happens for:

- **Broadcast frames** (destination `FFFF.FFFF.FFFF`) — always flooded to every port.
- **Multicast frames** — flooded unless multicast-aware filtering (e.g. IGMP snooping) is configured.
- **Unknown unicast** — as above, until the switch learns that MAC's real port.

6.4 A Full Example: Learn → Flood → Forward

T1: Host A sends a frame to Host B (switch doesn't know Host B's port yet)
 → Switch learns Host A's MAC ↔ Port 1
 → Switch FLOODS the frame out every other port (Host B's real port is one of them)

T2: Host B replies to Host A
 → Switch learns Host B's MAC ↔ Port 3
 → Switch already knows Host A is on Port 1 → FORWARDS directly, no flooding needed

T3 onward: Switch now knows BOTH MACs
 → All further traffic between A and B is forwarded directly, port to port

One-liners:

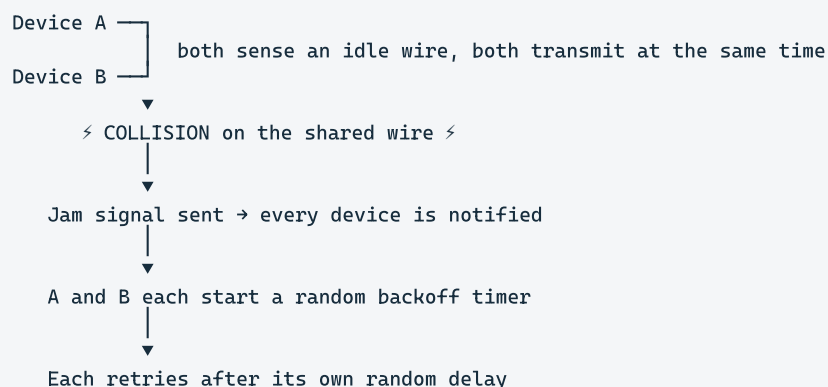
- The MAC table maps **MAC address** ↔ **switch port**, learned from the **source MAC** of incoming frames.
- **Known destination MAC** → forward out that one port only.
- **Unknown destination MAC, broadcast, or unhandled multicast** → flood out every port except the source port.
- Flooding is a **fallback**, not the switch's normal behavior — it stops as soon as the destination's real port is learned.

7. CSMA/CD — Carrier Sense Multiple Access with Collision Detection

CSMA/CD is the access method traditionally used by Ethernet on **shared, half-duplex media** (old hub-based networks or coax Ethernet) to decide when a device may transmit, and to recover when two devices transmit at once (a **collision**).

7.1 How CSMA/CD Works, Step by Step

1. **Carrier Sense** : Before transmitting, the device **LISTENS** to the wire.
Idle? → proceed. Busy? → wait.
2. **Multiple Access** : Many devices share the **SAME** medium and can all attempt to transmit – there's no central controller granting turns.
3. **Collision Detect**: If two devices transmit at the **SAME** time, their signals collide and corrupt on the wire. Both devices detect this.
4. **Jam Signal** : The colliding devices send a jam signal so that **EVERY** device on the segment knows a collision occurred.
5. **Backoff & Retry** : Each device waits a **RANDOM** amount of time, then retries – reducing the odds of an immediate repeat collision.



7.2 When Is CSMA/CD Actually Used?

- It matters on **half-duplex** links, where collisions are physically possible (Section 8).
- It's **not needed** on **full-duplex** links (standard on modern switched Ethernet) — since sending and receiving use separate paths, **collisions can't happen**, so CSMA/CD is effectively irrelevant there.

One-liners:

- CSMA/CD = Carrier Sense (listen first) + Multiple Access (shared medium) + Collision Detection (detect and recover from simultaneous transmissions).
- Collision-handling sequence: **sense** → **transmit** → **detect collision** → **jam signal** → **random backoff** → **retry**.
- Mostly a **legacy** concept today — largely irrelevant on modern full-duplex switched networks, but still an important exam topic.

8. Half-Duplex vs Full-Duplex

Feature	Half-Duplex	Full-Duplex
Data flow	One direction at a time (send <i>or</i> receive)	Both directions at once (send <i>and</i> receive)
Collisions possible?	✓ Yes	✗ No
CSMA/CD needed?	✓ Yes	✗ No
Typical use today	Legacy hubs, some wireless links	Standard for modern switched Ethernet (switch-to-host, switch-to-switch)
Throughput	Lower — shared channel, contention overhead	Higher — dedicated send/receive paths, no contention

Half-Duplex:

A → send →
A ← recv ← (not at the same time)

Full-Duplex:

A → send →
A ← recv ← (send & recv happen SIMULTANEOUSLY, on separate wire pairs)

Why modern networks run full-duplex:

- A switch (unlike an old hub) gives each device its **own dedicated collision domain** per port.
- With a direct switch-to-host link and no shared medium, a collision **can't physically happen** — so both ends can safely send and receive at the same time.

One-liners:

- **Half-duplex** = one direction at a time, collisions possible, needs CSMA/CD.
- **Full-duplex** = both directions at once, no collisions, CSMA/CD not needed.
- Modern switched links (switch ↔ host, switch ↔ switch) run **full-duplex** by default.
- A **duplex mismatch** (one side set to half, the other to full) is a classic real-world cause of poor performance and errors on a link.

9. Quick Revision Sheet

SWITCHING TYPES (network-wide, Part A)

Circuit Switching : dedicated path, reserved end-to-end, wastes idle bandwidth
 Message Switching : store-and-forward whole message, high delay, obsolete
 Datagram Switching : connectionless, no fixed path, packets may arrive out of order
 Virtual Circuit : connection-oriented, fixed logical path, VCI-based, in order

ETHERNET FORWARDING METHODS (switch-internal, Part B)

Store-and-Forward : buffers WHOLE frame, full CRC check, safest, slowest
 Fast-Forward : buffers only 6 bytes (Dst MAC), no check, fastest, riskiest
 Fragment-Free : buffers 64 bytes, filters runs only, balanced

MAC TABLE / FLOODING / FORWARDING (Part C)

MAC table learns : source MAC → incoming port
 Known destination : forward out that one port
 Unknown/broadcast : flood out all ports except the source port

CSMA/CD (Part C)

Steps : Carrier Sense → transmit → Collision Detect → Jam Signal → Random Backoff → Retry
 Needed only on: Half-Duplex links (shared medium, collisions possible)

DUPLEX (Part C)

Half-Duplex : one direction at a time, collisions possible, needs CSMA/CD

Full-Duplex : both directions at once, no collisions, CSMA/CD not needed

Modern switched Ethernet = Full-Duplex by default

Switching Types, Ethernet Forwarding & Layer 2 Switching — Reference Notes

Related Notes

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [01A - OSI Model](#)
- [08 - Spanning Tree Protocol](#)
- [05 - VLANs and Inter-VLAN Routing](#)
- [09 - Infrastructure Security ACL AAA and Port Security](#)

Source note: 11 - Layer 2 Switching and Ethernet Forwarding.md

CHAPTER 13 · WAN, SWITCHING & ROUTER OPERATIONS

Router Fundamentals: IOS, Boot Process, Memory, CLI Modes, Remote Access, Password Recovery & Troubleshooting

Cisco IOS, hardware and memory, CLI modes, boot flow, secure access, recovery, logs, and debugging.

- Topic 13
- computer networks
- cisco ios
- routers
- troubleshooting

Table of Contents

- 0. Full Forms / Abbreviations
- PART A — What Is Cisco IOS?
- PART B — Router Hardware Components (The "Big 5")
- PART C — Cisco IOS CLI Modes
- PART D — Router Boot Sequence
- 1. Boot Sequence Overview
- 1.1 Detailed Step-by-Step Flow
- PART E — Telnet vs SSH (Remote Access Security)

- PART F — Ways to Access the Cisco IOS CLI
- PART G — Configuration Registers
- What Is the Configuration Register?
- PART H — Cisco IOS Password Recovery (Full Procedure)
- PART I — Debugging & Logging
- Why Debugging and Logging Matter
- PART J — Master Quick Revision
- Related Notes

0. Full Forms / Abbreviations

Abbreviation	Full Form
POST	Power-On Self-Test
IOS	Internetwork Operating System (Cisco)
ROM	Read-Only Memory
RAM	Random-Access Memory
NVRAM	Non-Volatile Random-Access Memory
Flash	Flash Memory
SSH	Secure Shell
Telnet	Teletype Network
CLI	Command-Line Interface
TFTP	Trivial File Transfer Protocol
ROMmon	ROM Monitor (also written RXBOOT)
VTY	Virtual Teletype (virtual terminal line, used for remote CLI sessions)

Abbreviation	Full Form
AUX	Auxiliary (port)

PART A — WHAT IS CISCO IOS?

Cisco IOS is the operating system that runs on Cisco routers and switches. It is responsible for:

- **Routing** — moving packets between networks
- **Switching** — forwarding frames within a network
- **Network device management** — CLI, configuration, monitoring
- **Security** — passwords, ACLs, AAA, encryption features (e.g., SSH)

PART B — ROUTER HARDWARE COMPONENTS (THE "BIG 5")

Component	Role
CPU	Executes IOS instructions and processes
RAM	Stores the running-config, routing tables, ARP cache, buffers, and the currently <i>loaded/running</i> IOS
ROM	Stores POST routines, the Bootstrap program, and a bare-bones mini-IOS (ROMmon)
NVRAM	Stores the startup-config
Flash	Stores the full IOS image file (the OS itself)

Memory Cheat Sheet

Memory Type	Stores	Volatile?	Analogy
ROM	Bootstrap program, POST routines, mini-IOS (ROMmon)	✗ No — permanent	The router's built-in "BIOS"
Flash	The full IOS image	✗ No — persists across reboots	The router's "hard drive"
NVRAM	The startup-config	✗ No — persists across reboots	Saved settings file
RAM	The running-config + currently running IOS + routing tables/ARP cache/buffers	✓ Yes — wiped on reboot/power loss	The router's "working memory"

Memory mnemonic: *"Real Networks Feel Real"* → **RAM** (Running-config) → **NVRAM** (Startup-config) → **Flash** (Full IOS) → **ROM** (ROMmon/Bootstrap).

Key Distinctions

Concept	Lives In	Notes
<code>show running-config</code>	RAM	The currently active config — lost on reboot unless saved
<code>show startup-config</code>	NVRAM	The config that will load at the next boot

Concept	Lives In	Notes
<code>copy running-config startup-config</code> (or <code>write memory</code>)	RAM → NVRAM	Saves the active config so it survives a reboot
IOS image file (<code>.bin</code>)	Flash	The actual OS software; Flash can hold multiple IOS versions
ROMmon / mini-IOS	ROM	Emergency recovery mode if the real IOS can't load

PART C — CISCO IOS CLI MODES

Mode	Purpose	Prompt	How to Enter
A. User EXEC	Limited access — basic monitoring commands only (e.g., <code>ping</code> , <code>show version</code>)	<code>Router></code>	Default mode on login
B. Privileged EXEC	Full administrative access — debugging, copying/saving config, entering config mode	<code>Router#</code>	<code>Router> enable</code>
C. Global Configuration	Configure system-wide settings (hostname, passwords, routing, etc.)	<code>Router(config)#</code>	<code>Router# configure terminal</code>
D. Interface Configuration	Configure a specific interface (IP address, description, etc.)	<code>Router(config-if)#</code>	<code>Router(config)# interface <name></code>
E. Line Configuration	Configure console / VTY / AUX lines (passwords, login method, transport)	<code>Router(config-line)#</code>	<code>Router(config)# line vty 0 4</code> (etc.)

Common User EXEC Commands

```
Router> show ?
Router> show history
Router> show version
Router> ping 192.168.1.1
Router> traceroute 8.8.8.8
Router> telnet 192.168.1.10
Router> logout
```

TEXT

Privileged EXEC — Common Uses

Once in `Router#`, you can:

- View detailed information (`show running-config`, `show startup-config`, `show version`)
- Copy/save configuration
- Run `debug` commands
- Enter global configuration mode

```
Router# show running-config
```

TEXT

Shows the **active** configuration in RAM. Includes: interfaces, hostname, passwords, routing configuration, VTY line settings, and enabled services.

```
Router# show startup-config
```

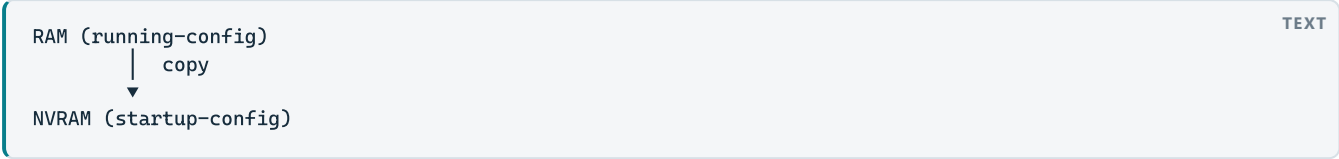
TEXT

Shows the configuration **saved in NVRAM** that will be used on the next reboot.

Router# copy running-config startup-config

TEXT

Saves the current (active) configuration to NVRAM so it survives a reboot:



Types of Line Passwords

- **Console password** — protects local console-port access (`line console 0`)
- **Telnet / SSH password** — protects remote CLI access (`line vty 0 4`)

PART D — ROUTER BOOT SEQUENCE

1. Boot Sequence Overview

POST → Bootstrap → Locate & Load IOS → Load Configuration

Stage	What Happens	Stored In
1. POST (Power-On Self-Test)	Hardware diagnostic check — CPU, RAM, interfaces, and other components are tested for basic functionality	Executed from ROM
2. Bootstrap	A small program that locates and loads the IOS image	Stored in ROM
3. IOS Load	The bootstrap program loads the Cisco IOS image into RAM, following the order set by the configuration register	Loaded from Flash (default) into RAM
4. Configuration	The router loads its saved configuration (startup-config) into running memory to become operational	Loaded from NVRAM into RAM

1.1 Detailed Step-by-Step Flow



STEP 3: IOS Image Is Located and Loaded

Default search order:

1. Flash memory (most common default location)
 2. TFTP server (if configured / Flash fails)
 3. ROM (mini-IOS) (fallback if all else fails)
- IOS image is loaded (decompressed if needed) INTO RAM
 - Router now has a running, functional operating system

**STEP 4: Configuration File Is Loaded**

- Router looks in NVRAM for startup-config
 - Found? → copied into RAM as running-config → router is fully configured and operational
 - Not found? → tries TFTP → if still not found, enters SETUP MODE (interactive Q&A wizard) with a blank/default configuration

Simplified Flow (as commonly drawn)

TEXT

**Exam One-Liners**

- Order to remember: **POST** → **Bootstrap** → **IOS** → **Config** ("Please **B**ring **I**ce **C**ream").
- POST and Bootstrap both run from **ROM**.
- IOS is normally loaded from **Flash**; startup-config is loaded from **NVRAM**.
- If startup-config is missing, the router tries **TFTP**, and if that also fails, it enters **Setup Mode**.

PART E — TELNET VS SSH (REMOTE ACCESS SECURITY)

Both protocols allow **remote CLI access** to a router/switch through **VTY lines**, but they differ fundamentally in security.

By default a router has 5 VTY lines (`line vty 0 4` = VTY 0 through VTY 4), meaning up to 5 simultaneous remote CLI sessions.

Feature	Telnet	SSH
Encryption	❌ None — sends everything, including passwords, in plain text	✅ Full encryption of the entire session

Feature	Telnet	SSH
Port	TCP 23	TCP 22
Authentication security	Weak — credentials easily captured via packet sniffing	Strong — supports password AND public-key authentication
Data integrity checking	✗ No	✓ Yes (cryptographic integrity checks)
Vulnerable to eavesdropping?	✓ Very — anyone on the path can read the traffic	✗ No — traffic is encrypted end-to-end
Recommended today?	✗ No — legacy/insecure, mainly seen in labs/older environments	✓ Yes — industry standard for remote device management

Why Telnet Is Insecure — Visualized

Telnet Session (PLAIN TEXT):

```
Admin PC — "Username: admin" —> Router
Admin PC — "Password: Cisco123" —> Router
```

↓

Anyone capturing packets on this path
(e.g., via Wireshark) can read the
password DIRECTLY – no decryption needed.

Example command: `telnet 192.168.1.1`

Why SSH Is Secure — Visualized

SSH Session (ENCRYPTED):

```
Admin PC — [encrypted blob: xK9$#mP2...] —> Router
```

↓

An eavesdropper only sees scrambled
ciphertext – username/password/commands
are unreadable without the correct key.

Example command: `ssh admin@192.168.1.1`

Basic SSH Configuration (Cisco IOS)

Step 1 – Set hostname and domain name (required for RSA key generation):

```
Router(config)# hostname R1
R1(config)# ip domain-name mylab.com
```

Step 2 – Generate the RSA key pair:

```
R1(config)# crypto key generate rsa
(choose a modulus size, e.g. 1024 or 2048 bits)
```

Step 3 – Create a local user account:

```
R1(config)# username admin secret StrongPass123
```

Step 4 – Configure the VTY lines to use SSH + local login:

```
R1(config)# line vty 0 4
R1(config-line)# transport input ssh
R1(config-line)# login local
```

TEXT

`transport input ssh` **disables Telnet** on the VTY lines (only SSH is accepted). Using `transport input telnet ssh` would allow both — not recommended for security.

Exam One-Liners

- Telnet = **unencrypted**, TCP port 23 — never use on production networks.
- SSH = **encrypted**, TCP port 22 — the modern standard for remote device administration.
- SSH requires a **hostname + domain name + RSA key pair** before it can be enabled.
- Best practice: `transport input ssh` on VTY lines to **disable Telnet entirely**.
- Default VTY lines: **0 to 4** (5 lines total).

PART F — WAYS TO ACCESS THE CISCO IOS CLI

There are four common ways an administrator can reach the CLI of a router:

1. Console — Local, Out-of-Band Access

PC/Laptop — Console cable — Router

TEXT

Direct physical connection to the router's console port, accessed via a terminal program.

Used when:

- Router is being configured for the first time (no network config exists yet)
- Network access is unavailable
- Password recovery is needed

2. SSH — Remote, Encrypted Access (Preferred)

Admin PC — SSH → Router

TEXT

Example: `ssh admin@192.168.1.1`

Used when: administering the router remotely and secure access is required. SSH encrypts the entire communication (see Part E).

3. Telnet — Remote, Unencrypted Access

Admin PC — Telnet → Router

TEXT

Example: `telnet 192.168.1.1`

Used when: remote CLI access is needed — typically only in labs or older environments. ⚠️ Not recommended for production since credentials and data are sent in plain text.

4. AUX Port — Out-of-Band Remote/Modem Access

Remote Admin — Modem — AUX port — Router

TEXT

Some Cisco routers have an **AUX (Auxiliary) port**, traditionally connected to a modem for **out-of-band management** — i.e., accessing the router when the normal in-band network path is down or unavailable. It is not used for day-to-day remote administration like SSH/Telnet, but as a backup access path.

Exam One-Liners

- Console and AUX are physical/out-of-band access methods; SSH and Telnet are network/in-band (VTY) access methods.
- Console access is required for the **very first configuration** of a router (no IP/network config exists yet).
- SSH is preferred over Telnet for all remote in-band access.

PART G — CONFIGURATION REGISTERS

What Is the Configuration Register?

The **configuration register** is a **16-bit (4 hex-digit) value** stored in **NVRAM** that tells the router **how to boot** — where to look for the IOS, whether to load the startup-config, and other boot-time behaviors.

View current value:
Router# show version
(look for the line: "Configuration register is 0x XXXX")

TEXT

Common Configuration Register Values

Value	Meaning
0x2102	Default — boot normally: load IOS from Flash, load startup-config from NVRAM
0x2142	Ignore startup-config at boot — router boots into Setup Mode with a blank running-config (commonly used for password recovery)
0x2100	Boot into ROMmon (ROM Monitor) mode — minimal recovery environment, does not load IOS normally

Changing the Configuration Register

Router(config)# config-register 0x2142

TEXT

- Takes effect **after the next reload** — not immediately.
- Commonly used during **password recovery**: set to **0x2142** to skip loading the (password-protected) startup-config, log in with no password, then manually reload the old config and change the password, and finally set the register **back to 0x2102** before the final reload/save.

Exam One-Liners

- Config register = 16-bit value in **NVRAM**, controls boot behavior.
- **0x2102** = normal/default boot.
- **0x2142** = ignore (skip loading) startup-config — the classic password-recovery value.
- **0x2100** = boot straight into ROMmon.

- Changes to the register only take effect **after a reload**.
- Always remember to set it **back to 0x2102** after password recovery — otherwise the router will keep ignoring its saved config on every future boot.

PART H — CISCO IOS PASSWORD RECOVERY (FULL PROCEDURE)

If you forget a Cisco router's password, you can recover access using **ROMmon mode**, since physical/console access effectively lets you bypass password protection by controlling the boot process.

Step 1 — Restart and Enter ROMmon

Power off/on the router, and during boot send the **Break** signal to interrupt the normal boot and drop into ROMmon:

```
rommon 1 >
```

TEXT

Step 2 — Change the Configuration Register

Tell the router to **ignore the startup-config** on the next boot:

```
rommon 1 > confreg 0x2142
```

TEXT

Why this works:

- `0x2102` → normally loads startup-config (including the password)
- `0x2142` → **ignores** startup-config, so the router boots with no password set

Step 3 — Reset/Reload

```
rommon 2 > reset
```

TEXT

The router reboots without loading the old (password-protected) configuration.

Step 4 — Enter Privileged Mode

Since the old configuration was skipped, no password is currently applied:

```
Router> enable
Router#
```

TEXT

Step 5 — Copy the Old Configuration into RAM

The old configuration is still safely stored in NVRAM — it was only skipped, not deleted:

```
Router# copy startup-config running-config
```

TEXT

```
NVRAM (startup-config)
  |
  | copy
  v
RAM (running-config)
```

TEXT

This restores your old configuration (interfaces, routing, etc.) **without** re-locking you out with the old password.

Step 6 — Change the Password

```
Router# configure terminal
Router(config)# enable secret NewPassword123
```

TEXT

If needed, also change the console password:

```
Router(config)# line console 0
Router(config-line)# password NewPassword123
Router(config-line)# login
```

TEXT

Step 7 — Restore Normal Boot Behavior

This step is critical — don't skip it:

```
Router(config)# config-register 0x2102
```

TEXT

This ensures the router will load the startup-config normally on all future reboots.

Step 8 — Save the Configuration

```
Router# copy running-config startup-config
```

TEXT

Complete Sequence to Remember

```
Power ON
↓
Break
↓
ROMmon
↓
confreg 0x2142
↓
reset
↓
Router> enable
↓
copy startup-config running-config
↓
configure terminal → change password
↓
config-register 0x2102
↓
copy running-config startup-config
```

TEXT

★ Exam Shortcut

- **2142 = Ignore startup-config** (skip password)

- **2102 = Normal boot**

2142 → Recover → 2102 → Save.

⚠ Important exam trap: password recovery via ROMmon requires **physical console access** to the router — it cannot be done remotely via Telnet/SSH, since it relies on interrupting the boot process at the console.

PART I — DEBUGGING & LOGGING

Why Debugging and Logging Matter

Routers and switches constantly generate information about their internal operations, errors, and state changes. **Logging** captures this passively; **debugging** actively surfaces detailed real-time information about specific processes — both are essential for troubleshooting.

Logging (`show logging` / Syslog)

- The router keeps an internal **log buffer** of system messages (interface up/down, config changes, errors, etc.) by default.
- Logs can be sent to multiple destinations simultaneously:

Destination	Command	Notes
Console	(default, always on unless disabled)	Shown directly on the console session
Internal buffer	<code>logging buffered</code>	Stored in RAM; view with <code>show logging</code>
Terminal (VTY sessions)	<code>terminal monitor</code>	Needed to see log messages when connected via Telnet/SSH (not shown by default!)
External Syslog server	<code>logging <syslog-server-ip></code>	Centralizes logs from many devices for long-term storage/analysis

Log Message Severity Levels (0 = most severe → 7 = least severe):

TEXT

0 Emergency	1 Alert	2 Critical	3 Error
4 Warning	5 Notice	6 Informational	7 Debugging

⚠ **Common gotcha:** if you're remotely connected via Telnet/SSH and don't see log messages that appear on the console, run `terminal monitor` to enable them for your session.

Debugging (`debug` commands)

- `debug` commands provide **real-time, detailed** output about a specific process as it happens (e.g., `debug ip routing`, `debug ip icmp`, `debug spanning-tree events`).
- Extremely useful for diagnosing **active, in-progress** problems (e.g., watching routing updates arrive, or watching an authentication exchange fail step-by-step).

Router# debug ip routing
Router# debug ip icmp
Router# undebug all ← or "u all" – disables ALL active debug output

TEXT

 **Warning:** `debug` commands are **CPU-intensive** — running heavy debugs (like `debug ip packet` on a busy router) can significantly load the CPU and, in extreme cases, cause the router to become unresponsive. Always disable debugging (`undebug all`) as soon as you're done.

Why This Matters — Practical Importance

Without Debugging/Logging	With Debugging/Logging
Problems are invisible until they cause an outage	Early warning signs (interface flaps, failed logins, routing changes) are visible before they escalate
Root-cause analysis after an incident is guesswork	Log history + timestamps let you reconstruct exactly what happened and when
No audit trail of configuration/security events	Logs provide accountability — who did what, and when (especially combined with AAA accounting)
Diagnosing live/intermittent issues is nearly impossible	<code>debug</code> shows the process happening in real time, letting you catch transient issues

Best Practices

- Always timestamp log messages: `service timestamps log datetime msec`
- Send logs to a **centralized syslog server** in production — local buffers are limited in size and lost on reload.
- Use `debug` **sparingly and temporarily** in production — prefer testing in a lab first, and always run `undebug all` when finished.
- Match the **logging severity level** to what's actually needed (`logging console <level>` , `logging buffered <level>`) to avoid being flooded with noise.

Exam One-Liners

- **Logging** = passive record-keeping of system events; **Debugging** = active, real-time, detailed process tracing.
- Logging severity levels run **0 (Emergency)** → **7 (Debugging)** — lower number = more severe.
- `terminal monitor` is required to see log messages over a remote (Telnet/SSH) session.
- `debug` commands are CPU-intensive — always `undebug all` when finished.
- Centralized **Syslog servers** are the production-standard destination for logs (local buffers are volatile and size-limited).

PART J — MASTER QUICK REVISION

CISCO IOS
Operating system for Cisco routers/switches: routing, switching, management, security

HARDWARE COMPONENTS
CPU : executes instructions
RAM : running-config, routing table, ARP cache, buffers, running IOS
ROM : POST + Bootstrap + ROMmon (permanent)
NVRAM : startup-config (persists across reboot)
Flash : full IOS image (persists across reboot)

TEXT

CLI MODES

User EXEC	Router>	basic/limited commands
Privileged EXEC	Router#	full admin access (enable)
Global Config	Router(config)#	configure terminal
Interface Config	Router(config-if)#	interface <name>
Line Config	Router(config-line)#	line vty/console/aux

BOOT SEQUENCE

POST → Bootstrap → Locate/Load IOS → Load Config
 POST & Bootstrap : run from ROM
 IOS : loaded from Flash → into RAM (fallback: TFTP, then ROM)
 Startup-config : loaded from NVRAM → into RAM (as running-config)
 No startup-config found → try TFTP → Setup Mode

ACCESS METHODS

Console : local, out-of-band, cable – first-time setup/password recovery
 SSH : remote, encrypted, TCP 22 – preferred
 Telnet : remote, unencrypted, TCP 23 – legacy/labs only
 AUX : out-of-band via modem – backup access when network is down

TELNET vs SSH

Telnet : TCP 23, PLAIN TEXT, insecure – avoid in production
 SSH : TCP 22, ENCRYPTED, secure – industry standard
 SSH setup requires: hostname + domain-name + RSA key pair
 `transport input ssh` on VTY lines disables Telnet
 Default VTY lines: 0–4 (5 lines)

CONFIG REGISTER

16-bit value in NVRAM controlling boot behavior
 0x2102 : normal/default boot
 0x2142 : SKIP startup-config (used for password recovery)
 0x2100 : boot to ROMmon
 Takes effect only AFTER reload

PASSWORD RECOVERY (requires physical console access)

Break → ROMmon → confreg 0x2142 → reset → enable →
 copy startup-config running-config → change password →
 config-register 0x2102 → copy running-config startup-config

DEBUGGING & LOGGING

Logging : passive, ongoing record of events (0=Emergency ... 7=Debugging)
 Debugging : active, real-time, CPU-intensive process tracing
 terminal monitor : needed to see logs over Telnet/SSH session
 undebg all : stops all active debug output – always run when done
 Best practice : centralized Syslog server for production logging

*Router Fundamentals Reference Notes — CDAC Exam Prep***Related Notes**

- [00 - Syllabus and Interview Checklist](#)
- [Computer Networks Index](#)
- [04A - Network Routing Fundamentals](#)
- [04B - Routing Protocols and Administrative Distance](#)

Source note: 13 - Router IOS and Management.md

PART A

Application-Layer Appendix

Trace the evolution of the web stack from one-line HTTP to encrypted, multiplexed QUIC connections.

A1 · HTTP Evolution & TLS

CHAPTER A1 · APPLICATION-LAYER APPENDIX

HTTP Evolution — Interview Notes (0.9 → 1.0 → 1.1 → 2 → 3)

HTTP/0.9 through HTTP/3, TLS handshakes, QUIC, methods, status codes, flags, and headers.

computer networks

http

tls

application layer

Table of Contents

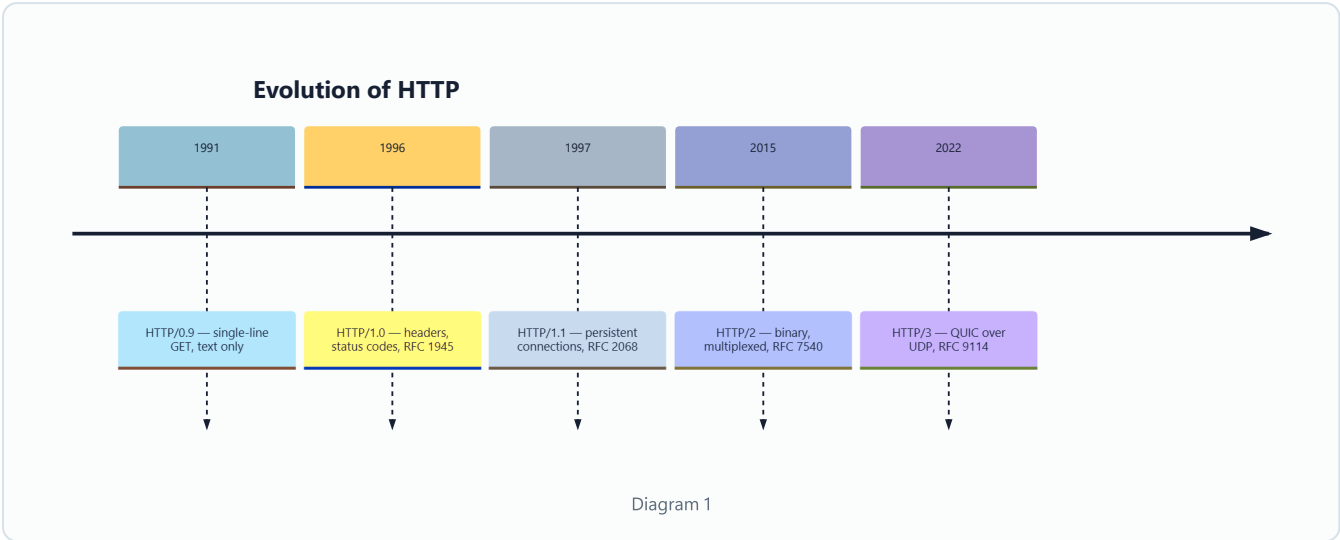
- What is HTTP?
- Timeline at a Glance
- Version Timeline (Quick Recall)
- HTTP/0.9 — The "One-Line" Protocol
- HTTP/1.0 — Extensibility Begins (1996)
- TLS Handshake — Detailed Breakdown
- TLS 1.2 Handshake — Deep Dive
- HTTP Connection Closing (by Version)
- HTTP/1.1 — The Long-Lived Workhorse (1997)
- Chunked Transfer Encoding
- HTTP/2 — Binary, Multiplexed HTTP (2015)
- HTTP/3 — HTTP over QUIC/UDP (2022)
- How QUIC Ensures Reliable Data Transfer
- HTTP Methods
- HTTP Status Codes
- TCP Flags (Control Bits)
- Likely Interview Questions
- Important HTTP Request and Response Headers
- Sources
- Related Notes

What is HTTP?

- HTTP (HyperText Transfer Protocol) is the application-layer protocol used to transfer data between a client (browser, API client) and a server.
- Created by Tim Berners-Lee and his team at CERN around 1989–1991, originally to transfer simple hypertext documents in a small, trusted environment.
- Over time it evolved to efficiently carry large images, video, JSON APIs, file downloads, etc., while keeping its core semantics (methods, status codes, headers) largely stable.

Content evolution (good one-liner for interviews): HTTP started out carrying **only plain text** (HTML pages). As the web grew, it had to support **images, audio, video, and arbitrary file transfer** — this is exactly why **Content-Type** and **Content-Length** headers became necessary starting with HTTP/1.0.

Timeline at a Glance



Version Timeline (Quick Recall)

Version	RFC	Year	Key Idea
HTTP/0.9	None (informal)	~1990–91	Single-line GET, no headers
HTTP/1.0	RFC 1945	1996	Headers + status codes
HTTP/1.1	RFC 2068 → RFC 2616 → RFC 9110/9112	1997 (updated 1999, 2022)	Persistent connections
HTTP/2	RFC 7540	2015	Binary, multiplexed
HTTP/3	RFC 9114	2022	Runs over QUIC/UDP

HTTP/0.9 — The "One-Line" Protocol

Context: Earliest experimental HTTP, used with the first web browsers/servers, pre-standardization.

Characteristics:

- Only one method existed: `GET`. No `POST`, `HEAD`, etc.
- The entire request was a single line, e.g. `GET /path` — no HTTP version included.
- No headers in the request or response; the server returned the raw HTML body directly.
- No status line or status codes — errors could only be communicated through HTML content itself.
- Limited to plain HTML text; there was no concept of content-type negotiation or binary formats.

Limitations:

- No caching control, no content-type, no virtual hosting, no authentication, no extensibility via headers.
- Every response was just raw bytes with zero metadata, making automation or advanced features practically impossible.

HTTP/1.0 — Extensibility Begins (1996)

Standardized as: RFC 1945 (May 1996) — it formalized common practice already in use by existing implementations.

Key improvements over 0.9:

- HTTP version added to the request line: `GET /index.html HTTP/1.0`.
- Introduced request/response **headers**, enabling metadata like `Content-Type`, `Content-Length`, `User-Agent`.
- Added a **status line and status codes** (e.g., `HTTP/1.0 200 OK`, `404 Not Found`) for machine-readable success/failure.
- Supported multiple methods: `GET`, `HEAD`, `POST`.
- `Content-Type` allowed images, video, and application data — not just HTML.

Limitations:

- Connections were typically **non-persistent** — one new TCP (and, for HTTPS, TLS) handshake per request, causing high latency.
- Caching and virtual hosting existed but were basic compared to HTTP/1.1.

HTTPS context: Netscape introduced SSL in the mid-1990s; HTTPS at this stage meant HTTP/1.0 running over SSL/TLS for confidentiality and integrity.

Connection steps for one request (the classic interview walk-through):

1. **TCP handshake** — `SYN → SYN/ACK → ACK`.
2. **Certificate check / TLS (SSL) handshake** — server presents its certificate, client verifies it against a trusted CA.
3. **Key exchange** — client and server agree on a shared session key (RSA or (EC)DHE) to encrypt the rest of the conversation.
4. **Data transfer** — the actual HTTP request/response is sent over the now-encrypted channel.
5. (*HTTP/1.0 problem*): connection closes after this — **every single request repeats steps 1–4 from scratch**, which is the main reason HTTP/1.1 made connections persistent.

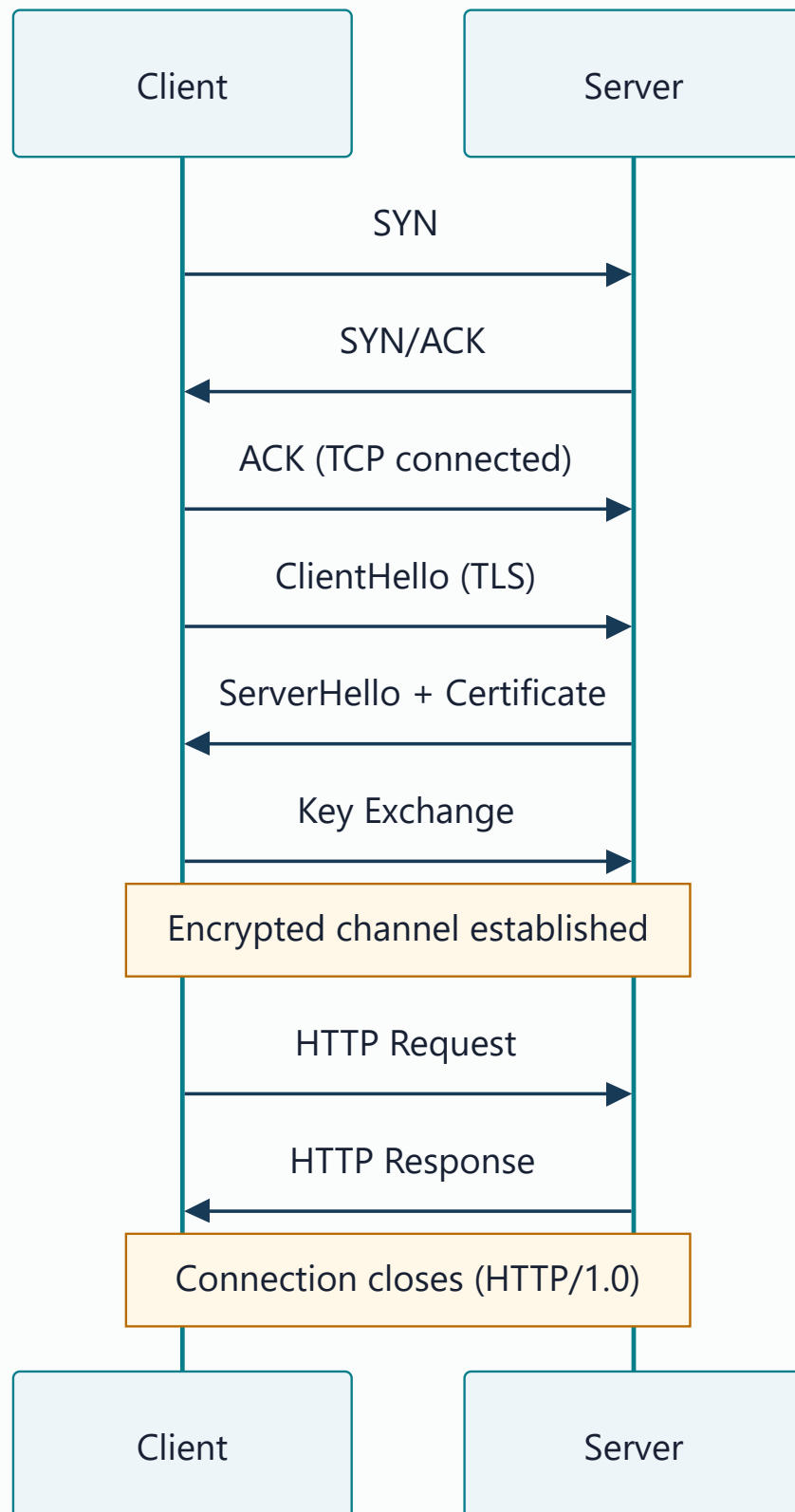


Diagram 2

TLS Handshake — Detailed Breakdown

Where does TLS fit? TLS sits between the Transport layer (TCP) and the Application layer (HTTP). It is what turns `http://` into `https://`. The TLS handshake happens **after** the TCP 3-way handshake and **before** any HTTP data is sent.

Phase-by-Phase Walkthrough

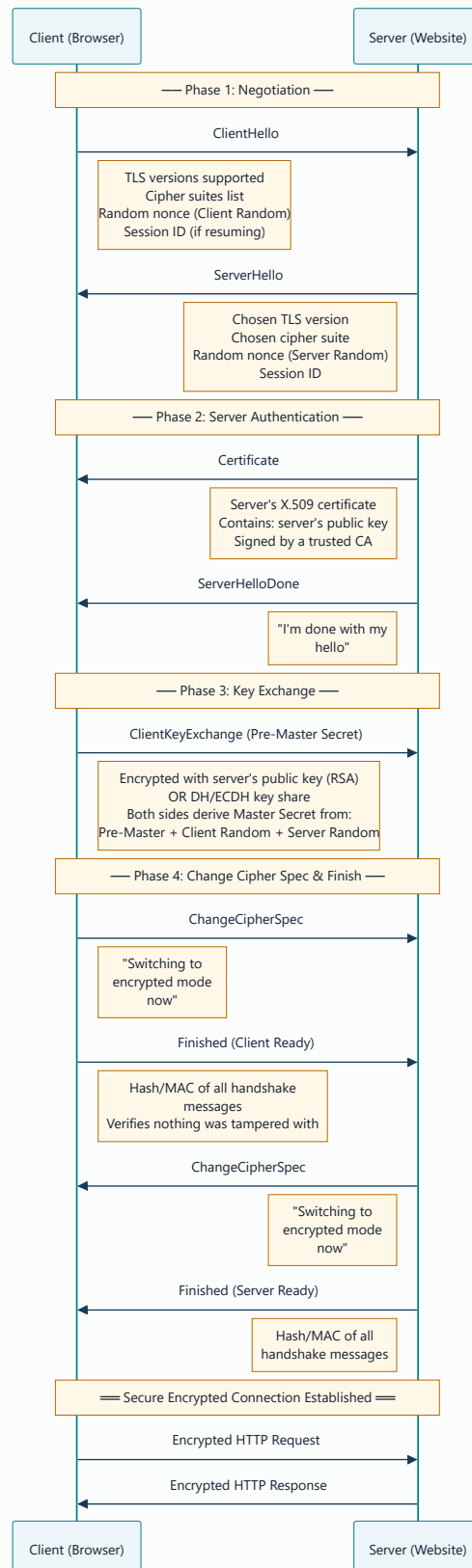


Diagram 3

TLS 1.2 Handshake — Deep Dive

This is the full story of how your browser and a server go from strangers to a secure, encrypted tunnel. Every step builds on the last, so nothing can be skipped.

Step 1 — ClientHello (Client → Server)

The client kicks off the conversation by saying: *"Here's what I support — you pick."*

- **Supported TLS versions:** e.g., TLS 1.0, 1.1, 1.2 (or even 1.3)
- **Cipher suites:** An ordered list like `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` — each entry defines the key exchange algo, auth method, symmetric cipher, and MAC algo
- **Client Random:** A 32-byte nonce (timestamp + random bytes) — critical for key derivation later
- **Session ID:** If the client has a previous session, it sends this to attempt resumption (skip the full handshake)
- **Extensions:** SNI (Server Name Indication so one server can host many domains), supported elliptic curves, compression methods

Think of it as the client handing over a menu and saying "order anything from this list."

Step 2 — ServerHello (Server → Client)

The server responds by making its choices from the client's menu.

- **Chosen TLS version:** The highest version both sides support
- **Chosen cipher suite:** e.g., `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`
- **Server Random:** Another independent 32-byte nonce — also critical for key derivation
- **Session ID:** Either echoes back the client's ID (resumption) or creates a new one

Both randoms (**Client Random + Server Random**) are now known to both parties and will be mixed into all key material — this is why replaying a captured handshake doesn't work.

Step 3 — Certificate (Server → Client)

The server proves its identity by sending its **X.509 certificate**.

What the certificate contains:

- Server's **public key** (RSA or EC)
- Server's **domain name** (Common Name / SAN)
- **Digital signature** (by the CA private key) by a Certificate Authority (CA)
- Validity period, serial number, etc.

What the client does with it:

1. Checks the CA signature — verifies it chains up to a **root CA** in its trusted store (built into your OS/browser)
2. Validates the domain matches what it connected to (prevents MITM)
3. Checks the certificate isn't expired or revoked (via CRL or OCSP)

If any check fails → **handshake aborted**. No exceptions.

Note: The client verifies the CA's digital signature by using the CA's public key to check that the certificate was really signed by that CA and was not changed. First, it hashes the certificate data itself, then it compares that hash with the one recovered from the CA's signature; if both match, the signature is valid.

Step 4 — ServerHelloDone (Server → Client)

A simple signal: *"I'm done with my part of the negotiation. Your turn."*

In mutual TLS (mTLS), the server would also send a **CertificateRequest** here, asking the client to prove its identity too. In standard HTTPS, this step is skipped.

Step 5 — ClientKeyExchange (Client → Server)

This is the most cryptographically important step — where the **shared secret** is established.

RSA Mode (older, not forward-secret):

- Client generates a **Pre-Master Secret** (48 random bytes)
- Encrypts it with the **server's public key** from the certificate
- Sends it — only the server can decrypt it with its private key

ECDHE/DHE Mode (modern, forward-secret):

- Client generates its own ephemeral key pair
- Sends its **DH/ECDH public share** to the server
- Server does the same (sent earlier in a `ServerKeyExchange` message)
- Both sides independently compute the same Pre-Master Secret using Diffie-Hellman math:

```
Pre-Master Secret = Client_Private × Server_Public = Server_Private × Client_Public
```

Master Secret Derivation:

Once both sides have the Pre-Master Secret, they compute:

```
Master Secret = PRF(pre_master_secret, "master secret", ClientRandom || ServerRandom)
```

The **PRF (Pseudorandom Function)** expands this into 48 bytes. Then a second expansion generates the actual **session keys**:

- Client write key (client → server encryption)
- Server write key (server → client encryption)
- Client MAC key
- Server MAC key
- IVs (initialization vectors)

Both sides compute all of this **independently** — no keys are ever transmitted.

Step 6 — ChangeCipherSpec (Client → Server)

A simple **1-byte message** (value `0x01`) on a separate TLS record layer type.

It means: *"Everything I send from this point forward will be encrypted using the negotiated cipher and the session keys we just derived."*

This is NOT a handshake message — it's a protocol-level signal to flip the encryption switch.

Step 7 — Finished (Client → Server)

The first **encrypted** message of the handshake. It contains:

```
verify_data = PRF(master_secret, "client finished", MD5(H) || SHA1(H))
```

Where **H** = a hash of **all handshake messages sent so far**.

This proves two things:

1. The client derived the **correct master secret** (otherwise the MAC won't match)
2. The handshake messages were **not tampered with** in transit (MITM detection)

If the server can't verify this MAC → connection dropped immediately.

Step 8 — ChangeCipherSpec (Server → Client)

Same 1-byte signal from the server: *"I'm also switching to encrypted mode now."*

Step 9 — Finished (Server → Client)

The server sends its own **Finished** message — a MAC over all handshake messages **including the client's Finished**.

At this point:

- Client verifies the server's Finished → confirms server has the same master secret
- **Both sides have mutually verified the entire handshake**
- The secure channel is now fully established

Step 10 — Encrypted Application Data (Client ↔ Server)

HTTP requests and responses now flow inside TLS records, encrypted with the agreed symmetric cipher (e.g., **AES-256-GCM**).

AES-GCM provides **AEAD** (Authenticated Encryption with Associated Data) — meaning it encrypts *and* authenticates simultaneously, so you get both confidentiality and integrity in one operation.

Key Concepts Within the TLS Handshake

Random Nonces (Client Random + Server Random)

- Both client and server generate a 32-byte random value at the start.
- These are combined with the Pre-Master Secret to derive the **Master Secret**.
- Purpose: ensures that even if the same Pre-Master Secret were used twice, each session's keys are different — prevents replay attacks.

Pre-Master Secret → Master Secret → Session Keys



Client Write IV Server Write IV

Certificate Verification (Chain of Trust)

```

Root CA (self-signed, pre-installed in your OS/browser)
├── Intermediate CA (signed by Root CA)
│   └── Server Certificate (signed by Intermediate CA)
│       └── Contains: server's domain, public key, validity dates
  
```

- Client checks: Is the certificate signed by a CA it trusts? Is the domain name correct? Is it within the validity period? Has it been revoked (CRL/OCSP check)?
- If any check fails → TLS alert, connection aborted.

Cipher Suite (what the ServerHello chooses):

Example: `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`

Part	Meaning
TLS	Protocol
ECDHE	Key exchange algorithm (Elliptic Curve Diffie-Hellman Ephemeral — provides forward secrecy)
RSA	Authentication algorithm (server cert is RSA-signed)
AES_128_GCM	Symmetric encryption (AES 128-bit in Galois/Counter Mode)
SHA256	MAC / hash function for message integrity

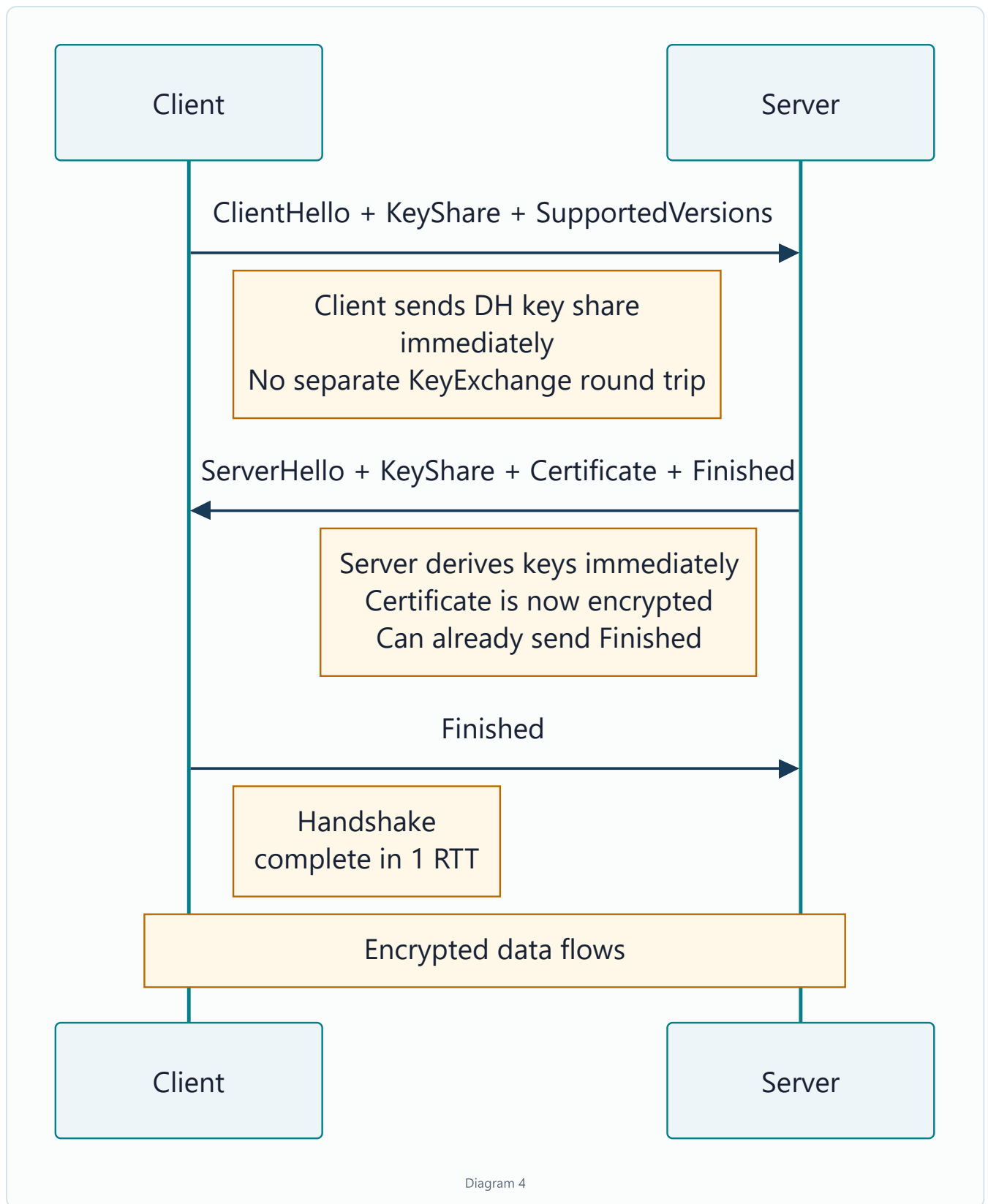
Forward Secrecy (DHE / ECDHE):

- If RSA is used for key exchange (not just authentication), a stolen private key can decrypt **all past sessions** (since the Pre-Master Secret was encrypted with that key).
- ECDHE/DHE generate **ephemeral** (temporary, per-session) DH key pairs. Even if the server's private key is later stolen, past session keys cannot be derived — each session's key exists only in memory during that session.
- **Interview trap:** DHE/ECDHE provide forward secrecy; plain RSA key exchange does not.

TLS 1.2 vs TLS 1.3 — Key Differences

Feature	TLS 1.2	TLS 1.3
Handshake RTTs	2-RTT (full), 1-RTT (resumption)	1-RTT (full), 0-RTT (resumption)
Cipher suites	Many (including weak ones like RC4, 3DES)	Only 5 strong suites (AEAD only)
Key exchange	RSA or DHE/ECDHE	DHE/ECDHE only (forward secrecy mandatory)
ChangeCipherSpec	Explicit message	Removed (implicit)
Certificate encryption	Certificate sent in plaintext	Certificate is encrypted
0-RTT	Not supported	Supported (with replay risk)
Status	Still in use	Recommended; default in modern browsers

TLS 1.3 Handshake (simplified — only 1-RTT):



TLS vs SSL — Terminology Clarification

Term	Reality
SSL 2.0	Deprecated, insecure — do not use
SSL 3.0	Deprecated (POODLE attack, 2014) — do not use

Term	Reality
TLS 1.0	Deprecated (RFC 8996, 2021)
TLS 1.1	Deprecated (RFC 8996, 2021)
TLS 1.2	Still widely used, acceptable with strong cipher suites
TLS 1.3	Current standard — preferred

"SSL" is colloquially still used (e.g., "SSL certificate", "SSL termination") but technically all modern implementations use **TLS**. Saying "TLS" in interviews is correct.

Where TLS Sits in the Full HTTPS Request Flow

1. DNS resolution → resolve domain to IP
2. TCP handshake → SYN / SYN-ACK / ACK (3 messages, ~1 RTT)
3. TLS handshake → ClientHello ... Finished (~2 RTT for TLS 1.2, ~1 RTT for TLS 1.3)
4. HTTP request → GET /index.html HTTP/1.1
5. HTTP response → 200 OK + body
6. (HTTP/1.0 only) → Connection closes, repeat steps 2–5 for next resource
(HTTP/1.1+) → Connection reused for next request

Total round trips before first byte of data:

Scenario	RTTs
TLS 1.2 over HTTP/1.1 (new conn)	~3.5 RTT
TLS 1.3 over HTTP/2	~2.5 RTT
HTTP/3 / QUIC (new connection)	~1 RTT
HTTP/3 / QUIC (0-RTT resume)	~0 RTT

Interview Questions on TLS

1. What happens if the certificate is expired or untrusted?

- The TLS handshake aborts with a fatal alert (`certificate_expired` or `unknown_ca`). The browser shows a security warning.

2. What is SNI (Server Name Indication)?

- An extension in ClientHello where the client sends the hostname it's trying to reach. Allows one server with one IP to host multiple TLS certificates (multiple domains). Without SNI, the server doesn't know which certificate to send before the TLS handshake completes.

3. What is OCSP Stapling?

- Instead of the client contacting the CA to check if a certificate is revoked (slow, privacy leak), the server periodically fetches a signed OCSP response from the CA and "staples" it to the TLS handshake — faster and more private.

4. What is mutual TLS (mTLS)?

- In standard TLS, only the server presents a certificate. In mTLS, the **client also presents a certificate** that the server verifies. Used for machine-to-machine authentication (microservices, API gateways, zero-trust networks).

5. Why can't you just skip TLS for internal services?

- Internal network traffic is still vulnerable to insider threats, compromised internal hosts, and lateral movement after a breach. TLS encrypts in transit even on internal networks — the foundation of zero-trust architecture.

HTTP Connection Closing (by Version)

TCP flags (`FIN` / `RST`) close the *transport* connection — but each HTTP version has its own *application-layer* signal for when a connection should be closed, layered on top of that.

HTTP/1.0 — closes by default after every response (no persistence). To opt into reuse, both sides had to explicitly send `Connection: keep-alive`.

HTTP/1.1 — persistent by default. Closes when:

- Either side sends the `Connection: close` header, or
- The connection sits idle past a timeout, or
- An error occurs.

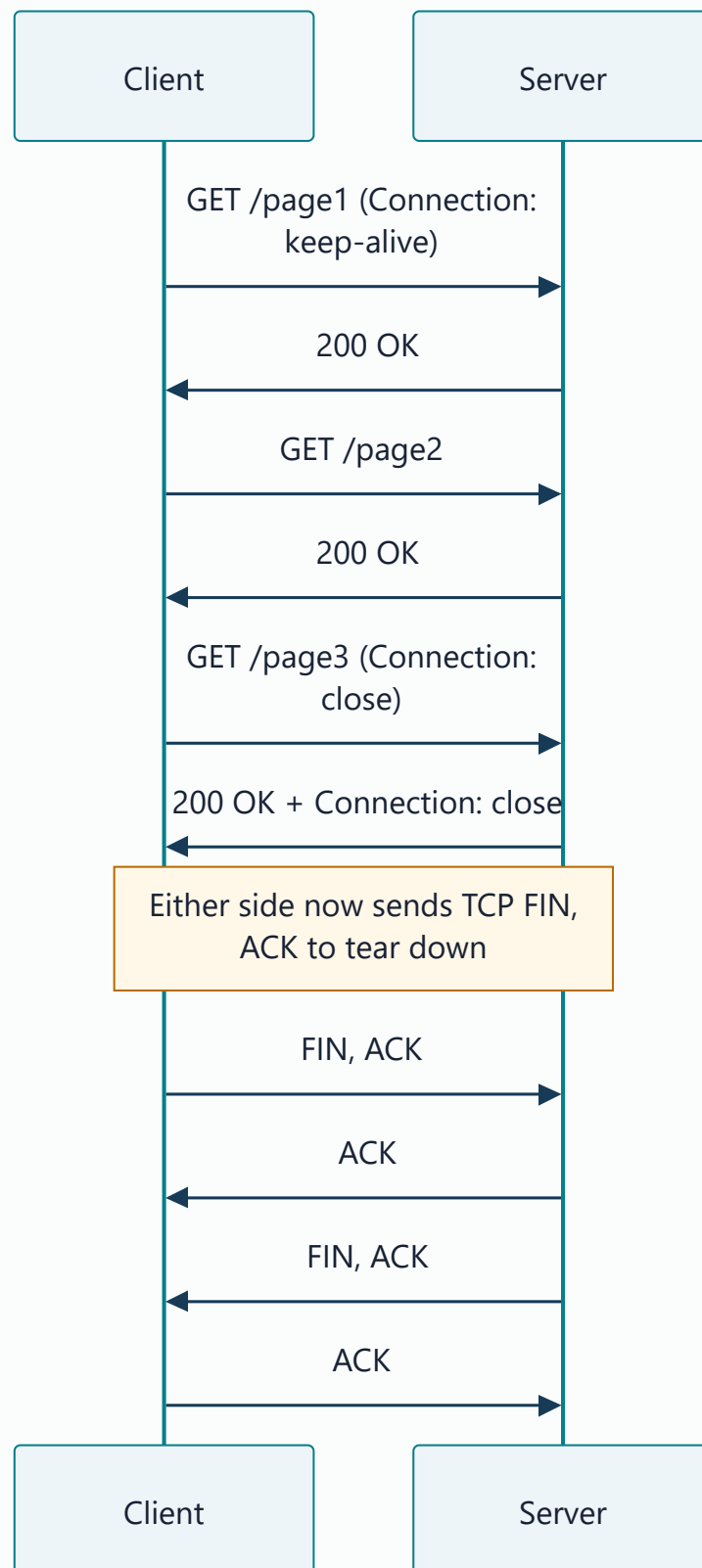


Diagram 5

HTTP/2 — uses a `GOAWAY` frame instead of relying on `Connection: close` (that header is actually disallowed in HTTP/2). The server announces it won't accept new streams, lets existing in-flight streams finish, then the underlying TCP connection closes.

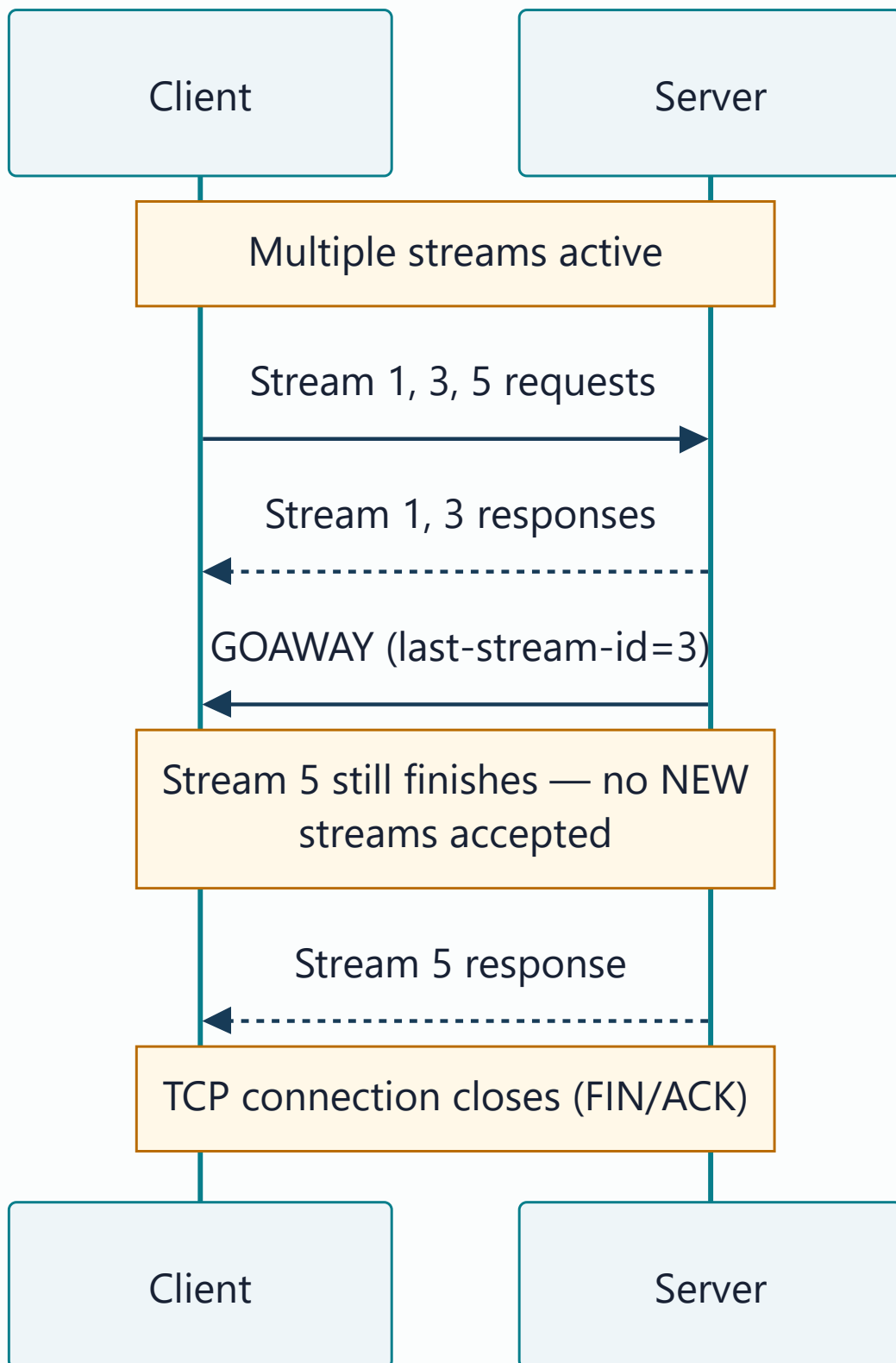


Diagram 6

HTTP/3 — since there's no TCP, closing works differently:

- A single stream can be closed independently (`STREAM` with a `FIN` bit) without affecting others.
- The whole QUIC connection is closed with a `CONNECTION_CLOSE` frame, which can carry an application-level error code/reason — this replaces the TCP FIN/RST handshake entirely.

One-line summary for interviews: *Closing happens at two layers — TCP closes the pipe (`FIN` / `RST`), while HTTP decides **when** to close it (`Connection: close` in 1.x, `GOAWAY` in HTTP/2, `CONNECTION_CLOSE` in HTTP/3 since QUIC has no separate TCP layer to close).*

HTTP/1.1 — The Long-Lived Workhorse (1997)

Standardized as: RFC 2068 (Jan 1997), refined by RFC 2616 (1999). Today its wire format is RFC 9112 and its semantics are RFC 9110.

It remained the dominant version of HTTP on the web for **over 15 years** before HTTP/2 arrived.

Major features:

- **Persistent connections by default** — a single TCP connection can serve multiple request/response pairs, so there's **no repeated TCP/TLS handshake** for every object on a page (the big fix over HTTP/1.0).
- **Pipelining** — client can send multiple requests back-to-back without waiting for each response (intended to cut latency).
- **Chunked transfer encoding** — see full explanation below.
- **Better caching** — headers like `Cache-Control`, `ETag`, and validators for more precise cache rules.
- **Mandatory `Host` header** — enables name-based virtual hosting (many domains on one IP).
- **Congestion control improvements** — since one TCP connection now carries many requests, TCP's congestion-control behaviour (slow start, window scaling) matters more and is used more efficiently than opening a fresh, "cold" connection per request as in HTTP/1.0.

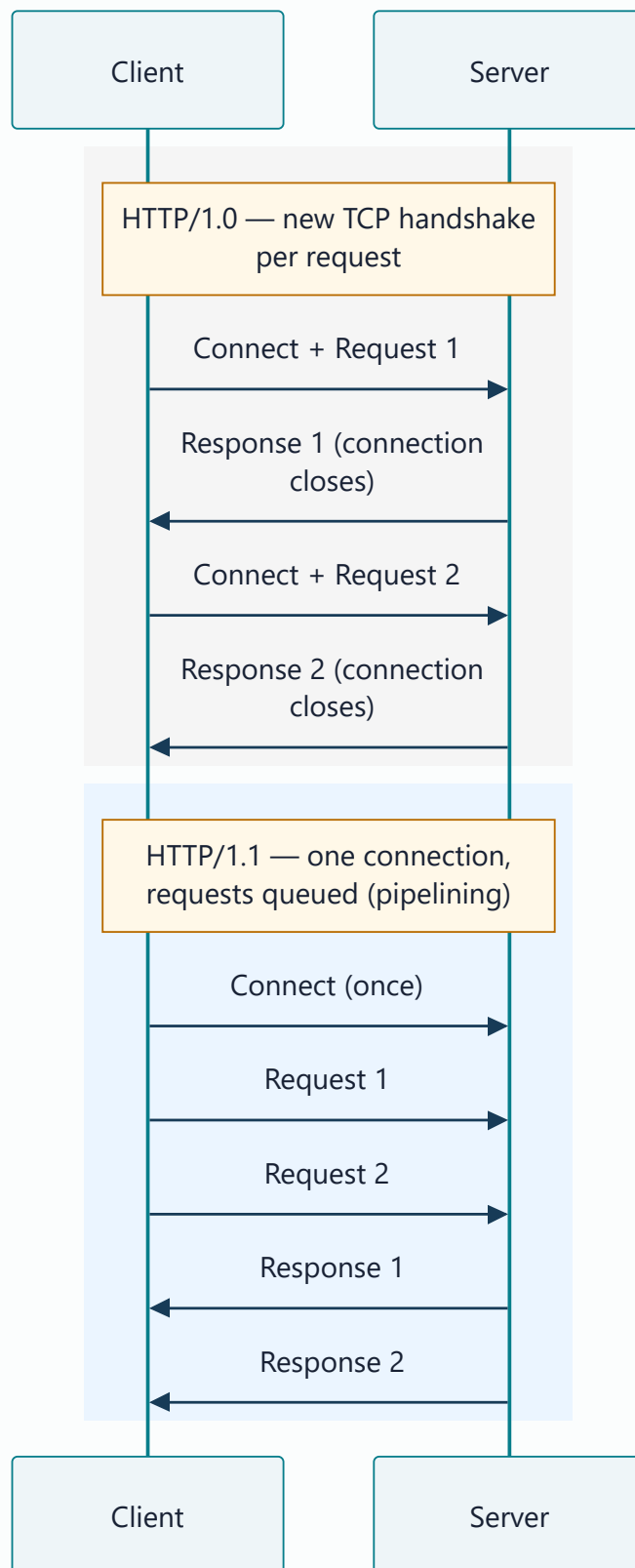


Diagram 7

Head-of-line (HOL) blocking & workaround:

- Pipelining's drawback: if one response is slow, every later response on that connection is stuck behind it (application-layer HOL blocking).

- Browsers worked around this by opening several parallel TCP connections per origin (commonly 4–6) and using domain sharding.

TLS handshake flow over HTTP/1.x (good interview talking point):

1. TCP 3-way handshake: `SYN → SYN/ACK → ACK`.
2. TLS handshake: negotiate protocol version + cipher suite, exchange keys (RSA or (EC)DHE), verify server certificate, optionally authenticate client.
3. Once TLS is established, encrypted HTTP/1.x requests/responses flow over the secure channel.

Chunked Transfer Encoding

What it is:

In normal HTTP, the server sends `Content-Length` to tell the client how many bytes the body has. But sometimes the server **cannot know the total size before sending** (e.g., streaming data, dynamic reports, live logs). In these cases, HTTP/1.1 uses **chunked transfer encoding**.

Instead of `Content-Length`, the server sends:

```
Transfer-Encoding: chunked
```

HTTP

The body is sent as a series of **chunks**, each formatted as:

1. Chunk size in **hex** + `\r\n`
2. Chunk data (that many bytes)
3. `\r\n`

The response ends with a **zero-length chunk** to signal end of body:

```
0\r\n
\r\n
```

Example on the wire:

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked
```

HTTP

```
7\r\n
Mozilla\r\n
9\r\n
Developer\r\n
7\r\n
Network\r\n
0\r\n
\r\n
```

(Chunk sizes: `7` = "Mozilla", `9` = "Developer", `7` = "Network", `0` = end)

When is it used?

- Live log streaming, server-sent events, dynamic page rendering, large file downloads where total size isn't known upfront.
- The client knows the response is finished only when it receives the `0\r\n\r\n` terminator — not from a byte count.

HTTP/2 — Binary, Multiplexed HTTP (2015)

Standardized as: RFC 7540 (May 2015), based on Google's experimental **SPDY** protocol — but keeps HTTP semantics (methods, URLs, headers, status codes) unchanged.

Key features:

- **Binary framing layer** — messages are encoded as binary frames instead of human-readable text, improving parsing efficiency and extensibility.
- **Full multiplexing** — multiple streams share one TCP connection, with frames interleaved; this removes HTTP/1.1's application-layer HOL blocking.
- **Stream prioritization** — client can assign priority/dependency to streams (e.g., load CSS before images).
- **Header compression (HPACK)** — compresses repetitive headers (cookies, user-agent, etc.), cutting overhead significantly.
 - **Important contrast:** in HTTP/1.x, only the **body** could be compressed (e.g. via `Content-Encoding: gzip`) — headers were always sent as plain repeated text on every request. HPACK in HTTP/2 is the first version to compress the **headers** too.
- **Server push** — server can proactively push resources (CSS/JS) into the client's cache before they're explicitly requested.

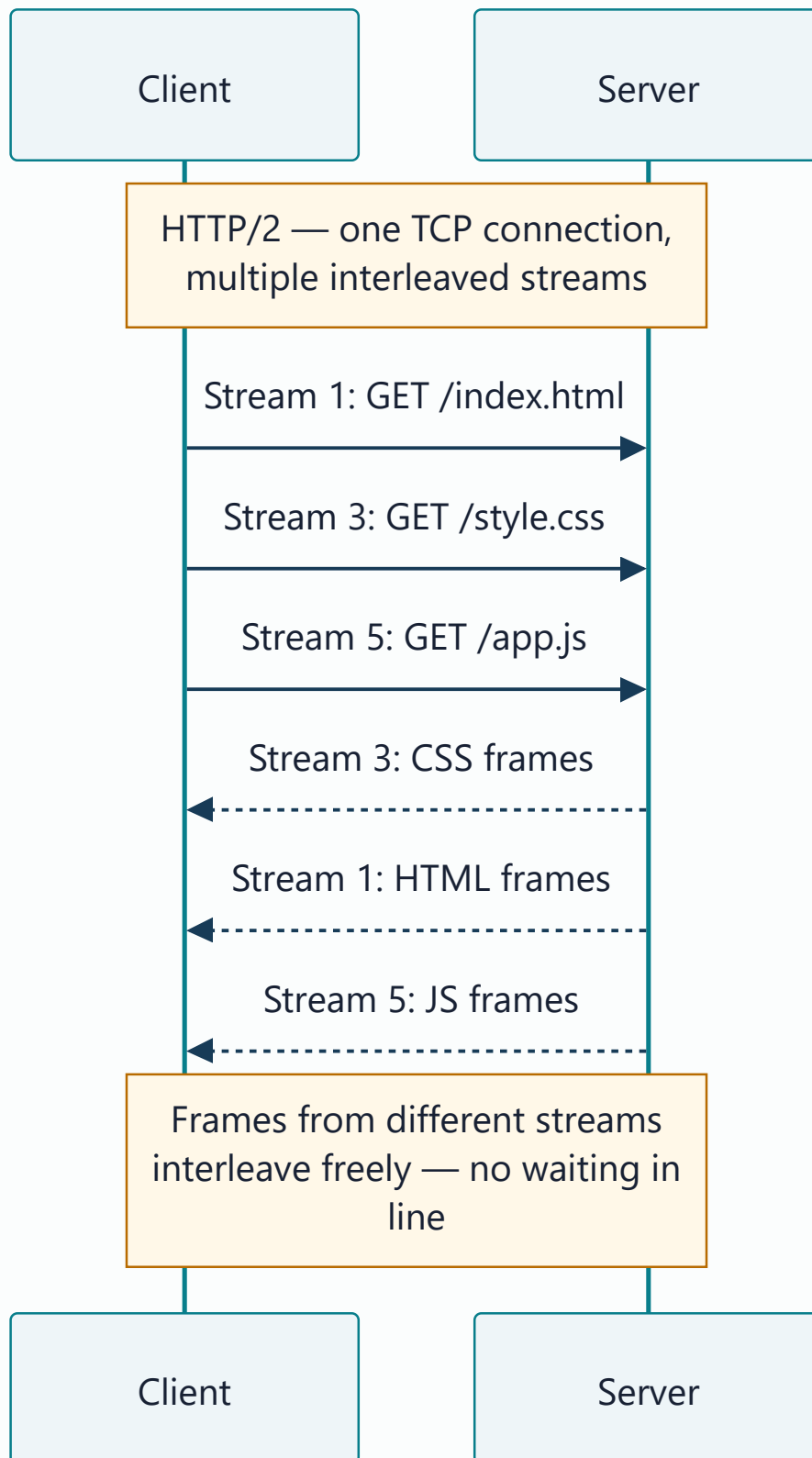


Diagram 8

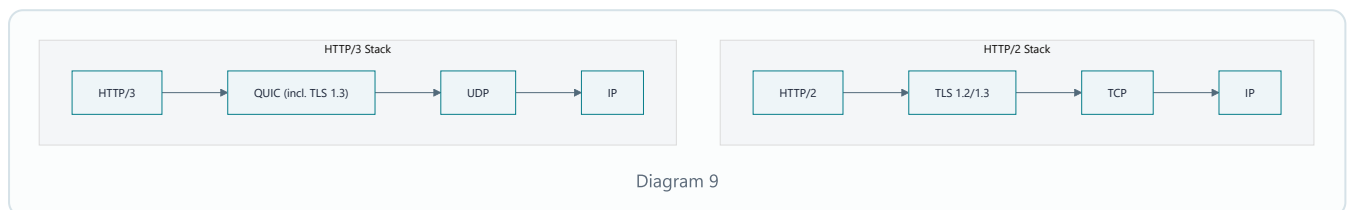
Remaining limitation:

- Streams are multiplexed at the HTTP layer but still share **one TCP connection**. If a TCP segment is lost, *all* streams stall waiting for retransmission — this is **transport-layer HOL blocking**, and it's the main reason HTTP/3 exists.

HTTP/3 — HTTP over QUIC/UDP (2022)

Standardized as: RFC 9114 (June 2022). Runs over **QUIC**, a transport protocol built on UDP (RFC 9000 for transport, RFC 9001 for QUIC + TLS 1.3).

Origin: QUIC was originally designed and deployed by **Google** (used internally in Chrome and Google's own services) before being handed over to the IETF, which standardized it as RFC 9000 — the same pattern as SPDY → HTTP/2.



Core concepts:

- **QUIC replaces TCP** — retransmission, congestion control, and reliability are implemented in user space over UDP, not in the kernel's TCP stack.
- **No transport-layer HOL blocking** — QUIC multiplexes independent streams within one connection; packet loss on one stream does not block the others.
- **Mandatory TLS 1.3** — QUIC integrates TLS 1.3 directly; all HTTP/3 traffic is encrypted, with no plaintext mode.

Performance features:

- **Faster connection setup** — QUIC combines the transport and crypto handshakes, typically achieving **1-RTT** for new connections and **0-RTT** for resumed sessions.
- **Connection migration** — QUIC connections are identified by Connection IDs (not IP/port), so a device can switch networks (Wi-Fi → 4G) without dropping the connection.
- **Better mobile/wireless performance** — per-stream loss isolation makes HTTP/3 more robust on lossy, high-latency mobile networks.

Deployment status: Supported by major browsers (Chrome, Edge, Firefox) and most CDNs; a significant share of web traffic has shifted from HTTP/2 to HTTP/3 since 2022.

How QUIC Ensures Reliable Data Transfer

UDP on its own is unreliable — no ordering, no retransmission, no delivery guarantee. QUIC re-implements all of TCP's reliability mechanisms **in user space**, per-stream, on top of UDP. Here's exactly how:

1. Packet Numbering (always increasing)

Every QUIC packet gets a **monotonically increasing packet number** — never reused, even on retransmission. This is a key improvement over TCP sequence numbers, which reuse the same number for retransmitted data and make it hard to distinguish "original" from "retransmit".

- TCP problem: ACK for seq=500 — was that the original or the retransmit? (the **retransmission ambiguity problem**)
- QUIC fix: each retransmit gets a **new, higher packet number**, so the sender always knows exactly which packet was acknowledged.

2. Stream Offsets (ordering within a stream)

Packet numbers alone don't define ordering of data. QUIC uses **stream offsets** (like TCP sequence numbers, but per-stream) to reassemble data in the correct order within each stream, independently of other streams.

```
Stream 1: [offset 0-99] [offset 100-199] [offset 200-299]
Stream 3: [offset 0-49] [offset 50-99]
→ loss of a Stream 1 packet does NOT stall Stream 3
```

3. ACKs and Retransmission

- Receiver sends **ACK frames** listing which packet numbers were received.
- QUIC supports **SACK-like selective acknowledgement by default** — it can ACK non-contiguous ranges (e.g., "got 1–5 and 8–10, missing 6–7"), so the sender only retransmits the specific missing packets, not everything after the gap.
- Lost packets are detected via **ACK delay + packet number gaps**, then retransmitted with a new packet number.

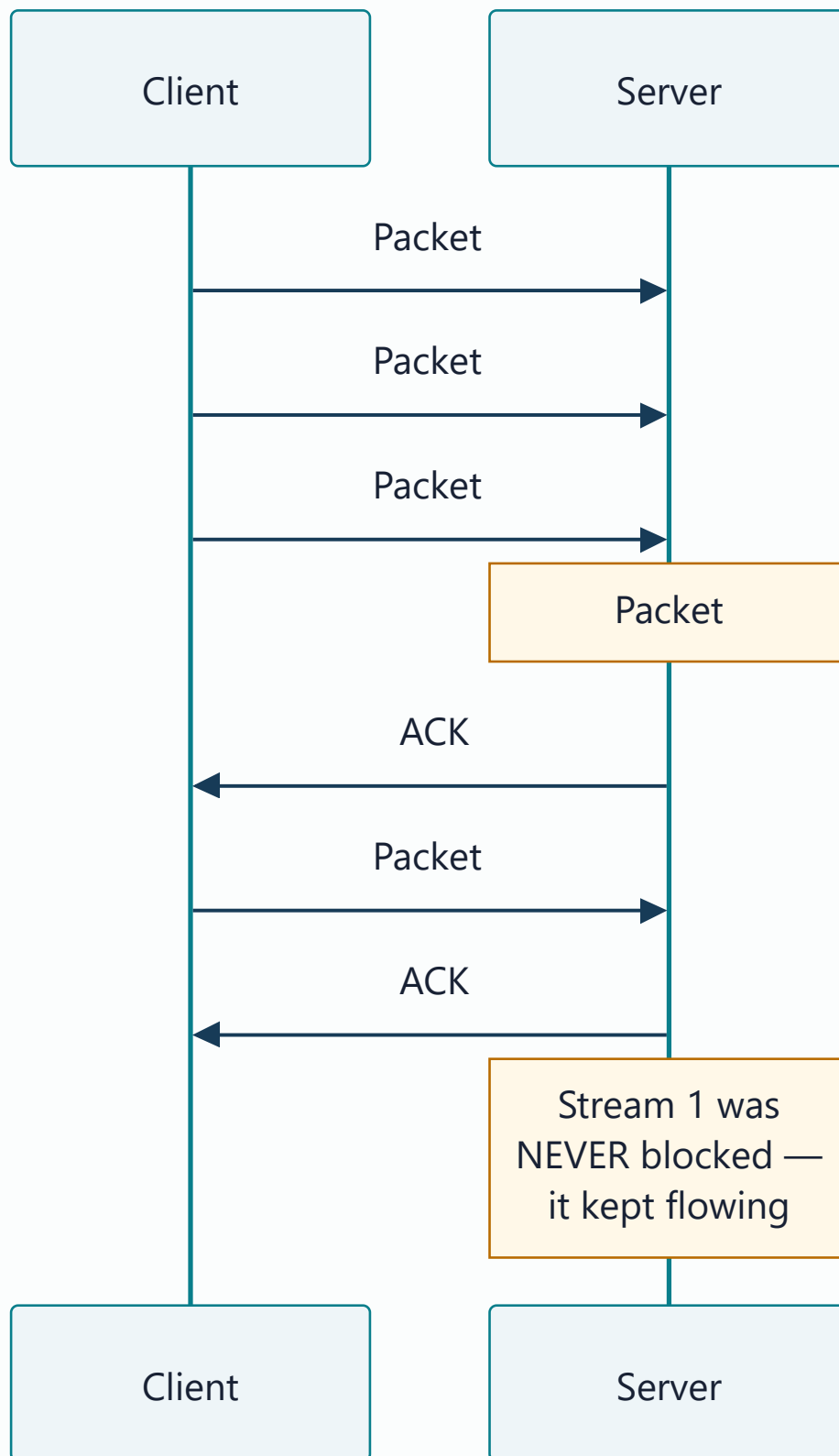


Diagram 10

4. Congestion Control (in user space)

QUIC implements congestion control algorithms (default: **CUBIC** or **BBR**, same as modern TCP) entirely in user space. This means:

- It can be **updated without OS kernel patches** — just update the QUIC library.

- Different connections or applications can use different congestion control algorithms simultaneously.

5. Flow Control (two levels)

QUIC has flow control at **two levels**, unlike TCP which only has connection-level flow control:

Level	What it controls
Stream-level	How much data can be buffered per individual stream
Connection-level	Total data across all streams on one QUIC connection

This prevents a single fast stream from starving others, and prevents the receiver's buffer from being overwhelmed overall.

6. Connection Migration (no reconnect needed)

TCP identifies a connection by the 4-tuple: `src IP + src port + dst IP + dst port`. Change any one of those (e.g., switch from Wi-Fi to 4G) and TCP breaks — you need a full new handshake.

QUIC uses **Connection IDs** instead. When the network changes:

- The same Connection ID continues on the new network path.
- QUIC performs a **path validation** (sends a `PATH_CHALLENGE`, waits for `PATH_RESPONSE`) to verify the new path works.
- No new handshake, no data loss, seamless to the application.

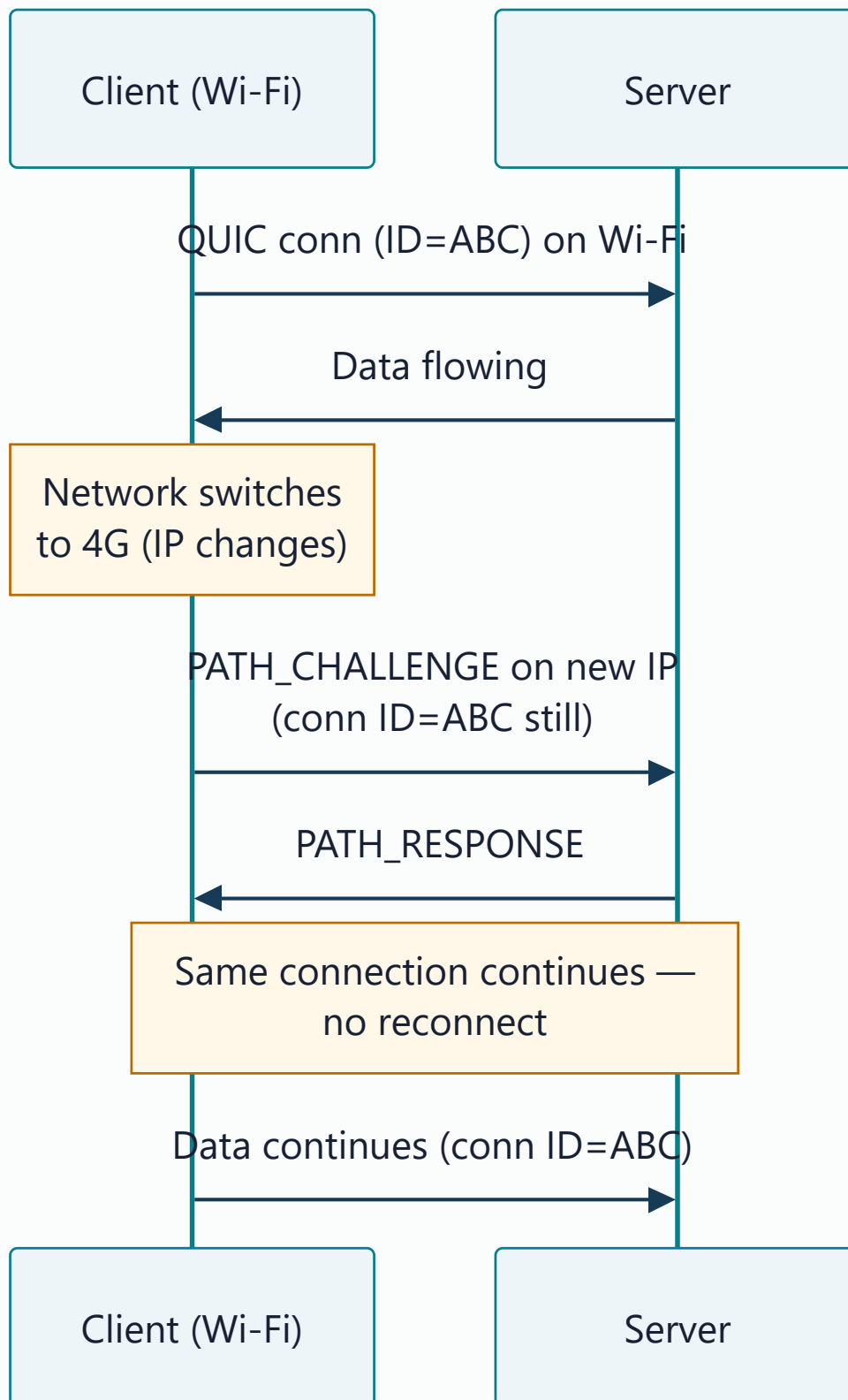


Diagram 11

7. 0-RTT Resumption

For sessions that were previously established, QUIC + TLS 1.3 allows the client to send application data on the **very first packet** (0 round trips before data) using a cached session ticket. This skips the handshake entirely for returning connections.

Tradeoff: 0-RTT data is vulnerable to **replay attacks** (an attacker could replay the first packet), so safe usage is limited to idempotent requests (e.g., `GET`, not `POST`).

HTTP Methods

Method	Purpose	Safe?	Idempotent?	Has Body?
<code>GET</code>	Retrieve a resource	Yes	Yes	No (request body ignored)
<code>HEAD</code>	Like GET, but headers only — no response body	Yes	Yes	No
<code>POST</code>	Create a resource / submit data	No	No	Yes
<code>PUT</code>	Replace a resource entirely	No	Yes	Yes
<code>PATCH</code>	Partially update a resource	No	No	Yes
<code>DELETE</code>	Remove a resource	No	Yes	Usually no
<code>OPTIONS</code>	Ask server which methods/headers are allowed (used in CORS preflight)	Yes	Yes	No
<code>TRACE</code>	Echoes back the request for debugging (rarely used, security risk)	Yes	Yes	No
<code>CONNECT</code>	Establish a tunnel to the server (used for HTTPS via proxies)	No	No	No

- **Safe** = doesn't change server state (read-only).
- **Idempotent** = calling it once or N times has the same effect on the server (`GET`, `PUT`, `DELETE` are idempotent; `POST` and `PATCH` are not).
- **Interview trap:** `PUT` vs `POST` — `PUT` is idempotent (same request repeated = same result, e.g. "set resource to X"), `POST` is not (repeating it can create duplicates, e.g. "create a new order").

HTTP Status Codes

Class	Range	Meaning	Common Codes
1xx	100–199	Informational	<code>100 Continue</code> , <code>101 Switching Protocols</code>
2xx	200–299	Success	<code>200 OK</code> , <code>201 Created</code> , <code>202 Accepted</code> , <code>204 No Content</code>
3xx	300–399	Redirection	<code>301 Moved Permanently</code> , <code>302 Found</code> , <code>304 Not Modified</code> , <code>307 Temporary Redirect</code>
4xx	400–499	Client error	<code>400 Bad Request</code> , <code>401 Unauthorized</code> , <code>403 Forbidden</code> , <code>404 Not Found</code> , <code>405 Method Not Allowed</code> , <code>409 Conflict</code> , <code>429 Too Many Requests</code>
5xx	500–599	Server error	<code>500 Internal Server Error</code> , <code>502 Bad Gateway</code> , <code>503 Service Unavailable</code> , <code>504 Gateway Timeout</code>

Quick distinctions interviewers love to ask:

- `401 Unauthorized` vs `403 Forbidden` — 401 = "you haven't authenticated" (no/invalid credentials), 403 = "you're authenticated, but not allowed."
- `301` vs `302` — 301 is permanent (browsers/search engines cache the redirect), 302 is temporary (re-check the original URL next time).

- **502** vs **503** vs **504** — 502 = upstream server sent an invalid response, 503 = server is overloaded/down for maintenance, 504 = upstream server took too long to respond.

TCP Flags (Control Bits)

Flag	Name	Purpose
SYN	Synchronize	Initiates a connection, synchronizes sequence numbers
ACK	Acknowledge	Confirms receipt of data/segment
FIN	Finish	Politely requests connection termination (no more data to send)
RST	Reset	Abruptly terminates a connection (e.g., port closed, error)
PSH	Push	Tells the receiver to push buffered data to the application immediately, don't wait to fill the buffer
URG	Urgent	Marks data as urgent — process it ahead of the queue (rarely used today)
ECN-E	ECN-Echo	Signals that the network experienced congestion (Explicit Congestion Notification)
CWR	Congestion Window Reduced	Sender confirms it has reduced its congestion window in response to ECE

Connection lifecycle, flag by flag:

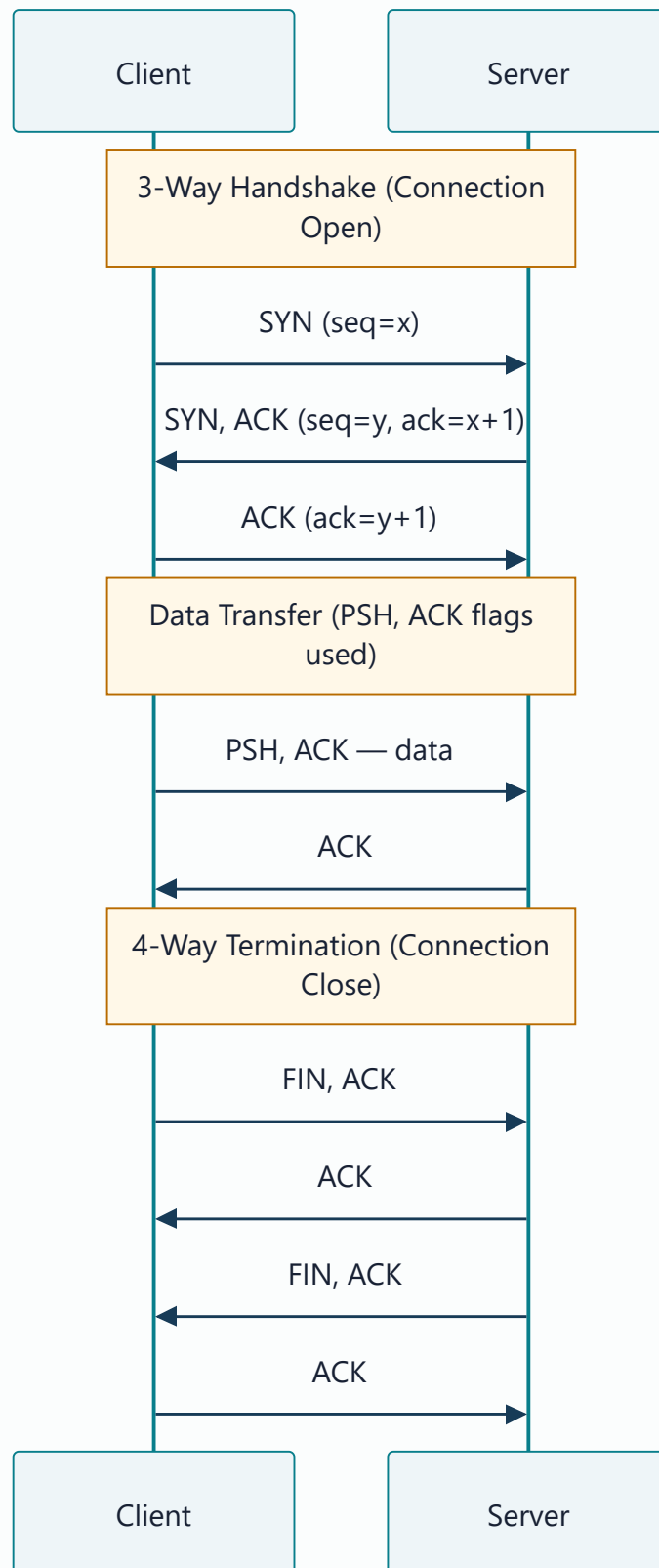


Diagram 12

- **Handshake uses:** `SYN` → `SYN, ACK` → `ACK` (3-way).
- **Graceful close uses:** `FIN, ACK` from each side → 4 messages total (sometimes optimized to 3 if combined).
- **Abrupt close uses:** a single `RST` from either side — skips the polite FIN exchange entirely (e.g., connecting to a closed port, or a crashed application).

Likely Interview Questions

1. What problem did each HTTP version solve over the previous one?

- 0.9 → 1.0: added headers, status codes, methods.
- 1.0 → 1.1: added persistent connections, pipelining, caching, virtual hosting.
- 1.1 → 2: solved app-layer HOL blocking via multiplexing + binary framing.
- 2 → 3: solved transport-layer HOL blocking by replacing TCP with QUIC/UDP.

2. What is Head-of-Line blocking, and at which layer does it occur in HTTP/1.1 vs HTTP/2?

- HTTP/1.1: application layer (pipelining stalls behind one slow response).
- HTTP/2: transport layer (shared single TCP connection stalls on packet loss).
- HTTP/3: solved via QUIC's independent per-stream delivery over UDP.

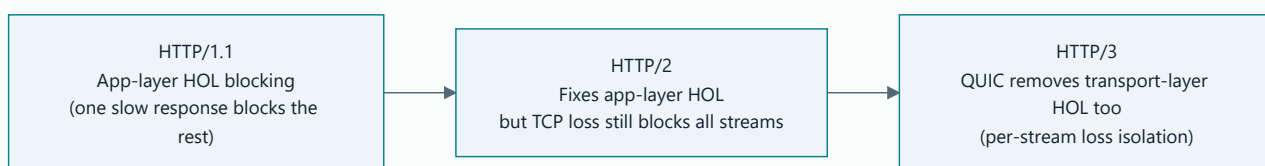


Diagram 13

3. Why does HTTP/3 use UDP instead of TCP?

- To avoid TCP's in-order, single-stream delivery guarantee, which causes HOL blocking when multiplexing. QUIC re-implements reliability and congestion control over UDP per-stream.

4. What is HPACK and why does it matter?

- Header compression in HTTP/2 that reduces overhead from repetitive headers (cookies, user-agent) sent on every request.

5. What is 0-RTT and why is it useful?

- Lets a client resume a previous QUIC/TLS 1.3 session and send data on the very first packet, cutting connection setup latency — especially valuable on high-latency/mobile networks.

Important HTTP Request and Response Headers

Common Request Headers

Header	Purpose
Accept	Media types the client will accept (e.g., <code>text/html</code> , <code>application/json</code>)
User-Agent	Client software info (browser name, version, OS)
Authorization	Credentials for authentication (e.g., <code>Basic</code> , <code>Bearer <token></code>)
Host	Domain name of the server — mandatory in HTTP/1.1
Cookie	Stored cookies sent to the server (<code>name=value</code> pairs)
Referer	URL of the page that linked to the current request

Header	Purpose
Accept-Encoding	Compression algorithms the client accepts (e.g., <code>gzip</code> , <code>deflate</code> , <code>br</code>)
Connection	Controls connection persistence (<code>keep-alive</code> or <code>close</code>)

Common Response Headers

Header	Purpose
Content-Type	Media type of the response body (e.g., <code>text/html</code> , <code>image/png</code> , <code>application/json</code>)
Content-Length	Size of the response body in bytes
Cache-Control	Caching directives: <code>max-age</code> , <code>no-cache</code> , <code>no-store</code> , <code>public</code> , <code>private</code>
ETag	Unique identifier for a specific version of a resource (used for cache validation)
Last-Modified	Timestamp of when the resource was last changed
Set-Cookie	Sends cookies from server to client
Location	Used with <code>301</code> / <code>302</code> redirects to indicate the new URL
Server	Software running on the origin server (e.g., <code>nginx/1.24</code>)
Date	Date and time when the response was generated
Content-Encoding	Compression used on the body (e.g., <code>gzip</code>) — note this is body compression , different from HPACK which is header compression in HTTP/2

Interview note on compression scope:

- `Content-Encoding: gzip` — compresses the **body** only. Works in HTTP/1.x and HTTP/2.
- HPACK** — compresses the **headers** in HTTP/2. Not available in HTTP/1.x at all.

Sources

- MDN Web Docs — Evolution of HTTP
- HTTP Archive / HPBN.co — High Performance Browser Networking
- RFC 1945, RFC 2068, RFC 2616, RFC 7540, RFC 9000, RFC 9001, RFC 9110, RFC 9112, RFC 9114
- RFC 8446 (TLS 1.3), RFC 5246 (TLS 1.2), RFC 8996 (deprecating TLS 1.0/1.1)

Related Notes

- 00 - Syllabus and Interview Checklist
- Computer Networks Index
- 01A - OSI Model
- 02 - TCP-IP Model

Source note: A1 - HTTP Evolution and TLS.md